

School Management System (SMS)

Module: Complete Finance & Fees Codebase (Frontend, Backend & Database Models)

Generated Date: 9/24/2026, 10:07:25 AM

Architecture: C# .NET 8 Web API | MySQL Entity Framework Core | React + TypeScript Frontend

Table of Contents

1. Database & Entity Models (C# .NET EF Core)

- [Models/FinanceManagement/FeeHead.cs](#)
- [Models/FinanceManagement/DynamicFeeStructure.cs](#)
- [Models/FinanceManagement/FeeStructureItem.cs](#)
- [Models/FinanceManagement/FeePayment.cs](#)
- [Models/FinanceManagement/FeeSchedule.cs](#)
- [Models/FinanceManagement/Scholarship.cs](#)
- [Models/FinanceManagement/StudentScholarship.cs](#)
- [Models/FinanceManagement/DiscountRule.cs](#)
- [Models/FinanceManagement/StudentDiscount.cs](#)
- [Models/FinanceManagement/StudentFeeAssignment.cs](#)

2. Backend DTOs & Services (C# .NET 8 API)

- [Dtos/FinanceManagement/FeeHeadDto.cs](#)
- [Dtos/FinanceManagement/FinanceDtos.cs](#)
- [Dtos/FinanceManagement/FinanceMasterDtos.cs](#)
- [Dtos/FinanceManagement/FeeCollectionDtos.cs](#)
- [Controllers/FinanceManagement/FinanceController.cs](#)
- [Controllers/FinanceManagement/FinanceMasterController.cs](#)
- [Controllers/FinanceManagement/FeeCollectionController.cs](#)
- [Services/Implementations/FinanceManagement/FinanceService.cs](#)
- [Services/Implementations/FinanceManagement/FinanceMasterService.cs](#)
- [Services/Implementations/FinanceManagement/FeeCollectionService.cs](#)

3. Frontend API & Core Components (React + TypeScript)

- [src/api/finance.ts](#)
- [Finance/FeeHeadsView.tsx](#)
- [Finance/FeeStructuresView.tsx](#)
- [Finance/StudentFeeAssignmentView.tsx](#)
- [Finance/FeeCollectionView.tsx](#)
- [Finance/DueFeesView.tsx](#)

[Finance/TransactionsMasterLedgerView.tsx](#)

[Finance/ScholarshipsView.tsx](#)

[Finance/DiscountsView.tsx](#)

[Finance/FineRulesView.tsx](#)

[Finance/FinanceDashboardView.tsx](#)

[Finance/FinanceReportsView.tsx](#)

1. Database & Entity Models (C# .NET EF Core)

 Models/FinanceManagement/FeeHead.cs

CSHARP

```
using System.ComponentModel.DataAnnotations;
namespace SMS.Api.Models.FinanceManagement;

public class FeeHead
{
    [Key]
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string Category { get; set; } = string.Empty;
    public string Frequency { get; set; } = string.Empty;
    public decimal DefaultAmount { get; set; }
    public bool Mandatory { get; set; } = true;
    public bool IsRefundable { get; set; }
    public bool IsTaxable { get; set; }
    public decimal TaxPercentage { get; set; }
    public int DisplayOrder { get; set; } = 1;
    public string Status { get; set; } = "Active";
}
```

 Models/FinanceManagement/DynamicFeeStructure.cs

CSHARP

```
using System.ComponentModel.DataAnnotations;
namespace SMS.Api.Models.FinanceManagement;

public class DynamicFeeStructure
{
    [Key]
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string TargetAudience { get; set; } = string.Empty;
    public string AcademicYear { get; set; } = string.Empty;
    public string Branch { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public string StudentCategory { get; set; } = string.Empty;
    public decimal TotalAmount { get; set; }
    public string Status { get; set; } = "Active";
    public string? ItemsJson { get; set; }
}
```

 Models/FinanceManagement/FeeStructureItem.cs

CSHARP

```
using System.ComponentModel.DataAnnotations;
namespace SMS.Api.Models.FinanceManagement;

public class FeeStructureItem
{
    [Key]
    public int Id { get; set; }
    public int DynamicFeeStructureId { get; set; }
    public int FeeHeadId { get; set; }
    public decimal Amount { get; set; }
}
```

 Models/FinanceManagement/FeePayment.cs

CSHARP

```
using System.ComponentModel.DataAnnotations;
namespace SMS.Api.Models.FinanceManagement;

public class FeePayment
{
    [Key]
    public int Id { get; set; }
    public string ReceiptNo { get; set; } = string.Empty;
    public string StudentId { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public decimal DiscountAmount { get; set; }
    public decimal FineAmount { get; set; }
    public decimal TransportFee { get; set; }
    public string TransactionId { get; set; } = string.Empty;
    public System.DateTime PaymentDate { get; set; }
    public string PaymentMethod { get; set; } = string.Empty;
    public string Status { get; set; } = "Completed";
}
```

 Models/FinanceManagement/FeeSchedule.cs

CSHARP

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SMS.Api.Models.FinanceManagement;

[Table("fee_schedules")]
public class FeeSchedule
{
    [Key]
    public string Id { get; set; } = string.Empty;

    public string AcademicYear { get; set; } = "2026-2027";

    public int NumberOfTerms { get; set; } = 4;

    public string Status { get; set; } = "Published";

    public string AnnualDueDate { get; set; } = "2026-04-15";

    public string OneTimeDueDate { get; set; } = "2026-04-15";

    public bool ApplySameDayToAllMonths { get; set; } = true;

    public int MonthlyDueDay { get; set; } = 5;

    public string? TermsJson { get; set; }

    public string? MonthlyConfigJson { get; set; }

    public List<FeeScheduleTerm> Terms { get; set; } = new();

    public List<FeeScheduleMonthlyDate> MonthlyDates { get; set; } = new();

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}
```

 Models/FinanceManagement/Scholarship.cs

CSHARP

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SMS.Api.Models.FinanceManagement;

[Table("scholarships")]
public class Scholarship
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]
    [MaxLength(150)]
    public string Name { get; set; } = string.Empty;

    [Required]
    [MaxLength(50)]
    public string Code { get; set; } = string.Empty;

    [Required]
    [MaxLength(50)]
    public string Type { get; set; } = "Merit";

    [Required]
    [MaxLength(30)]
    public string DiscountType { get; set; } = "Percentage";

    [Column(TypeName = "decimal(10,2)")]
    public decimal Percentage { get; set; } = 0m;

    [Column(TypeName = "decimal(10,2)")]
    public decimal FixedAmount { get; set; } = 0m;

    public string? ApplicableFeeHeadIdsJson { get; set; } = "[]";

    public string? ApplicableClassesJson { get; set; } = "[]";

    [MaxLength(30)]
    public string StartDate { get; set; } = "2026-04-01";

    [MaxLength(30)]
    public string EndDate { get; set; } = "2027-03-31";

    [MaxLength(255)]
    public string? Eligibility { get; set; } = string.Empty;

    public string? Description { get; set; } = string.Empty;

    [MaxLength(30)]
    public string Status { get; set; } = "Active";

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}
```

 Models/FinanceManagement/StudentScholarship.cs

CSHARP

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SMS.Api.Models.FinanceManagement;

[Table("student_scholarships")]
public class StudentScholarship
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string StudentId { get; set; } = string.Empty;

    [Required]
    [MaxLength(150)]
    public string StudentName { get; set; } = string.Empty;

    [MaxLength(50)]
    public string? AdmissionNo { get; set; } = string.Empty;

    [MaxLength(50)]
    public string? ClassName { get; set; } = string.Empty;

    [MaxLength(20)]
    public string? Section { get; set; } = string.Empty;

    public int ScholarshipId { get; set; }

    [Required]
    [MaxLength(150)]
    public string ScholarshipName { get; set; } = string.Empty;

    [Required]
    [MaxLength(50)]
    public string ScholarshipCode { get; set; } = string.Empty;

    [Required]
    [MaxLength(30)]
    public string DiscountType { get; set; } = "Percentage";

    [Column(TypeName = "decimal(10,2)")]
    public decimal DiscountValue { get; set; } = 0m;

    [MaxLength(30)]
    public string AppliedDate { get; set; } = string.Empty;

    [MaxLength(30)]
    public string Status { get; set; } = "Active";

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

 Models/FinanceManagement/DiscountRule.cs

CSHARP

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SMS.Api.Models.FinanceManagement;

[Table("discounts")]
public class DiscountRule
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]
    [MaxLength(150)]
    public string Name { get; set; } = string.Empty;

    [Required]
    [MaxLength(50)]
    public string Code { get; set; } = string.Empty;

    [Required]
    [MaxLength(50)]
    public string Type { get; set; } = "Sibling Discount";

    [Required]
    [MaxLength(30)]
    public string Mode { get; set; } = "Percentage";

    [Column(TypeName = "decimal(10,2)")]
    public decimal Value { get; set; } = 10m;

    public string? Description { get; set; } = string.Empty;

    [MaxLength(30)]
    public string Status { get; set; } = "Active";

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}
```

 Models/FinanceManagement/StudentDiscount.cs

CSHARP

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace SMS.Api.Models.FinanceManagement;

[Table("student_discounts")]
public class StudentDiscount
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string StudentId { get; set; } = string.Empty;

    [Required]
    [MaxLength(150)]
    public string StudentName { get; set; } = string.Empty;

    [MaxLength(50)]
    public string? AdmissionNo { get; set; } = string.Empty;

    [MaxLength(50)]
    public string? ClassName { get; set; } = string.Empty;

    [MaxLength(20)]
    public string? Section { get; set; } = string.Empty;

    public int DiscountId { get; set; }

    [Required]
    [MaxLength(150)]
    public string DiscountName { get; set; } = string.Empty;

    [Required]
    [MaxLength(50)]
    public string DiscountCode { get; set; } = string.Empty;

    [Required]
    [MaxLength(30)]
    public string Mode { get; set; } = "Percentage";

    [Column(TypeName = "decimal(10,2)")]
    public decimal Value { get; set; } = 0m;

    [MaxLength(30)]
    public string AppliedDate { get; set; } = string.Empty;

    [MaxLength(30)]
    public string Status { get; set; } = "Active";

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

 Models/FinanceManagement/StudentFeeAssignment.cs

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace SMS.Api.Models.FinanceManagement;

[Table("student_fee_assignments")]
public class StudentFeeAssignment
{
    [Key]
    public int Id { get; set; }
    public string StudentId { get; set; } = string.Empty;
    public int? DynamicFeeStructureId { get; set; }
    public decimal TotalAmount { get; set; }
    public decimal PaidAmount { get; set; }
    public decimal DueAmount { get; set; }
    public string Status { get; set; } = "Active";
    public string FeePolicy { get; set; } = string.Empty;
}
```

2. Backend DTOs & Services (C# .NET 8 API)

 Dtos/FinanceManagement/FeeHeadDto.cs

CSHARP

```
namespace SMS.Api.Dtos.FinanceManagement;

using System.Collections.Generic;

public class FeeHeadDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public string Category { get; set; } = "Tuition";
    public string Frequency { get; set; } = "Quarterly";
    public decimal DefaultAmount { get; set; }
    public bool Mandatory { get; set; } = true;
    public bool IsRefundable { get; set; }
    public bool IsTaxable { get; set; }
    public decimal TaxPercentage { get; set; }
    public int DisplayOrder { get; set; } = 1;
    public string Status { get; set; } = "Active";
    public string Description { get; set; } = string.Empty;
    public List<string> ApplicableClasses { get; set; } = new();
    public List<string> ApplicableBranches { get; set; } = new() { "All Branches" };
}
```

 Dtos/FinanceManagement/FinanceDtos.cs

CSHARP

```
namespace SMS.Api.Dtos.FinanceManagement;

public class DynamicFeeStructureDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string TargetAudience { get; set; } = string.Empty;
    public string AcademicYear { get; set; } = string.Empty;
    public string Branch { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public string StudentCategory { get; set; } = string.Empty;
    public decimal TotalAmount { get; set; }
    public string Status { get; set; } = "Active";
    public System.Collections.Generic.List<FeeStructureItemDto> Items { get; set; } = new();
}

public class StringOrIntConverter : System.Text.Json.Serialization.JsonConverter<string>
{
    public override string Read(ref System.Text.Json.Utf8JsonReader reader, System.Type typeToConvert, System.Text.Json.JsonSerializerOptions options)
    {
        if (reader.TokenType == System.Text.Json.JsonTokenType.Number)
        {
            return reader.GetInt64().ToString();
        }
        if (reader.TokenType == System.Text.Json.JsonTokenType.String)
        {
            return reader.GetString() ?? string.Empty;
        }
        return string.Empty;
    }

    public override void Write(System.Text.Json.Utf8JsonWriter writer, string value, System.Text.Json.JsonSerializerOptions options)
    {
        writer.WriteStringValue(value ?? string.Empty);
    }
}

public class FeeStructureItemDto
{
    [System.Text.Json.Serialization.JsonConverter(typeof(StringOrIntConverter))]
    public string FeeHeadId { get; set; } = string.Empty;
    public string FeeHeadName { get; set; } = string.Empty;
    public string Category { get; set; } = string.Empty;
    public decimal Amount { get; set; }
}

public class StudentFeeAssignmentDto
{
    public int Id { get; set; }
    public string StudentId { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string AdmissionNo { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public int? DynamicFeeStructureId { get; set; }
```

```
public decimal TotalAmount { get; set; }
public decimal PaidAmount { get; set; }
public decimal DueAmount { get; set; }
public string Status { get; set; } = "Active";
public string FeePolicy { get; set; } = "Full Annual Fee";
public string? AdjustmentReason { get; set; }
public string? AssignedDate { get; set; }
}

public class BulkFeeAssignmentRequestDto
{
    public List<string> StudentIds { get; set; } = new();
    public int? DynamicFeeStructureId { get; set; }
    public string? ClassName { get; set; }
    public string? FeePolicy { get; set; } = "Full Annual Fee";
    public decimal? TotalAmount { get; set; }
}

public class CustomFeeAssignmentRequestDto
{
    public string StudentId { get; set; } = string.Empty;
    public int? DynamicFeeStructureId { get; set; }
    public string FeePolicy { get; set; } = "Monthly Pro-rated Fee";
    public string? AdmissionDate { get; set; }
    public string? AdjustmentReason { get; set; }
    public decimal TotalAmount { get; set; }
    public List<FeeHeadAssignmentItemDto>? Breakdown { get; set; }
}

public class FeeHeadAssignmentItemDto
{
    public string FeeHeadId { get; set; } = string.Empty;
    public string FeeHeadName { get; set; } = string.Empty;
    public string Category { get; set; } = string.Empty;
    public decimal OriginalAmount { get; set; }
    public decimal AssignedAmount { get; set; }
    public decimal AdjustmentAmount { get; set; }
}

public class FeePaymentDto
{
    public int Id { get; set; }
    public string ReceiptNo { get; set; } = string.Empty;
    public string StudentId { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public decimal DiscountAmount { get; set; }
    public decimal FineAmount { get; set; }
    public decimal TransportFee { get; set; }
    public string TransactionId { get; set; } = string.Empty;
    public System.DateTime PaymentDate { get; set; }
    public string PaymentMethod { get; set; } = string.Empty;
    public string Status { get; set; } = "Completed";
}
```

 Dtos/FinanceManagement/FinanceMasterDtos.cs

CSHARP

```

namespace SMS.Api.Dtos.FinanceManagement;

using System;
using System.Collections.Generic;

// =====
// 1. GENERAL LEDGER & TRANSACTIONS DTOS
// =====

public class FinanceTransactionDto
{
    public int Id { get; set; }
    public string TransactionId { get; set; } = string.Empty;
    public string Type { get; set; } = "Income"; // "Income" or "Expense"
    public string SourceModule { get; set; } = "Manual"; // "Student Fee Collection", "Admissions", "Payrol
1", "Hostel", "Transport", "Uniform", "Inventory", "Manual"
    public string Category { get; set; } = "General";
    public string Description { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public string FormattedAmount => $"₹{Amount:N0}";
    public string PaymentMode { get; set; } = "Bank Transfer"; // "Cash", "Bank Transfer", "UPI", "Cheque",
"POS / Card"
    public string Account { get; set; } = "Main Bank Account";
    public DateTime TransactionDate { get; set; }
    public string FormattedDate => TransactionDate.ToString("yyyy-MM-dd");
    public string Date => FormattedDate;
    public string Time => TransactionDate.ToString("hh:mm tt");
    public string Status { get; set; } = "Completed"; // "Completed", "Pending", "Reversed", "Cancelled"
    public string ReferenceNumber { get; set; } = string.Empty;
    public string CreatedBy { get; set; } = "Admin";
    public string ApprovedBy { get; set; } = "Chief Accountant";
    public string Branch { get; set; } = "Main Campus";
    public string AcademicYear { get; set; } = "2025-2026";
    public string Notes { get; set; } = string.Empty;
    public string AttachmentName { get; set; } = string.Empty;
}

public class CreateTransactionRequestDto
{
    public string Type { get; set; } = "Income";
    public string SourceModule { get; set; } = "Manual";
    public string Category { get; set; } = "General";
    public string Description { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public string PaymentMode { get; set; } = "Bank Transfer";
    public string Account { get; set; } = "Main Bank Account";
    public string TransactionDate { get; set; } = string.Empty;
    public string Branch { get; set; } = "Main Campus";
    public string AcademicYear { get; set; } = "2025-2026";
    public string Notes { get; set; } = string.Empty;
    public string AttachmentName { get; set; } = string.Empty;
}

public class ReverseTransactionRequestDto
{
    public string ReversalReason { get; set; } = string.Empty;
    public string AuthorizedBy { get; set; } = "Admin";
}

```

```

public class FinanceTransactionSummaryDto
{
    public decimal TotalInflow { get; set; }
    public decimal TotalOutflow { get; set; }
    public decimal NetBalance { get; set; }
    public decimal TodayInflow { get; set; }
    public decimal PendingClearances { get; set; }
    public string FormattedInflow => $"₹{TotalInflow:N0}";
    public string FormattedOutflow => $"₹{TotalOutflow:N0}";
    public string FormattedNetBalance => $"₹{NetBalance:N0}";
    public string FormattedTodayInflow => $"₹{TodayInflow:N0}";
}

// =====
// 2. BANK ACCOUNTS, CATEGORIES & BUDGETS DTOS
// =====

public class FinancialCategoryDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Type { get; set; } = "Income"; // "Income" or "Expense"
    public string SourceModule { get; set; } = "Manual";
    public string Status { get; set; } = "Active";
    public bool IsSystem { get; set; } = false;
}

public class FinancialAccountDto
{
    public int Id { get; set; }
    public string AccountName { get; set; } = string.Empty;
    public string AccountType { get; set; } = "Main Bank Account"; // "Main Bank Account", "Petty Cash", "Gateway Account", "Escrow Account", "Hostel Account", "Transport Account", "Cash"
    public string AccountNumber { get; set; } = string.Empty;
    public string BankName { get; set; } = string.Empty;
    public string BranchName { get; set; } = string.Empty;
    public decimal CurrentBalance { get; set; }
    public string Currency { get; set; } = "INR";
    public string FormattedBalance => $"₹{CurrentBalance:N0}";
    public string Status { get; set; } = "Active";
}

public class FinancialBudgetDto
{
    public int Id { get; set; }
    public string CategoryName { get; set; } = string.Empty;
    public string Department { get; set; } = string.Empty;
    public string AcademicYear { get; set; } = "2025-2026";
    public string Branch { get; set; } = "Main Campus";
    public decimal AllocatedAmount { get; set; }
    public decimal ConsumedAmount { get; set; }
    public decimal RemainingAmount => Math.Max(0m, AllocatedAmount - ConsumedAmount);
    public decimal AllocatedBudget { get => AllocatedAmount; set => AllocatedAmount = value; }
    public decimal UtilizedBudget { get => ConsumedAmount; set => ConsumedAmount = value; }
    public decimal RemainingBudget => RemainingAmount;
    public double UtilizationPercentage => AllocatedAmount > 0 ? Math.Round((double)(ConsumedAmount / AllocatedAmount) * 100, 1) : 0;
    public string FormattedAllocated => $"₹{AllocatedAmount:N0}";
    public string FormattedUtilized => $"₹{ConsumedAmount:N0}";
    public string FormattedRemaining => $"₹{RemainingAmount:N0}";
    public string Status { get; set; } = "Active";
}

```

```
// =====
// 3. REFUND MANAGEMENT DTOs
// =====

public class FeeRefundRequestDto
{
    public int Id { get; set; }
    public string RefundRequestId { get; set; } = string.Empty;
    public string RefundNo { get; set; } = string.Empty;
    public string ReceiptNo { get; set; } = string.Empty;
    public int StudentId { get; set; }
    public string AdmissionNo { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public decimal RefundAmount { get; set; }
    public string FormattedAmount => $"₹{RefundAmount:N0}";
    public string Reason { get; set; } = string.Empty;
    public string Status { get; set; } = "Pending"; // "Pending", "Approved", "Rejected", "Disbursed"
    public string RequestedBy { get; set; } = "Admin";
    public string ApprovedBy { get; set; } = string.Empty;
    public DateTime RequestedDate { get; set; } = DateTime.UtcNow;
    public DateTime? ProcessedDate { get; set; }
    public string PaymentMode { get; set; } = "Bank Transfer";
    public string Remarks { get; set; } = string.Empty;
}

public class CreateRefundRequestDto
{
    public int StudentId { get; set; }
    public string? StudentName { get; set; }
    public string? AdmissionNo { get; set; }
    public string? ClassName { get; set; }
    public string? Section { get; set; }
    public string? ReceiptNo { get; set; }
    public decimal RefundAmount { get; set; }
    public string Reason { get; set; } = "Scholarship Adjustment";
    public string PaymentMode { get; set; } = "Bank Transfer";
    public string Remarks { get; set; } = string.Empty;
}

public class ProcessRefundRequestDto
{
    public string Status { get; set; } = "Approved"; // "Approved" or "Rejected"
    public string Remarks { get; set; } = string.Empty;
    public string ProcessedBy { get; set; } = "Admin";
}

// =====
// 4. FINANCE SETUP & MASTER DTOs
// =====

public class FeeScheduleTermDto
{
    public string Id { get; set; } = string.Empty;
    public int Sequence { get; set; } = 1;
    public string TermName { get; set; } = "Term 1";
    public string StartDate { get; set; } = "2026-04-01";
    public string EndDate { get; set; } = "2026-06-30";
    public string DueDate { get; set; } = "2026-04-15";
    public string Status { get; set; } = "Active";
    public double PercentageShare { get; set; } = 25.0;
}
```

```

public class MonthDueDateItemDto
{
    public int MonthIndex { get; set; }
    public string MonthName { get; set; } = string.Empty;
    public string DueDate { get; set; } = string.Empty;
}

public class MonthlyDueDateConfigDto
{
    public bool ApplySameDayToAllMonths { get; set; } = true;
    public int DueDay { get; set; } = 10;
    public List<MonthDueDateItemDto> MonthDueDates { get; set; } = new();
}

public class FeeScheduleConfigDto
{
    public string Id { get; set; } = string.Empty;
    public string AcademicYear { get; set; } = "2026-2027";
    public int NumberOfTerms { get; set; } = 4;
    public string Status { get; set; } = "Published";
    public List<FeeScheduleTermDto> Terms { get; set; } = new();
    public MonthlyDueDateConfigDto? MonthlyConfig { get; set; }
    public string AnnualDueDate { get; set; } = "2026-04-15";
    public string OneTimeDueDate { get; set; } = "2026-04-15";
}

public class FinanceSettingsTaxDto
{
    public bool Enabled { get; set; } = true;
    public string TaxName { get; set; } = "GST";
    public decimal Percentage { get; set; } = 0m;
}

public class FinanceSettingsDto
{
    public string AcademicYear { get; set; } = "2025-2026";
    public string? ActiveAcademicYear { get; set; } = "2025-2026";
    public string FinancialYear { get; set; } = "2025-2026";
    public string DefaultCurrency { get; set; } = "INR";
    public string Currency { get; set; } = "INR";
    public string LateFeeRuleId { get; set; } = "1";
    public string ReceiptPrefix { get; set; } = "REC-2026-";
    public string InvoicePrefix { get; set; } = "INV-2026-";
    public string ReceiptFormat { get; set; } = "{PREFIX}{YEAR}-{NUMBER}";
    public bool AutoReceiptNo { get; set; } = true;
    public bool AutoEnforceLateFines { get; set; } = true;
    public decimal DefaultLateFinePerDay { get; set; } = 50m;
    public int DefaultGracePeriodDays { get; set; } = 7;
    public bool EnablePartialFeePayments { get; set; } = true;
    public bool EnableOnlinePaymentGateway { get; set; } = true;
    public string PaymentGatewayProvider { get; set; } = "Razorpay";
    public List<string> PaymentModes { get; set; } = new() { "Bank Transfer", "Cash", "Cheque", "UPI / Online" };
    public FinanceSettingsTaxDto TaxSettings { get; set; } = new();
}

// =====
// 5. FINANCE REPORTS DTOS
// =====

public class FinanceReportsSummaryDto
{

```

```

    public decimal TodayCollection { get; set; }
    public decimal MonthlyCollection { get; set; }
    public decimal PendingDues { get; set; }
    public int StudentsPaidCount { get; set; }
    public decimal ScholarshipsAndDiscounts { get; set; }
    public decimal TransportAndHostel { get; set; }

    public string FormattedToday => $"₹{TodayCollection:N0}";
    public string FormattedMonthly => $"₹{MonthlyCollection:N0}";
    public string FormattedPending => $"₹{PendingDues:N0}";
    public string FormattedConcessions => $"₹{ScholarshipsAndDiscounts:N0}";
    public string FormattedServices => $"₹{TransportAndHostel:N0}";
}

public class DailyCollectionReportRowDto
{
    public string ReceiptNo { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string AdmissionNo { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string PaymentMode { get; set; } = "Cash";
    public decimal Amount { get; set; }
    public string FormattedAmount => $"₹{Amount:N0}";
    public string CollectedBy { get; set; } = "Counter 1";
    public DateTime PaymentTime { get; set; }
}

public class DailyCollectionReportResponseDto
{
    public string ReportDate { get; set; } = string.Empty;
    public decimal TotalCash { get; set; }
    public decimal TotalOnline { get; set; }
    public decimal TotalCheque { get; set; }
    public decimal GrandTotal { get; set; }
    public string FormattedGrandTotal => $"₹{GrandTotal:N0}";
    public List<DailyCollectionReportRowDto> Transactions { get; set; } = new();
}

public class ClassWiseCollectionReportRowDto
{
    public string ClassName { get; set; } = string.Empty;
    public int TotalStudents { get; set; }
    public decimal ExpectedRevenue { get; set; }
    public decimal CollectedRevenue { get; set; }
    public decimal OutstandingDues { get; set; }
    public double CollectionPercentage => ExpectedRevenue > 0 ? Math.Round((double)(CollectedRevenue / ExpectedRevenue) * 100, 1) : 0;
    public string FormattedExpected => $"₹{ExpectedRevenue:N0}";
    public string FormattedCollected => $"₹{CollectedRevenue:N0}";
    public string FormattedOutstanding => $"₹{OutstandingDues:N0}";
}

// =====
// 6. SCHOLARSHIP MASTER & STUDENT SCHOLARSHIP DTOS
// =====

public class ScholarshipMasterDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public string Type { get; set; } = "Merit";
    public string DiscountType { get; set; } = "Percentage"; // "Percentage" or "Fixed Amount"

```

```

    public decimal Percentage { get; set; } = 15m;
    public decimal FixedAmount { get; set; } = 0m;
    public List<string> ApplicableFeeHeadIds { get; set; } = new();
    public List<string> ApplicableClasses { get; set; } = new();
    public string StartDate { get; set; } = "2026-04-01";
    public string EndDate { get; set; } = "2027-03-31";
    public string Eligibility { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string Status { get; set; } = "Active";
}

public class StudentScholarshipAwardDto
{
    public int Id { get; set; }
    public string StudentId { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string AdmissionNo { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public int ScholarshipId { get; set; }
    public string ScholarshipName { get; set; } = string.Empty;
    public string ScholarshipCode { get; set; } = string.Empty;
    public string DiscountType { get; set; } = "Percentage";
    public decimal DiscountValue { get; set; }
    public string AppliedDate { get; set; } = string.Empty;
    public string Status { get; set; } = "Active";
}

public class AwardScholarshipRequestDto
{
    public string StudentId { get; set; } = string.Empty;
    public int ScholarshipId { get; set; }
    public string? Remarks { get; set; }
}

// =====
// 7. DISCOUNTS & STUDENT CONCESSIONS DTOS
// =====

public class DiscountRuleDto
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public string Type { get; set; } = "Sibling Discount";
    public string Mode { get; set; } = "Percentage"; // "Percentage" or "Fixed Amount"
    public decimal Value { get; set; } = 10m;
    public string? Description { get; set; } = string.Empty;
    public string Status { get; set; } = "Active";
}

public class StudentDiscountDto
{
    public int Id { get; set; }
    public string StudentId { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string AdmissionNo { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public int DiscountId { get; set; }
    public string DiscountName { get; set; } = string.Empty;
    public string DiscountCode { get; set; } = string.Empty;
    public string Mode { get; set; } = "Percentage";
}

```

```

    public decimal Value { get; set; }
    public string AppliedDate { get; set; } = string.Empty;
    public string Status { get; set; } = "Active";
}

public class GrantDiscountRequestDto
{
    public string StudentId { get; set; } = string.Empty;
    public int DiscountId { get; set; }
    public string? Remarks { get; set; }
}

// =====
// 8. LATE FINE RULES DTOS
// =====

public class FineRuleDto
{
    public int Id { get; set; }
    public string RuleName { get; set; } = string.Empty;
    public string DueDate { get; set; } = "2026-08-15";
    public int GraceDays { get; set; } = 5;
    public string FineType { get; set; } = "Daily Fine"; // "Daily Fine", "Fixed Fine", "Percentage"
    public decimal DailyFine { get; set; } = 50m;
    public decimal FixedFine { get; set; } = 200m;
    public decimal MaximumFine { get; set; } = 1500m;
    public string Status { get; set; } = "Active";
}

// =====
// 9. HOSTEL FEE CONFIGURATIONS DTOS
// =====

public class FinanceHostelConfigDto
{
    public int Id { get; set; }
    public string HostelId { get; set; } = string.Empty;
    public string HostelName { get; set; } = string.Empty;
    public string RoomTypeId { get; set; } = string.Empty;
    public string RoomTypeName { get; set; } = string.Empty;
    public string RoomId { get; set; } = string.Empty;
    public string RoomNo { get; set; } = "All Rooms";
    public string FeePlan { get; set; } = "Annual"; // "Monthly", "Quarterly", "Half Yearly", "Annual"
    public decimal HostelFee { get; set; } = 40000m;
    public decimal SecurityDeposit { get; set; } = 5000m;
    public decimal TotalFee => HostelFee + SecurityDeposit;
    public string EffectiveFrom { get; set; } = "2026-09-02";
    public string Status { get; set; } = "Active"; // "Active", "Inactive"
}

public class CreateFinanceHostelConfigDto
{
    public string HostelId { get; set; } = string.Empty;
    public string HostelName { get; set; } = string.Empty;
    public string RoomTypeId { get; set; } = string.Empty;
    public string RoomTypeName { get; set; } = string.Empty;
    public string? RoomId { get; set; }
    public string? RoomNo { get; set; } = "All Rooms";
    public string FeePlan { get; set; } = "Annual";
    public decimal HostelFee { get; set; } = 40000m;
    public decimal SecurityDeposit { get; set; } = 5000m;
    public string? EffectiveFrom { get; set; }
    public string Status { get; set; } = "Active";
}

```

```
}

// =====
// 10. UNIFORM FEE CONFIGURATIONS DTOS
// =====

public class FinanceUniformConfigDto
{
    public int Id { get; set; }
    public string AcademicYear { get; set; } = "2026-2027";
    public string Branch { get; set; } = "Main Campus";
    public string ClassName { get; set; } = "Class 10";
    public string Gender { get; set; } = "Unisex"; // "Male", "Female", "Unisex"
    public string UniformPackage { get; set; } = "Full Kit";
    public string? UniformItemId { get; set; }
    public string FeePlan { get; set; } = "Annual"; // "One Time", "Annual", "Term-wise"
    public decimal FeeAmount { get; set; } = 3500m;
    public string EffectiveFrom { get; set; } = "2026-09-02";
    public string Status { get; set; } = "Active"; // "Active", "Inactive"
}

public class CreateFinanceUniformConfigDto
{
    public string? AcademicYear { get; set; } = "2026-2027";
    public string? Branch { get; set; } = "Main Campus";
    public string? ClassName { get; set; } = "Class 10";
    public string? Gender { get; set; } = "Unisex";
    public string? UniformPackage { get; set; } = "Full Kit";
    public string? UniformItemId { get; set; }
    public string? FeePlan { get; set; } = "Annual";
    public decimal? FeeAmount { get; set; } = 3500m;
    public string? EffectiveFrom { get; set; }
    public string? Status { get; set; } = "Active";
}
```

 Dtos/FinanceManagement/FeeCollectionDtos.cs

CSHARP

```
namespace SMS.Api.Dtos.FinanceManagement;

using System;
using System.Collections.Generic;

// =====
// 1. FEE COLLECTION STUDENT ROSTER DTOS
// =====

public class FeeCollectionStudentSummaryDto
{
    public int StudentId { get; set; }
    public string Id => StudentId.ToString();
    public string AdmissionNo { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public int ClassId { get; set; }
    public string ClassName { get; set; } = string.Empty;
    public int SectionId { get; set; }
    public string Section { get; set; } = string.Empty;
    public string StudentType { get; set; } = "Day Scholar";
    public string Branch { get; set; } = "Main Campus";
    public string AcademicYear { get; set; } = "2026-2027";
    public string FatherName { get; set; } = string.Empty;
    public string FatherMobile { get; set; } = string.Empty;
    public decimal TotalFee { get; set; }
    public decimal PaidFee { get; set; }
    public decimal TotalOutstanding { get; set; }
    public decimal CurrentYearDue { get; set; }
    public decimal PreviousYearsDue { get; set; }
    public string FormattedTotalOutstanding => $"₹{TotalOutstanding:N0}";
    public string Status { get; set; } = "Active";
}

public class FeeCollectionStudentRosterResponseDto
{
    public int TotalRecords { get; set; }
    public int Page { get; set; }
    public int PageSize { get; set; }
    public List<FeeCollectionStudentSummaryDto> Items { get; set; } = new();
}

// =====
// 2. DETAILED STUDENT FEE PROFILE & CALCULATION DTOS
// =====

public class FeeTermItemDto
{
    public string TermId { get; set; } = string.Empty;
    public string TermName { get; set; } = string.Empty;
    public int TermNumber { get; set; }
    public string DueDate { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public decimal PaidAmount { get; set; }
    public decimal RemainingAmount { get; set; }
    public string Status { get; set; } = "PENDING";
    public bool IsOverdue { get; set; }
    public int DaysOverdue { get; set; }
```

```

    public string FormattedAmount => $"₹{Amount:N0}";
    public string FormattedPaidAmount => $"₹{PaidAmount:N0}";
    public string FormattedRemainingAmount => $"₹{RemainingAmount:N0}";
}

public class FeeLineItemDto
{
    public string FeeHeadId { get; set; } = string.Empty;
    public string HeadName { get; set; } = string.Empty;
    public string Frequency { get; set; } = "Term-Wise";
    public string DueDate { get; set; } = string.Empty;
    public decimal TotalAmount { get; set; }
    public decimal PaidAmount { get; set; }
    public decimal RemainingAmount { get; set; }
    public string Status { get; set; } = "PENDING";
    public bool IsOverdue { get; set; }
    public int DaysOverdue { get; set; }
    public string FormattedTotal => $"₹{TotalAmount:N0}";
    public string FormattedPaid => $"₹{PaidAmount:N0}";
    public string FormattedRemaining => $"₹{RemainingAmount:N0}";
    public List<FeeTermItemDto> Terms { get; set; } = new();
}

public class LateFineRuleDetailDto
{
    public string RuleName { get; set; } = "Standard Monthly Late Fine Rule";
    public int DaysOverdue { get; set; } = 0;
    public decimal CalculatedFineAmount { get; set; } = 0m;
    public bool IsWaived { get; set; } = false;
    public string Description => $"{RuleName} ({DaysOverdue} Days Overdue)";
    public string FormattedFineAmount => $"₹{CalculatedFineAmount:N0}";
}

public class ConcessionOptionDto
{
    public string Id { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Type { get; set; } = "Percentage";
    public decimal Value { get; set; }
    public string ApplicableHead { get; set; } = "Tuition Fee";
}

public class StudentPaymentReceiptSummaryDto
{
    public string ReceiptNo { get; set; } = string.Empty;
    public DateTime PaymentDate { get; set; }
    public string FormattedDate => PaymentDate.ToString("yyyy-MM-dd");
    public string PaymentMethod { get; set; } = "Cash";
    public decimal AmountPaid { get; set; }
    public string FormattedAmount => $"₹{AmountPaid:N0}";
    public string Status { get; set; } = "Paid";
    public string TransactionId { get; set; } = string.Empty;
    public string Remarks { get; set; } = string.Empty;
    public List<string> PaidHeads { get; set; } = new();
}

public class StudentFeeProfileResponseDto
{
    public int StudentId { get; set; }
    public string AdmissionNo { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
}

```

```

    public string StudentType { get; set; } = "Day Scholar";
    public string Branch { get; set; } = "Main Campus";
    public string AcademicYear { get; set; } = "2026-2027";
    public string FatherName { get; set; } = string.Empty;
    public string FatherMobile { get; set; } = string.Empty;

    public decimal CurrentYearDues { get; set; }
    public decimal PreviousYearsArrears { get; set; }
    public decimal TotalConcessions { get; set; }
    public decimal TotalOutstandingBalance { get; set; }

    public string FormattedCurrentDues => $"₹{CurrentYearDues:N0}";
    public string FormattedPreviousArrears => $"₹{PreviousYearsArrears:N0}";
    public string FormattedConcessions => $"₹{TotalConcessions:N0}";
    public string FormattedTotalOutstanding => $"₹{TotalOutstandingBalance:N0}";

    public LateFineRuleDetailDto FineRule { get; set; } = new();
    public List<ConcessionOptionDto> AvailableScholarships { get; set; } = new();
    public List<ConcessionOptionDto> AvailableDiscounts { get; set; } = new();
    public List<FeeLineItemDto> CurrentAcademicYearFees { get; set; } = new();
    public List<StudentPaymentReceiptSummaryDto> RecordedReceipts { get; set; } = new();
}

// =====
// 3. PAYMENT COLLECTION & RECEIPT PROCESSING DTOS
// =====

public class SelectedFeeItemPaymentDto
{
    public string FeeHeadId { get; set; } = string.Empty;
    public string HeadName { get; set; } = string.Empty;
    public string TermId { get; set; } = string.Empty;
    public string TermName { get; set; } = string.Empty;
    public decimal Amount { get; set; }
}

public class CollectFeePaymentRequestDto
{
    public int StudentId { get; set; }
    public string AdmissionNo { get; set; } = string.Empty;
    public string AcademicYear { get; set; } = "2026-2027";
    public decimal TotalAmountPaid { get; set; }
    public decimal ConcessionDiscountAmount { get; set; } = 0m;
    public decimal FineAmount { get; set; } = 0m;
    public bool IsFineWaived { get; set; } = false;
    public string PaymentMethod { get; set; } = "Cash";
    public string TransactionId { get; set; } = string.Empty;
    public string ChequeNo { get; set; } = string.Empty;
    public string ChequeDate { get; set; } = string.Empty;
    public string BankName { get; set; } = string.Empty;
    public string Remarks { get; set; } = string.Empty;
    public List<SelectedFeeItemPaymentDto> SelectedItems { get; set; } = new();
}

public class CollectFeePaymentResponseDto
{
    public bool Success { get; set; } = true;
    public string Message { get; set; } = "Payment collected and receipt generated successfully.";
    public string ReceiptNo { get; set; } = string.Empty;
    public int PaymentId { get; set; }
    public DateTime PaymentDate { get; set; }
    public decimal AmountPaid { get; set; }
    public decimal RemainingOutstanding { get; set; }
}

```

```

        public string FormattedAmountPaid => $"₹{AmountPaid:N0}";
        public string FormattedRemaining => $"₹{RemainingOutstanding:N0}";
    }

// =====
// 4. DUE FEES & DEFAULTERS DTOs
// =====

public class DueFeeStudentDto
{
    public int StudentId { get; set; }
    public string AdmissionNo { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public string ParentName { get; set; } = string.Empty;
    public string ParentMobile { get; set; } = string.Empty;
    public decimal TotalDueAmount { get; set; }
    public int MaxDaysOverdue { get; set; }
    public string OverdueStatus => MaxDaysOverdue > 90 ? "Critical (>90 Days)" : MaxDaysOverdue > 30 ? "Overdue (>30 Days)" : "Due Soon";
    public string FormattedDueAmount => $"₹{TotalDueAmount:N0}";
    public List<string> OverdueHeads { get; set; } = new();
}

public class DueFeesSummaryResponseDto
{
    public int TotalOverdueStudents { get; set; }
    public decimal TotalOverdueAmount { get; set; }
    public int CriticalDefaultersCount { get; set; }
    public string FormattedTotalOverdue => $"₹{TotalOverdueAmount:N0}";
    public List<DueFeeStudentDto> Items { get; set; } = new();
}

public class SendFeeReminderRequestDto
{
    public int StudentId { get; set; }
    public string ReminderType { get; set; } = "SMS";
    public string CustomMessage { get; set; } = string.Empty;
}

// =====
// 5. PROMOTED STUDENTS DUES DTOs
// =====

public class PromotedDueItemDto
{
    public string Id { get; set; } = string.Empty;
    public string FeeHeadName { get; set; } = string.Empty;
    public string TermName { get; set; } = string.Empty;
    public string DueDate { get; set; } = string.Empty;
    public decimal OriginalAmount { get; set; }
    public decimal PaidAmount { get; set; }
    public decimal DueAmount { get; set; }
    public string Status { get; set; } = "Due";
    public string FormattedOriginal => $"₹{OriginalAmount:N0}";
    public string FormattedPaid => $"₹{PaidAmount:N0}";
    public string FormattedDue => $"₹{DueAmount:N0}";
}

public class PromotedDueBreakdownGroupDto
{
    public string AcademicYear { get; set; } = string.Empty;

```

```

    public string ClassName { get; set; } = string.Empty;
    public decimal TotalPending { get; set; }
    public string FormattedPending => $"₹{TotalPending:N0}";
    public List<PromotedDueItemDto> Items { get; set; } = new();
}

public class PromotedDueStudentDto
{
    public int StudentId { get; set; }
    public string StudentIdStr => StudentId.ToString();
    public string AdmissionNo { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string CurrentClass { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public string PreviousClass { get; set; } = string.Empty;
    public List<string> PreviousAcademicYears { get; set; } = new();
    public string PreviousAcademicYear { get; set; } = string.Empty;
    public decimal PreviousArrearsAmount { get; set; }
    public decimal PreviousYearPendingAmount => PreviousArrearsAmount;
    public string FormattedArrears => $"₹{PreviousArrearsAmount:N0}";
    public int PendingComponentsCount { get; set; }
    public string Status { get; set; } = string.Empty;
    public string FatherName { get; set; } = string.Empty;
    public string FatherMobile { get; set; } = string.Empty;
    public List<PromotedDueBreakdownGroupDto> BreakdownByYear { get; set; } = new();
}

// =====
// 6. FEE RECEIPTS REGISTER DTOS
// =====

public class FeeReceiptDetailDto
{
    public int PaymentId { get; set; }
    public string ReceiptNo { get; set; } = string.Empty;
    public DateTime PaymentDate { get; set; }
    public string FormattedDate => PaymentDate.ToString("yyyy-MM-dd HH:mm");
    public int StudentId { get; set; }
    public string AdmissionNo { get; set; } = string.Empty;
    public string StudentName { get; set; } = string.Empty;
    public string ClassName { get; set; } = string.Empty;
    public string Section { get; set; } = string.Empty;
    public string AcademicYear { get; set; } = "2026-2027";
    public string Branch { get; set; } = "Main Campus";
    public decimal AmountPaid { get; set; }
    public decimal DiscountAmount { get; set; }
    public decimal FineAmount { get; set; }
    public string PaymentMethod { get; set; } = "Cash";
    public string TransactionId { get; set; } = string.Empty;
    public string Status { get; set; } = "Completed";
    public string Remarks { get; set; } = string.Empty;
    public string FormattedAmount => $"₹{AmountPaid:N0}";
    public List<FeeBreakdownItemDto> ItemizedBreakdown { get; set; } = new();
}

public class FeeReceiptsRegisterResponseDto
{
    public int TotalRecords { get; set; }
    public int Page { get; set; }
    public int PageSize { get; set; }
    public decimal TotalCollectedAmount { get; set; }
    public string FormattedTotalCollected => $"₹{TotalCollectedAmount:N0}";
    public List<FeeReceiptDetailDto> Items { get; set; } = new();
}

```

```

}

// =====
// 7. FINANCE DASHBOARD & REPORTS DTOS
// =====

public class MonthlyCollectionTrendDto
{
    public string Month { get; set; } = string.Empty;
    public decimal TargetAmount { get; set; }
    public decimal CollectedAmount { get; set; }
}

public class FeeHeadCollectionShareDto
{
    public string HeadName { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public double Percentage { get; set; }
    public string Color { get; set; } = "#3b82f6";
}

public class PaymentModeSplitDto
{
    public string Mode { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public int TransactionsCount { get; set; }
}

public class ClassWiseCollectionShareDto
{
    public string ClassName { get; set; } = string.Empty;
    public decimal ExpectedAmount { get; set; }
    public decimal CollectedAmount { get; set; }
    public decimal PendingAmount => Math.Max(0m, ExpectedAmount - CollectedAmount);
    public double Percentage => ExpectedAmount > 0 ? Math.Min(100.0, Math.Round((double)(CollectedAmount /
ExpectedAmount) * 100, 1)) : 0;
    public string FormattedExpected => $"₹{ExpectedAmount:N0}";
    public string FormattedCollected => $"₹{CollectedAmount:N0}";
}

public class FinanceDashboardStatsDto
{
    public decimal TotalExpectedRevenue { get; set; }
    public decimal TotalCollectedRevenue { get; set; }
    public decimal TotalOutstandingDues { get; set; }
    public decimal TotalConcessionsGranted { get; set; }
    public decimal TodayCollectionAmount { get; set; }
    public decimal MonthlyCollectionAmount { get; set; }
    public int StudentsPaidCount { get; set; }
    public double CollectionEfficiencyPercentage { get; set; }

    public decimal TransportRevenue { get; set; }
    public decimal HostelRevenue { get; set; }
    public decimal UniformRevenue { get; set; }
    public decimal ScholarshipsGranted { get; set; }
    public decimal FineCollected { get; set; }

    public string FormattedExpected => $"₹{TotalExpectedRevenue:N0}";
    public string FormattedCollected => $"₹{TotalCollectedRevenue:N0}";
    public string FormattedOutstanding => $"₹{TotalOutstandingDues:N0}";
    public string FormattedConcessions => $"₹{TotalConcessionsGranted:N0}";
    public string FormattedToday => $"₹{TodayCollectionAmount:N0}";
    public string FormattedMonthly => $"₹{MonthlyCollectionAmount:N0}";
}

```

```
public string FormattedTransport => $"₹{TransportRevenue:N0}";
public string FormattedHostel => $"₹{HostelRevenue:N0}";
public string FormattedUniform => $"₹{UniformRevenue:N0}";
public string FormattedScholarships => $"₹{ScholarshipsGranted:N0}";
public string FormattedFine => $"₹{FineCollected:N0}";

public List<ClassWiseCollectionShareDto> ClassWiseRevenue { get; set; } = new();
public List<MonthlyCollectionTrendDto> MonthlyTrends { get; set; } = new();
public List<FeeHeadCollectionShareDto> HeadWiseDistribution { get; set; } = new();
public List<PaymentModeSplitDto> PaymentModeDistribution { get; set; } = new();
public List<FeeReceiptDetailDto> RecentTransactions { get; set; } = new();
}
```

 Controllers/FinanceManagement/FinanceController.cs

CSHARP

```
namespace SMS.Api.Controllers.FinanceManagement;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SMS.Api.Dtos.FinanceManagement;
using SMS.Api.Services.Interfaces.FinanceManagement;
using System.Threading.Tasks;

[ApiController]
[Route("api/finance")]
[Authorize]
[Tags("Finance & Fee Management (Full)")]
public class FinanceController : ControllerBase
{
    private readonly IFinanceService _service;
    public FinanceController(IFinanceService service) { _service = service; }

    [HttpGet("fee-heads")]
    public async Task<IActionResult> GetFeeHeads(
        [FromQuery] string? search,
        [FromQuery] string? category,
        [FromQuery] string? frequency,
        [FromQuery] string? status)
    {
        var result = await _service.GetFeeHeadsAsync();
        if (!string.IsNullOrEmpty(search))
        {
            string s = search.Trim().ToLower();
            result = result.Where(h => h.Name.ToLower().Contains(s) || h.Code.ToLower().Contains(s));
        }
        if (!string.IsNullOrEmpty(category) && !category.Equals("ALL", System.StringComparison.OrdinalIgnoreCase))
            result = result.Where(h => h.Category.Equals(category, System.StringComparison.OrdinalIgnoreCase));
        if (!string.IsNullOrEmpty(frequency) && !frequency.Equals("ALL", System.StringComparison.OrdinalIgnoreCase))
            result = result.Where(h => h.Frequency.Equals(frequency, System.StringComparison.OrdinalIgnoreCase));
        if (!string.IsNullOrEmpty(status) && !status.Equals("ALL", System.StringComparison.OrdinalIgnoreCase))
        {
            string cleanStatus = status.Replace(" Only", "", System.StringComparison.OrdinalIgnoreCase).Trim();
            result = result.Where(h => h.Status.Equals(cleanStatus, System.StringComparison.OrdinalIgnoreCase));
        }

        return Ok(new { success = true, data = result });
    }

    [HttpPost("fee-heads")]
    public async Task<IActionResult> CreateFeeHead([FromBody] FeeHeadDto dto)
    {
        try
        {
            var data = await _service.CreateFeeHeadAsync(dto);
            return Ok(new { success = true, data });
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(ex.Message);
        }
    }
}
```

```

        return BadRequest(new { success = false, message = ex.Message });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { success = false, message = ex.Message });
    }
}

[HttpPut("fee-heads/{id}")]
public async Task<IActionResult> UpdateFeeHead(int id, [FromBody] FeeHeadDto dto)
{
    try
    {
        var data = await _service.UpdateFeeHeadAsync(id, dto);
        return Ok(new { success = true, data });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { success = false, message = ex.Message });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { success = false, message = ex.Message });
    }
}

[HttpPatch("fee-heads/{id}/toggle-status")]
public async Task<IActionResult> ToggleFeeHeadStatus(int id)
{
    var updated = await _service.ToggleFeeHeadStatusAsync(id);
    if (updated == null) return NotFound(new { success = false, message = "Fee Head not found." });
    return Ok(new { success = true, data = updated });
}

[HttpDelete("fee-heads/{id}")]
public async Task<IActionResult> DeleteFeeHead(int id) { await _service.DeleteFeeHeadAsync(id); return
Ok(new { success = true }); }

[HttpGet("fee-structures")]
public async Task<IActionResult> GetDynamicFeeStructures(
    [FromQuery] string? search,
    [FromQuery] string? className,
    [FromQuery] string? academicYear)
{
    var list = await _service.GetDynamicFeeStructuresAsync();
    if (!string.IsNullOrEmpty(search))
    {
        string s = search.Trim().ToLower();
        list = list.Where(x => x.Name.ToLower().Contains(s) || x.ClassName.ToLower().Contains(s));
    }
    if (!string.IsNullOrEmpty(className) && !className.Equals("ALL", System.StringComparison.OrdinalIgnoreCase))
        list = list.Where(x => x.ClassName.Equals(className, System.StringComparison.OrdinalIgnoreCase));
    if (!string.IsNullOrEmpty(academicYear) && !academicYear.Equals("ALL", System.StringComparison.OrdinalIgnoreCase))
        list = list.Where(x => string.IsNullOrEmpty(x.AcademicYear) || x.AcademicYear.Equals(academicYear, System.StringComparison.OrdinalIgnoreCase));

    return Ok(new { success = true, data = list });
}

[HttpGet("fee-structures/{id:int}")]

```

```

public async Task<IActionResult> GetDynamicFeeStructureById(int id)
{
    var item = await _service.GetDynamicFeeStructureByIdAsync(id);
    if (item == null) return NotFound(new { success = false, message = "Fee Structure not found." });
    return Ok(new { success = true, data = item });
}

[HttpPost("fee-structures")]
public async Task<IActionResult> CreateDynamicFeeStructure([FromBody] DynamicFeeStructureDto dto) => Ok
(new { success = true, data = await _service.CreateDynamicFeeStructureAsync(dto) });

[HttpPut("fee-structures/{id:int}")]
public async Task<IActionResult> UpdateDynamicFeeStructure(int id, [FromBody] DynamicFeeStructureDto dt
o)
{
    var item = await _service.UpdateDynamicFeeStructureAsync(id, dto);
    if (item == null) return NotFound(new { success = false, message = "Fee Structure not found." });
    return Ok(new { success = true, data = item });
}

[HttpDelete("fee-structures/{id:int}")]
public async Task<IActionResult> DeleteDynamicFeeStructure(int id)
{
    await _service.DeleteDynamicFeeStructureAsync(id);
    return Ok(new { success = true });
}

[HttpGet("fee-assignments")]
public async Task<IActionResult> GetStudentFeeAssignments() => Ok(new { success = true, data = await _s
ervice.GetStudentFeeAssignmentsAsync() });

[HttpGet("fee-assignments/{id:int}")]
public async Task<IActionResult> GetStudentFeeAssignmentById(int id)
{
    var item = await _service.GetStudentFeeAssignmentByIdAsync(id);
    if (item == null) return NotFound(new { success = false, message = "Fee assignment not found." });
    return Ok(new { success = true, data = item });
}

[HttpPost("fee-assignments")]
public async Task<IActionResult> CreateStudentFeeAssignment([FromBody] StudentFeeAssignmentDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.StudentId))
        return BadRequest(new { success = false, message = "Student ID is required." });
    var res = await _service.CreateStudentFeeAssignmentAsync(dto);
    return Ok(new { success = true, message = "Fee assignment saved successfully.", data = res });
}

[HttpPost("fee-assignments/bulk")]
public async Task<IActionResult> BulkAssignStudentFees([FromBody] BulkFeeAssignmentRequestDto request)
{
    if (request == null || request.StudentIds == null || !request.StudentIds.Any())
        return BadRequest(new { success = false, message = "At least one student must be selected." });
    var list = await _service.BulkAssignStudentFeesAsync(request);
    return Ok(new { success = true, message = $"Successfully assigned fee structure to {list.Count} stu
dents.", data = list });
}

[HttpPost("fee-assignments/custom")]
public async Task<IActionResult> SaveCustomStudentFeeAssignment([FromBody] CustomFeeAssignmentRequestDt
o request)
{
    if (request == null || string.IsNullOrEmpty(request.StudentId))

```

```
        return BadRequest(new { success = false, message = "Student ID is required." });
        var res = await _service.SaveCustomStudentFeeAssignmentAsync(request);
        return Ok(new { success = true, message = "Custom fee policy assignment saved successfully.", data
= res });
    }

    [HttpPut("fee-assignments/{id:int}")]
    public async Task<IActionResult> UpdateStudentFeeAssignment(int id, [FromBody] StudentFeeAssignmentDto
dto)
    {
        var res = await _service.UpdateStudentFeeAssignmentAsync(id, dto);
        if (res == null) return NotFound(new { success = false, message = "Fee assignment not found." });
        return Ok(new { success = true, message = "Fee assignment updated successfully.", data = res });
    }

    [HttpDelete("fee-assignments/{id:int}")]
    public async Task<IActionResult> DeleteStudentFeeAssignment(int id)
    {
        var success = await _service.DeleteStudentFeeAssignmentAsync(id);
        if (!success) return NotFound(new { success = false, message = "Fee assignment not found." });
        return Ok(new { success = true, message = "Fee assignment deleted successfully." });
    }

    [HttpGet("fee-payments")]
    public async Task<IActionResult> GetFeePayments() => Ok(new { success = true, data = await _service.Get
FeePaymentsAsync() });

    [HttpPost("fee-payments")]
    public async Task<IActionResult> CreateFeePayment([FromBody] FeePaymentDto dto) => Ok(new { success = t
rue, data = await _service.CreateFeePaymentAsync(dto) });
}
```

 Controllers/FinanceManagement/FinanceMasterController.cs

CSHARP

```

namespace SMS.Api.Controllers.FinanceManagement;

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SMS.Api.Dtos.FinanceManagement;
using SMS.Api.Services.Interfaces.FinanceManagement;
using System.Threading.Tasks;

[ApiController]
[Route("api/finance")]
[Authorize]
[Tags("Finance Master, General Ledger & Reports Hub")]
public class FinanceMasterController : ControllerBase
{
    private readonly IFinanceMasterService _masterService;

    public FinanceMasterController(IFinanceMasterService masterService)
    {
        _masterService = masterService;
    }

    // =====
    // 1. GENERAL LEDGER & TRANSACTIONS
    // =====

    [HttpGet("transactions")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetTransactions(
        [FromQuery] string? search,
        [FromQuery] string? type,
        [FromQuery] string? module,
        [FromQuery] string? category,
        [FromQuery] string? paymentMode,
        [FromQuery] string? status,
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 50)
    {
        var result = await _masterService.GetTransactionsAsync(
            search, type, module, category, paymentMode, status, page, pageSize);
        return Ok(new { success = true, data = result });
    }

    [HttpGet("transactions/summary")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetTransactionSummary()
    {
        var result = await _masterService.GetTransactionSummaryAsync();
        return Ok(new { success = true, data = result });
    }

    [HttpPost("transactions")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> CreateTransaction([FromBody] CreateTransactionRequestDto request)
    {
        if (request == null || request.Amount <= 0)
            return BadRequest(new { success = false, message = "Valid transaction details and positive amount are required." });

        var result = await _masterService.CreateTransactionAsync(request);
    }
}

```

```

        return Ok(new { success = true, message = "Transaction recorded successfully in General Ledger.", data = result });
    }

    [HttpPost("transactions/{id:int}/reverse")]
    [Authorize(Roles = "Admin,SuperAdmin")]
    public async Task<IActionResult> ReverseTransaction(int id, [FromBody] ReverseTransactionRequestDto request)
    {
        var success = await _masterService.ReverseTransactionAsync(id, request ?? new ReverseTransactionRequestDto());
        if (!success)
            return NotFound(new { success = false, message = "Transaction not found or could not be reversed." });

        return Ok(new { success = true, message = "Transaction reversed and contra-entry logged." });
    }

    // =====
    // 2. BANK ACCOUNTS & CATEGORIES
    // =====

    [HttpGet("accounts")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetAccounts()
    {
        var result = await _masterService.GetAccountsAsync();
        return Ok(new { success = true, data = result });
    }

    [HttpPost("accounts")]
    [Authorize(Roles = "Admin,SuperAdmin,Accountant")]
    public async Task<IActionResult> CreateAccount([FromBody] FinancialAccountDto account)
    {
        if (account == null || string.IsNullOrEmpty(account.AccountName))
            return BadRequest(new { success = false, message = "Account name is required." });

        var result = await _masterService.CreateAccountAsync(account);
        return Ok(new { success = true, message = "Financial account created successfully.", data = result });
    }

    [HttpPut("accounts/{id:int}")]
    [Authorize(Roles = "Admin,SuperAdmin,Accountant")]
    public async Task<IActionResult> UpdateAccount(int id, [FromBody] FinancialAccountDto account)
    {
        var success = await _masterService.UpdateAccountAsync(id, account);
        if (!success)
            return NotFound(new { success = false, message = "Account not found." });

        return Ok(new { success = true, message = "Financial account updated successfully." });
    }

    [HttpDelete("accounts/{id:int}")]
    [Authorize(Roles = "Admin,SuperAdmin")]
    public async Task<IActionResult> DeleteAccount(int id)
    {
        var success = await _masterService.DeleteAccountAsync(id);
        if (!success)
            return NotFound(new { success = false, message = "Account not found." });

        return Ok(new { success = true, message = "Financial account deleted successfully." });
    }

```

```

[HttpGet("categories")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetCategories([FromQuery] string? type)
{
    var result = await _masterService.GetCategoriesAsync(type);
    return Ok(new { success = true, data = result });
}

[HttpPost("categories")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> CreateCategory([FromBody] FinancialCategoryDto category)
{
    if (category == null || string.IsNullOrEmpty(category.Name))
        return BadRequest(new { success = false, message = "Category name is required." });

    var result = await _masterService.CreateCategoryAsync(category);
    return Ok(new { success = true, message = "Category created successfully.", data = result });
}

[HttpPut("categories/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> UpdateCategory(int id, [FromBody] FinancialCategoryDto category)
{
    var success = await _masterService.UpdateCategoryAsync(id, category);
    if (!success)
        return NotFound(new { success = false, message = "Category not found." });

    return Ok(new { success = true, message = "Category updated successfully." });
}

[HttpDelete("categories/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin")]
public async Task<IActionResult> DeleteCategory(int id)
{
    var success = await _masterService.DeleteCategoryAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Category not found." });

    return Ok(new { success = true, message = "Category deleted successfully." });
}

// =====
// 3. BUDGETS
// =====

[HttpGet("budgets")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetBudgets([FromQuery] string? academicYear = "2025-2026")
{
    var result = await _masterService.GetBudgetsAsync(academicYear);
    return Ok(new { success = true, data = result });
}

[HttpPost("budgets")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> SaveBudget([FromBody] FinancialBudgetDto budget)
{
    if (budget == null || (string.IsNullOrEmpty(budget.Department) && string.IsNullOrEmpty(budget.CategoryName)))
        return BadRequest(new { success = false, message = "Category/Department and allocated budget are required." });
}

```

```

        var result = await _masterService.SaveBudgetAsync(budget);
        return Ok(new { success = true, message = "Departmental budget allocation saved.", data = result
    });
}

[HttpPut("budgets/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> UpdateBudget(int id, [FromBody] FinancialBudgetDto budget)
{
    var success = await _masterService.UpdateBudgetAsync(id, budget);
    if (!success)
        return NotFound(new { success = false, message = "Budget not found." });

    return Ok(new { success = true, message = "Budget allocation updated successfully." });
}

// =====
// 4. REFUND MANAGEMENT
// =====

[HttpGet("refunds")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetRefundRequests([FromQuery] string? status)
{
    var result = await _masterService.GetRefundRequestsAsync(status);
    return Ok(new { success = true, data = result });
}

[HttpPost("refunds/request")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant,Parent")]
public async Task<IActionResult> CreateRefundRequest([FromBody] CreateRefundRequestDto request)
{
    if (request == null || request.RefundAmount <= 0)
        return BadRequest(new { success = false, message = "Valid student details and positive refund a
mount are required." });

    var result = await _masterService.CreateRefundRequestAsync(request);
    return Ok(new { success = true, message = "Refund request submitted successfully.", data = result
});
}

[HttpPost("refunds/{id:int}/process")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> ProcessRefundRequest(int id, [FromBody] ProcessRefundRequestDto reques
t)
{
    var success = await _masterService.ProcessRefundRequestAsync(id, request ?? new ProcessRefundReques
tDto());
    if (!success)
        return NotFound(new { success = false, message = "Refund request not found." });

    return Ok(new { success = true, message = $"Refund request marked as {request?.Status}." });
}

// =====
// 5. FINANCE SETUP & SETTINGS
// =====

[HttpGet("fee-schedules")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant,Parent,Student")]
public async Task<IActionResult> GetFeeSchedule([FromQuery] string? academicYear = "2026-2027")
{
    var result = await _masterService.GetFeeScheduleAsync(academicYear);

```

```

        return Ok(new { success = true, data = result });
    }

    [HttpPost("fee-schedules")]
    [Authorize(Roles = "Admin,SuperAdmin,Accountant,Staff")]
    public async Task<IActionResult> SaveFeeSchedule([FromBody] FeeScheduleConfigDto schedule)
    {
        var success = await _masterService.SaveFeeScheduleAsync(schedule);
        return Ok(new { success = true, message = "Fee installment schedule updated." });
    }

    [HttpGet("settings")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetFinanceSettings()
    {
        var result = await _masterService.GetFinanceSettingsAsync();
        return Ok(new { success = true, data = result });
    }

    [HttpPut("settings")]
    [Authorize(Roles = "Admin,SuperAdmin")]
    public async Task<IActionResult> UpdateFinanceSettings([FromBody] FinanceSettingsDto settings)
    {
        var success = await _masterService.UpdateFinanceSettingsAsync(settings);
        return Ok(new { success = true, message = "Finance settings saved successfully." });
    }

    // =====
    // 6. REPORTS HUB
    // =====

    [HttpGet("reports/summary")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetReportsSummary([FromQuery] string? academicYear = "2025-2026")
    {
        var result = await _masterService.GetReportsSummaryAsync(academicYear);
        return Ok(new { success = true, data = result });
    }

    [HttpGet("reports/daily-collection")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetDailyCollectionReport([FromQuery] string? date)
    {
        var result = await _masterService.GetDailyCollectionReportAsync(date);
        return Ok(new { success = true, data = result });
    }

    [HttpGet("reports/class-wise-collection")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetClassWiseCollectionReport([FromQuery] string? academicYear = "2026-2027")
    {
        var result = await _masterService.GetClassWiseCollectionReportAsync(academicYear);
        return Ok(new { success = true, data = result });
    }

    // =====
    // 7. SCHOLARSHIPS MASTER & STUDENT AWARDS
    // =====

    [HttpGet("scholarships")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetScholarships(

```

```

        [FromQuery] string? search,
        [FromQuery] string? type,
        [FromQuery] string? status)
    {
        var result = await _masterService.GetScholarshipsAsync(search, type, status);
        return Ok(new { success = true, data = result });
    }

[HttpGet("scholarships/{id:int}")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetScholarshipById(int id)
{
    var item = await _masterService.GetScholarshipByIdAsync(id);
    if (item == null)
        return NotFound(new { success = false, message = "Scholarship not found." });
    return Ok(new { success = true, data = item });
}

[HttpPost("scholarships")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> CreateScholarship([FromBody] ScholarshipMasterDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.Name))
        return BadRequest(new { success = false, message = "Scholarship name is required." });

    var result = await _masterService.CreateScholarshipAsync(dto);
    return Ok(new { success = true, message = "Scholarship created successfully.", data = result });
}

[HttpPut("scholarships/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> UpdateScholarship(int id, [FromBody] ScholarshipMasterDto dto)
{
    var result = await _masterService.UpdateScholarshipAsync(id, dto);
    if (result == null)
        return NotFound(new { success = false, message = "Scholarship not found." });

    return Ok(new { success = true, message = "Scholarship updated successfully.", data = result });
}

[HttpDelete("scholarships/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> DeleteScholarship(int id)
{
    var success = await _masterService.DeleteScholarshipAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Scholarship not found." });

    return Ok(new { success = true, message = "Scholarship deleted successfully." });
}

[HttpGet("student-scholarships")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetStudentScholarships(
    [FromQuery] string? search,
    [FromQuery] string? className,
    [FromQuery] int? scholarshipId)
{
    var result = await _masterService.GetStudentScholarshipsAsync(search, className, scholarshipId);
    return Ok(new { success = true, data = result });
}

[HttpPost("student-scholarships")]

```

```

[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> AwardStudentScholarship([FromBody] AwardScholarshipRequestDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.StudentId) || dto.ScholarshipId <= 0)
        return BadRequest(new { success = false, message = "Valid Student and Scholarship are required." });

    var result = await _masterService.AwardScholarshipToStudentAsync(dto);
    return Ok(new { success = true, message = "Scholarship awarded to student successfully.", data = result });
}

[HttpDelete("student-scholarships/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> RevokeStudentScholarship(int id)
{
    var success = await _masterService.RevokeStudentScholarshipAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Scholarship record not found." });

    return Ok(new { success = true, message = "Scholarship revoked successfully." });
}

// =====
// 8. DISCOUNTS MASTER & STUDENT CONCESSIONS
// =====

[HttpGet("discounts")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetDiscounts(
    [FromQuery] string? search,
    [FromQuery] string? type,
    [FromQuery] string? mode,
    [FromQuery] string? status)
{
    var result = await _masterService.GetDiscountsAsync(search, type, mode, status);
    return Ok(new { success = true, data = result });
}

[HttpGet("discounts/{id:int}")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetDiscountById(int id)
{
    var item = await _masterService.GetDiscountByIdAsync(id);
    if (item == null)
        return NotFound(new { success = false, message = "Discount rule not found." });
    return Ok(new { success = true, data = item });
}

[HttpPost("discounts")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> CreateDiscount([FromBody] DiscountRuleDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.Name))
        return BadRequest(new { success = false, message = "Discount name is required." });

    var result = await _masterService.CreateDiscountAsync(dto);
    return Ok(new { success = true, message = "Discount rule created successfully.", data = result });
}

[HttpPut("discounts/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> UpdateDiscount(int id, [FromBody] DiscountRuleDto dto)

```

```

{
    var result = await _masterService.UpdateDiscountAsync(id, dto);
    if (result == null)
        return NotFound(new { success = false, message = "Discount rule not found." });

    return Ok(new { success = true, message = "Discount rule updated successfully.", data = result });
}

[HttpDelete("discounts/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> DeleteDiscount(int id)
{
    var success = await _masterService.DeleteDiscountAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Discount rule not found." });

    return Ok(new { success = true, message = "Discount rule deleted successfully." });
}

[HttpGet("student-discounts")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetStudentDiscounts(
    [FromQuery] string? search,
    [FromQuery] string? className,
    [FromQuery] int? discountId)
{
    var result = await _masterService.GetStudentDiscountsAsync(search, className, discountId);
    return Ok(new { success = true, data = result });
}

[HttpPost("student-discounts")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> GrantStudentDiscount([FromBody] GrantDiscountRequestDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.StudentId) || dto.DiscountId <= 0)
        return BadRequest(new { success = false, message = "Valid Student and Discount rule are required." });

    var result = await _masterService.GrantDiscountToStudentAsync(dto);
    return Ok(new { success = true, message = "Concession granted to student successfully.", data = result });
}

[HttpDelete("student-discounts/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> RemoveStudentDiscount(int id)
{
    var success = await _masterService.RemoveStudentDiscountAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Concession record not found." });

    return Ok(new { success = true, message = "Concession removed successfully." });
}

// =====
// 9. LATE FINE RULES
// =====

[HttpGet("fine-rules")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetFineRules(
    [FromQuery] string? search,
    [FromQuery] string? status)

```

```

{
    var result = await _masterService.GetFineRulesAsync(search, status);
    return Ok(new { success = true, data = result });
}

[HttpGet("fine-rules/{id:int}")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetFineRuleById(int id)
{
    var item = await _masterService.GetFineRuleByIdAsync(id);
    if (item == null)
        return NotFound(new { success = false, message = "Fine rule not found." });
    return Ok(new { success = true, data = item });
}

[HttpPost("fine-rules")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> CreateFineRule([FromBody] FineRuleDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.RuleName))
        return BadRequest(new { success = false, message = "Rule name is required." });

    var result = await _masterService.CreateFineRuleAsync(dto);
    return Ok(new { success = true, message = "Fine rule created successfully.", data = result });
}

[HttpPut("fine-rules/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> UpdateFineRule(int id, [FromBody] FineRuleDto dto)
{
    var result = await _masterService.UpdateFineRuleAsync(id, dto);
    if (result == null)
        return NotFound(new { success = false, message = "Fine rule not found." });

    return Ok(new { success = true, message = "Fine rule updated successfully.", data = result });
}

[HttpDelete("fine-rules/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> DeleteFineRule(int id)
{
    var success = await _masterService.DeleteFineRuleAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Fine rule not found." });

    return Ok(new { success = true, message = "Fine rule deleted successfully." });
}

// =====
// 9. HOSTEL FEE CONFIGURATIONS
// =====

[HttpGet("hostel-fees")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetHostelFeeConfigs(
    [FromQuery] string? search,
    [FromQuery] string? hostelId,
    [FromQuery] string? status)
{
    var result = await _masterService.GetHostelFeeConfigsAsync(search, hostelId, status);
    return Ok(new { success = true, data = result });
}

```

```

[HttpGet("hostel-fees/{id:int}")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetHostelFeeConfigById(int id)
{
    var item = await _masterService.GetHostelFeeConfigByIdAsync(id);
    if (item == null)
        return NotFound(new { success = false, message = "Hostel fee configuration not found." });
    return Ok(new { success = true, data = item });
}

[HttpPost("hostel-fees")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> CreateHostelFeeConfig([FromBody] CreateFinanceHostelConfigDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.HostelId) || dto.HostelFee < 0)
        return BadRequest(new { success = false, message = "Valid hostel block and non-negative hostel fee are required." });

    var result = await _masterService.CreateHostelFeeConfigAsync(dto);
    return Ok(new { success = true, message = "Hostel fee configuration created successfully.", data = result });
}

[HttpPut("hostel-fees/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> UpdateHostelFeeConfig(int id, [FromBody] CreateFinanceHostelConfigDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.HostelId) || dto.HostelFee < 0)
        return BadRequest(new { success = false, message = "Valid hostel block and non-negative hostel fee are required." });

    var result = await _masterService.UpdateHostelFeeConfigAsync(id, dto);
    if (result == null)
        return NotFound(new { success = false, message = "Hostel fee configuration not found." });

    return Ok(new { success = true, message = "Hostel fee configuration updated successfully.", data = result });
}

[HttpDelete("hostel-fees/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> DeleteHostelFeeConfig(int id)
{
    var success = await _masterService.DeleteHostelFeeConfigAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Hostel fee configuration not found." });

    return Ok(new { success = true, message = "Hostel fee configuration deleted successfully." });
}

// =====
// 10. UNIFORM FEE CONFIGURATIONS
// =====

[HttpGet("uniform-fees")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetUniformFeeConfigs(
    [FromQuery] string? search,
    [FromQuery] string? className,
    [FromQuery] string? academicYear,
    [FromQuery] string? status)
{

```

```

        var result = await _masterService.GetUniformFeeConfigsAsync(search, className, academicYear, status);
    };

    return Ok(new { success = true, data = result });
}

[HttpGet("uniform-fees/{id:int}")]
[Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
public async Task<IActionResult> GetUniformFeeConfigById(int id)
{
    var item = await _masterService.GetUniformFeeConfigByIdAsync(id);
    if (item == null)
        return NotFound(new { success = false, message = "Uniform fee configuration not found." });
    return Ok(new { success = true, data = item });
}

[HttpPost("uniform-fees")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> CreateUniformFeeConfig([FromBody] CreateFinanceUniformConfigDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.ClassName) || string.IsNullOrEmpty(dto.UniformPackage) || dto.FeeAmount < 0)
        return BadRequest(new { success = false, message = "Class name, uniform package/item, and non-negative fee amount are required." });

    var result = await _masterService.CreateUniformFeeConfigAsync(dto);
    return Ok(new { success = true, message = "Uniform fee configuration created successfully.", data = result });
}

[HttpPut("uniform-fees/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> UpdateUniformFeeConfig(int id, [FromBody] CreateFinanceUniformConfigDto dto)
{
    if (dto == null || string.IsNullOrEmpty(dto.ClassName) || string.IsNullOrEmpty(dto.UniformPackage) || dto.FeeAmount < 0)
        return BadRequest(new { success = false, message = "Class name, uniform package/item, and non-negative fee amount are required." });

    var result = await _masterService.UpdateUniformFeeConfigAsync(id, dto);
    if (result == null)
        return NotFound(new { success = false, message = "Uniform fee configuration not found." });

    return Ok(new { success = true, message = "Uniform fee configuration updated successfully.", data = result });
}

[HttpDelete("uniform-fees/{id:int}")]
[Authorize(Roles = "Admin,SuperAdmin,Accountant")]
public async Task<IActionResult> DeleteUniformFeeConfig(int id)
{
    var success = await _masterService.DeleteUniformFeeConfigAsync(id);
    if (!success)
        return NotFound(new { success = false, message = "Uniform fee configuration not found." });

    return Ok(new { success = true, message = "Uniform fee configuration deleted successfully." });
}
}

```

 Controllers/FinanceManagement/FeeCollectionController.cs

CSHARP

```

namespace SMS.Api.Controllers.FinanceManagement;

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SMS.Api.Dtos.FinanceManagement;
using SMS.Api.Services.Interfaces.FinanceManagement;
using System.Threading.Tasks;

[ApiController]
[Route("api/finance")]
[Authorize]
[Tags("Finance & Fee Collection Module")]
public class FeeCollectionController : ControllerBase
{
    private readonly IFeeCollectionService _feeCollectionService;

    public FeeCollectionController(IFeeCollectionService feeCollectionService)
    {
        _feeCollectionService = feeCollectionService;
    }

    // =====
    // 1. FEE COLLECTION ROSTER & DETAIL
    // =====

    [HttpGet("fee-collection/students")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetStudentRoster(
        [FromQuery] string? search,
        [FromQuery] string? className,
        [FromQuery] string? sectionName,
        [FromQuery] string? studentType,
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 50)
    {
        var result = await _feeCollectionService.GetStudentRosterAsync(
            search, className, sectionName, studentType, page, pageSize);
        return Ok(new { success = true, data = result });
    }

    [HttpGet("fee-collection/student/{studentId:int}")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant,Parent,Student")]
    public async Task<IActionResult> GetStudentFeeProfile(
        int studentId,
        [FromQuery] string? academicYear = "2026-2027")
    {
        var result = await _feeCollectionService.GetStudentFeeProfileAsync(studentId, academicYear);
        if (result == null)
            return NotFound(new { success = false, message = "Student not found." });

        return Ok(new { success = true, data = result });
    }

    [HttpPost("fee-collection/collect")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> CollectFeePayment([FromBody] CollectFeePaymentRequestDto request)
    {
        if (request == null || request.TotalAmountPaid <= 0)
            return BadRequest(new { success = false, message = "Valid payment details and amount are requir

```

```

ed." });

        var response = await _feeCollectionService.CollectPaymentAsync(request);
        return Ok(response);
    }

    // =====
    // 2. DUE FEES & DEFAULTERS
    // =====

    [HttpGet("due-fees")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetDueFeesSummary(
        [FromQuery] string? className,
        [FromQuery] string? sectionName,
        [FromQuery] int minDaysOverdue = 0)
    {
        var result = await _feeCollectionService.GetDueFeesSummaryAsync(className, sectionName, minDaysOver
due);
        return Ok(new { success = true, data = result });
    }

    [HttpPost("due-fees/send-reminder")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public IActionResult SendFeeReminder([FromBody] SendFeeReminderRequestDto request)
    {
        if (request == null || request.StudentId <= 0)
            return BadRequest(new { success = false, message = "Student ID is required." });

        return Ok(new
        {
            success = true,
            message = $"Payment reminder {request.ReminderType} sent successfully to student's registered p
arent phone number."
        });
    }

    // =====
    // 3. PROMOTED STUDENTS DUES
    // =====

    [HttpGet("promoted-students-dues")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetPromotedStudentsDues(
        [FromQuery] string? search,
        [FromQuery] string? className,
        [FromQuery] string? previousAcademicYear,
        [FromQuery] string? status)
    {
        var result = await _feeCollectionService.GetPromotedStudentsDuesAsync(search, className, previousAc
ademicYear, status);
        return Ok(new { success = true, data = result });
    }

    // =====
    // 4. RECEIPTS REGISTER & PRINTABLE RECEIPT
    // =====

    [HttpGet("receipts")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant,Parent")]
    public async Task<IActionResult> GetReceiptsRegister(
        [FromQuery] string? search,
        [FromQuery] string? paymentMode,

```

```

        [FromQuery] string? fromDate,
        [FromQuery] string? toDate,
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 50)
    {
        var result = await _feeCollectionService.GetReceiptsRegisterAsync(
            search, paymentMode, fromDate, toDate, page, pageSize);
        return Ok(new { success = true, data = result });
    }

    [HttpGet("receipts/{receiptNo}")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant,Parent,Student")]
    public async Task<IActionResult> GetReceiptByNo(string receiptNo)
    {
        var result = await _feeCollectionService.GetReceiptByNoAsync(receiptNo);
        if (result == null)
            return NotFound(new { success = false, message = "Receipt not found." });

        return Ok(new { success = true, data = result });
    }

    [HttpPost("receipts/{receiptNo}/cancel")]
    [Authorize(Roles = "Admin,SuperAdmin")]
    public async Task<IActionResult> CancelReceipt(string receiptNo, [FromBody] string? reason)
    {
        var success = await _feeCollectionService.CancelReceiptAsync(receiptNo, reason ?? "Administrative c
ancellation");
        if (!success)
            return NotFound(new { success = false, message = "Receipt not found." });

        return Ok(new { success = true, message = "Receipt cancelled successfully." });
    }

    // =====
    // 5. FINANCE DASHBOARD & REPORTS
    // =====

    [HttpGet("dashboard")]
    [Authorize(Roles = "Admin,Staff,SuperAdmin,Accountant")]
    public async Task<IActionResult> GetDashboardStats()
    {
        var result = await _feeCollectionService.GetDashboardStatsAsync();
        return Ok(new { success = true, data = result });
    }
}

```

 Services/Implementations/FinanceManagement/FinanceService.cs

CSHARP

```

namespace SMS.Api.Services.Implementations.FinanceManagement;

using SMS.Api.Dtos.FinanceManagement;
using SMS.Api.Models.FinanceManagement;
using SMS.Api.Repositories.Interfaces.FinanceManagement;
using SMS.Api.Services.Interfaces.FinanceManagement;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.Json;
using System.Threading.Tasks;

public class FinanceService : IFinanceService
{
    private readonly IFinanceRepository _repo;

    public FinanceService(IFinanceRepository repo)
    {
        _repo = repo;
    }

    private class FeeHeadMetadata
    {
        public string? code { get; set; }
        public bool? mandatory { get; set; }
        public int? displayOrder { get; set; }
        public decimal? taxPercentage { get; set; }
        public List<string>? applicableClasses { get; set; }
        public List<string>? applicableBranches { get; set; }
    }

    public async Task<IEnumerable<FeeHeadDto>> GetFeeHeadsAsync()
    {
        var list = await _repo.GetFeeHeadsAsync();
        var dtos = new List<FeeHeadDto>();

        foreach (var x in list)
        {
            var dto = new FeeHeadDto
            {
                Id = x.Id,
                Name = x.Name,
                Code = $"FH-{x.Id:D3}",
                Category = x.Category,
                Frequency = x.Frequency,
                DefaultAmount = x.DefaultAmount,
                IsRefundable = x.IsRefundable,
                IsTaxable = x.IsTaxable,
                Status = x.Status ?? "Active",
                Description = x.Description ?? "",
                Mandatory = true,
                DisplayOrder = x.Id,
                TaxPercentage = 0,
                ApplicableClasses = new List<string> { "Nursery", "LKG", "UKG", "Class 1", "Class 2", "Class 3", "Class 4", "Class 5", "Class 6", "Class 7", "Class 8", "Class 9", "Class 10" },
                ApplicableBranches = new List<string> { "All Branches" }
            };

            if (!string.IsNullOrEmpty(x.Description) && x.Description.Trim().StartsWith("{"))

```

```

        {
            try
            {
                var meta = JsonSerializer.Deserialize<FeeHeadMetadata>(x.Description);
                if (meta != null)
                {
                    if (!string.IsNullOrEmpty(meta.code)) dto.Code = meta.code;
                    if (meta.mandatory.HasValue) dto.Mandatory = meta.mandatory.Value;
                    if (meta.displayOrder.HasValue) dto.DisplayOrder = meta.displayOrder.Value;
                    if (meta.taxPercentage.HasValue) dto.TaxPercentage = meta.taxPercentage.Value;
                    if (meta.applicableClasses != null && meta.applicableClasses.Count > 0) dto.ApplicableClasses = meta.applicableClasses;
                    if (meta.applicableBranches != null && meta.applicableBranches.Count > 0) dto.ApplicableBranches = meta.applicableBranches;
                }
            }
            catch { }
        }

        dtos.Add(dto);
    }

    return dtos.OrderBy(d => d.DisplayOrder);
}

public async Task<FeeHeadDto> CreateFeeHeadAsync(FeeHeadDto dto)
{
    if (string.IsNullOrEmpty(dto.Name))
        throw new InvalidOperationException("Fee Head name is required.");
    if (string.IsNullOrEmpty(dto.Code))
        throw new InvalidOperationException("Fee Head code is required.");

    var existingList = await _repo.GetFeeHeadsAsync();
    string cleanCode = dto.Code.Trim().ToLower();
    string cleanName = dto.Name.Trim().ToLower();
    string cleanCategory = (dto.Category ?? "").Trim().ToLower();

    foreach (var item in existingList)
    {
        string itemCode = "";
        if (!string.IsNullOrEmpty(item.Description) && item.Description.Trim().StartsWith("{"))
        {
            try
            {
                var metaObj = JsonSerializer.Deserialize<FeeHeadMetadata>(item.Description);
                if (metaObj != null && !string.IsNullOrEmpty(metaObj.code)) itemCode = metaObj.code.Trim().ToLower();
            }
            catch { }
        }
        if (string.IsNullOrEmpty(itemCode)) itemCode = $"fh-{item.Id:d3}";

        if (!string.IsNullOrEmpty(cleanCode) && itemCode == cleanCode)
        {
            throw new InvalidOperationException($"Duplicate Error: A Fee Head with code '{dto.Code}' already exists.");
        }
        if (!string.IsNullOrEmpty(cleanName) && item.Name.Trim().ToLower() == cleanName && (item.Category ?? "").Trim().ToLower() == cleanCategory)
        {
            throw new InvalidOperationException($"Duplicate Error: A Fee Head with name '{dto.Name}' in category '{dto.Category}' already exists.");
        }
    }
}

```

```

    }

    var meta = new FeeHeadMetadata
    {
        code = string.IsNullOrEmpty(dto.Code) ? $"FH-{Random.Shared.Next(100, 999)}" : dto.Code,
        mandatory = dto.Mandatory,
        displayOrder = dto.DisplayOrder > 0 ? dto.DisplayOrder : 1,
        taxPercentage = dto.TaxPercentage,
        applicableClasses = dto.ApplicableClasses ?? new List<string>(),
        applicableBranches = (dto.ApplicableBranches != null && dto.ApplicableBranches.Count > 0) ? dt
o.ApplicableBranches : new List<string> { "All Branches" }
    };

    var model = new FeeHead
    {
        Name = dto.Name,
        Category = dto.Category,
        Frequency = dto.Frequency,
        DefaultAmount = dto.DefaultAmount,
        IsRefundable = dto.IsRefundable,
        IsTaxable = dto.IsTaxable || dto.TaxPercentage > 0,
        Status = dto.Status ?? "Active",
        Description = JsonSerializer.Serialize(meta)
    };

    var res = await _repo.CreateFeeHeadAsync(model);
    dto.Id = res.Id;
    dto.Code = meta.code;
    return dto;
}

public async Task<FeeHeadDto> UpdateFeeHeadAsync(int id, FeeHeadDto dto)
{
    if (string.IsNullOrWhiteSpace(dto.Name))
        throw new InvalidOperationException("Fee Head name is required.");
    if (string.IsNullOrWhiteSpace(dto.Code))
        throw new InvalidOperationException("Fee Head code is required.");

    var existingList = await _repo.GetFeeHeadsAsync();
    string cleanCode = dto.Code.Trim().ToLower();
    string cleanName = dto.Name.Trim().ToLower();
    string cleanCategory = (dto.Category ?? "").Trim().ToLower();

    foreach (var item in existingList)
    {
        if (item.Id == id) continue;

        string itemCode = "";
        if (!string.IsNullOrEmpty(item.Description) && item.Description.Trim().StartsWith("{"))
        {
            try
            {
                var metaObj = JsonSerializer.Deserialize<FeeHeadMetadata>(item.Description);
                if (metaObj != null && !string.IsNullOrEmpty(metaObj.code)) itemCode = metaObj.code.Tri
m().ToLower();
            }
            catch { }
        }
        if (string.IsNullOrEmpty(itemCode)) itemCode = $"fh-{item.Id:d3}";

        if (!string.IsNullOrEmpty(cleanCode) && itemCode == cleanCode)
        {
            throw new InvalidOperationException($"Duplicate Error: A Fee Head with code '{dto.Code}' al

```

```

ready exists.");
    }
    if (!string.IsNullOrEmpty(cleanName) && item.Name.Trim().ToLower() == cleanName && (item.Category ?? "").Trim().ToLower() == cleanCategory)
    {
        throw new InvalidOperationException($"Duplicate Error: A Fee Head with name '{dto.Name}' in category '{dto.Category}' already exists.");
    }
}

var meta = new FeeHeadMetadata
{
    code = string.IsNullOrEmpty(dto.Code) ? $"FH-{id:D3}" : dto.Code,
    mandatory = dto.Mandatory,
    displayOrder = dto.DisplayOrder > 0 ? dto.DisplayOrder : id,
    taxPercentage = dto.TaxPercentage,
    applicableClasses = dto.ApplicableClasses ?? new List<string>(),
    applicableBranches = (dto.ApplicableBranches != null && dto.ApplicableBranches.Count > 0) ? dto.ApplicableBranches : new List<string> { "All Branches" }
};

var model = new FeeHead
{
    Id = id,
    Name = dto.Name,
    Category = dto.Category,
    Frequency = dto.Frequency,
    DefaultAmount = dto.DefaultAmount,
    IsRefundable = dto.IsRefundable,
    IsTaxable = dto.IsTaxable || dto.TaxPercentage > 0,
    Status = dto.Status ?? "Active",
    Description = JsonSerializer.Serialize(meta)
};

await _repo.UpdateFeeHeadAsync(model);
dto.Id = id;
return dto;
}

public async Task<FeeHeadDto?> ToggleFeeHeadStatusAsync(int id)
{
    var heads = await _repo.GetFeeHeadsAsync();
    var item = heads.FirstOrDefault(h => h.Id == id);
    if (item == null) return null;

    item.Status = item.Status == "Active" ? "Inactive" : "Active";
    await _repo.UpdateFeeHeadAsync(item);

    var dtos = await GetFeeHeadsAsync();
    return dtos.FirstOrDefault(d => d.Id == id);
}

public async Task DeleteFeeHeadAsync(int id)
{
    await _repo.DeleteFeeHeadAsync(id);
}

public async Task<IEnumerable<DynamicFeeStructureDto>> GetDynamicFeeStructuresAsync()
{
    var list = await _repo.GetDynamicFeeStructuresAsync();
    return list.Select(x => new DynamicFeeStructureDto
    {
        Id = x.Id,

```

```

        Name = x.Name,
        Description = x.Description,
        TargetAudience = x.TargetAudience,
        AcademicYear = x.AcademicYear,
        Branch = x.Branch,
        ClassName = x.ClassName,
        Section = x.Section,
        StudentCategory = x.StudentCategory,
        TotalAmount = x.TotalAmount,
        Status = x.Status,
        Items = string.IsNullOrEmpty(x.ItemsJson)
            ? new List<FeeStructureItemDto>()
            : JsonSerializer.Deserialize<List<FeeStructureItemDto>>(x.ItemsJson) ?? new List<FeeStructureItemDto>();
    });
}

public async Task<DynamicFeeStructureDto?> GetDynamicFeeStructureByIdAsync(int id)
{
    var x = await _repo.GetDynamicFeeStructureByIdAsync(id);
    if (x == null) return null;
    return new DynamicFeeStructureDto
    {
        Id = x.Id,
        Name = x.Name,
        Description = x.Description,
        TargetAudience = x.TargetAudience,
        AcademicYear = x.AcademicYear,
        Branch = x.Branch,
        ClassName = x.ClassName,
        Section = x.Section,
        StudentCategory = x.StudentCategory,
        TotalAmount = x.TotalAmount,
        Status = x.Status,
        Items = string.IsNullOrEmpty(x.ItemsJson)
            ? new List<FeeStructureItemDto>()
            : JsonSerializer.Deserialize<List<FeeStructureItemDto>>(x.ItemsJson) ?? new List<FeeStructureItemDto>();
    };
}

public async Task<DynamicFeeStructureDto> CreateDynamicFeeStructureAsync(DynamicFeeStructureDto dto)
{
    var model = new DynamicFeeStructure
    {
        Name = string.IsNullOrEmpty(dto.Name) ? $"{dto.ClassName} Fee Structure" : dto.Name,
        Description = dto.Description,
        TargetAudience = dto.TargetAudience,
        AcademicYear = string.IsNullOrEmpty(dto.AcademicYear) ? "2026-2027" : dto.AcademicYear,
        Branch = string.IsNullOrEmpty(dto.Branch) ? "Main Campus" : dto.Branch,
        ClassName = dto.ClassName,
        Section = string.IsNullOrEmpty(dto.Section) ? "All Sections" : dto.Section,
        StudentCategory = string.IsNullOrEmpty(dto.StudentCategory) ? "General" : dto.StudentCategory,
        TotalAmount = (dto.Items != null && dto.Items.Count > 0) ? dto.Items.Sum(i => i.Amount) : dto.TotalAmount,
        Status = dto.Status ?? "Active",
        ItemsJson = JsonSerializer.Serialize(dto.Items ?? new List<FeeStructureItemDto>());
    };
    var res = await _repo.CreateDynamicFeeStructureAsync(model);
    dto.Id = res.Id;
    dto.TotalAmount = model.TotalAmount;
    dto.Name = model.Name;
    return dto;
}

```

```

    }

    public async Task<DynamicFeeStructureDto> UpdateDynamicFeeStructureAsync(int id, DynamicFeeStructureDt
o dto)
    {
        var model = new DynamicFeeStructure
        {
            Id = id,
            Name = string.IsNullOrEmpty(dto.Name) ? $"{dto.ClassName} Fee Structure" : dto.Name,
            Description = dto.Description,
            TargetAudience = dto.TargetAudience,
            AcademicYear = string.IsNullOrEmpty(dto.AcademicYear) ? "2026-2027" : dto.AcademicYear,
            Branch = string.IsNullOrEmpty(dto.Branch) ? "Main Campus" : dto.Branch,
            ClassName = dto.ClassName,
            Section = string.IsNullOrEmpty(dto.Section) ? "All Sections" : dto.Section,
            StudentCategory = string.IsNullOrEmpty(dto.StudentCategory) ? "General" : dto.StudentCategory,
            TotalAmount = (dto.Items != null && dto.Items.Count > 0) ? dto.Items.Sum(i => i.Amount) : dto.T
otalAmount,
            Status = dto.Status ?? "Active",
            ItemsJson = JsonSerializer.Serialize(dto.Items ?? new List<FeeStructureItemDto>())
        };
        var res = await _repo.UpdateDynamicFeeStructureAsync(model);
        dto.Id = id;
        dto.TotalAmount = model.TotalAmount;
        dto.Name = model.Name;
        return dto;
    }

    public async Task DeleteDynamicFeeStructureAsync(int id)
    {
        await _repo.DeleteDynamicFeeStructureAsync(id);
    }

    public async Task<IEnumerable<StudentFeeAssignmentDto>> GetStudentFeeAssignmentsAsync()
    {
        var list = await _repo.GetStudentFeeAssignmentsAsync();
        return list.Select(x => new StudentFeeAssignmentDto
        {
            Id = x.Id,
            StudentId = x.StudentId,
            DynamicFeeStructureId = x.DynamicFeeStructureId,
            TotalAmount = x.TotalAmount,
            PaidAmount = x.PaidAmount,
            DueAmount = x.DueAmount,
            Status = x.Status,
            FeePolicy = string.IsNullOrEmpty(x.FeePolicy) ? "Full Annual Fee" : x.FeePolicy
        });
    }

    public async Task<StudentFeeAssignmentDto> GetStudentFeeAssignmentByIdAsync(int id)
    {
        var x = await _repo.GetStudentFeeAssignmentByIdAsync(id);
        if (x == null) return null;
        return new StudentFeeAssignmentDto
        {
            Id = x.Id,
            StudentId = x.StudentId,
            DynamicFeeStructureId = x.DynamicFeeStructureId,
            TotalAmount = x.TotalAmount,
            PaidAmount = x.PaidAmount,
            DueAmount = x.DueAmount,
            Status = x.Status,
            FeePolicy = string.IsNullOrEmpty(x.FeePolicy) ? "Full Annual Fee" : x.FeePolicy
        }
    }

```

```

    });
}

public async Task<StudentFeeAssignmentDto> CreateStudentFeeAssignmentAsync(StudentFeeAssignmentDto dto)
{
    var existing = (await _repo.GetStudentFeeAssignmentsAsync()).FirstOrDefault(x => x.StudentId == dt
o.StudentId);
    if (existing != null)
    {
        existing.DynamicFeeStructureId = dto.DynamicFeeStructureId ?? existing.DynamicFeeStructureId;
        existing.TotalAmount = dto.TotalAmount > 0 ? dto.TotalAmount : existing.TotalAmount;
        existing.DueAmount = existing.TotalAmount - existing.PaidAmount;
        existing.FeePolicy = !string.IsNullOrEmpty(dto.FeePolicy) ? dto.FeePolicy : existing.FeePo
licy;

        existing.Status = !string.IsNullOrEmpty(dto.Status) ? dto.Status : "Active";
        await _repo.UpdateStudentFeeAssignmentAsync(existing);
        dto.Id = existing.Id;
        return dto;
    }

    var model = new StudentFeeAssignment
    {
        StudentId = dto.StudentId,
        DynamicFeeStructureId = dto.DynamicFeeStructureId,
        TotalAmount = dto.TotalAmount,
        PaidAmount = dto.PaidAmount,
        DueAmount = dto.TotalAmount - dto.PaidAmount,
        Status = string.IsNullOrEmpty(dto.Status) ? "Active" : dto.Status,
        FeePolicy = string.IsNullOrEmpty(dto.FeePolicy) ? "Full Annual Fee" : dto.FeePolicy
    };
    var res = await _repo.CreateStudentFeeAssignmentAsync(model);
    dto.Id = res.Id;
    return dto;
}

public async Task<List<StudentFeeAssignmentDto>> BulkAssignStudentFeesAsync(BulkFeeAssignmentRequestDto
request)
{
    var results = new List<StudentFeeAssignmentDto>();
    decimal defaultTotal = request.TotalAmount ?? 0m;

    if (defaultTotal <= 0 && request.DynamicFeeStructureId.HasValue)
    {
        var dfs = await _repo.GetDynamicFeeStructureByIdAsync(request.DynamicFeeStructureId.Value);
        if (dfs != null)
        {
            defaultTotal = dfs.TotalAmount;
        }
    }

    foreach (var studentId in request.StudentIds)
    {
        var dto = new StudentFeeAssignmentDto
        {
            StudentId = studentId,
            DynamicFeeStructureId = request.DynamicFeeStructureId,
            TotalAmount = defaultTotal,
            PaidAmount = 0,
            DueAmount = defaultTotal,
            Status = "Active",
            FeePolicy = string.IsNullOrEmpty(request.FeePolicy) ? "Full Annual Fee" : request.FeeP
olicy,
            AssignedDate = DateTime.Now.ToString("yyyy-MM-dd")
        };
    }
}

```

```

        };

        var saved = await CreateStudentFeeAssignmentAsync(dto);
        results.Add(saved);
    }

    return results;
}

public async Task<StudentFeeAssignmentDto> SaveCustomStudentFeeAssignmentAsync(CustomFeeAssignmentRequestDto request)
{
    decimal total = request.TotalAmount;
    if (total <= 0 && request.Breakdown != null && request.Breakdown.Any())
    {
        total = request.Breakdown.Sum(b => b.AssignedAmount);
    }

    var dto = new StudentFeeAssignmentDto
    {
        StudentId = request.StudentId,
        DynamicFeeStructureId = request.DynamicFeeStructureId,
        TotalAmount = total,
        PaidAmount = 0,
        DueAmount = total,
        Status = "Active",
        FeePolicy = string.IsNullOrEmpty(request.FeePolicy) ? "Monthly Pro-rated Fee" : request.FeePolicy,
        AdjustmentReason = request.AdjustmentReason,
        AssignedDate = request.AdmissionDate ?? DateTime.Now.ToString("yyyy-MM-dd")
    };

    return await CreateStudentFeeAssignmentAsync(dto);
}

public async Task<StudentFeeAssignmentDto> UpdateStudentFeeAssignmentAsync(int id, StudentFeeAssignmentDto dto)
{
    var existing = await _repo.GetStudentFeeAssignmentByIdAsync(id);
    if (existing == null) return null;

    existing.DynamicFeeStructureId = dto.DynamicFeeStructureId ?? existing.DynamicFeeStructureId;
    existing.TotalAmount = dto.TotalAmount;
    existing.PaidAmount = dto.PaidAmount;
    existing.DueAmount = dto.TotalAmount - dto.PaidAmount;
    existing.Status = string.IsNullOrEmpty(dto.Status) ? existing.Status : dto.Status;
    existing.FeePolicy = string.IsNullOrEmpty(dto.FeePolicy) ? existing.FeePolicy : dto.FeePolicy;

    var updated = await _repo.UpdateStudentFeeAssignmentAsync(existing);
    dto.Id = updated.Id;
    return dto;
}

public async Task<bool> DeleteStudentFeeAssignmentAsync(int id)
{
    var existing = await _repo.GetStudentFeeAssignmentByIdAsync(id);
    if (existing == null) return false;

    await _repo.DeleteStudentFeeAssignmentAsync(id);
    return true;
}

public async Task<IEnumerable<FeePaymentDto>> GetFeePaymentsAsync()

```

```
{
    var list = await _repo.GetFeePaymentsAsync();
    return list.Select(x => new FeePaymentDto
    {
        Id = x.Id,
        ReceiptNo = x.ReceiptNo,
        StudentId = x.StudentId,
        Amount = x.Amount,
        DiscountAmount = x.DiscountAmount,
        FineAmount = x.FineAmount,
        TransportFee = x.TransportFee,
        PaymentMethod = x.PaymentMethod,
        TransactionId = x.TransactionId,
        PaymentDate = x.PaymentDate,
        Status = x.Status
    });
}

public async Task<FeePaymentDto> CreateFeePaymentAsync(FeePaymentDto dto)
{
    var model = new FeePayment
    {
        ReceiptNo = dto.ReceiptNo,
        StudentId = dto.StudentId,
        Amount = dto.Amount,
        DiscountAmount = dto.DiscountAmount,
        FineAmount = dto.FineAmount,
        TransportFee = dto.TransportFee,
        PaymentMethod = dto.PaymentMethod,
        TransactionId = dto.TransactionId,
        PaymentDate = dto.PaymentDate == default ? DateTime.UtcNow : dto.PaymentDate,
        Status = dto.Status
    };
    var res = await _repo.CreateFeePaymentAsync(model);
    dto.Id = res.Id;
    return dto;
}
}
```

 Services/Implementations/FinanceManagement/FinanceMasterService.cs

CSHARP

```
namespace SMS.Api.Services.Implementations.FinanceManagement;

using SMS.Api.Dtos.FinanceManagement;
using SMS.Api.Repositories.Interfaces.FinanceManagement;
using SMS.Api.Services.Interfaces.FinanceManagement;
using System.Collections.Generic;
using System.Threading.Tasks;

public class FinanceMasterService : IFinanceMasterService
{
    private readonly IFinanceMasterRepository _repository;

    public FinanceMasterService(IFinanceMasterRepository repository)
    {
        _repository = repository;
    }

    public Task<List<FinanceTransactionDto>> GetTransactionsAsync(
        string? search, string? type, string? module, string? category, string? paymentMode, string? status, int page, int pageSize)
    {
        return _repository.GetTransactionsAsync(search, type, module, category, paymentMode, status, page, pageSize);
    }

    public Task<FinanceTransactionSummaryDto> GetTransactionSummaryAsync()
    {
        return _repository.GetTransactionSummaryAsync();
    }

    public Task<FinanceTransactionDto> CreateTransactionAsync(CreateTransactionRequestDto request)
    {
        return _repository.CreateTransactionAsync(request);
    }

    public Task<bool> ReverseTransactionAsync(int id, ReverseTransactionRequestDto request)
    {
        return _repository.ReverseTransactionAsync(id, request);
    }

    public Task<List<FinancialAccountDto>> GetAccountsAsync()
    {
        return _repository.GetAccountsAsync();
    }

    public Task<FinancialAccountDto> CreateAccountAsync(FinancialAccountDto account)
    {
        return _repository.CreateAccountAsync(account);
    }

    public Task<bool> UpdateAccountAsync(int id, FinancialAccountDto account)
    {
        return _repository.UpdateAccountAsync(id, account);
    }

    public Task<bool> DeleteAccountAsync(int id)
    {
        return _repository.DeleteAccountAsync(id);
    }
}
```

```
public Task<List<FinancialCategoryDto>> GetCategoriesAsync(string? type)
{
    return _repository.GetCategoriesAsync(type);
}

public Task<FinancialCategoryDto> CreateCategoryAsync(FinancialCategoryDto category)
{
    return _repository.CreateCategoryAsync(category);
}

public Task<bool> UpdateCategoryAsync(int id, FinancialCategoryDto category)
{
    return _repository.UpdateCategoryAsync(id, category);
}

public Task<bool> DeleteCategoryAsync(int id)
{
    return _repository.DeleteCategoryAsync(id);
}

public Task<List<FinancialBudgetDto>> GetBudgetsAsync(string? academicYear)
{
    return _repository.GetBudgetsAsync(academicYear);
}

public Task<FinancialBudgetDto> SaveBudgetAsync(FinancialBudgetDto budget)
{
    return _repository.SaveBudgetAsync(budget);
}

public Task<bool> UpdateBudgetAsync(int id, FinancialBudgetDto budget)
{
    return _repository.UpdateBudgetAsync(id, budget);
}

public Task<List<FeeRefundRequestDto>> GetRefundRequestsAsync(string? status)
{
    return _repository.GetRefundRequestsAsync(status);
}

public Task<FeeRefundRequestDto> CreateRefundRequestAsync(CreateRefundRequestDto request)
{
    return _repository.CreateRefundRequestAsync(request);
}

public Task<bool> ProcessRefundRequestAsync(int id, ProcessRefundRequestDto request)
{
    return _repository.ProcessRefundRequestAsync(id, request);
}

public Task<FeeScheduleConfigDto> GetFeeScheduleAsync(string? academicYear)
{
    return _repository.GetFeeScheduleAsync(academicYear);
}

public Task<bool> SaveFeeScheduleAsync(FeeScheduleConfigDto schedule)
{
    return _repository.SaveFeeScheduleAsync(schedule);
}

public Task<FinanceSettingsDto> GetFinanceSettingsAsync()
{

```

```
        return _repository.GetFinanceSettingsAsync();
    }

    public Task<bool> UpdateFinanceSettingsAsync(FinanceSettingsDto settings)
    {
        return _repository.UpdateFinanceSettingsAsync(settings);
    }

    public Task<FinanceReportsSummaryDto> GetReportsSummaryAsync(string? academicYear)
    {
        return _repository.GetReportsSummaryAsync(academicYear);
    }

    public Task<DailyCollectionReportResponseDto> GetDailyCollectionReportAsync(string? date)
    {
        return _repository.GetDailyCollectionReportAsync(date);
    }

    public Task<List<ClassWiseCollectionReportRowDto>> GetClassWiseCollectionReportAsync(string? academicYear)
    {
        return _repository.GetClassWiseCollectionReportAsync(academicYear);
    }

    public Task<List<ScholarshipMasterDto>> GetScholarshipsAsync(string? search, string? type, string? status)
    {
        return _repository.GetScholarshipsAsync(search, type, status);
    }

    public Task<ScholarshipMasterDto?> GetScholarshipByIdAsync(int id)
    {
        return _repository.GetScholarshipByIdAsync(id);
    }

    public Task<ScholarshipMasterDto> CreateScholarshipAsync(ScholarshipMasterDto scholarship)
    {
        return _repository.CreateScholarshipAsync(scholarship);
    }

    public Task<ScholarshipMasterDto?> UpdateScholarshipAsync(int id, ScholarshipMasterDto scholarship)
    {
        return _repository.UpdateScholarshipAsync(id, scholarship);
    }

    public Task<bool> DeleteScholarshipAsync(int id)
    {
        return _repository.DeleteScholarshipAsync(id);
    }

    public Task<List<StudentScholarshipAwardDto>> GetStudentScholarshipsAsync(string? search, string? className, int? scholarshipId)
    {
        return _repository.GetStudentScholarshipsAsync(search, className, scholarshipId);
    }

    public Task<StudentScholarshipAwardDto> AwardScholarshipToStudentAsync(AwardScholarshipRequestDto request)
    {
        return _repository.AwardScholarshipToStudentAsync(request);
    }

    public Task<bool> RevokeStudentScholarshipAsync(int id)
```

```
{
    return _repository.RevokeStudentScholarshipAsync(id);
}

public Task<List<DiscountRuleDto>> GetDiscountsAsync(string? search, string? type, string? mode, string? status)
{
    return _repository.GetDiscountsAsync(search, type, mode, status);
}

public Task<DiscountRuleDto?> GetDiscountByIdAsync(int id)
{
    return _repository.GetDiscountByIdAsync(id);
}

public Task<DiscountRuleDto> CreateDiscountAsync(DiscountRuleDto discount)
{
    return _repository.CreateDiscountAsync(discount);
}

public Task<DiscountRuleDto?> UpdateDiscountAsync(int id, DiscountRuleDto discount)
{
    return _repository.UpdateDiscountAsync(id, discount);
}

public Task<bool> DeleteDiscountAsync(int id)
{
    return _repository.DeleteDiscountAsync(id);
}

public Task<List<StudentDiscountDto>> GetStudentDiscountsAsync(string? search, string? className, int? discountId)
{
    return _repository.GetStudentDiscountsAsync(search, className, discountId);
}

public Task<StudentDiscountDto> GrantDiscountToStudentAsync(GrantDiscountRequestDto request)
{
    return _repository.GrantDiscountToStudentAsync(request);
}

public Task<bool> RemoveStudentDiscountAsync(int id)
{
    return _repository.RemoveStudentDiscountAsync(id);
}

public Task<List<FineRuleDto>> GetFineRulesAsync(string? search, string? status)
{
    return _repository.GetFineRulesAsync(search, status);
}

public Task<FineRuleDto?> GetFineRuleByIdAsync(int id)
{
    return _repository.GetFineRuleByIdAsync(id);
}

public Task<FineRuleDto> CreateFineRuleAsync(FineRuleDto rule)
{
    return _repository.CreateFineRuleAsync(rule);
}

public Task<FineRuleDto?> UpdateFineRuleAsync(int id, FineRuleDto rule)
{

```

```
        return _repository.UpdateFineRuleAsync(id, rule);
    }

    public Task<bool> DeleteFineRuleAsync(int id)
    {
        return _repository.DeleteFineRuleAsync(id);
    }

    public Task<List<FinanceHostelConfigDto>> GetHostelFeeConfigsAsync(string? search, string? hostelId, string? status)
    {
        return _repository.GetHostelFeeConfigsAsync(search, hostelId, status);
    }

    public Task<FinanceHostelConfigDto?> GetHostelFeeConfigByIdAsync(int id)
    {
        return _repository.GetHostelFeeConfigByIdAsync(id);
    }

    public Task<FinanceHostelConfigDto> CreateHostelFeeConfigAsync(CreateFinanceHostelConfigDto dto)
    {
        return _repository.CreateHostelFeeConfigAsync(dto);
    }

    public Task<FinanceHostelConfigDto?> UpdateHostelFeeConfigAsync(int id, CreateFinanceHostelConfigDto dto)
    {
        return _repository.UpdateHostelFeeConfigAsync(id, dto);
    }

    public Task<bool> DeleteHostelFeeConfigAsync(int id)
    {
        return _repository.DeleteHostelFeeConfigAsync(id);
    }

    public Task<List<FinanceUniformConfigDto>> GetUniformFeeConfigsAsync(string? search, string? className, string? academicYear, string? status)
    {
        return _repository.GetUniformFeeConfigsAsync(search, className, academicYear, status);
    }

    public Task<FinanceUniformConfigDto?> GetUniformFeeConfigByIdAsync(int id)
    {
        return _repository.GetUniformFeeConfigByIdAsync(id);
    }

    public Task<FinanceUniformConfigDto> CreateUniformFeeConfigAsync(CreateFinanceUniformConfigDto dto)
    {
        return _repository.CreateUniformFeeConfigAsync(dto);
    }

    public Task<FinanceUniformConfigDto?> UpdateUniformFeeConfigAsync(int id, CreateFinanceUniformConfigDto dto)
    {
        return _repository.UpdateUniformFeeConfigAsync(id, dto);
    }

    public Task<bool> DeleteUniformFeeConfigAsync(int id)
    {
        return _repository.DeleteUniformFeeConfigAsync(id);
    }
}
```

 Services/Implementations/FinanceManagement/FeeCollectionService.cs

CSHARP

```
namespace SMS.Api.Services.Implementations.FinanceManagement;

using SMS.Api.Dtos.FinanceManagement;
using SMS.Api.Repositories.Interfaces.FinanceManagement;
using SMS.Api.Services.Interfaces.FinanceManagement;
using System.Collections.Generic;
using System.Threading.Tasks;

public class FeeCollectionService : IFeeCollectionService
{
    private readonly IFeeCollectionRepository _repository;

    public FeeCollectionService(IFeeCollectionRepository repository)
    {
        _repository = repository;
    }

    public Task<FeeCollectionStudentRosterResponseDto> GetStudentRosterAsync(
        string? search, string? className, string? sectionName, string? studentType, int page, int pageSiz
e)
    {
        return _repository.GetStudentRosterAsync(search, className, sectionName, studentType, page, pageSiz
e);
    }

    public Task<StudentFeeProfileResponseDto?> GetStudentFeeProfileAsync(int studentId, string? academicYea
r)
    {
        return _repository.GetStudentFeeProfileAsync(studentId, academicYear);
    }

    public Task<CollectFeePaymentResponseDto> CollectPaymentAsync(CollectFeePaymentRequestDto request)
    {
        return _repository.CollectPaymentAsync(request);
    }

    public Task<DueFeesSummaryResponseDto> GetDueFeesSummaryAsync(
        string? className, string? sectionName, int minDaysOverdue)
    {
        return _repository.GetDueFeesSummaryAsync(className, sectionName, minDaysOverdue);
    }

    public Task<List<PromotedDueStudentDto>> GetPromotedStudentsDuesAsync(
        string? search = null, string? className = null, string? previousAcademicYear = null, string? statu
s = null)
    {
        return _repository.GetPromotedStudentsDuesAsync(search, className, previousAcademicYear, status);
    }

    public Task<FeeReceiptsRegisterResponseDto> GetReceiptsRegisterAsync(
        string? search, string? paymentMode, string? fromDate, string? toDate, int page, int pageSize)
    {
        return _repository.GetReceiptsRegisterAsync(search, paymentMode, fromDate, toDate, page, pageSize);
    }

    public Task<FeeReceiptDetailDto?> GetReceiptByNoAsync(string receiptNo)
    {
        return _repository.GetReceiptByNoAsync(receiptNo);
    }
}
```

```
public Task<bool> CancelReceiptAsync(string receiptNo, string reason)
{
    return _repository.CancelReceiptAsync(receiptNo, reason);
}

public Task<FinanceDashboardStatsDto> GetDashboardStatsAsync()
{
    return _repository.GetDashboardStatsAsync();
}
}
```

3. Frontend API & Core Components (React + TypeScript)

src/api/finance.ts

TYPESCRIPT

```

import { apiClient } from './client';
import { FeeHead, DynamicFeeStructure, StudentFeeAssignment, FeePayment } from '../types';

export const fetchDynamicFeeStructuresApi = async () => {
  const response: any = await apiClient('/api/finance/fee-structures', { method: 'GET' });
  const items = Array.isArray(response)
    ? response
    : response?.data?.items || response?.data || [];
  if (Array.isArray(items)) {
    return items.map((item: any) => ({
      id: item.id?.toString() || "",
      academicYear: item.academicYear || "2026-2027",
      branch: item.branch || "Main Campus",
      className: item.className || "",
      section: item.section || "",
      studentCategory: item.studentCategory || "General",
      items: (item.items || []).map((x: any) => ({
        feeHeadId: x.feeHeadId?.toString() || "",
        feeHeadName: x.feeHeadName || "",
        category: x.category || "",
        amount: x.amount ?? 0
      })),
      totalAmount: item.totalAmount ?? 0,
      status: item.status || "Active"
    }));
  }
  return [];
};

export const createDynamicFeeStructureApi = async (data: Omit<DynamicFeeStructure, 'id'>) => {
  const payload = {
    name: `${data.className} Structure`,
    description: `Dynamic fee structure for ${data.className}`,
    targetAudience: data.studentCategory || "All",
    academicYear: data.academicYear || "2026-2027",
    branch: data.branch || "Main Campus",
    className: data.className || "",
    section: data.section || "",
    studentCategory: data.studentCategory || "General",
    totalAmount: Number(data.totalAmount) || 0,
    status: data.status || "Active",
    items: (data.items || []).map((x: any) => ({
      feeHeadId: (x.feeHeadId || "").toString(),
      feeHeadName: x.feeHeadName || "",
      category: x.category || "",
      amount: Number(x.amount) || 0
    })))
  };
  return apiClient('/api/finance/fee-structures', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateDynamicFeeStructureApi = async (id: string, data: Partial<DynamicFeeStructure>) => {
  const cleanId = id.toString().replace(/\D/g, '') || id;
  const payload = {
    ...data,
    id: parseInt(cleanId, 10) || 0,
  };

```

```

    totalAmount: Number(data.totalAmount) || 0,
    items: (data.items || []).map((x: any) => ({
      feeHeadId: (x.feeHeadId || "").toString(),
      feeHeadName: x.feeHeadName || "",
      category: x.category || "",
      amount: Number(x.amount) || 0
    })))
  });
  return apiClient(`/api/finance/fee-structures/${cleanId}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const deleteDynamicFeeStructureApi = async (id: string) => {
  return apiClient(`/api/finance/fee-structures/${id}`, { method: 'DELETE' });
};

export const fetchStudentFeeAssignmentsApi = async () => {
  const response: any = await apiClient('/api/finance/fee-assignments', { method: 'GET' });
  const items = Array.isArray(response)
    ? response
    : response?.data?.items || response?.data || [];
  return items;
};

export const createStudentFeeAssignmentApi = async (data: Omit<StudentFeeAssignment, 'id'>) => {
  const payload = {
    studentId: data.studentId || "",
    dynamicFeeStructureId: parseInt(data.feeStructureId) || 0,
    totalAmount: data.baseFeeTotal ?? data.originalFeeTotal ?? 0,
    paidAmount: 0,
    dueAmount: data.baseFeeTotal ?? data.originalFeeTotal ?? 0,
    status: data.status || "Active",
    feePolicy: data.feePolicy || "Full Annual Fee"
  };
  return apiClient('/api/finance/fee-assignments', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const bulkAssignStudentFeesApi = async (payload: {
  studentIds: string[];
  dynamicFeeStructureId?: number | string;
  className?: string;
  feePolicy?: string;
  totalAmount?: number;
}) => {
  return apiClient('/api/finance/fee-assignments/bulk', {
    method: 'POST',
    body: JSON.stringify({
      ...payload,
      dynamicFeeStructureId: payload.dynamicFeeStructureId ? parseInt(String(payload.dynamicFeeStructureI
d)) : undefined
    })
  });
};

export const saveCustomStudentFeeAssignmentApi = async (payload: {
  studentId: string;
  dynamicFeeStructureId?: number | string;
  feePolicy: string;

```

```

    admissionDate?: string;
    adjustmentReason?: string;
    totalAmount: number;
    breakdown?: any[];
  }) => {
    return apiClient('/api/finance/fee-assignments/custom', {
      method: 'POST',
      body: JSON.stringify({
        ...payload,
        dynamicFeeStructureId: payload.dynamicFeeStructureId ? parseInt(String(payload.dynamicFeeStructureId)) : undefined
      })
    });
  });
};

export const updateStudentFeeAssignmentApi = async (id: string, data: Partial<StudentFeeAssignment>) => {
  return apiClient(`/api/finance/fee-assignments/${id}`, {
    method: 'PUT',
    body: JSON.stringify(data)
  });
};

export const deleteStudentFeeAssignmentApi = async (id: string) => {
  return apiClient(`/api/finance/fee-assignments/${id}`, { method: 'DELETE' });
};

export const fetchFeePaymentsApi = async () => {
  const response: any = await apiClient('/api/finance/fee-payments', { method: 'GET' });
  const items = Array.isArray(response)
    ? response
    : response?.data?.items || response?.data || [];
  if (Array.isArray(items)) {
    return items.map((item: any) => ({
      id: item.id?.toString() || "",
      receiptNo: item.receiptNo || "",
      studentId: item.studentId || "",
      studentName: item.studentName || "",
      className: item.className || "",
      amountPaid: Number(item.amount ?? item.amountPaid ?? 0),
      discount: Number(item.discountAmount ?? item.discount ?? 0),
      fine: Number(item.fineAmount ?? item.fine ?? 0),
      transportFee: Number(item.transportFee ?? 0),
      paymentMode: item.paymentMethod || item.paymentMode || "Cash",
      transactionId: item.transactionId || "",
      paymentDate: item.paymentDate || "",
      academicYear: item.academicYear || "2026-2027",
      status: item.status || "Completed",
      remarks: item.remarks || ""
    }));
  }
  return [];
};

export const createFeePaymentApi = async (data: Omit<FeePayment, 'id' | 'receiptNo'>) => {
  const payload = {
    receiptNo: (data as any).receiptNo || "",
    studentId: data.studentId || "",
    amount: data.amountPaid ?? (data as any).amount ?? 0,
    discountAmount: data.discount ?? (data as any).discountAmount ?? 0,
    fineAmount: data.fine ?? (data as any).fineAmount ?? 0,
    transportFee: data.transportFee ?? 0,
    transactionId: data.transactionId || "",
    paymentDate: data.paymentDate || new Date().toISOString(),
  };

```

```

    paymentMethod: data.paymentMode || (data as any).paymentMethod || "Cash",
    status: data.status || "Completed"
  };
  return apiClient('/api/finance/fee-payments', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

// =====
// NEW FINANCE MODULE ENDPOINTS (Fee Collection, Due Fees, Receipts, Dashboard)
// =====

export const fetchFeeCollectionStudentsApi = async (params?: {
  search?: string;
  className?: string;
  sectionName?: string;
  studentType?: string;
  page?: number;
  pageSize?: number;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.className) query.append('className', params.className);
  if (params?.sectionName) query.append('sectionName', params.sectionName);
  if (params?.studentType) query.append('studentType', params.studentType);
  if (params?.page) query.append('page', params.page.toString());
  if (params?.pageSize) query.append('pageSize', params.pageSize.toString());

  const qs = query.toString();
  return apiClient(`/api/finance/fee-collection/students${qs ? `?${qs}` : ''}`, { method: 'GET' });
};

export const fetchStudentFeeProfileApi = async (studentId: number | string, academicYear = "2026-2027") => {
  return apiClient(`/api/finance/fee-collection/student/${studentId}?academicYear=${encodeURIComponent(academicYear)}`, { method: 'GET' });
};

export const collectFeePaymentApi = async (payload: {
  studentId: number;
  admissionNo: string;
  academicYear?: string;
  totalAmountPaid: number;
  concessionDiscountAmount?: number;
  fineAmount?: number;
  isFineWaived?: boolean;
  paymentMethod: string;
  transactionId?: string;
  chequeNo?: string;
  chequeDate?: string;
  bankName?: string;
  remarks?: string;
  selectedItems?: Array<{
    feeHeadId: string;
    headName: string;
    termId: string;
    termName: string;
    amount: number;
  }>;
}) => {
  return apiClient('/api/finance/fee-collection/collect', {
    method: 'POST',

```

```

    body: JSON.stringify(payload)
  });
};

export const fetchDueFeesApi = async (params?: {
  className?: string;
  sectionName?: string;
  minDaysOverdue?: number;
}) => {
  const query = new URLSearchParams();
  if (params?.className) query.append('className', params.className);
  if (params?.sectionName) query.append('sectionName', params.sectionName);
  if (params?.minDaysOverdue !== undefined) query.append('minDaysOverdue', params.minDaysOverdue.toString());

  const qs = query.toString();
  return apiClient(`/api/finance/due-fees${qs ? `?` + qs : ''}`, { method: 'GET' });
};

export const sendFeeReminderApi = async (payload: {
  studentId: number;
  reminderType?: string;
  customMessage?: string;
}) => {
  return apiClient(`/api/finance/due-fees/send-reminder`, {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const fetchPromotedDuesApi = async (params?: {
  search?: string;
  className?: string;
  previousAcademicYear?: string;
  status?: string;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.className) query.append('className', params.className);
  if (params?.previousAcademicYear) query.append('previousAcademicYear', params.previousAcademicYear);
  if (params?.status) query.append('status', params.status);

  const qs = query.toString();
  return apiClient(`/api/finance/promoted-students-dues${qs ? `?` + qs : ''}`, { method: 'GET' });
};

export const fetchFeeReceiptsRegisterApi = async (params?: {
  search?: string;
  paymentMode?: string;
  fromDate?: string;
  toDate?: string;
  page?: number;
  pageSize?: number;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.paymentMode) query.append('paymentMode', params.paymentMode);
  if (params?.fromDate) query.append('fromDate', params.fromDate);
  if (params?.toDate) query.append('toDate', params.toDate);
  if (params?.page) query.append('page', params.page.toString());
  if (params?.pageSize) query.append('pageSize', params.pageSize.toString());

  const qs = query.toString();

```

```

    return apiClient(`/api/finance/receipts${qs ? `?` + `${qs}` : ''}`, { method: 'GET' });
  };

export const fetchReceiptByNoApi = async (receiptNo: string) => {
  return apiClient(`/api/finance/receipts/${encodeURIComponent(receiptNo)}`, { method: 'GET' });
};

export const cancelReceiptApi = async (receiptNo: string, reason: string) => {
  return apiClient(`/api/finance/receipts/${encodeURIComponent(receiptNo)}/cancel`, {
    method: 'POST',
    body: JSON.stringify(reason)
  });
};

export const fetchFinanceDashboardStatsApi = async () => {
  return apiClient(`/api/finance/dashboard`, { method: 'GET' });
};

// =====
// FEE HEADS / FEE TYPES CRUD APIS
// =====

export const fetchFeeHeadsApi = async (params?: {
  search?: string;
  category?: string;
  frequency?: string;
  status?: string;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.category) query.append('category', params.category);
  if (params?.frequency) query.append('frequency', params.frequency);
  if (params?.status) query.append('status', params.status);

  const qs = query.toString();
  return apiClient(`/api/finance/fee-heads${qs ? `?` + `${qs}` : ''}`, { method: 'GET' });
};

export const createFeeHeadApi = async (payload: any) => {
  return apiClient(`/api/finance/fee-heads`, {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateFeeHeadApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/fee-heads/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const toggleFeeHeadStatusApi = async (id: number | string) => {
  return apiClient(`/api/finance/fee-heads/${id}/toggle-status`, {
    method: 'PATCH'
  });
};

export const deleteFeeHeadApi = async (id: number | string) => {
  return apiClient(`/api/finance/fee-heads/${id}`, {
    method: 'DELETE'
  });
};

```

```
// =====
// GENERAL LEDGER, TRANSACTIONS, ACCOUNTS, REFUNDS & REPORTS APIS
// =====

export const fetchFinanceTransactionsApi = async (params?: {
  search?: string;
  type?: string;
  module?: string;
  category?: string;
  paymentMode?: string;
  status?: string;
  page?: number;
  pageSize?: number;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.type) query.append('type', params.type);
  if (params?.module) query.append('module', params.module);
  if (params?.category) query.append('category', params.category);
  if (params?.paymentMode) query.append('paymentMode', params.paymentMode);
  if (params?.status) query.append('status', params.status);
  if (params?.page) query.append('page', params.page.toString());
  if (params?.pageSize) query.append('pageSize', params.pageSize.toString());

  const qs = query.toString();
  return apiClient(`/api/finance/transactions${qs ? `?${qs}` : ''}`, { method: 'GET' });
};

export const fetchFinanceTransactionsSummaryApi = async () => {
  return apiClient(`/api/finance/transactions/summary`, { method: 'GET' });
};

export const createFinanceTransactionApi = async (payload: any) => {
  return apiClient(`/api/finance/transactions`, {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const reverseFinanceTransactionApi = async (id: number | string, reason: string) => {
  return apiClient(`/api/finance/transactions/${id}/reverse`, {
    method: 'POST',
    body: JSON.stringify({ reversalReason: reason, authorizedBy: 'Admin' })
  });
};

export const fetchFinancialAccountsApi = async () => {
  return apiClient(`/api/finance/accounts`, { method: 'GET' });
};

export const createFinancialAccountApi = async (payload: any) => {
  return apiClient(`/api/finance/accounts`, {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateFinancialAccountApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/accounts/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};
```

```
};

export const deleteFinancialAccountApi = async (id: number | string) => {
  return apiClient(`/api/finance/accounts/${id}`, {
    method: 'DELETE'
  });
};

export const fetchFinancialCategoriesApi = async (type?: string) => {
  const qs = type ? `?type=${encodeURIComponent(type)}` : '';
  return apiClient(`/api/finance/categories${qs}`, { method: 'GET' });
};

export const createFinancialCategoryApi = async (payload: any) => {
  return apiClient('/api/finance/categories', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateFinancialCategoryApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/categories/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const deleteFinancialCategoryApi = async (id: number | string) => {
  return apiClient(`/api/finance/categories/${id}`, {
    method: 'DELETE'
  });
};

export const fetchFinancialBudgetsApi = async (academicYear = '2025-2026') => {
  return apiClient(`/api/finance/budgets?academicYear=${encodeURIComponent(academicYear)}`, { method: 'GET' });
};

export const saveFinancialBudgetApi = async (payload: any) => {
  return apiClient('/api/finance/budgets', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateFinancialBudgetApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/budgets/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const fetchFeeRefundRequestsApi = async (status?: string) => {
  const qs = status ? `?status=${encodeURIComponent(status)}` : '';
  return apiClient(`/api/finance/refunds${qs}`, { method: 'GET' });
};

export const createFeeRefundRequestApi = async (payload: any) => {
  return apiClient('/api/finance/refunds/request', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};
```

```

export const processFeeRefundRequestApi = async (id: number | string, payload: { status: string; remarks?: string }) => {
  return apiClient(`/api/finance/refunds/${id}/process`, {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const fetchFeeScheduleConfigApi = async (academicYear = '2026-2027') => {
  return apiClient(`/api/finance/fee-schedules?academicYear=${encodeURIComponent(academicYear)}`, { method: 'GET' });
};

export const saveFeeScheduleConfigApi = async (payload: any) => {
  return apiClient(`/api/finance/fee-schedules`, {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const fetchFinanceGlobalSettingsApi = async () => {
  return apiClient(`/api/finance/settings`, { method: 'GET' });
};

export const updateFinanceGlobalSettingsApi = async (payload: any) => {
  return apiClient(`/api/finance/settings`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const fetchFinanceReportsSummaryApi = async (academicYear = '2025-2026') => {
  return apiClient(`/api/finance/reports/summary?academicYear=${encodeURIComponent(academicYear)}`, { method: 'GET' });
};

export const fetchDailyCollectionReportApi = async (date?: string) => {
  const qs = date ? `?date=${encodeURIComponent(date)}` : '';
  return apiClient(`/api/finance/reports/daily-collection${qs}`, { method: 'GET' });
};

export const fetchClassWiseCollectionReportApi = async (academicYear = '2025-2026') => {
  return apiClient(`/api/finance/reports/class-wise-collection?academicYear=${encodeURIComponent(academicYear)}`, { method: 'GET' });
};

// =====
// SCHOLARSHIPS & STUDENT AWARDS APIS
// =====

export const fetchScholarshipsApi = async (params?: {
  search?: string;
  type?: string;
  status?: string;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.type) query.append('type', params.type);
  if (params?.status) query.append('status', params.status);

  const qs = query.toString();
  return apiClient(`/api/finance/scholarships${qs ? `?${qs}` : ''}`, { method: 'GET' });
};

```

```

});

export const fetchScholarshipByIdApi = async (id: number | string) => {
  return apiClient(`/api/finance/scholarships/${id}`, { method: 'GET' });
};

export const createScholarshipApi = async (payload: any) => {
  return apiClient('/api/finance/scholarships', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateScholarshipApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/scholarships/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const deleteScholarshipApi = async (id: number | string) => {
  return apiClient(`/api/finance/scholarships/${id}`, {
    method: 'DELETE'
  });
};

export const fetchStudentScholarshipsApi = async (params?: {
  search?: string;
  className?: string;
  scholarshipId?: number | string;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.className) query.append('className', params.className);
  if (params?.scholarshipId) query.append('scholarshipId', params.scholarshipId.toString());

  const qs = query.toString();
  return apiClient(`/api/finance/student-scholarships${qs ? `?` + qs : ''}`, { method: 'GET' });
};

export const awardStudentScholarshipApi = async (payload: { studentId: string; scholarshipId: number; remarks?: string }) => {
  return apiClient('/api/finance/student-scholarships', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const revokeStudentScholarshipApi = async (id: number | string) => {
  return apiClient(`/api/finance/student-scholarships/${id}`, {
    method: 'DELETE'
  });
};

// =====
// DISCOUNTS & STUDENT CONCESSIONS APIS
// =====

export const fetchDiscountsApi = async (params?: {
  search?: string;
  type?: string;
  mode?: string;
  status?: string;

```

```

  }) => {
    const query = new URLSearchParams();
    if (params?.search) query.append('search', params.search);
    if (params?.type) query.append('type', params.type);
    if (params?.mode) query.append('mode', params.mode);
    if (params?.status) query.append('status', params.status);

    const qs = query.toString();
    return apiClient(`/api/finance/discounts${qs ? `?` + qs : ''}`, { method: 'GET' });
  };

export const fetchDiscountByIdApi = async (id: number | string) => {
  return apiClient(`/api/finance/discounts/${id}`, { method: 'GET' });
};

export const createDiscountApi = async (payload: any) => {
  return apiClient('/api/finance/discounts', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateDiscountApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/discounts/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const deleteDiscountApi = async (id: number | string) => {
  return apiClient(`/api/finance/discounts/${id}`, {
    method: 'DELETE'
  });
};

export const fetchStudentDiscountsApi = async (params?: {
  search?: string;
  className?: string;
  discountId?: number | string;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.className) query.append('className', params.className);
  if (params?.discountId) query.append('discountId', params.discountId.toString());

  const qs = query.toString();
  return apiClient(`/api/finance/student-discounts${qs ? `?` + qs : ''}`, { method: 'GET' });
};

export const grantStudentDiscountApi = async (payload: { studentId: string; discountId: number; remarks?: string }) => {
  return apiClient('/api/finance/student-discounts', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const removeStudentDiscountApi = async (id: number | string) => {
  return apiClient(`/api/finance/student-discounts/${id}`, {
    method: 'DELETE'
  });
};

```

```
// =====  
// LATE FINE RULES APIS  
// =====  
  
export const fetchFineRulesApi = async (params?: {  
  search?: string;  
  status?: string;  
}) => {  
  const query = new URLSearchParams();  
  if (params?.search) query.append('search', params.search);  
  if (params?.status) query.append('status', params.status);  
  
  const qs = query.toString();  
  return apiClient(`/api/finance/fine-rules${qs ? `?` + qs : ''}`, { method: 'GET' });  
};  
  
export const fetchFineRuleByIdApi = async (id: number | string) => {  
  return apiClient(`/api/finance/fine-rules/${id}`, { method: 'GET' });  
};  
  
export const createFineRuleApi = async (payload: any) => {  
  return apiClient(`/api/finance/fine-rules`, {  
    method: 'POST',  
    body: JSON.stringify(payload)  
  });  
};  
  
export const updateFineRuleApi = async (id: number | string, payload: any) => {  
  return apiClient(`/api/finance/fine-rules/${id}`, {  
    method: 'PUT',  
    body: JSON.stringify(payload)  
  });  
};  
  
export const deleteFineRuleApi = async (id: number | string) => {  
  return apiClient(`/api/finance/fine-rules/${id}`, {  
    method: 'DELETE'  
  });  
};  
  
// =====  
// HOSTEL FEE CONFIGURATIONS APIS  
// =====  
  
export const fetchHostelFeeConfigsApi = async (params?: {  
  search?: string;  
  hostelId?: string;  
  status?: string;  
}) => {  
  const query = new URLSearchParams();  
  if (params?.search) query.append('search', params.search);  
  if (params?.hostelId) query.append('hostelId', params.hostelId);  
  if (params?.status) query.append('status', params.status);  
  
  const qs = query.toString();  
  return apiClient(`/api/finance/hostel-fees${qs ? `?` + qs : ''}`, { method: 'GET' });  
};  
  
export const fetchHostelFeeConfigByIdApi = async (id: number | string) => {  
  return apiClient(`/api/finance/hostel-fees/${id}`, { method: 'GET' });  
};  
  
export const createHostelFeeConfigApi = async (payload: any) => {
```

```
return apiClient('/api/finance/hostel-fees', {
  method: 'POST',
  body: JSON.stringify(payload)
});
};

export const updateHostelFeeConfigApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/hostel-fees/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const deleteHostelFeeConfigApi = async (id: number | string) => {
  return apiClient(`/api/finance/hostel-fees/${id}`, {
    method: 'DELETE'
  });
};

// =====
// UNIFORM FEE CONFIGURATIONS APIS
// =====

export const fetchUniformFeeConfigsApi = async (params?: {
  search?: string;
  className?: string;
  academicYear?: string;
  status?: string;
}) => {
  const query = new URLSearchParams();
  if (params?.search) query.append('search', params.search);
  if (params?.className) query.append('className', params.className);
  if (params?.academicYear) query.append('academicYear', params.academicYear);
  if (params?.status) query.append('status', params.status);

  const qs = query.toString();
  return apiClient(`/api/finance/uniform-fees${qs ? `?` + qs : ''}`, { method: 'GET' });
};

export const fetchUniformFeeConfigByIdApi = async (id: number | string) => {
  return apiClient(`/api/finance/uniform-fees/${id}`, { method: 'GET' });
};

export const createUniformFeeConfigApi = async (payload: any) => {
  return apiClient('/api/finance/uniform-fees', {
    method: 'POST',
    body: JSON.stringify(payload)
  });
};

export const updateUniformFeeConfigApi = async (id: number | string, payload: any) => {
  return apiClient(`/api/finance/uniform-fees/${id}`, {
    method: 'PUT',
    body: JSON.stringify(payload)
  });
};

export const deleteUniformFeeConfigApi = async (id: number | string) => {
  return apiClient(`/api/finance/uniform-fees/${id}`, {
    method: 'DELETE'
  });
};
```



 Finance/FeeHeadsView.tsx

TSX

```
import React, { useState, useMemo } from 'react';
import { Tag, Plus, Search, Edit, Trash2, CheckCircle2, XCircle, ArrowUpDown } from 'lucide-react';
import { FeeHead, FeeHeadCategory, FeeHeadFrequency } from '../types';
import { useData } from '../context/DataContext';
import { useToast } from '../context/ToastContext';
import { Badge } from '../common/Badge';
import { ExportButton } from '../common/ExportButton';
import { ConfirmModal } from '../common/ConfirmModal';
import { compareClassesAscending } from '../utils/classSorter';

const CATEGORIES: FeeHeadCategory[] = [
  'Tuition', 'Admission', 'Books', 'Uniform', 'Lab', 'Computer',
  'Library', 'Sports', 'Activity', 'Exam', 'Transport', 'Hostel', 'Miscellaneous'
];

const FREQUENCIES: FeeHeadFrequency[] = [
  'One Time', 'Monthly', 'Quarterly', 'Half Yearly', 'Annual', 'Custom'
];

const DEFAULT_CLASSES = [
  'Playgroup', 'Nursery', 'LKG', 'UKG',
  'Class 1', 'Class 2', 'Class 3', 'Class 4', 'Class 5',
  'Class 6', 'Class 7', 'Class 8', 'Class 9', 'Class 10', 'Class 11', 'Class 12'
];

export const FeeHeadsView: React.FC = () => {
  const { feeHeads, addFeeHead, updateFeeHead, deleteFeeHead, toggleFeeHeadStatus, academicClasses, branches } = useData();
  const { addToast } = useToast();

  const classOptions = useMemo(() => {
    if (academicClasses && academicClasses.length > 0) {
      const list = academicClasses.map(c => c.name || (c as any).className).filter(Boolean);
      return Array.from(new Set(list)).sort(compareClassesAscending);
    }
    return DEFAULT_CLASSES;
  }, [academicClasses]);

  const branchOptions = useMemo(() => {
    if (branches && branches.length > 0) {
      const names = branches.map((b: any) => typeof b === 'string' ? b : (b.name || b.branchName || b.branch)).filter(Boolean);
      const unique = Array.from(new Set(names));
      if (!unique.includes('All Branches')) unique.unshift('All Branches');
      return unique;
    }
    return ['All Branches', 'Main Campus', 'Madhapur Branch'];
  }, [branches]);

  const formatClassDisplayName = (cls: string) => {
    if (!cls) return '';
    return cls.replace(/^(Class\s+)/i, '').trim();
  };

  const formatClassListForDisplay = (classes?: string[]) => {
    if (!classes || classes.length === 0) return 'None';
    return classes.map(formatClassDisplayName).join(', ');
  };
};
```

```

const [query, setQuery] = useState('');
const [selectedCategory, setSelectedCategory] = useState<string>('All');
const [selectedFrequency, setSelectedFrequency] = useState<string>('All');
const [selectedStatus, setSelectedStatus] = useState<string>('All');

const [isModalOpen, setIsModalOpen] = useState(false);
const [editingHead, setEditingHead] = useState<FeeHead | null>(null);
const [deletingHead, setDeletingHead] = useState<FeeHead | null>(null);

const [formData, setFormData] = useState<Partial<FeeHead>>>({
  name: '',
  code: '',
  category: 'Tuition',
  frequency: 'Quarterly',
  mandatory: true,
  applicableClasses: classOptions,
  applicableBranches: ['All Branches'],
  taxPercentage: 0,
  displayOrder: 1,
  status: 'Active'
});

const filteredHeads = feeHeads.filter(h => {
  const matchesQuery = h.name.toLowerCase().includes(query.toLowerCase()) || h.code.toLowerCase().includes(query.toLowerCase());
  const matchesCategory = selectedCategory === 'All' || h.category === selectedCategory;
  const matchesFrequency = selectedFrequency === 'All' || h.frequency === selectedFrequency;
  const matchesStatus = selectedStatus === 'All' || h.status === selectedStatus;
  return matchesQuery && matchesCategory && matchesFrequency && matchesStatus;
}).sort((a, b) => {
  // Mandatory fee types always display at top
  if (a.mandatory && !b.mandatory) return -1;
  if (!a.mandatory && b.mandatory) return 1;
  return (a.displayOrder || 0) - (b.displayOrder || 0);
});

const handleOpenAdd = () => {
  setEditingHead(null);
  setFormData({
    name: '',
    code: 'FH-' + Math.floor(100 + Math.random() * 900),
    category: 'Tuition',
    frequency: 'Quarterly',
    mandatory: true,
    applicableClasses: classOptions,
    applicableBranches: ['All Branches'],
    taxPercentage: 0,
    displayOrder: feeHeads.length + 1,
    status: 'Active'
  });
  setIsModalOpen(true);
};

const handleOpenEdit = (h: FeeHead) => {
  setEditingHead(h);
  setFormData({
    ...h,
    mandatory: h.mandatory === true,
    applicableClasses: h.applicableClasses && h.applicableClasses.length > 0 ? [...h.applicableClasses] : [...classOptions],
    applicableBranches: h.applicableBranches && h.applicableBranches.length > 0 ? [...h.applicableBranches] : ['All Branches'],
  });
};

```

```

    setIsModalOpen(true);
  };

  const handleSubmit = async (e: React.SyntheticEvent) => {
    e.preventDefault();
    try {
      if (!formData.name || !formData.name.trim()) {
        throw new Error('Fee type name is required.');
```

```

      }
      if (!formData.code || !formData.code.trim()) {
        throw new Error('Fee type code is required.');
```

```

      }
      if (!formData.applicableClasses || formData.applicableClasses.length === 0) {
        throw new Error('Please select at least one applicable class for this fee type.');
```

```

      }

      const cleanName = formData.name.trim().toLowerCase();
      const cleanCode = formData.code.trim().toLowerCase();
```

```

      // Duplicate validation check using Exception Handling
```

```

      const existingDuplicate = feeHeads.find(h => {
        if (editingHead && String(h.id) === String(editingHead.id)) return false;
        const sameCode = h.code && h.code.trim().toLowerCase() === cleanCode;
        const sameNameCat = h.name && h.name.trim().toLowerCase() === cleanName && h.category === formData.
category;
        return sameCode || sameNameCat;
      });
```

```

      if (existingDuplicate) {
        if (existingDuplicate.code && existingDuplicate.code.trim().toLowerCase() === cleanCode) {
          throw new Error(`Duplicate Validation Error: Fee type code "${formData.code}" already exists.`);
        } else {
          throw new Error(`Duplicate Validation Error: Fee type "${formData.name}" in category "${formData.
category}" already exists.`);
        }
      }
    }
  }

  if (editingHead) {
    await updateFeeHead(editingHead.id, {
      ...formData,
      mandatory: formData.mandatory === true,
    });
    addToast('success', 'Fee Head Updated', `Updated ${formData.name}`);
  } else {
    await addFeeHead({
      ...formData,
      mandatory: formData.mandatory === true,
    } as Omit<FeeHead, 'id'>);
    addToast('success', 'Fee Head Created', `Created ${formData.name}`);
  }
  setIsModalOpen(false);
} catch (err: any) {
  addToast('error', 'Validation Error', err.message || 'Validation failed for Fee Type.');
```

```

}
};
```

```

return (
  <div className="space-y-6 animate-in fade-in">
    {/* Header Controls */}
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
      <div>
        <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center ga
p-2">
```

```

        <Tag className="w-6 h-6 text-sky-500" /> Fee Types
      </h2>
    </div>

    <div className="flex items-center gap-3">
      <button
        onClick={handleOpenAdd}
        className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-500 text-white text-xs font-bold shadow
        -lg shadow-sky-500/20 flex items-center gap-2 transition-all"
        >
        <Plus className="w-4 h-4" /> Add Fee Category
      </button>
      <ExportButton data={filteredHeads} filename="fee_heads" />
    </div>
  </div>

  {/ * Search & Filter Bar */}
  <div className="glass-card p-4 rounded-2xl grid grid-cols-1 sm:grid-cols-4 gap-3">
    <div className="relative">
      <Search className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
      <input
        type="text"
        placeholder="Search name or code..."
        value={query}
        onChange={e => setQuery(e.target.value)}
        className="w-full pl-9 pr-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-s
        late-900 dark:text-white outline-none"
        />
    </div>

    <select
      value={selectedCategory}
      onChange={e => setSelectedCategory(e.target.value)}
      className="px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-slate-900 dark:
      text-white outline-none"
    >
      <option value="All">All Categories</option>
      {CATEGORIES.map(c => <option key={c} value={c}>{c}</option>)}
    </select>

    <select
      value={selectedFrequency}
      onChange={e => setSelectedFrequency(e.target.value)}
      className="px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-slate-900 dark:
      text-white outline-none"
    >
      <option value="All">All Frequencies</option>
      {FREQUENCIES.map(f => <option key={f} value={f}>{f}</option>)}
    </select>

    <select
      value={selectedStatus}
      onChange={e => setSelectedStatus(e.target.value)}
      className="px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-slate-900 dark:
      text-white outline-none"
    >
      <option value="All">All Statuses</option>
      <option value="Active">Active Only</option>
      <option value="Inactive">Inactive Only</option>
    </select>
  </div>

  {/ * Table */}

```

```

<div className="glass-card rounded-2xl overflow-hidden border border-slate-200/80 dark:border-slate-800">
  <div className="overflow-x-auto">
    <table className="w-full text-left border-collapse text-xs">
      <thead>
        <tr className="bg-slate-100/70 dark:bg-slate-800/60 text-slate-500 font-bold uppercase tracking-wider border-b border-slate-200 dark:border-slate-800">
          <th className="py-3.5 px-4">Order</th>
          <th className="py-3.5 px-4">Fee Type</th>
          <th className="py-3.5 px-4">Code</th>
          <th className="py-3.5 px-4">Category</th>
          <th className="py-3.5 px-4">Frequency</th>
          <th className="py-3.5 px-4">Mandatory</th>
          <th className="py-3.5 px-4">Classes</th>
          <th className="py-3.5 px-4">Status</th>
          <th className="py-3.5 px-4 text-right">Actions</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-100 dark:divide-slate-800/80 font-medium">
        {filteredHeads.length === 0 ? (
          <tr>
            <td colspan={9} className="py-12 text-center text-slate-400">
              <Tag className="w-10 h-10 mx-auto text-slate-300 dark:text-slate-600 mb-2 opacity-50" />
              <p className="font-bold text-slate-700 dark:text-slate-300">No fee types configured</p>
              <p className="text-xs text-slate-400 mt-1">Click "+ Add Fee Type" above to create fee heads dynamically.</p>
            </td>
          </tr>
        ) : (
          filteredHeads.map((h, idx) => (
            <tr key={h.id} className="hover:bg-slate-50/80 dark:hover:bg-slate-800/40">
              <td className="py-3 px-4 text-slate-400 font-mono">{idx + 1}</td>
              <td className="py-3 px-4 font-bold text-slate-900 dark:text-white">{h.name}</td>
              <td className="py-3 px-4 font-mono text-slate-600 dark:text-slate-300">{h.code}</td>
              <td className="py-3 px-4">
                <span className="px-2 py-0.5 rounded-lg bg-sky-50 dark:bg-sky-950 text-sky-600 dark:text-sky-400 font-bold">
                  {h.category}
                </span>
              </td>
              <td className="py-3 px-4 font-semibold text-slate-700 dark:text-slate-300">{h.frequency}</td>
              <td className="py-3 px-4">
                {h.mandatory ? (
                  <span className="text-rose-500 font-bold flex items-center gap-1"><CheckCircle2 className="w-3.5 h-3.5" /> Mandatory</span>
                ) : (
                  <span className="text-slate-400 font-medium">Optional</span>
                )}
              </td>
              <td className="py-3 px-4 text-slate-500 font-medium">{formatClassListForDisplay(h.applicableClasses)}</td>
              <td className="py-3 px-4">
                <button
                  onClick={() => toggleFeeHeadStatus(h.id)}
                  className={`px-2.5 py-1 rounded-full text-[10px] font-extrabold transition-all ${h.status === 'Active' ? 'bg-emerald-100 text-emerald-700 dark:bg-emerald-950 dark:text-emerald-300' : 'bg-slate-100 text-slate-500 dark:bg-slate-800'} `}
                >
                  {h.status}
                </button>
              </td>
            </tr>
          )
        )}
      </tbody>
    </table>
  </div>
</div>

```

```

        </td>
        <td className="py-3 px-4 text-right">
            <div className="flex items-center justify-end gap-1">
                <button onClick={() => handleOpenEdit(h)} className="p-1.5 rounded-lg hover:bg-slate-100 dark:hover:bg-slate-800 text-sky-600">
                    <Edit className="w-3.5 h-3.5" />
                </button>
                <button onClick={() => setDeletingHead(h)} className="p-1.5 rounded-lg hover:bg-rose-50 dark:hover:bg-rose-950 text-rose-500">
                    <Trash2 className="w-3.5 h-3.5" />
                </button>
            </div>
        </td>
    </tr>
  </tbody>
</table>
</div>
</div>

{ /* Modal */}
{isModalOpen && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/60 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-3xl max-w-md w-full p-6 shadow-2xl space-y-4">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-3">
        <h3 className="text-base font-bold text-slate-900 dark:text-white">
          {editingHead ? 'Edit Fee Type' : 'Add Fee Type'}
        </h3>
        <button onClick={() => setIsModalOpen(false)} className="text-slate-400 hover:text-slate-600">X</button>
      </div>

      <form onSubmit={handleSubmit} className="space-y-3 text-xs">
        <div className="grid grid-cols-2 gap-3">
          <div>
            <label className="block font-semibold mb-1">Fee Type Name <span className="text-rose-500 font-bold ml-0.5">*</span></label>
            <input
              type="text"
              required
              value={formData.name}
              onChange={e => setFormData({ ...formData, name: e.target.value })}
              placeholder="e.g. Tuition Fee"
              className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
            />
          </div>
          <div>
            <label className="block font-semibold mb-1">Fee Code <span className="text-rose-500 font-bold ml-0.5">*</span></label>
            <input
              type="text"
              required
              value={formData.code}
              onChange={e => setFormData({ ...formData, code: e.target.value })}
              placeholder="e.g. TUIT-101"
              className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-mono"
            />
          </div>
        </div>
      </form>
    </div>
  </div>
)
}

```

```

<div className="grid grid-cols-2 gap-3">
  <div>
    <label className="block font-semibold mb-1">Category</label>
    <select
      value={formData.category}
      onChange={e => setFormData({ ...formData, category: e.target.value as any })}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
    >
      {CATEGORIES.map(c => <option key={c} value={c}>{c}</option>)}
    </select>
  </div>
  <div>
    <label className="block font-semibold mb-1">Frequency</label>
    <select
      value={formData.frequency}
      onChange={e => setFormData({ ...formData, frequency: e.target.value as any })}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
    >
      {FREQUENCIES.map(f => <option key={f} value={f}>{f}</option>)}
    </select>
  </div>
</div>

<div className="grid grid-cols-2 gap-3">
  <div>
    <label className="block font-semibold mb-1">Display Order</label>
    <input
      type="number"
      value={formData.displayOrder}
      onChange={e => setFormData({ ...formData, displayOrder: Number(e.target.value) })}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
    />
  </div>
  <div>
    <label className="block font-semibold mb-1">Tax (%)</label>
    <input
      type="number"
      value={formData.taxPercentage || 0}
      onChange={e => setFormData({ ...formData, taxPercentage: Number(e.target.value) })}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
    />
  </div>
</div>

/* Applicable Classes Configuration */
<div className="space-y-2 p-3 rounded-2xl bg-slate-50 dark:bg-slate-800/60 border border-slate-200/80 dark:border-slate-700">
  <div className="flex items-center justify-between">
    <label className="font-extrabold text-slate-800 dark:text-slate-200">
      Applicable Classes ({(formData.applicableClasses || []).length}/{classOptions.length})
    </label>
    <span className="text-rose-500 font-bold ml-0.5">*</span>
  </div>
  <div className="flex items-center gap-2 text-[11px]">
    <button
      type="button"
      onClick={() => setFormData({ ...formData, applicableClasses: [...classOptions] })}
      className="text-sky-600 hover:text-sky-700 dark:text-sky-400 font-bold hover:underline cursor-pointer"
    >
      Select All
    </button>
    <span className="text-slate-300">|</span>
    <button

```

```

        type="button"
        onClick={() => setFormData({ ...formData, applicableClasses: [] })}
        className="text-slate-400 hover:text-slate-600 font-bold hover:underline cursor-point
er"

        >
        Deselect All
        </button>
      </div>
    </div>

    <div className="grid grid-cols-2 sm:grid-cols-3 gap-1.5 max-h-36 overflow-y-auto p-1">
      {classOptions.map((cls) => {
        const isChecked = (formData.applicableClasses || []).includes(cls);
        return (
          <label
            key={cls}
            className={`flex items-center gap-2 p-1.5 rounded-xl border text-[11px] font-semibo
ld cursor-pointer transition-all ${
              isChecked
                ? 'bg-sky-50 dark:bg-sky-950/60 border-sky-300 dark:border-sky-800 text-sky-900
dark:text-sky-200'
                : 'bg-white dark:bg-slate-900 border-slate-200 dark:border-slate-800 text-slate
-600 dark:text-slate-400 hover:bg-slate-100'
            }`}
          >
            <input
              type="checkbox"
              checked={isChecked}
              onChange={(e) => {
                const current = formData.applicableClasses || [];
                if (e.target.checked) {
                  setFormData({ ...formData, applicableClasses: [...current, cls] });
                } else {
                  setFormData({ ...formData, applicableClasses: current.filter(c => c !== cls)
});
                }
              }}
              className="w-3.5 h-3.5 rounded text-sky-600 focus:ring-sky-500"
            />
            <span className="truncate">{formatClassDisplayName(cls)}</span>
          </label>
        );
      })}
    </div>
  </div>

  {/* Applicable Branches Configuration */}
  <div className="space-y-2 p-3 rounded-2xl bg-slate-50 dark:bg-slate-800/60 border border-slat
e-200/80 dark:border-slate-700">
    <div className="flex items-center justify-between">
      <label className="font-extrabold text-slate-800 dark:text-slate-200">
        Applicable Branches ({(formData.applicableBranches || []).includes('All Branches')} ? 'A
ll' : (formData.applicableBranches || []).length})
      </label>
    </div>

    <div className="flex flex-wrap gap-1.5 p-1">
      {branchOptions.map((br) => {
        const isAll = br === 'All Branches';
        const currentBranches = formData.applicableBranches || ['All Branches'];
        const isChecked = currentBranches.includes(br) || (isAll && currentBranches.length ===
0);

        return (

```

```

        <label
          key={br}
          className={`flex items-center gap-1.5 px-2.5 py-1 rounded-xl border text-[11px] font-semibold cursor-pointer transition-all ${
            isChecked
              ? 'bg-sky-50 dark:bg-sky-950/60 border-sky-300 dark:border-sky-800 text-sky-900 dark:text-sky-200'
              : 'bg-white dark:bg-slate-900 border-slate-200 dark:border-slate-800 text-slate-600 dark:text-slate-400 hover:bg-slate-100'
            }`}
        >
        <input
          type="checkbox"
          checked={isChecked}
          onChange={(e) => {
            if (isAll) {
              setFormData({ ...formData, applicableBranches: ['All Branches'] });
            } else {
              let next = currentBranches.filter(b => b !== 'All Branches');
              if (e.target.checked) {
                next.push(br);
              } else {
                next = next.filter(b => b !== br);
              }
              if (next.length === 0) next = ['All Branches'];
              setFormData({ ...formData, applicableBranches: next });
            }
          }}
          className="w-3.5 h-3.5 rounded text-sky-600 focus:ring-sky-500"
        />
        <span>{br}</span>
      </label>
    );
  }}}
</div>
</div>

<label className="flex items-center justify-between p-3 rounded-xl bg-slate-50 dark:bg-slate-800 cursor-pointer hover:bg-slate-100/80 dark:hover:bg-slate-700/50 transition-colors">
  <div>
    <span className="font-semibold text-slate-700 dark:text-slate-300 block">Mandatory Fee Type</span>
    <span className="text-[11px] text-slate-400">If checked, this fee is mandatory and included by default.</span>
  </div>
  <input
    type="checkbox"
    checked={formData.mandatory === true}
    onChange={e => setFormData({ ...formData, mandatory: e.target.checked })}
    className="w-4 h-4 rounded text-sky-600 cursor-pointer"
  />
</label>

<div className="flex items-center justify-end gap-3 pt-3 border-t border-slate-100 dark:border-slate-800">
  <button type="button" onClick={() => setIsModalOpen(false)} className="px-4 py-2 font-semibold bg-slate-100 dark:bg-slate-800 rounded-xl">Cancel</button>
  <button type="submit" className="px-4 py-2 font-bold bg-sky-600 text-white rounded-xl">Save</button>
</div>
</form>
</div>
</div>

```

```
    )}

    <ConfirmModal
      isOpen={!deletingHead}
      title="Delete Fee Type"
      message={`Are you sure you want to delete ${deletingHead?.name}?`}
      onConfirm={() => {
        if (deletingHead) {
          deleteFeeHead(deletingHead.id);
          addToast('success', 'Fee Head Removed');
          setDeletingHead(null);
        }
      }}
      onCancel={() => setDeletingHead(null)}
    />
  </div>
);
};
```

 Finance/FeeStructuresView.tsx

TSX

```
import React, { useState } from "react";
import { formatCurrency } from "../../utils/currency";
import {
  Layers,
  Plus,
  Search,
  Edit,
  Trash2,
  Calculator,
  CheckCircle,
} from "lucide-react";
import { DynamicFeeStructure, FeeStructureItem } from "../../types";
import { useData } from "../../context/DataContext";
import { useAuth } from "../../context/AuthContext";
import { useToast } from "../../context/ToastContext";
import { ExportButton } from "../../common/ExportButton";
import { ConfirmModal } from "../../common/ConfirmModal";
import { compareClassesAscending } from "../../utils/classSorter";
import { getUniformFeeForClass } from "../../utils/uniformUtils";

export const FeeStructuresView: React.FC = () => {
  const {
    feeHeads,
    dynamicFeeStructures,
    addDynamicFeeStructure,
    updateDynamicFeeStructure,
    deleteDynamicFeeStructure,
    academicClasses,
    financeUniformConfigs,
  } = useData();
  const { selectedBranch, selectedAcademicYear } = useAuth();
  const { addToast } = useToast();

  const [query, setQuery] = useState("");
  const [selectedClassFilter, setSelectedClassFilter] = useState<string>("All");

  const [isModalOpen, setIsModalOpen] = useState(false);
  const [editingStruct, setEditingStruct] =
    useState<DynamicFeeStructure | null>(null);
  const [deletingStruct, setDeletingStruct] =
    useState<DynamicFeeStructure | null>(null);

  const [className, setClassName] = useState("");
  const [selectedHeadIds, setSelectedHeadIds] = useState<string[]>([]);
  const [selectedHeadAmounts, setSelectedHeadAmounts] = useState<
    Record<string, string>
  >({});
  const [isLoadingFeeTypes, setIsLoadingFeeTypes] = useState(false);

  const activeFeeHeads = feeHeads.filter((h) => h.status === "Active");

  const totalCalculated = selectedHeadIds.reduce((sum, id) => {
    const val = Number(selectedHeadAmounts[id]) || 0;
    return sum + val;
  }, 0);

  // Sorted & Filtered Fee Structures (Displayed in ascending class order and filtered by academic year)
  const filteredStructures = dynamicFeeStructures
    .filter((s) => {
```

```

    const matchesYear =
      !selectedAcademicYear ||
      !s.academicYear ||
      s.academicYear === selectedAcademicYear;
    const matchesQuery = s.className
      .toLowerCase()
      .includes(query.toLowerCase());
    const matchesClass =
      selectedClassFilter === "All" || s.className === selectedClassFilter;
    return matchesYear && matchesQuery && matchesClass;
  })
  .sort((a, b) => compareClassesAscending(a.className, b.className));

// Sorted Academic Classes for top-bar Filter
const sortedClassesForFilter = [...academicClasses].sort((a, b) =>
  compareClassesAscending(a.name, b.name),
);

// Filter out classes that already have a fee structure created for the current academic year (unless editing current one) & sort in ascending order
const existingFeeStructClasses = dynamicFeeStructures
  .filter(
    (s) =>
      (!editingStruct || s.id !== editingStruct.id) &&
      (!selectedAcademicYear ||
        !s.academicYear ||
        s.academicYear === selectedAcademicYear),
  )
  .map((s) => s.className);

const availableClassesForModal = academicClasses
  .filter((c) => !existingFeeStructClasses.includes(c.name))
  .sort((a, b) => compareClassesAscending(a.name, b.name));

const handleOpenAdd = () => {
  setEditingStruct(null);
  setClassName("");
  setSelectedHeadIds([]);
  setSelectedHeadAmounts({});
  setIsLoadingFeeTypes(false);
  setIsModalOpen(true);
};

const handleOpenEdit = (s: DynamicFeeStructure) => {
  setEditingStruct(s);
  setClassName(s.className);
  const ids: string[] = [];
  const amounts: Record<string, string> = {};
  s.items.forEach((item) => {
    const existsInFeeHeads = feeHeads.some(
      (h) =>
        h.id === item.feeHeadId ||
        h.name.toLowerCase() === item.feeHeadName.toLowerCase(),
    );
    if (existsInFeeHeads && item.feeHeadName !== "Fee Head") {
      const head = feeHeads.find(
        (h) =>
          h.id === item.feeHeadId ||
          h.name.toLowerCase() === item.feeHeadName.toLowerCase(),
      );
      const realId = head ? head.id : item.feeHeadId;
      ids.push(realId);
      amounts[realId] = String(item.amount);
    }
  });
};

```

```

    }
  });
  setSelectedHeadIds(ids);
  setSelectedHeadAmounts(amounts);
  setIsLoadingFeeTypes(false);
  setIsModalOpen(true);
};

const handleClassChange = (newClass: string) => {
  setClassName(newClass);

  if (newClass) {
    setIsLoadingFeeTypes(true);

    const uniFee = getUniformFeeForClass(newClass, '', financeUniformConfigs);
    const uniHead = activeFeeHeads.find((h) => (h.name || "").toLowerCase().includes("uniform") || (h.name || "").toLowerCase().includes("package") || h.id === 'FH-04');

    const initialHeadIds: string[] = [];
    const initialAmounts: Record<string, string> = {};

    if (uniHead) {
      initialHeadIds.push(uniHead.id);
      if (uniFee && uniFee > 0) {
        initialAmounts[uniHead.id] = String(uniFee);
      }
    }

    // Auto-select mandatory fee heads when class changes
    activeFeeHeads.forEach((h) => {
      if (h.mandatory !== false && !initialHeadIds.includes(h.id)) {
        initialHeadIds.push(h.id);
      }
    });

    setSelectedHeadIds(initialHeadIds);
    setSelectedHeadAmounts(initialAmounts);

    setTimeout(() => {
      setIsLoadingFeeTypes(false);
    }, 250);
  } else {
    setSelectedHeadIds([]);
    setSelectedHeadAmounts({});
    setIsLoadingFeeTypes(false);
  }
};

const handleToggleHead = (headId: string) => {
  if (selectedHeadIds.includes(headId)) {
    // Unchecked: remove from selection and clear amount
    setSelectedHeadIds((prev) => prev.filter((id) => id !== headId));
    setSelectedHeadAmounts((prev) => {
      const copy = { ...prev };
      delete copy[headId];
      return copy;
    });
  } else {
    // Checked: enable input
    setSelectedHeadIds((prev) => [...prev, headId]);
    const headObj = activeFeeHeads.find((h) => h.id === headId);
    const isUniform = headObj && ((headObj.name || "").toLowerCase().includes("uniform") || (headObj.name || "").toLowerCase().includes("package") || headId === 'FH-04');
  }
};

```

```

    const uniFee = isUniform && className ? getUniformFeeForClass(className, '', financeUniformConfigs) :
0;

    setSelectedHeadAmounts((prev) => ({
        ...prev,
        [headId]: uniFee && uniFee > 0 ? String(uniFee) : (prev[headId] || ""),
    }));
}
};

const handleAmountChange = (headId: string, valStr: string) => {
    // Only accept numeric inputs (digits and optional single decimal point)
    if (valStr === "" || !/^d*\.\?d*$/.test(valStr)) {
        setSelectedHeadAmounts((prev) => ({
            ...prev,
            [headId]: valStr,
        }));
    }
};

const handleSubmit = (e: React.SyntheticEvent) => {
    e.preventDefault();
    if (!className) {
        addToast("warning", "Validation Error", "Please select a Class Grade.");
        return;
    }

    if (selectedHeadIds.length === 0) {
        addToast(
            "warning",
            "Validation Error",
            "Please select at least one applicable master fee type.",
        );
        return;
    }

    // Validate that every checked fee type has a valid amount > 0
    for (const headId of selectedHeadIds) {
        const head = feeHeads.find((h) => h.id === headId);
        const headName = head ? head.name : "Fee Type";
        const amtStr = (selectedHeadAmounts[headId] || "").trim();

        if (amtStr === "") {
            addToast(
                "warning",
                "Validation Error",
                `Please enter an amount for ${headName}.`,
            );
            return;
        }

        const amtNum = Number(amtStr);
        if (isNaN(amtNum) || amtNum <= 0) {
            addToast(
                "warning",
                "Validation Error",
                `Please enter a valid amount greater than 0 for ${headName}.`,
            );
            return;
        }
    }

    const itemsList: FeeStructureItem[] = selectedHeadIds
        .map((headId) => {

```

```

    const head = feeHeads.find((h) => h.id === headId);
    if (!head) return null;
    return {
      feeHeadId: headId,
      feeHeadName: head.name,
      category: head.category,
      amount: Number(selectedHeadAmounts[headId]),
    };
  });
}

.filter(Boolean) as FeeStructureItem[];

const payload: Omit<DynamicFeeStructure, "id"> = {
  academicYear: editingStruct
    ? editingStruct.academicYear
    : selectedAcademicYear || "2026-2027",
  branch: editingStruct
    ? editingStruct.branch
    : selectedBranch || "Main Campus",
  className,
  section: "A",
  studentCategory: "General",
  items: itemsList,
  totalAmount: totalCalculated,
  status: "Active",
};

if (editingStruct) {
  updateDynamicFeeStructure(editingStruct.id, payload);
  addToast(
    "success",
    "Fee Structure Updated",
    `Updated structure for ${className}`,
  );
} else {
  addDynamicFeeStructure(payload);
  addToast(
    "success",
    "Fee Structure Configured",
    `Configured structure for ${className}`,
  );
}
setIsModalOpen(false);
};

// Filter applicable fee heads for selected class grade
const applicableFeeHeads = activeFeeHeads.filter((h) => {
  if (!className) return false;
  if (!h.applicableClasses || h.applicableClasses.length === 0) return true;
  return (
    h.applicableClasses.includes(className) ||
    h.applicableClasses.includes("All")
  );
});

const displayFeeHeads =
  applicableFeeHeads.length > 0 ? applicableFeeHeads : activeFeeHeads;

return (
  <div className="space-y-6 animate-in fade-in">
    { /* Header */ }
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
      <div>
        <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center ga

```

```

p-2">
    <Layers className="w-6 h-6 text-sky-500" /> Class Fee Structures
  </h2>
</div>

<div className="flex items-center gap-3">
  <button
    onClick={handleOpenAdd}
    className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-500 text-white text-xs font-bold shadow
-1lg shadow-sky-500/20 flex items-center gap-2 transition-all cursor-pointer"
  >
    <Plus className="w-4 h-4" /> Create Fee Structure
  </button>
  <ExportButton data={filteredStructures} filename="fee_structures" />
</div>
</div>

{/* Filter Bar */}
<div className="glass-card p-4 rounded-2xl flex flex-col sm:flex-row items-center justify-between gap
-4">
  <div className="relative w-full sm:w-72">
    <Search className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
    <input
      type="text"
      placeholder="Search class or category..."
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      className="w-full pl-9 pr-4 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-s
late-900 dark:text-white outline-none"
    />
  </div>

  <select
    value={selectedClassFilter}
    onChange={(e) => setSelectedClassFilter(e.target.value)}
    className="w-full sm:w-48 px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-
slate-900 dark:text-white outline-none cursor-pointer"
  >
    <option value="All">All Class Grades</option>
    {sortedClassesForFilter.map((c) => (
      <option key={c.id} value={c.name}>
        {c.name}
      </option>
    ))}
  </select>
</div>

{/* Grid of Structures */}
{filteredStructures.length === 0 ? (
  <div className="glass-card p-12 text-center rounded-3xl border border-slate-200 dark:border-slate-8
00 space-y-4">
    <div className="w-12 h-12 rounded-2xl bg-sky-50 dark:bg-sky-950 text-sky-600 dark:text-sky-400 fl
ex items-center justify-center mx-auto">
      <Layers className="w-6 h-6" />
    </div>
    <div>
      <h3 className="font-bold text-base text-slate-900 dark:text-white">
        No Fee Structures Found
      </h3>
      <p className="text-xs text-slate-500 dark:text-slate-400 mt-1 max-w-sm mx-auto">
        No fee structures configured for the selected search filters.
        Click below to create a new class fee structure.
      </p>
    </div>
  </div>
) : (
  <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
    {filteredStructures.map((s) => (
      <div key={s.id} className="glass-card p-4 rounded-2xl flex flex-col items-center justify-center gap-2">
        <div className="flex justify-between w-full">
          <div>{s.class}</div>
          <div>{s.fee}</div>
        </div>
        <div>{s.description}</div>
        <div>{s.action}</div>
      </div>
    ))}
  </div>
)
}

```

```

    </div>
    <button
      onClick={handleOpenAdd}
      className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-500 text-white text-xs font-bold shadow
        -lg shadow-sky-500/20 inline-flex items-center gap-2 transition-all cursor-pointer"
    >
      <Plus className="w-4 h-4" /> Create Fee Structure
    </button>
  </div>
) : (
  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    {filteredStructures.map((s) => (
      <div key={s.id} className="glass-card p-5 rounded-2xl space-y-3">
        <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate
          -800 pb-2">
          <div>
            <h3 className="font-bold text-sm text-slate-900 dark:text-white">
              {s.className} Fee Structure
            </h3>
          </div>
          <div>
            <div className="flex items-center gap-1">
              <button
                onClick={() => handleOpenEdit(s)}
                className="p-1 rounded hover:bg-slate-100 text-sky-600 cursor-pointer"
                title="Edit"
              >
                <Edit className="w-3.5 h-3.5" />
              </button>
              <button
                onClick={() => setDeletingStruct(s)}
                className="p-1 rounded hover:bg-rose-50 text-rose-600 cursor-pointer"
                title="Delete"
              >
                <Trash2 className="w-3.5 h-3.5" />
              </button>
            </div>
          </div>
        </div>
      </div>
    ))}
  </div>

  <div className="space-y-1.5 text-xs">
    {s.items.map((item) => {
      const head = feeHeads.find(
        (h) =>
          h.id === item.feeHeadId ||
          h.name.toLowerCase().trim() ===
            item.feeHeadName.toLowerCase().trim(),
      );
      const isMandatory =
        head && head.mandatory !== undefined
          ? head.mandatory
          : item.feeHeadName.toLowerCase().includes("tuition") ||
            item.feeHeadName.toLowerCase().includes("admission") ||
            item.feeHeadName.toLowerCase().includes("book") ||
            item.feeHeadName.toLowerCase().includes("exam");
      const freq = head ? head.frequency : "Quarterly";
      const displayName =
        (item.feeHeadName || "")
          .toLowerCase()
          .includes("uniform") ||
        (item.feeHeadName || "").toLowerCase().includes("package")
          ? "Uniform & Accessories"
          : item.feeHeadName;
    })}
  </div>

```

```

        return (
          <div
            key={item.feeHeadId}
            className="flex items-center justify-between text-slate-600 dark:text-slate-300 py-0.5"
          >
            <div className="flex items-center gap-2">
              <span className="font-semibold text-slate-900 dark:text-white">
                {displayName}
              </span>
              <span
                className={`px-1.5 py-0.5 rounded text-[9px] font-extrabold uppercase tracking-wider ${
                  isMandatory
                    ? "bg-rose-100 text-rose-700 dark:bg-rose-950 dark:text-rose-300"
                    : "bg-sky-100 text-sky-700 dark:bg-sky-950 dark:text-sky-300"
                }`}
              >
                {isMandatory ? "Mandatory" : "Optional"}
              </span>
              <span className="text-[10px] text-slate-400 font-medium">
                ({freq})
              </span>
            </div>
            <span className="font-mono font-bold text-slate-900 dark:text-white">
              {formatCurrency(item.amount)}
            </span>
          </div>
        );
      }}}
    </div>
  )}

  <div className="pt-2 border-t border-slate-100 dark:border-slate-800 flex justify-between font-extrabold text-sm text-slate-900 dark:text-white">
    <span>Total Standard Base Fee:</span>
    <span className="text-emerald-600 dark:text-emerald-400">
      {formatCurrency(s.totalAmount)}
    </span>
  </div>
</div>
)}}
</div>
)}

{/* Modal */}
{isModalOpen && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/60 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-3xl max-w-lg w-full p-6 shadow-2xl space-y-4 max-h-[90vh] flex flex-col">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-3">
        <h3 className="text-base font-bold text-slate-900 dark:text-white">
          {editingStruct
            ? "Edit Dynamic Fee Structure"
            : "Configure Dynamic Fee Structure"}
        </h3>
        <button
          onClick={() => setIsModalOpen(false)}
          className="text-slate-400 hover:text-slate-600 cursor-pointer"
        >
          ✕
        </button>
      </div>

```

```

</div>

<form
  onSubmit={handleSubmit}
  className="space-y-4 text-xs overflow-y-auto pr-1 flex-1"
>
  <div>
    <label className="block font-semibold mb-1 text-slate-700 dark:text-slate-300">
      Class Grade <span className="text-rose-500 font-bold ml-0.5">*</span></label>
    <select
      value={className}
      onChange={(e) => handleClassChange(e.target.value)}
      className="w-full px-3 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 text-xs font-bold text-slate-900 dark:text-white outline-none focus:ring-2 focus:ring-sky-500/20 cursor-pointer"
    >
      <option value="">Select Class</option>
      {availableClassesForModal.map((c) => (
        <option key={c.id} value={c.name}>
          {c.name}
        </option>
      ))}
    </select>
  </div>

  {/* Dynamic Fee Heads Selection – Displayed ONLY after class grade is selected */}
  {className !== "" && (
    <div className="space-y-3 border-t border-slate-100 dark:border-slate-800 pt-3">
      <h4 className="font-extrabold text-slate-800 dark:text-slate-200 uppercase tracking-wider text-[10px]">
        SELECT APPLICABLE MASTER FEE TYPES
      </h4>

      {isLoadingFeeTypes ? (
        <div className="p-6 text-center text-slate-400 space-y-2">
          <div className="w-5 h-5 border-2 border-sky-500 border-t-transparent rounded-full animate-spin mx-auto" />
          <p className="text-xs font-medium italic">
            Loading applicable fee types...
          </p>
        </div>
      ) : (
        <div className="space-y-2.5">
          {displayFeeHeads.map((head) => {
            const isChecked = selectedHeadIds.includes(head.id);
            const amountVal = selectedHeadAmounts[head.id] ?? "";

            return (
              <div
                key={head.id}
                className={`flex items-center justify-between p-3 rounded-2xl transition-all border ${
                  isChecked
                    ? "bg-sky-50/50 dark:bg-sky-950/20 border-sky-200 dark:border-sky-850"
                    : "bg-slate-50 dark:bg-slate-800/40 border-slate-100 dark:border-slate-800"
                }`}
              >
                <label className="flex items-center gap-3 cursor-pointer flex-1 select-none">
                  <input
                    type="checkbox"
                    checked={isChecked}
                    onChange={() => handleToggleHead(head.id)}
                    className="w-4 h-4 rounded text-sky-600 border-slate-300 dark:border-slate-

```

```

700 bg-white focus:ring-sky-500 cursor-pointer"
    />
    <div>
      <div className="flex items-center gap-1.5">
        <p className="font-bold text-slate-900 dark:text-white text-xs">
          {(head.name || "")
            .toLowerCase()
            .includes("uniform") ||
            (head.name || "")
            .toLowerCase()
            .includes("package")
            ? "Uniform & Accessories"
            : head.name}
        </p>
        <span
          className={`px-1.5 py-0.5 rounded text-[8px] font-extrabold uppercase t
racking-wider ${
            head.mandatory !== false
            ? "bg-rose-100 text-rose-700 dark:bg-rose-950 dark:text-rose-300"
            : "bg-sky-100 text-sky-700 dark:bg-sky-950 dark:text-sky-300"
          }`}
        >
          {head.mandatory !== false
            ? "Mandatory"
            : "Optional"}
        </span>
      </div>
      <p className="text-[10px] text-slate-400 font-medium">
        {head.category} • {head.frequency}
      </p>
    </div>
  </label>

  <div className="flex items-center gap-1.5 ml-2">
    <span
      className={`font-bold text-xs ${isChecked ? "text-slate-600 dark:text-slate
-300" : "text-slate-300 dark:text-slate-600"}`}
    >
      ₹
    </span>
    <input
      type="text"
      disabled={!isChecked}
      value={isChecked ? amountVal : ""}
      placeholder=""
      onChange={(e) =>
        handleAmountChange(head.id, e.target.value)
      }
      className={`w-28 px-3 py-1.5 rounded-xl border text-right font-mono font-bo
ld text-xs outline-none transition-all ${
        isChecked
        ? "bg-white dark:bg-slate-900 border-slate-200 dark:border-slate-700 te
xt-sky-600 dark:text-sky-400 focus:ring-2 focus:ring-sky-500/20"
        : "bg-slate-100 dark:bg-slate-800/20 border-slate-200/50 dark:border-sl
ate-800 text-slate-300 dark:text-slate-600 cursor-not-allowed"
      }`}
    />
  </div>
</div>
);
}}
</div>
)}

```

```

        { /* Total Auto-Calculated Fee Box */ }
        <div className="p-3 rounded-2xl bg-emerald-50 dark:bg-emerald-950/60 border border-emerald-200 dark:border-emerald-800 flex items-center justify-between mt-3">
            <span className="font-bold text-xs text-emerald-800 dark:text-emerald-300 flex items-center gap-1.5">
                <Calculator className="w-4 h-4" /> Total Auto-Calculated Fee:
            </span>
            <span className="font-extrabold text-sm text-emerald-600 dark:text-emerald-400 font-monospace">
                {formatCurrency(totalCalculated)}
            </span>
        </div>
    </div>
)}

<div className="flex items-center justify-end gap-3 pt-3 border-t border-slate-100 dark:border-slate-800">
    <button
        type="button"
        onClick={() => setIsModalOpen(false)}
        className="px-4 py-2 font-semibold bg-slate-100 dark:bg-slate-800 text-slate-700 dark:text-slate-300 hover:bg-slate-200 dark:hover:bg-slate-700 rounded-xl text-xs transition-all cursor-pointer"
    >
        Cancel
    </button>
    <button
        type="submit"
        className="px-4 py-2 font-bold bg-sky-600 hover:bg-sky-500 text-white rounded-xl text-xs shadow-lg shadow-sky-500/20 transition-all cursor-pointer"
    >
        Save Fee Structure
    </button>
</div>
</form>
</div>
</div>
)}

<ConfirmModal
    isOpen={!deletingStruct}
    title="Delete Structure"
    message={`Are you sure you want to delete structure for ${deletingStruct?.className}?`}
    onConfirm={() => {
        if (deletingStruct) {
            deleteDynamicFeeStructure(deletingStruct.id);
            addToast("success", "Fee Structure Removed");
            setDeletingStruct(null);
        }
    }}
    onCancel={() => setDeletingStruct(null)}
/>
</div>
);
};

```

 Finance/StudentFeeAssignmentView.tsx

TSX

```
import React, { useState } from 'react';
import { formatCurrency } from '../../../utils/currency';
import { UserPlus, Search, CheckSquare, Square, CheckCircle, CheckCircle2, ArrowRight, Settings2, RefreshCw, X, AlertTriangle, Info, Check } from 'lucide-react';
import { useData } from '../../../context/DataContext';
import { useAuth } from '../../../context/AuthContext';
import { useToast } from '../../../context/ToastContext';
import { FeePolicyType, FeeHeadAssignmentBreakdown, Student } from '../../../types';

export const StudentFeeAssignmentView: React.FC = () => {
  const {
    students,
    updateStudent,
    dynamicFeeStructures,
    studentFeeAssignments,
    assignFeeStructure,
    assignCustomFeeStructure,
    bulkAssignFeeStructure,
    academicClasses,
    academicYears,
    generateInstallmentsForStudent,
    getStudentFeeLedger
  } = useData();

  const { selectedAcademicYear } = useAuth();
  const { addToast } = useToast();

  const [selectedClass, setSelectedClass] = useState('');
  const [selectedSection, setSelectedSection] = useState('');
  const [query, setQuery] = useState('');
  const [selectedStudentIds, setSelectedStudentIds] = useState<string[]>([]);
  const [targetStructureId, setTargetStructureId] = useState<string>('');

  // Configure Policy Modal State for Individual Student
  const [configStudent, setConfigStudent] = useState<Student | null>(null);
  const [isConfigModalOpen, setIsConfigModalOpen] = useState(false);
  const [modalPolicy, setModalPolicy] = useState<FeePolicyType>('Full Annual Fee');
  const [modalAdmissionDate, setModalAdmissionDate] = useState<string>('');
  const [modalAdjustmentReason, setModalAdjustmentReason] = useState<string>('');
  const [modalStructureId, setModalStructureId] = useState<string>('');
  const [customBreakdown, setCustomBreakdown] = useState<FeeHeadAssignmentBreakdown[]>([]);
  const [isPreviewOpen, setIsPreviewOpen] = useState(false);
  const [previewInstallments, setPreviewInstallments] = useState<any[]>([]);

  const activeAY = selectedAcademicYear || '2026-2027';

  // Academic Year Start/End Date Range Check
  const currentAYObj = academicYears.find((ay) => ay.academicYear === activeAY);
  const ayStartDate = currentAYObj?.startDate || `${activeAY.slice(0, 4)}-06-01`;
  const ayEndDate = currentAYObj?.endDate || `${parseInt(activeAY.slice(0, 4), 10) + 1}-05-31`;

  const filteredStudents = students.filter((s) => {
    if (!selectedClass || !selectedSection) return false;
    const matchesClass = s.className === selectedClass;
    const matchesSection = selectedSection === 'All' || s.section === selectedSection;
    const matchesQuery =
      `${s.firstName} ${s.lastName}`.toLowerCase().includes(query.toLowerCase()) ||
      s.admissionNo.toLowerCase().includes(query.toLowerCase());
    return matchesClass && matchesSection && matchesQuery;
  });

```

```

});

const handleSelectAll = () => {
  if (selectedStudentIds.length === filteredStudents.length) {
    setSelectedStudentIds([]);
  } else {
    setSelectedStudentIds(filteredStudents.map((s) => s.id));
  }
};

const handleToggleSelect = (id: string) => {
  setSelectedStudentIds((prev) =>
    prev.includes(id) ? prev.filter((i) => i !== id) : [...prev, id]
  );
};

const handleBulkAssign = () => {
  if (selectedStudentIds.length === 0) {
    addToast('warning', 'No Students Selected', 'Please select at least one student for bulk fee assignment.');
    return;
  }

  const classDfs =
    dynamicFeeStructures.find((d) => d.className === selectedClass && d.academicYear === activeAY) ||
    dynamicFeeStructures.find((d) => d.className === selectedClass) ||
    dynamicFeeStructures.find((d) => d.id === targetStructureId) ||
    dynamicFeeStructures[0];

  if (!classDfs) {
    addToast('warning', 'Fee Structure Missing', `No active fee structure found for ${selectedClass} || 'the selected class'. Please configure one in Fee Structures.`);
    return;
  }

  bulkAssignFeeStructure(selectedStudentIds, classDfs.id);
  addToast('success', 'Fee Structure Assigned', `Assigned ${classDfs.className} structure (${formatCurrency(classDfs.totalAmount)}) to ${selectedStudentIds.length} students.`);
  setSelectedStudentIds([]);
};

// Open Policy Configuration Modal
const handleOpenConfigModal = (st: Student) => {
  setConfigStudent(st);

  // Find structure for student's class
  const matchingDfs =
    dynamicFeeStructures.find((d) => d.className === st.className && d.academicYear === activeAY) ||
    dynamicFeeStructures.find((d) => d.className === st.className) ||
    dynamicFeeStructures[0];

  const structId = matchingDfs ? matchingDfs.id : '';
  setModalStructureId(structId);

  const admDate = st.joiningDate || (st as any).admissionDate || `${activeAY.slice(0, 4)}-08-15`;
  setModalAdmissionDate(admDate);

  // Check existing assignment
  const existingAssign = studentFeeAssignments.find(
    (a) => a.studentId === st.id && a.academicYear === activeAY
  );

  let initialPolicy: FeePolicyType = 'Full Annual Fee';

```

```

    if (existingAssign) {
        const p = existingAssign.feePolicy === 'Pro-rata' ? 'Monthly Pro-rated Fee' : (existingAssign.feePolicy || 'Full Annual Fee');
        initialPolicy = p as FeePolicyType;
        setModalPolicy(initialPolicy);
        setModalAdjustmentReason(existingAssign.adjustmentReason || '');
    } else {
        setModalPolicy('Full Annual Fee');
        setModalAdjustmentReason('');
    }

    // Initialize Breakdown Table
    buildModalBreakdown(matchingDfs, initialPolicy, admDate);
    setIsPreviewOpen(false);
    setIsConfigModalOpen(true);
};

// Build Breakdown Table
const buildModalBreakdown = (dfsObj: any, policy: FeePolicyType, admDateStr: string) => {
    if (!dfsObj || !dfsObj.items) {
        setCustomBreakdown([]);
        return;
    }

    const itemsList: FeeHeadAssignmentBreakdown[] = dfsObj.items.map((item: any) => {
        const orig = item.amount;
        const hNameLower = (item.feeHeadName || '').toLowerCase();
        const categoryLower = (item.category || '').toLowerCase();
        const freq = item.frequency || item.feePlan || '';

        const isFixedFullAmountHead =
            freq === 'One Time' ||
            freq === 'Annual' ||
            freq === 'One Term' ||
            freq === 'Single Term' ||
            hNameLower.includes('admission') ||
            categoryLower.includes('admission') ||
            hNameLower.includes('caution') ||
            categoryLower.includes('caution') ||
            hNameLower.includes('annual');

        let assigned = orig;

        if (isFixedFullAmountHead || policy === 'Full Annual Fee') {
            // One Time, Annual, One Term -> Full amount
            assigned = orig;
        } else if (
            policy === 'Monthly Pro-rated Fee' ||
            policy === 'Monthly Pro-rated' ||
            policy === 'Pro-rata' ||
            (policy as string) === 'Monthly'
        ) {
            // Late Admission = Monthly -> Apply remaining months
            if (admDateStr) {
                const admMonth = new Date(admDateStr).getMonth() + 1; // 1-12 (Jan=1, Apr=4, Jun=6)
                const startMonthIdx = admMonth >= 4 ? admMonth - 4 : admMonth + 8;
                const remainingMonths = Math.max(1, 12 - startMonthIdx);
                assigned = Math.round((orig / 12) * remainingMonths);
            }
        } else if (policy === 'Term-wise' || policy === 'Term-wise Fee') {
            // Late Admission = Term-wise -> Apply remaining terms
            if (admDateStr) {
                const admMonth = new Date(admDateStr).getMonth() + 1;

```

```

        let remainingTermsRatio = 1.0;
        if (admMonth >= 4 && admMonth <= 6) remainingTermsRatio = 1.0;
        else if (admMonth >= 7 && admMonth <= 9) remainingTermsRatio = 0.75;
        else if (admMonth >= 10 && admMonth <= 12) remainingTermsRatio = 0.50;
        else remainingTermsRatio = 0.25;

        assigned = Math.round(orig * remainingTermsRatio);
    }
} else if (policy === 'Custom' || policy === 'Custom Amount') {
    assigned = orig;
}

return {
    feeHeadId: item.feeHeadId,
    feeHeadName: item.feeHeadName,
    category: item.feeHeadName.includes('Tuition')
        ? 'Tuition Fee'
        : item.feeHeadName.includes('Transport')
        ? 'Transport Fee'
        : 'Other Fee',
    billingType: isFixedFullAmountHead ? 'One-time' : 'Monthly',
    originalAmount: orig,
    assignedAmount: assigned,
    adjustmentAmount: assigned - orig,
    isEligibleForProRata: !isFixedFullAmountHead
};
});

setCustomBreakdown(itemsList);
};

const handlePolicyChange = (newPolicy: FeePolicyType) => {
    setModalPolicy(newPolicy);
    const dfsObj = dynamicFeeStructures.find((d) => d.id === modalStructureId);
    buildModalBreakdown(dfsObj, newPolicy, modalAdmissionDate);
};

const handleAdmissionDateChange = (newDate: string) => {
    setModalAdmissionDate(newDate);
    const dfsObj = dynamicFeeStructures.find((d) => d.id === modalStructureId);
    buildModalBreakdown(dfsObj, modalPolicy, newDate);
};

const handleCustomAmountChange = (headId: string, valStr: string) => {
    const numericVal = parseFloat(valStr) || 0;
    setCustomBreakdown((prev) =>
        prev.map((item) =>
            item.feeHeadId === headId
                ? {
                    ...item,
                    assignedAmount: Math.max(0, numericVal),
                    adjustmentAmount: Math.max(0, numericVal) - item.originalAmount
                }
                : item
        )
    );
};

const handleSaveModalAssignment = () => {
    if (!configStudent || !modalStructureId) return;

    // Validate Admission Date range
    if (modalAdmissionDate < ayStartDate || modalAdmissionDate > ayEndDate) {

```

```

        addToast(
            'error',
            'Invalid Admission Date',
            `Admission date (${modalAdmissionDate}) must fall within current academic year dates (${ayStartDate} to ${ayEndDate}).`
        );
        return;
    }

    if (modalPolicy === 'Custom' && !modalAdjustmentReason.trim()) {
        addToast('warning', 'Reason Required', 'Please provide an adjustment reason for Custom Amount policy.');
```

```

        return;
    }

    updateStudent(configStudent.id, {
        joiningDate: modalAdmissionDate,
        feeCalculationMethod: modalPolicy as any
    });

    assignCustomFeeStructure(
        configStudent.id,
        modalStructureId,
        modalPolicy,
        customBreakdown,
        modalAdjustmentReason,
        modalAdmissionDate
    );

    addToast(
        'success',
        'Fee Assignment Saved',
        `Assigned ${modalPolicy === 'Custom' ? 'Custom Amount' : modalPolicy} policy for ${configStudent.firstName}.`
    );

    setIsConfigModalOpen(false);
};

const handleGenerateSchedulePreview = () => {
    if (!configStudent || !modalStructureId) return;

    // Validate Admission Date range
    if (modalAdmissionDate < ayStartDate || modalAdmissionDate > ayEndDate) {
        addToast(
            'error',
            'Invalid Admission Date',
            `Admission date (${modalAdmissionDate}) must fall within current academic year dates (${ayStartDate} to ${ayEndDate}).`
        );
        return;
    }

    if (modalPolicy === 'Custom' && !modalAdjustmentReason.trim()) {
        addToast('warning', 'Reason Required', 'Please provide an adjustment reason for Custom Amount policy.');
```

```

        return;
    }

    const tempAssignment = {
        id: 'TEMP-ASSIGN',
        studentId: configStudent.id,
        studentName: `${configStudent.firstName} ${configStudent.lastName}`,

```

```

    admissionNo: configStudent.admissionNo,
    branch: configStudent.branch || 'Main Campus',
    academicYear: activeAY,
    className: configStudent.className,
    section: configStudent.section,
    feeStructureId: modalStructureId,
    assignedFeeHeads: customBreakdown.map(i => ({
      feeHeadId: i.feeHeadId,
      feeHeadName: i.feeHeadName,
      category: i.category,
      amount: i.assignedAmount
    })),
    baseFeeTotal: assignedTotalSum,
    assignedDate: new Date().toISOString().split('T')[0],
    status: 'Active' as const,
    feePolicy: modalPolicy
  });

const tempLedger = {
  id: `LED-TEMP-${configStudent.id}`,
  studentId: configStudent.id,
  studentName: `${configStudent.firstName} ${configStudent.lastName}`,
  admissionNo: configStudent.admissionNo,
  className: configStudent.className,
  section: configStudent.section,
  studentType: (configStudent.studentType === 'Residential' ? 'Hosteller' : 'Day Scholar') as 'Day Scholar' | 'Hosteller',
  academicYear: activeAY,
  feeItems: customBreakdown.map(i => ({
    headId: i.feeHeadId,
    headName: i.feeHeadName,
    category: i.category,
    originalAmount: i.originalAmount,
    scholarshipDeduction: 0,
    discountDeduction: 0,
    fineAmount: 0,
    finalAmount: i.assignedAmount,
    isApplicable: true,
    status: 'Pending' as const
  })),
  totalOriginalAmount: originalTotalSum,
  grossAmount: originalTotalSum,
  totalScholarship: 0,
  totalDiscount: 0,
  totalFine: 0,
  totalPayable: assignedTotalSum,
  paidAmount: 0,
  dueBalance: assignedTotalSum,
  createdAt: new Date().toISOString().split('T')[0],
  updatedAt: new Date().toISOString().split('T')[0],
  scholarshipAmount: 0,
  discountAmount: 0,
  fineAmount: 0,
  previousDue: 0
};

const insts = generateInstallmentsForStudent(configStudent.id, activeAY, tempAssignment, tempLedger);
setPreviewInstallments(insts);
setIsPreviewOpen(true);
};

const originalTotalSum = customBreakdown.reduce((sum, i) => sum + i.originalAmount, 0);
const assignedTotalSum = customBreakdown.reduce((sum, i) => sum + i.assignedAmount, 0);

```

```
const adjustmentTotalSum = assignedTotalSum - originalTotalSum;

return (
  <div className="space-y-6 animate-in fade-in">
    { /* Header */ }
    <div>
      <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center gap-2">
        <UserPlus className="w-6 h-6 text-sky-500" /> Student Fee Assignment & Policy Management
      </h2>
      <p className="text-xs text-slate-500 dark:text-slate-400 mt-0.5 font-medium">
        Allocate standard fee structures or configure mid-year admission adjustments (Full Annual Fee, Monthly Pro-rated Fee, Term-wise Fee, Custom Amount).
      </p>
    </div>

    { /* Filter & Bulk Control Bar */ }
    <div className="glass-card p-4 rounded-2xl flex flex-col lg:flex-row lg:items-end justify-between gap-4">
      <div className="flex flex-wrap items-end gap-3 w-full lg:w-auto text-xs">
        <div>
          <label className="block font-semibold text-slate-500 dark:text-slate-400 mb-1 text-[11px]">Class Grade</label>
          <select
            value={selectedClass}
            onChange={(e) => {
              setSelectedClass(e.target.value);
              setSelectedSection('');
            }}
            className="px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold text-slate-900 dark:text-white outline-none cursor-pointer">
            <option value="">Select Class</option>
            {academicClasses.map((c) => (
              <option key={c.id} value={c.name}>
                {c.name}
              </option>
            ))}
          </select>
        </div>

        <div>
          <label className="block font-semibold text-slate-500 dark:text-slate-400 mb-1 text-[11px]">Section</label>
          <select
            value={selectedSection}
            onChange={(e) => setSelectedSection(e.target.value)}
            className="px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold text-slate-900 dark:text-white outline-none cursor-pointer"
            disabled={!selectedClass}>
            <option value="">Select Section</option>
            <option value="A">Section A</option>
            <option value="B">Section B</option>
            <option value="C">Section C</option>
            <option value="All">All Sections</option>
          </select>
        </div>

        <div>
          <label className="block font-semibold text-slate-500 dark:text-slate-400 mb-1 text-[11px]">Search Student</label>
          <div className="relative">
```

```

        <Search className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
        <input
            type="text"
            placeholder="Search student or adm no..."
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            className="pl-9 pr-4 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-200
dark:border-slate-700 text-xs text-slate-900 dark:text-white outline-none w-56 font-medium"
        />
    </div>
</div>
</div>

{/* Bulk Action */}
<div className="flex items-center gap-3 shrink-0 border-t lg:border-t-0 pt-3 lg:pt-0 border-slate-1
00 dark:border-slate-800">
    <button
        onClick={handleBulkAssign}
        disabled={selectedStudentIds.length === 0}
        className={`px-4 py-2 rounded-xl text-xs font-bold shadow-md flex items-center gap-1.5 shrink-0
cursor-pointer h-[38px] transition-all ${
            selectedStudentIds.length > 0
                ? "bg-emerald-600 hover:bg-emerald-500 text-white shadow-emerald-600/20 active:scale-95"
                : "bg-slate-100 dark:bg-slate-800 text-slate-400 dark:text-slate-600 cursor-not-allowed opa
city-60"
            }`}
    >
        <CheckCircle className="w-4 h-4" /> Bulk Assign Class Fee ({selectedStudentIds.length})
    </button>
</div>
</div>

{/* Student List Table */}
<div className="glass-card rounded-2xl overflow-hidden border border-slate-200/80 dark:border-slate-8
00">
    <div className="overflow-x-auto">
        <table className="w-full text-left border-collapse text-xs">
            <thead>
                <tr className="bg-slate-100/70 dark:bg-slate-800/60 text-slate-500 font-bold uppercase tracki
ng-wider border-b border-slate-200 dark:border-slate-800">
                    <th className="py-3.5 px-4">
                        <button onClick={handleSelectAll} className="flex items-center gap-1 text-slate-600 dark:
text-slate-300 font-bold">
                            {selectedStudentIds.length === filteredStudents.length && filteredStudents.length > 0 ?
(
                                <CheckSquare className="w-4 h-4 text-sky-600" />
                            ) : (
                                <Square className="w-4 h-4" />
                            )}
                        </button>
                    </th>
                    <th className="py-3.5 px-4">Student Name</th>
                    <th className="py-3.5 px-4">Adm No</th>
                    <th className="py-3.5 px-4">Class & Sec</th>
                    <th className="py-3.5 px-4">Category</th>
                    <th className="py-3.5 px-4">Fee Policy</th>
                    <th className="py-3.5 px-4">Assigned Fee Payable</th>
                    <th className="py-3.5 px-4 text-right">Configure / Assign</th>
                </tr>
            </thead>
            <tbody className="divide-y divide-slate-100 dark:divide-slate-800/80 font-medium">
                {!selectedClass || !selectedSection ? (
                    <tr>

```

```

        <td colSpan={8} className="py-8 text-center text-slate-400 dark:text-slate-500 font-bold italic">
            Please select a Class Grade and Section to view and allocate student fee assignments.
        </td>
    </tr>
) : filteredStudents.length === 0 ? (
    <tr>
        <td colSpan={8} className="py-8 text-center text-slate-400 dark:text-slate-500 font-bold italic">
            No students found matching the selected class, section, or search criteria.
        </td>
    </tr>
) : (
    filteredStudents.map((st) => {
        const isSelected = selectedStudentIds.includes(st.id);
        const assignment = studentFeeAssignments.find(
            (a) => a.studentId === st.id && a.academicYear === activeAY
        );

        return (
            <tr
                key={st.id}
                className={`hover:bg-slate-50/80 dark:hover:bg-slate-800/40 ${
                    isSelected ? 'bg-sky-50/50 dark:bg-sky-950/20' : ''
                }}`
            >
                <td className="py-3 px-4">
                    <button onClick={() => handleToggleSelect(st.id)}>
                        {isSelected ? (
                            <CheckSquare className="w-4 h-4 text-sky-600" />
                        ) : (
                            <Square className="w-4 h-4 text-slate-400" />
                        )}
                    </button>
                </td>
                <td className="py-3 px-4 font-bold text-slate-900 dark:text-white flex items-center gap-2">
                    <div className="w-7 h-7 rounded-lg bg-sky-100 dark:bg-sky-950 text-sky-700 dark:text-slate-300 flex items-center justify-center font-bold text-xs shrink-0">
                        {st.firstName?.[0] || 'S'}
                    </div>
                    {st.firstName} {st.lastName}
                </td>
                <td className="py-3 px-4 font-mono text-slate-500">{st.admissionNo}</td>
                <td className="py-3 px-4 text-slate-700 dark:text-slate-300">
                    {st.className}-{st.section}
                </td>
                <td className="py-3 px-4 font-semibold text-slate-600 dark:text-slate-400">
                    {st.category || 'General'}
                </td>
                <td className="py-3 px-4">
                    {assignment ? (
                        <span className="px-2.5 py-1 rounded-lg bg-emerald-50 text-emerald-700 dark:bg-emerald-950 dark:text-emerald-300 font-bold text-[11px]">
                            {assignment.feePolicy === 'Pro-rata' || assignment.feePolicy === 'Monthly Pro-rated' || assignment.feePolicy === 'Monthly Pro-rated Fee'
                                ? 'Monthly Pro-rated Fee'
                                : (assignment.feePolicy === 'Custom' || assignment.feePolicy === 'Custom Amount' ? 'Custom Amount' : assignment.feePolicy || 'Full Annual Fee')}
                        </span>
                    ) : (
                        <span className="px-2 py-0.5 rounded bg-amber-50 text-amber-700 dark:bg-amber-950 dark:text-amber-300 font-bold text-[11px]">

```

```

                Default Annual
            </span>
        ))
    </td>
    <td className="py-3 px-4 font-extrabold text-slate-900 dark:text-white font-mono">
        {formatCurrency(assignment ? assignment.baseFeeTotal : st.totalFee || 0)}
    </td>
    <td className="py-3 px-4 text-right">
        <button
            onClick={() => handleOpenConfigModal(st)}
            className="px-3 py-1 rounded-xl bg-sky-50 text-sky-700 dark:bg-sky-950 dark:text-
sky-300 font-bold hover:bg-sky-100 flex items-center gap-1.5 ml-auto border border-sky-200 dark:border-sky-
800 transition-colors cursor-pointer"
        >
            <Settings2 className="w-3.5 h-3.5" /> Configure Policy
        </button>
    </td>
</tr>
</tbody>
</table>
</div>
</div>

{/* POLICY & MID-YEAR ASSIGNMENT MODAL */}
{isConfigModalOpen && configStudent && (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/70 backdrop-bl
ur-xs overflow-y-auto">
        <div className="w-full max-w-3xl rounded-3xl bg-white dark:bg-slate-900 border border-slate-200 d
ark:border-slate-800 shadow-2xl overflow-hidden flex flex-col max-h-[92vh]">

            {/* Modal Header */}
            <div className="flex items-center justify-between px-6 py-4 bg-gradient-to-r from-sky-900 via-s
ky-800 to-indigo-900 text-white shrink-0">
                <div className="flex items-center gap-3">
                    <div className="p-2.5 rounded-2xl bg-white/10 text-white backdrop-blur-md">
                        <Settings2 className="w-6 h-6" />
                    </div>
                    <div>
                        <h3 className="text-base font-black tracking-tight">Configure Student Fee Assignment & Po
licy</h3>
                        <p className="text-xs text-sky-200 font-medium">
                            {configStudent.firstName} {configStudent.lastName} • Adm No: <span className="font-mon
o">{configStudent.admissionNo}</span>
                        </p>
                    </div>
                </div>
                <button
                    onClick={() => setIsConfigModalOpen(false)}
                    className="p-1.5 rounded-xl hover:bg-white/20 text-white/80 hover:text-white transition-col
ors cursor-pointer"
                >
                    <X className="w-5 h-5" />
                </button>
            </div>

            {/* Modal Body */}
            <div className="flex-1 overflow-y-auto p-6 space-y-5 text-xs bg-slate-50/50 dark:bg-slate-950/4
0">
                {isPreviewOpen ? (
                    <div className="space-y-4">

```

```

        <div className="p-4 rounded-2xl bg-sky-50 dark:bg-sky-950/40 border border-sky-200 dark:b
order-sky-800 grid grid-cols-2 gap-4 font-bold text-slate-700 dark:text-slate-350">
            <div>
                <span className="text-[10px] text-slate-400 block uppercase tracking-wider">Student N
ame</span>
                <span className="text-slate-900 dark:text-white font-black text-sm">{configStudent.fi
rstName} {configStudent.lastName}</span>
            </div>
            <div>
                <span className="text-[10px] text-slate-400 block uppercase tracking-wider">Academic
Session</span>
                <span className="text-sky-600 dark:text-sky-400 font-extrabold text-sm font-mono">{ac
tiveAY}</span>
            </div>
        </div>

        <div className="rounded-2xl border border-slate-200 dark:border-slate-800 bg-white dark:b
g-slate-900 overflow-hidden shadow-xs">
            <table className="w-full text-left text-xs border-collapse">
                <thead>
                    <tr className="bg-slate-100 dark:bg-slate-800/80 text-[10px] font-black uppercase t
racking-wider text-slate-500 border-b">
                        <th className="p-3">Fee Head</th>
                        <th className="p-3 font-mono">Annual Assigned</th>
                        <th className="p-3">Frequency</th>
                        <th className="p-3">Term / Installment</th>
                        <th className="p-3">Due Date</th>
                        <th className="p-3 font-mono text-right">Amount</th>
                    </tr>
                </thead>
                <tbody className="divide-y divide-slate-100 dark:divide-slate-800 font-medium">
                    {previewInstallments.map((inst, idx) => (
                        <tr key={idx} className="hover:bg-slate-50/40 dark:hover:bg-slate-800/40">
                            <td className="p-3 font-bold text-slate-900 dark:text-white">{inst.feeHeadName}
                        </td>
                            <td className="p-3 font-mono text-slate-600 dark:text-slate-400">
                                {formatCurrency(inst.amount * (inst.frequency === 'One Time' ? 1 : (inst.freq
uency === 'Term-wise' ? previewInstallments.filter(i => i.feeHeadId === inst.feeHeadId).length : (inst.freq
uency === 'Quarterly' ? 4 : (inst.frequency === 'Half-Yearly' ? 2 : (inst.frequency === 'Monthly' ? 12 :
1))))))}
                            </td>
                            <td className="p-3">
                                <span className="px-2 py-0.5 rounded text-[10px] font-bold bg-slate-100 text-
slate-600 dark:bg-slate-800 dark:text-slate-400">
                                    {inst.frequency}
                                </span>
                            </td>
                            <td className="p-3 text-slate-800 dark:text-slate-200 font-bold">{inst.termName
|| 'Term 1'}</td>
                            <td className="p-3 font-mono text-slate-500">{inst.dueDate}</td>
                            <td className="p-3 font-mono text-right font-black text-emerald-600 dark:text-e
merald-400">{formatCurrency(inst.amount)}</td>
                        </tr>
                    )))}
                </tbody>
            </table>
        </div>
    </div>
) : (
    <>
        { /* Student Metadata Card */ }
        <div className="p-4 rounded-2xl bg-white dark:bg-slate-900 border border-slate-200 dark:b
order-slate-800 grid grid-cols-2 sm:grid-cols-4 gap-3">

```

```

        <div>
            <span className="text-slate-400 block font-semibold text-[10px]">Academic Session</span>
        </div>
        <div>
            <span className="font-extrabold font-mono text-sky-600">{activeAY}</span>
        </div>
        <div>
            <span className="text-slate-400 block font-semibold text-[10px]">Class & Section</span>
        </div>
        <div>
            <span className="font-bold text-slate-800 dark:text-slate-200">{configStudent.className} - {configStudent.section}</span>
        </div>
        <div>
            <span className="text-slate-400 block font-semibold text-[10px]">Student Type</span>
            <span className="font-bold text-slate-800 dark:text-slate-200">{configStudent.studentType} || 'Day Scholar'</span>
        </div>
        <div>
            <span className="text-slate-400 block font-semibold text-[10px]">Admission Date</span>
            <input
                type="date"
                value={modalAdmissionDate}
                onChange={(e) => handleAdmissionDateChange(e.target.value)}
                className="px-2 py-1 rounded-lg border border-slate-200 dark:border-slate-700 bg-slate-50 dark:bg-slate-800 font-bold text-slate-900 dark:text-white text-xs mt-0.5"
            />
        </div>
    </div>

    { /* Policy Selector */ }
    <div className="space-y-2">
        <label className="block font-black text-slate-900 dark:text-slate-100 uppercase tracking-wider text-[11px]">
            Select Fee Policy:
        </label>
        <div className="grid grid-cols-2 sm:grid-cols-4 gap-2.5">
            <label
                className={`p-3 rounded-2xl border cursor-pointer transition-all flex flex-col justify-between`}
                modalPolicy === 'Full Annual Fee'
                ? 'border-sky-600 bg-sky-50/80 dark:bg-sky-950/60 ring-2 ring-sky-500/20'
                : 'border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900'
            >
                <div className="flex items-center justify-between">
                    <span className="font-bold text-slate-900 dark:text-slate-100">Full Annual Fee</span>
                    <input
                        type="radio"
                        name="feePolicy"
                        checked={modalPolicy === 'Full Annual Fee'}
                        onChange={() => handlePolicyChange('Full Annual Fee')}
                    />
                </div>
                <span className="text-[10px] text-slate-400 mt-1">Standard full session fee</span>
            </label>
            <label
                className={`p-3 rounded-2xl border cursor-pointer transition-all flex flex-col justify-between`}
                modalPolicy === 'Monthly Pro-rated Fee' || modalPolicy === 'Monthly Pro-rated' || modalPolicy === 'Pro-rata'
                ? 'border-sky-600 bg-sky-50/80 dark:bg-sky-950/60 ring-2 ring-sky-500/20'
            >
                <div className="flex items-center justify-between">
                    <span className="font-bold text-slate-900 dark:text-slate-100">Monthly Pro-rated Fee</span>
                    <input
                        type="radio"
                        name="feePolicy"
                        checked={modalPolicy === 'Monthly Pro-rated Fee' || modalPolicy === 'Monthly Pro-rated' || modalPolicy === 'Pro-rata'}
                        onChange={() => handlePolicyChange('Monthly Pro-rated Fee')}
                    />
                </div>
                <span className="text-[10px] text-slate-400 mt-1">Monthly pro-rated session fee</span>
            </label>
        </div>
    </div>

```

```

        : 'border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900'
      }` }
    >
    <div className="flex items-center justify-between">
      <span className="font-bold text-slate-900 dark:text-slate-100">Monthly Pro-rated
</span>

      <input
        type="radio"
        name="feePolicy"
        checked={modalPolicy === 'Monthly Pro-rated Fee' || modalPolicy === 'Monthly Pr
o-rated' || modalPolicy === 'Pro-rata'}
        onChange={() => handlePolicyChange('Monthly Pro-rated Fee')}
      />
    </div>
    <span className="text-[10px] text-slate-400 mt-1">Pro-rated for remaining months</s
pan>

  </label>

  <label
    className={`p-3 rounded-2xl border cursor-pointer transition-all flex flex-col just
ify-between ${
      modalPolicy === 'Term-wise'
        ? 'border-sky-600 bg-sky-50/80 dark:bg-sky-950/60 ring-2 ring-sky-500/20'
        : 'border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900'
      }` }
  >
    <div className="flex items-center justify-between">
      <span className="font-bold text-slate-900 dark:text-slate-100">Term-wise Fee</spa
n>

      <input
        type="radio"
        name="feePolicy"
        checked={modalPolicy === 'Term-wise'}
        onChange={() => handlePolicyChange('Term-wise')}
      />
    </div>
    <span className="text-[10px] text-slate-400 mt-1">Applicable terms remaining</span>
  </label>

  <label
    className={`p-3 rounded-2xl border cursor-pointer transition-all flex flex-col just
ify-between ${
      modalPolicy === 'Custom'
        ? 'border-sky-600 bg-sky-50/80 dark:bg-sky-950/60 ring-2 ring-sky-500/20'
        : 'border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900'
      }` }
  >
    <div className="flex items-center justify-between">
      <span className="font-bold text-slate-900 dark:text-slate-100">Custom Amount</spa
n>

      <input
        type="radio"
        name="feePolicy"
        checked={modalPolicy === 'Custom'}
        onChange={() => handlePolicyChange('Custom')}
      />
    </div>
    <span className="text-[10px] text-slate-400 mt-1">Manual admin adjustments</span>
  </label>
</div>
</div>

  { /* Adjustment Reason input for Custom Amount */ }

```

```

        {modalPolicy === 'Custom' && (
          <div className="p-3.5 rounded-2xl bg-amber-50/60 dark:bg-amber-950/40 border border-amber-200 dark:border-amber-900/60 space-y-2">
            <label className="block font-bold text-amber-900 dark:text-amber-300">
              Adjustment Reason / Notes (Required for Custom Amount):
            </label>
            <input
              type="text"
              placeholder="e.g. Mid-Year Late Joining Discount, Special Financial Concession"
              value={modalAdjustmentReason}
              onChange={(e) => setModalAdjustmentReason(e.target.value)}
              className="w-full px-3 py-2 rounded-xl border border-amber-300 dark:border-amber-800 bg-white dark:bg-slate-900 font-medium text-xs text-slate-900 dark:text-slate-100"
            />
          </div>
        )}

    {/* Fee Head Breakdown Table */}
    <div className="space-y-2">
      <div className="flex items-center justify-between">
        <span className="font-black text-slate-900 dark:text-slate-100 uppercase tracking-wider text-[11px]">
          Fee Head Breakdown:
        </span>
        <button
          onClick={() => {
            const dfsObj = dynamicFeeStructures.find((d) => d.id === modalStructureId);
            buildModalBreakdown(dfsObj, modalPolicy, modalAdmissionDate);
          }}
          className="text-[11px] font-bold text-sky-600 hover:text-sky-700 flex items-center gap-1 cursor-pointer"
        >
          <RefreshCw className="w-3.5 h-3.5" /> Recalculate
        </button>
      </div>

      <div className="rounded-2xl border border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900 overflow-hidden shadow-xs">
        <table className="w-full text-left text-xs border-collapse">
          <thead>
            <tr className="bg-slate-100 dark:bg-slate-800/80 text-[10px] font-black uppercase tracking-wider text-slate-500 border-b">
              <th className="p-3">Fee Head</th>
              <th className="p-3">Billing Config</th>
              <th className="p-3 font-mono">Standard Amount</th>
              <th className="p-3 font-mono">Assigned Amount</th>
              <th className="p-3 font-mono text-right">Adjustment</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100 dark:divide-slate-800 font-medium">
            {customBreakdown.map((item) => (
              <tr key={item.feeHeadId}>
                <td className="p-3 font-bold text-slate-800 dark:text-slate-200">
                  {item.feeHeadName}
                </td>
                <td className="p-3">
                  <span
                    className={`px-2 py-0.5 rounded text-[10px] font-bold ${
                      item.isEligibleForProRata
                        ? 'bg-sky-100 text-sky-700 dark:bg-sky-950 dark:text-sky-300'
                        : 'bg-slate-100 text-slate-600 dark:bg-slate-800 dark:text-slate-400'
                    }`}
                  >

```

```

        {item.isEligibleForProRata ? 'Monthly (Pro-rata Eligible)' : 'One-Time (Fixed)'}
      </span>
    </td>
    <td className="p-3 font-mono font-bold text-slate-600 dark:text-slate-400">
      {formatCurrency(item.originalAmount)}
    </td>
    <td className="p-3">
      {modalPolicy === 'Custom' ? (
        <input
          type="number"
          value={item.assignedAmount}
          onChange={(e) => handleCustomAmountChange(item.feeHeadId, e.target.value)}
        >
        <div
          className="w-28 px-2 py-1 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-slate-800 font-bold font-mono text-slate-900 dark:text-white text-xs"
        >
          <div>
            <span>
              {formatCurrency(item.assignedAmount)}
            </span>
          </div>
        </div>
      ) : (
        <span className="font-mono font-extrabold text-slate-900 dark:text-white">
          {formatCurrency(item.assignedAmount)}
        </span>
      )}
    </td>
    <td className="p-3 font-mono font-bold text-right">
      <span
        className={
          item.adjustmentAmount < 0
            ? 'text-emerald-600 dark:text-emerald-400'
            : item.adjustmentAmount > 0
            ? 'text-rose-600'
            : 'text-slate-400'
        }
      >
        {item.adjustmentAmount === 0 ? '₹0' : formatCurrency(item.adjustmentAmount)}
      </span>
    </td>
  </tr>
</tbody>
<tfoot>
  <tr className="bg-slate-100 dark:bg-slate-800 font-black border-t text-xs">
    <td colspan={2} className="p-3 uppercase">Total Breakdown Summary:</td>
    <td className="p-3 font-mono text-slate-600 dark:text-slate-400">{formatCurrency(originalTotalSum)}</td>
    <td className="p-3 font-mono text-sky-700 dark:text-sky-300">{formatCurrency(assignedTotalSum)}</td>
    <td className="p-3 font-mono text-right">
      <span className={adjustmentTotalSum < 0 ? 'text-emerald-600' : 'text-slate-600'}>
        {formatCurrency(adjustmentTotalSum)}
      </span>
    </td>
  </tr>
</tfoot>
</table>
</div>
</div>
</>
  )}
</div>

```

```

    {/* Modal Footer */}
    <div className="flex items-center justify-between px-6 py-4 bg-slate-100 dark:bg-slate-850 border-t border-slate-200 dark:border-slate-800 shrink-0">
      {isPreviewOpen ? (
        <>
          <button
            onClick={() => setIsPreviewOpen(false)}
            className="px-4 py-2 rounded-xl border border-slate-200 dark:border-slate-700 text-slate-600 dark:text-slate-400 font-bold hover:bg-slate-200 dark:hover:bg-slate-800 transition-colors"
          >
            Recalculate / Adjust
          </button>
          <div className="flex items-center gap-3">
            <button
              onClick={handleGenerateSchedulePreview}
              className="px-4 py-2 rounded-xl border border-slate-200 dark:border-slate-700 text-slate-700 dark:text-slate-300 font-bold hover:bg-slate-200 transition-colors flex items-center gap-1.5 cursor-pointer"
            >
              <RefreshCw className="w-4 h-4" /> Refresh Preview
            </button>
            <button
              onClick={handleSaveModalAssignment}
              className="px-6 py-2 rounded-xl bg-emerald-600 hover:bg-emerald-700 text-white font-extrabold shadow-md transition-colors flex items-center gap-1.5 cursor-pointer"
            >
              <Check className="w-4 h-4" /> Save Assignment
            </button>
          </div>
        </>
      ) : (
        <>
          <button
            onClick={() => setIsConfigModalOpen(false)}
            className="px-4 py-2 rounded-xl border border-slate-200 dark:border-slate-700 text-slate-600 dark:text-slate-400 font-bold hover:bg-slate-200 dark:hover:bg-slate-800 transition-colors"
          >
            Cancel
          </button>
          <div className="flex items-center gap-3">
            <button
              onClick={() => {
                const dfsObj = dynamicFeeStructures.find((d) => d.id === modalStructureId);
                buildModalBreakdown(dfsObj, modalPolicy, modalAdmissionDate);
                addToast('info', 'Recalculated', 'Recalculated fee head breakdown.');
```

```
        </div>
    </div>
    })
</div>
);
};
```

 Finance/FeeCollectionView.tsx

TSX

```
// @ts-nocheck
import React, { useState, useEffect, useRef } from "react";
import { formatCurrency } from "../../utils/currency";
import {
  getUniformPackageFeeByClass,
  getUniformFeeForClass,
  getItemFeeFromFinanceConfig,
  calculateClothOrItemPrice,
} from "../../utils/uniformUtils";
import { matchesClassName } from "../../utils/classSorter";
import {
  IndianRupee,
  Search,
  Receipt,
  CheckCircle,
  AlertCircle,
  Calculator,
  History,
  ArrowRight,
  ArrowLeft,
  Printer,
  X,
  ChevronDown,
  ChevronUp,
  Users,
  Filter,
} from "lucide-react";
import {
  Student,
  FeePayment,
  StudentFeeLedger,
  StudentFeeInstallment,
} from "../../types";
import {
  useData,
  StudentCalculationResult,
} from "../../context/DataContext";
import { useAuth } from "../../context/AuthContext";
import { useToast } from "../../context/ToastContext";
import { Badge } from "../../common/Badge";

interface FeeCollectionViewProps {
  onPrintReceipt: (payment: FeePayment) => void;
  initialStudent?: Student | null;
  onClearInitialStudent?: () => void;
}

export const FeeCollectionView: React.FC<FeeCollectionViewProps> = ({
  onPrintReceipt,
  initialStudent,
  onClearInitialStudent,
}) => {
  const {
    students,
    admissions,
    feePayments,
    studentFeeLedgers,
    studentFeeInstallments,
    studentUniformIssues,
  }
```

```

    updateStudentUniformIssue,
    calculateStudentPayableFee,
    addFeePayment,
    financeSettings,
    financeUniformConfigs,
    feeStructures = [],
    feeHeads = [],
    dynamicFeeStructures = [],
    getStudentFeeLedger,
    getStudentFeeOutstandingSummary,
    getStudentInstallmentSummary,
    fetchFinanceData,
    scholarships,
    discounts,
    applyScholarshipToStudent,
    removeScholarshipFromStudent,
    applyDiscountToStudent,
    removeDiscountFromStudent,
  } = useData();
  const { selectedAcademicYear } = useAuth();
  const { addToast } = useToast();

  const [searchQuery, setSearchQuery] = useState("");
  const [selectedClass, setSelectedClass] = useState("ALL");
  const [selectedSection, setSelectedSection] = useState("ALL");
  const [selectedStudent, setSelectedStudent] = useState<Student | null>(null);
  const [calcResult, setCalcResult] = useState<StudentCalculationResult | null>(
    null,
  );

  const [tempScholarshipId, setTempScholarshipId] = useState("");
  const [tempDiscountId, setTempDiscountId] = useState("");
  const [isFineWaived, setIsFineWaived] = useState(false);
  const [expandedGroups, setExpandedGroups] = useState<Record<string, boolean>>({});

  const toggleGroupExpansion = (headName: string) => {
    setExpandedGroups((prev) => ({
      ...prev,
      [headName]: !prev[headName],
    }));
  };

  const handleWaiveFine = () => {
    setIsFineWaived(true);
    addToast(
      "info",
      "Late Fee Waived",
      "Waived late fee for this student collection.",
    );
  };

  const handleReapplyFine = () => {
    setIsFineWaived(false);
    addToast(
      "success",
      "Late Fee Re-applied",
      "Re-applied automatic late fee based on fine rule configuration.",
    );
  };

  const [selectedInstallments, setSelectedInstallments] = useState<string[]>(
    [],
  );

```

```

const [customCollectionAmounts, setCustomCollectionAmounts] = useState<
  Record<string, string>
>({});
const [paymentMode, setPaymentMode] = useState<
  FeePayment["paymentMode"] | ""
>("");
const [transactionId, setTransactionId] = useState("");
const [chequeNo, setChequeNo] = useState("");
const [chequeDate, setChequeDate] = useState("");
const [bankName, setBankName] = useState("");
const [remarks, setRemarks] = useState("");

const [showPaymentConfirmModal, setShowPaymentConfirmModal] = useState(false);
const [showPaymentHistoryModal, setShowPaymentHistoryModal] = useState(false);
const [isPreviousDuesOpen, setIsPreviousDuesOpen] = useState(true);
const [applyReturnCredit, setApplyReturnCredit] = useState(true);

const availableReturnCredit = (() => {
  if (!selectedStudent) return 0;
  const sId = selectedStudent.id;
  const admNo = selectedStudent.admissionNo;
  const sName =
    `${selectedStudent.firstName || ""} ${selectedStudent.lastName || ""}`
      .trim()
      .toLowerCase();

  const returnedPaidIssues = (studentUniformIssues || []).filter((i) => {
    const isStudentMatch = Boolean(
      (i.studentId && (i.studentId === sId || (admNo && i.studentId === admNo))) ||
      (i.admissionNo && (i.admissionNo === sId || (admNo && i.admissionNo === admNo))) ||
      (i.studentName && sName && (
        i.studentName.toLowerCase().trim() === sName ||
        i.studentName.toLowerCase().includes(sName) ||
        sName.includes(i.studentName.toLowerCase().trim())
      ))
    );
  });

  const notesLower = (i.notes || "").toLowerCase();
  if (!isStudentMatch || (i.status !== "Returned" && !notesLower.includes("returned"))) return false;

  const isExplicitlyPaidNote =
    (notesLower.includes("fees paid") ||
      notesLower.includes("paid at counter") ||
      notesLower.includes("already paid")) &&
    !notesLower.includes("unpaid") &&
    !notesLower.includes("not paid") &&
    !notesLower.includes("to be paid") &&
    !notesLower.includes("pending");

  const isPaidInFinance = Boolean(
    (i as any).wasPaid ||
    (i as any).previousStatus === "Paid" ||
    isExplicitlyPaidNote ||
    (feePayments || []).some((p) => {
      const isStd =
        p.studentId === sId ||
        (admNo && p.studentId === admNo) ||
        (p.receiptNo && ((admNo && p.receiptNo.includes(admNo)) || (sId && p.receiptNo.includes(sId))))
    })
  );

  if (!isStd || !p.amountPaid || p.amountPaid <= 0) return false;

```

```

const instId1 = `INST-UNIF-EXTRA-${i.id}`;
const instId2 = `FEE-UNI-EXTRA-${i.id}`;
const instId3 = `INST-UNIF-${i.id}`;

let isLegacyMatch = false;
const match = i.id.match(/-P(\d+)$/);
if (match) {
  const strippedId = i.id.replace(/-P\d+$/, '');
  const idx = parseInt(match[1], 10);
  const coversLegacy = p.amountPaid >= (i.price || i.unitPrice || 0) * idx;
  if (coversLegacy && (
    p.selectedInstallmentIds?.includes(`INST-UNIF-EXTRA-${strippedId}`) ||
    p.selectedInstallmentIds?.includes(`FEE-UNI-EXTRA-${strippedId}`) ||
    p.selectedInstallmentIds?.includes(`INST-UNIF-${strippedId}`) ||
    p.selectedInstallmentIds?.includes(strippedId) ||
    (p.receiptNo && (p.receiptNo.includes(`UNI-EXTRA-${strippedId}`) || p.receiptNo.includes(strippedId)))
  )) {
    isLegacyMatch = true;
  }
}

if (
  p.selectedInstallmentIds?.includes(instId1) ||
  p.selectedInstallmentIds?.includes(instId2) ||
  p.selectedInstallmentIds?.includes(instId3) ||
  p.selectedInstallmentIds?.includes(i.id) ||
  (p.receiptNo && (p.receiptNo.includes(`UNI-EXTRA-${i.id}`) || p.receiptNo.includes(i.id))) ||
  isLegacyMatch
) {
  return true;
}

if (p.paymentAllocation && p.paymentAllocation.length > 0) {
  return p.paymentAllocation.some((alloc) => {
    const head = String(alloc.feeHeadName || alloc.termName || (alloc as any).feeHeadId || "").toLowerCase();
    const itemLower = (i.itemName || i.itemCategory || "").toLowerCase().replace(/s*(extra)/g, "").trim();
    const allocInstId = String((alloc as any).installmentId || (alloc as any).feeHeadId || "");
    if (allocInstId === instId1 || allocInstId === instId2 || allocInstId === instId3 || allocInstId === i.id) return true;
    return Boolean(itemLower && itemLower.length > 3 && head.includes(itemLower));
  });
}

return false;
});
});

return isPaidInFinance;
});

if (returnedPaidIssues.length === 0) return 0;

return returnedPaidIssues.reduce((sum, item) => {
  const isPkg =
    item.type === "Base Package" ||
    (item.itemName &&
      (item.itemName.toLowerCase().includes("package") ||
        item.itemName.toLowerCase().includes("base")));
  // Use the stored price first – this is the most accurate value

```

```

let unitPrice = item.price && item.price > 0 && item.price !== 35 && item.price !== 85 ? item.price :
0;

if (unitPrice <= 0) {
  if (isPkg) {
    unitPrice =
      getUniformFeeForClass(
        selectedStudent.className,
        selectedStudent.gender,
        financeUniformConfigs,
      ) || 0;
  } else {
    // Strip (#N) suffix when looking up finance config price
    const cleanItemName = (item.itemName || "").replace(/\\(#\\d+\\)/g, "").trim();
    const catItem = (studentUniformIssues || []).find(
      (u) => u.category === cleanItemName || u.name === cleanItemName,
    );
    unitPrice =
      getItemFeeFromFinanceConfig(
        selectedStudent.className,
        cleanItemName,
        selectedStudent.gender,
        financeUniformConfigs,
        catItem?.price,
      ) || 0;
  }
}
return sum + unitPrice * (item.quantity || 1);
}, 0);
})();

const handlePaymentModeChange = (mode: FeePayment["paymentMode"] | "") => {
  setPaymentMode(mode);
  if (mode === "Cash") {
    setTransactionId("");
    setChequeNo("");
    setChequeDate("");
    setBankName("");
  } else if (mode === "Online" || mode === "Card") {
    setChequeNo("");
    setChequeDate("");
    setBankName("");
  } else if (mode === "Cheque") {
    setTransactionId("");
  }
};

// Use real enrolled students master roster, supplementing with non-duplicated admissions if any
const allEnrolledStudents = React.useMemo(() => {
  const map = new Map<string, Student>();

  // 1. Primary source for enrolled students: Enrolled admissions (e.g. REG-2044)
  (admissions || []).
    .filter((adm) => adm && (adm.status === "Enrolled" || adm.status === "Approved"))
    .forEach((adm) => {
      const admId = adm.id || adm.applicationNo || adm.registrationNo;
      const admNo = adm.registrationNo || adm.applicationNo || adm.id;
      if (!admNo) return;
      const fullName = (adm.applicantName || adm.studentName || `${adm.firstName || ''} ${adm.lastName || ''}`).trim();
      if (!fullName) return;

      const nameKey = fullName.toLowerCase().replace(/^[a-z0-9]/g, '');
      const nameParts = fullName.split(" ");

```

```

const fName = adm.firstName || nameParts[0] || "Student";
const lName = adm.lastName || nameParts.slice(1).join(" ") || "";
const targetCls = adm.appliedClass || adm.targetClass || adm.className || "Class 1";

map.set(nameKey, {
  id: String(admId),
  firstName: fName,
  lastName: lName,
  admissionNo: admNo,
  className: targetCls,
  section: adm.section || "A",
  gender: adm.gender || "Male",
  studentType: (adm.residentialStatus === "Residential" || adm.studentType === "Residential") ? "Homeschooler" : "Day Scholar",
  joiningDate: adm.admissionDate || new Date().toISOString().split("T")[0],
  dueFee: 0,
  paidFee: 0,
  totalFee: 0,
  rollNo: "0",
  fatherName: adm.parentName || adm.fatherName || "",
  motherName: adm.motherName || "",
  mobile: adm.mobile || adm.phone || "",
  email: adm.email || "",
  address: adm.address || "",
  status: "Active",
  academicYear: adm.academicYear || "2026-2027",
  branch: adm.branch || "Main Campus",
  selectedOptionalFees: adm.selectedOptionalFees || []
} as unknown as Student);
});

// 2. Secondary source: Master database students, skipping legacy dummy ADM-2026-2020 or students already present
(students || [])
  .filter(s => s && (!s.status || s.status === "Active" || s.status === "Enrolled"))
  .forEach((s) => {
    if (!s) return;
    const admNoUpper = (s.admissionNo || s.id || "").toUpperCase();
    if (admNoUpper === "ADM-2026-2020") return;

    const fullName = `${s.firstName || ''} ${s.lastName || ''}`.trim() || (s as any).studentName || '';
    const nameKey = fullName.toLowerCase().replace(/^[a-z0-9]/g, '');
    if (nameKey && map.has(nameKey)) return;

    const admKey = (s.admissionNo || s.id || "").toLowerCase().trim();
    if (admKey) {
      map.set(admKey, s);
    }
  });

return Array.from(map.values());
}, [students, admissions]);

const uniqueClassNames = React.useMemo(() => {
  const set = new Set<string>();
  allEnrolledStudents.forEach((s) => {
    if (s.className) set.add(String(s.className).trim());
  });
  return Array.from(set).sort((a, b) => {
    const numA = parseInt(a.replace(/\D/g, ""), 10);
    const numB = parseInt(b.replace(/\D/g, ""), 10);
    if (!isNaN(numA) && !isNaN(numB)) return numA - numB;
    return a.localeCompare(b);
  });
});

```

```

    });
  }, [allEnrolledStudents]);

const uniqueSections = React.useMemo(() => {
  const set = new Set<string>();
  allEnrolledStudents.forEach((s) => {
    if (s.section) set.add(String(s.section).trim());
  });
  return Array.from(set).sort();
}, [allEnrolledStudents]);

const filteredStudents = React.useMemo(() => {
  const q = searchQuery.toLowerCase().trim();
  return allEnrolledStudents.filter((s) => {
    const nameMatch =
      !q ||
      `${s.firstName} ${s.lastName}`.toLowerCase().includes(q) ||
      (s.admissionNo || "").toLowerCase().includes(q) ||
      (s.id || "").toLowerCase().includes(q);

    const classMatch =
      selectedClass === "ALL" ||
      matchesClassName(s.className, selectedClass);

    const sectionMatch =
      selectedSection === "ALL" ||
      String(s.section || "").toLowerCase() === selectedSection.toLowerCase();

    return nameMatch && classMatch && sectionMatch;
  });
}, [allEnrolledStudents, searchQuery, selectedClass, selectedSection]);

const updateCalculation = (
  studentId: string,
  freshCalcResult = calculateStudentPayableFee(studentId),
  ledgerOverride?: StudentFeeLedger,
) => {
  let calc = freshCalcResult;
  const ledger = ledgerOverride || getStudentFeeLedger(studentId);

  // Fallback fee calculation if freshCalcResult is null
  if (!calc) {
    const stObj =
      allEnrolledStudents.find(
        (s) => s?.id === studentId || s?.admissionNo === studentId,
      ) || selectedStudent;
    const dfs = (dynamicFeeStructures || []).find((d) =>
      matchesClassName(d.className, stObj?.className),
    );
    const feeVal = dfs?.totalAmount ?? 0;
    calc = {
      baseFee: feeVal,
      transportFee: 0,
      hostelFee: 0,
      fineAmount: 0,
      scholarshipDeduction: 0,
      discountDeduction: 0,
      totalPayable: feeVal,
      assignedFeeHeads: [],
    } as any;
  }

  if (ledger && calc) {

```

```

const scholarshipAmt =
  ledger.scholarshipAmount || ledger.totalScholarship || 0;
const discountAmt = ledger.discountAmount || ledger.totalDiscount || 0;

const fineAmt = isFineWaived ? 0 : (calc.fineAmount || 0);
const grossAmt =
  (calc.baseFee || 0) +
  (calc.transportFee || 0) +
  (calc.hostelFee || 0) +
  (calc.uniformFee || 0);
const updatedTotalPayable = Math.max(
  0,
  grossAmt + fineAmt - scholarshipAmt - discountAmt,
);
const updatedDueBalance = Math.max(
  0,
  updatedTotalPayable - (calc.paidAmount || 0),
);

calc = {
  ...calc,
  scholarshipId: ledger.scholarshipId,
  scholarshipName: ledger.scholarshipName,
  scholarshipDescription: ledger.scholarshipDescription,
  scholarshipDeduction: scholarshipAmt,
  discountId: ledger.discountId,
  discountName: ledger.discountName,
  discountDescription: ledger.discountDescription,
  discountDeduction: discountAmt,
  totalPayable: updatedTotalPayable,
  dueBalance: updatedDueBalance,
};
}
setCalcResult(calc);
if (calc && studentId) {
  setSelectedInstallments([]);
  setCustomCollectionAmounts({});
}
};

const handleSelectStudent = (st: Student) => {
  setSelectedStudent(st);
  setIsFineWaived(false);
  setTempScholarshipId("");
  setTempDiscountId("");
  setSelectedInstallments([]);
  setCustomCollectionAmounts({});
  updateCalculation(st.id);
};

const lastInitialStudentIdRef = useRef<string | null>(null);

useEffect(() => {
  if (initialStudent && (initialStudent.id || initialStudent.admissionNo)) {
    const studentKey = initialStudent.id || initialStudent.admissionNo;
    if (lastInitialStudentIdRef.current !== studentKey) {
      lastInitialStudentIdRef.current = studentKey;
      const matched =
        allEnrolledStudents.find(
          (s) =>
            s.id === initialStudent.id ||
            (initialStudent.admissionNo &&
              s.admissionNo === initialStudent.admissionNo),
        );
    }
  }
});

```

```
    ) || initialStudent;
    handleSelectStudent(matched);
  }
} else if (!initialStudent) {
  lastInitialStudentIdRef.current = null;
}
}, [initialStudent, allEnrolledStudents]));

const handleApplyScholarship = (scholarshipId: string) => {
  if (!selectedStudent || !calcResult) return;
  try {
    const updatedLedger = applyScholarshipToStudent(
      selectedStudent.id,
      scholarshipId,
    );
    updateCalculation(selectedStudent.id, undefined, updatedLedger);
    setTempScholarshipId("");
    addToast(
      "success",
      "Scholarship Applied",
      "Successfully applied scholarship.",
    );
  } catch (err: any) {
    addToast(
      "warning",
      "Already Applied",
      err.message || "Scholarship has already been applied.",
    );
  }
};

const handleRemoveScholarship = () => {
  if (!selectedStudent) return;
  try {
    const updatedLedger = removeScholarshipFromStudent(selectedStudent.id);
    updateCalculation(selectedStudent.id, undefined, updatedLedger);
    setTempScholarshipId("");
    addToast("info", "Scholarship Removed", "Removed scholarship.");
  } catch (err: any) {
    addToast(
      "warning",
      "Error",
      err.message || "Failed to remove scholarship.",
    );
  }
};

const handleApplyDiscount = (discountId: string) => {
  if (!selectedStudent || !calcResult) return;
  try {
    const updatedLedger = applyDiscountToStudent(
      selectedStudent.id,
      discountId,
    );
    updateCalculation(selectedStudent.id, undefined, updatedLedger);
    setTempDiscountId("");
    addToast("success", "Discount Applied", "Successfully applied discount.");
  } catch (err: any) {
    addToast(
      "warning",
      "Already Applied",
      err.message || "Discount has already been applied.",
    );
  }
};
```

```

    }
  };

  const handleRemoveDiscount = () => {
    if (!selectedStudent) return;
    try {
      const updatedLedger = removeDiscountFromStudent(selectedStudent.id);
      updateCalculation(selectedStudent.id, undefined, updatedLedger);
      setTempDiscountId("");
      addToast("info", "Discount Removed", "Removed discount.");
    } catch (err: any) {
      addToast("warning", "Error", err.message || "Failed to remove discount.");
    }
  };

  const handleCustomAmountChange = (id: string, valStr: string) => {
    if (valStr === "" || /^\\d*\\.?\\d*$/.test(valStr)) {
      setCustomCollectionAmounts((prev) => ({
        ...prev,
        [id]: valStr,
      }));
    }
  };

  const getInstallmentCollectionAmount = (inst: any): number => {
    if (!selectedInstallments.includes(inst.id)) return 0;
    const valStr = customCollectionAmounts[inst.id];
    if (valStr === undefined || valStr === "") return inst.dueAmount;
    const num = Number(valStr);
    return isNaN(num) ? 0 : num;
  };

  const getInstallmentAmountError = (inst: any): string | null => {
    if (!selectedInstallments.includes(inst.id)) return null;
    const valStr = customCollectionAmounts[inst.id];
    if (valStr === undefined) return null;
    if (valStr.trim() === "") return "Please enter collection amount.";
    const num = Number(valStr);
    if (isNaN(num) || num <= 0)
      return "Collection amount must be greater than ₹0.";
    if (num > inst.dueAmount)
      return `Collection amount cannot exceed pending amount of ${formatCurrency(inst.dueAmount)}.`;
    return null;
  };

  const handleSubmitPayment = (e: React.SyntheticEvent) => {
    e.preventDefault();
    if (!selectedStudent || !calcResult) return;

    if (amountPaying <= 0) {
      addToast(
        "warning",
        "Selection Required",
        "Please select at least one pending fee component to collect.",
      );
      return;
    }

    if (!paymentMode || paymentMode.trim() === "") {
      addToast(
        "warning",
        "Payment Mode Required",
        "Please select a Payment Mode before issuing receipt.",
      );
    }
  };

```

```

    );
    return;
  }

  setShowPaymentConfirmModal(true);
};

const executeProcessPayment = () => {
  if (!selectedStudent || !calcResult) return;
  setShowPaymentConfirmModal(false);

  const numericAmount = amountPaying;

  const paymentAllocations = selectedInstallments.map((id) => {
    const inst = allInstallments.find((i: any) => i.id === id);
    const customAmt = inst ? getInstallmentCollectionAmount(inst) : 0;
    return {
      academicYear: inst?.academicYear || currentYear,
      ledgerId: undefined,
      amount: customAmt,
      installmentId: id,
      feeHeadName: inst?.feeHeadName || "Fee",
      termName: inst?.termName || inst?.termId || "Installment",
    };
  });

  const payment = addFeePayment({
    studentId: selectedStudent.id,
    studentName: `${selectedStudent.firstName} ${selectedStudent.lastName}`,
    className: `${selectedStudent.className}-${selectedStudent.section}`,
    amountPaid: numericAmount,
    discount: calcResult.scholarshipDeduction + calcResult.discountDeduction,
    fine: isFineWaived ? 0 : calcResult.fineAmount,
    paymentMode: paymentMode as FeePayment["paymentMode"],
    transactionId:
      paymentMode === "Cheque"
        ? chequeNo
        : paymentMode !== "Cash"
        ? transactionId
        : undefined,
    chequeNo: paymentMode === "Cheque" ? chequeNo : undefined,
    chequeDate: paymentMode === "Cheque" ? chequeDate : undefined,
    bankName: paymentMode === "Cheque" ? bankName : undefined,
    paymentDate: new Date().toISOString().split("T")[0],
    status: numericAmount >= totalOutstanding ? "Paid" : "Partial",
    remarks,
    scholarshipId: calcResult.scholarshipId,
    scholarshipName: calcResult.scholarshipName,
    scholarshipDescription: calcResult.scholarshipDescription,
    scholarshipAmount: calcResult.scholarshipDeduction,
    discountId: calcResult.discountId,
    discountName: calcResult.discountName,
    discountDescription: calcResult.discountDescription,
    discountAmount: calcResult.discountDeduction,
    grossAmount:
      calcResult.baseFee + calcResult.transportFee + calcResult.hostelFee,
    previousDue: previousYearPending,
    selectedInstallmentIds: selectedInstallments,
    paymentAllocation: paymentAllocations,
  });

  // Mark uniform items as Paid in studentUniformIssues if collected
  if (updateStudentUniformIssue) {

```

```

selectedInstallments.forEach((instId) => {
  if (instId.startsWith("INST-UNIF-EXTRA-")) {
    const rawIssueId = instId.replace("INST-UNIF-EXTRA-", "");
    const targetIssue = (studentUniformIssues || []).find((i) => i.id === rawIssueId);
    if (targetIssue && !targetIssue.replacementDate && !targetIssue.notes?.toLowerCase().includes('ex
changed') && targetIssue.status !== 'Returned' && targetIssue.status !== 'Cancelled') {
      updateStudentUniformIssue(rawIssueId, {
        status: "Paid" as any,
        notes: `Fees Paid at Counter (${paymentMode}) – Receipt #${receiptNumber}`,
      });
    }
  } else if (instId.startsWith("INST-UNIF-BASE-") || instId.startsWith("INST-UNIF-") || instId === "F
H-04" || instId === "FH-UNI-BASE") {
    const cleanId = instId.replace("INST-UNIF-BASE-", "").replace("INST-UNIF-", "").split("-")[0];
    const baseIssue = (studentUniformIssues || []).find(
      (i) => (i.id === cleanId || i.studentId === selectedStudent.id || (selectedStudent.admissionNo
&& i.admissionNo === selectedStudent.admissionNo))
      && !i.replacementDate && !i.notes?.toLowerCase().includes('exchanged') && i.status !== 'Retur
ned' && i.status !== 'Cancelled'
    );
    if (baseIssue) {
      updateStudentUniformIssue(baseIssue.id, {
        status: "Paid" as any,
        notes: `Fees Paid at Counter (${paymentMode}) – Receipt #${receiptNumber}`,
      });
    } else if (addStudentUniformIssue) {
      addStudentUniformIssue({
        studentId: selectedStudent.id,
        studentName: `${selectedStudent.firstName} ${selectedStudent.lastName}`,
        admissionNo: selectedStudent.admissionNo || "ADM2026-000",
        className: selectedStudent.className || "Class 4",
        section: selectedStudent.section || "A",
        itemId: `pkg_${selectedStudent.className}`,
        itemName: `${selectedStudent.gender === 'Female' ? 'Girls' : 'Boys'} Base Package (Admission
Kit)`,
        size: "M",
        quantity: 1,
        issueDate: new Date().toISOString().split("T")[0],
        status: "Paid" as any,
        academicYear: selectedAcademicYear || financeSettings.academicYear || "2026-2027",
        type: "Base Package",
        price: paidAmount,
        notes: `Fees Paid at Counter (${paymentMode}) – Receipt #${receiptNumber}`,
      });
    }
  }
});
}

// Mark unpaid library fines as Paid in edu_db_library_fines if collected
try {
  const s = localStorage.getItem("edu_db_library_fines");
  if (s) {
    const fines = JSON.parse(s);
    const stAdm = selectedStudent.admissionNo || selectedStudent.id;
    const stFullName = `${selectedStudent.firstName} ${selectedStudent.lastName}`.toLowerCase();
    const updatedFines = fines.map((f: any) => {
      if (
        f.paymentStatus === "Unpaid" &&
        (f.memberId === selectedStudent.id ||
          f.memberId === stAdm ||
          (f.memberName || "").toLowerCase() === stFullName)
      ) {

```

```

        return {
            ...f,
            paymentStatus: "Paid",
            paidDate: new Date().toISOString().split("T")[0],
            remarks: "Paid via Admin Fee Collection",
        };
    }
    return f;
});
localStorage.setItem("edu_db_library_fines", JSON.stringify(updatedFines));
if (typeof window !== "undefined") {
    window.dispatchEvent(new Event("library_fines_updated"));
}
}
} catch {
    // ignore
}

addToast(
    "success",
    "Payment Processed",
    `Issued official receipt ${payment.receiptNo} for ${formatCurrency(numericAmount)}`,
);
onPrintReceipt(payment);

fetchFinanceData().catch(() => {});

setSelectedInstallments([]);
setCustomCollectionAmounts({});

// Immediate reactive feedback on calcResult
if (calcResult) {
    const newPaid = (calcResult.paidAmount || 0) + numericAmount;
    const newDue = Math.max(0, (calcResult.dueBalance ?? calcResult.totalPayable ?? 0) - numericAmount);
    setCalcResult({
        ...calcResult,
        paidAmount: newPaid,
        dueBalance: newDue,
    });
}

const updatedStudent =
    students.find(
        (s) =>
            s.id === selectedStudent.id ||
            (selectedStudent.admissionNo &&
                s.admissionNo === selectedStudent.admissionNo),
    ) || selectedStudent;
setSelectedStudent({
    ...updatedStudent,
    paidFee: (updatedStudent.paidFee || 0) + numericAmount,
    dueFee: Math.max(0, (updatedStudent.dueFee ?? calcResult?.dueBalance ?? numericAmount) - numericAmount),
});

updateCalculation(selectedStudent.id);

setPaymentMode("");
setTransactionId("");
setChequeNo("");
setChequeDate("");
setBankName("");
setRemarks("");

```

```

};

useEffect(() => {
  if (selectedStudent) {
    const freshCalc = calculateStudentPayableFee(selectedStudent.id);
    if (freshCalc) {
      setCalcResult((prev) => {
        if (!prev) return freshCalc;
        return {
          ...prev,
          paidAmount: freshCalc.paidAmount,
          dueBalance: freshCalc.dueBalance,
          totalPayable: freshCalc.totalPayable,
          paymentHistory: freshCalc.paymentHistory,
        };
      });
    }
  }
}, [feePayments, selectedStudent?.id]);

const getInstallmentStatus = (dueAmount: number, dueDate: string, instStatus?: string) => {
  if (instStatus === "Cancelled") return "CANCELLED";
  if (dueAmount <= 0) return "PAID";
  const todayStr = new Date().toISOString().split("T")[0];
  if (todayStr > dueDate) return "OVERDUE";
  return "UPCOMING";
};

const currentYear =
  selectedAcademicYear || financeSettings?.academicYear || "2026-2027";

const allInstallments = React.useMemo(() => {
  if (!selectedStudent) return [];

  // 1. Fetch ledger from getStudentFeeLedger which handles dynamic generation & admissions lookup
  const ledger = getStudentFeeLedger(selectedStudent.id, currentYear);
  let ledgerInstallments = ledger?.installments || [];

  // If ledgerInstallments is empty, construct installment checkboxes directly from ledger.feeItems
  if (
    ledgerInstallments.length === 0 &&
    ledger?.feeItems &&
    ledger.feeItems.length > 0
  ) {
    let remainingPaid = (feePayments || []).filter((p) => {
      const pSid = String(p.studentId || "").trim().toLowerCase();
      const sId = String(selectedStudent.id || "").trim().toLowerCase();
      const admNo = String(selectedStudent.admissionNo || "").trim().toLowerCase();
      return pSid === sId || (admNo && pSid === admNo);
    })
    .reduce((sum, p) => sum + (Number(p.amountPaid ?? (p as any).amount) || 0), 0);

    ledgerInstallments = ledger.feeItems
      .filter((item) => item.isApplicable && item.finalAmount > 0)
      .map((item, idx) => {
        const itemAmt = item.finalAmount > 0 ? item.finalAmount : item.originalAmount;
        const paidForThis = Math.min(itemAmt, remainingPaid);
        remainingPaid = Math.max(0, remainingPaid - paidForThis);
        const dueForThis = Math.max(0, itemAmt - paidForThis);

        return {
          id: `INST-FEE-${selectedStudent.id}-${item.headId || idx}`,

```

```

        studentId: selectedStudent.id,
        academicYear: currentYear,
        feeAssignmentId: `FA-LEAD-${selectedStudent.id}`,
        feeHeadId: item.headId || `FH-${idx}`,
        feeHeadName: item.headName,
        frequency: "One Time",
        termName: item.category || item.headName,
        dueDate: new Date().toISOString().split("T")[0],
        amount: item.originalAmount,
        paidAmount: paidForThis,
        dueAmount: dueForThis,
        status: dueForThis === 0 ? "Paid" : paidForThis > 0 ? "Partial" : "Pending",
        createdAt: new Date().toISOString(),
        updatedAt: new Date().toISOString(),
    });
});
}

const isResidentStudent =
    selectedStudent?.studentType === "Hosteller" ||
    (selectedStudent as any)?.studentType === "Residential" ||
    (selectedStudent as any)?.residentialStatus === "Residential" ||
    (selectedStudent as any)?.residentialStatus === "Resident";

const isNonResidentStudent = !isResidentStudent;

const admRecord = (admissions || []).find((a) => {
    if (!a) return false;
    const sId = selectedStudent.id ? String(selectedStudent.id).toLowerCase().trim() : '';
    const sAdm = selectedStudent.admissionNo ? String(selectedStudent.admissionNo).toLowerCase().trim() :
    '';
    const sName = `${selectedStudent.firstName || ''} ${selectedStudent.lastName || ''}`.toLowerCase().trim();

    const aId = a.id ? String(a.id).toLowerCase().trim() : '';
    const aApp = a.applicationNo ? String(a.applicationNo).toLowerCase().trim() : '';
    const aReg = a.registrationNo ? String(a.registrationNo).toLowerCase().trim() : '';
    const aName = (a.applicantName || (a as any).studentName || `${a.firstName || ''} ${a.lastName || ''}`
    `).toLowerCase().trim();

    if (sAdm && sAdm !== 'n/a' && (sAdm === aApp || sAdm === aReg || sAdm === aId)) return true;
    if (sId && sId !== '1' && sId !== 'stu-1' && sId !== 'n/a' && (sId === aId || sId === aApp || sId ===
    aReg)) return true;
    if (sName && sName !== 'student' && aName && (sName === aName || sName.includes(aName) || aName.inclu
    des(sName))) return true;
    return false;
});

const optList = admRecord ? admRecord.selectedOptionalFees : (selectedStudent as any)?.selectedOptional
Fees;

const isExplicitlyOptedIn = Boolean(
    (admRecord as any)?.uniformOpted === true ||
    (admRecord as any)?.isUniformOpted === true ||
    (selectedStudent as any)?.uniformOpted === true ||
    (selectedStudent as any)?.isUniformOpted === true ||
    (Array.isArray(optList) &&
        optList.some((id) => {
            const s = String(id).toLowerCase();
            return (
                s === "fh-04" ||
                s === "fh-004" ||
                s === "fh-uni-base" ||

```

```

        s.includes("uniform") ||
        s.includes("kit")
    );
    })))
);

const isExplicitlyOptedOut = !isExplicitlyOptedIn;

// 2. Filter baseline ledger installments based on Residential Status & Uniform Opt-In
const combined: StudentFeeInstallment[] = ledgerInstallments.filter(
    (inst) => {
        const headLower = (inst.feeHeadName || "").toLowerCase();
        const termLower = (inst.termName || "").toLowerCase();
        const catLower = ((inst as any).category || "").toLowerCase();

        const isHostel =
            inst.feeHeadId === "FH-HST" ||
            catLower.includes("hostel") ||
            headLower.includes("hostel") ||
            termLower.includes("hostel");

        const isTransport =
            inst.feeHeadId === "FH-TRP" ||
            catLower.includes("transport") ||
            headLower.includes("transport") ||
            termLower.includes("transport");

        // Rule 1: Non-Resident student -> DO NOT display Hostel Fee
        if (isNonResidentStudent && isHostel) {
            return false;
        }

        // Rule 2: Resident student -> DO NOT display Transport Fee
        if (isResidentStudent && isTransport) {
            return false;
        }

        // Rule 3: Hide non-applicable items (0 due amount & not applicable)
        if (!inst.isApplicable && inst.dueAmount <= 0) {
            return false;
        }

        // Rule 4: If student DID NOT OPT for uniform fee at admission, hide default baseline uniform fee
        const isUnifHead = inst.feeHeadId === "FH-04" || inst.feeHeadId === "FH-UNI-BASE" || headLower.includes("uniform");
        if (isUnifHead && !isExplicitlyOptedIn && !inst.id.startsWith("INST-UNIF-EXTRA-") && !inst.id.startsWith("INST-UNIF-UIS-")) {
            return false;
        }

        const isExtraItem =
            termLower.includes("extra") ||
            termLower.includes("shoes") ||
            termLower.includes("tracksuit") ||
            termLower.includes("belt") ||
            termLower.includes("blazer");
        return !isExtraItem;
    },
);

// 3. Build Base Package & Additional Purchases STRICTLY from active studentUniformIssues (Uniform Distribution)
const studentIssues = (studentUniformIssues || []).filter((issue) => {

```

```

    if (!issue) return false;
    const sId = selectedStudent.id ? String(selectedStudent.id).toLowerCase().trim() : '';
    const sAdm = selectedStudent.admissionNo ? String(selectedStudent.admissionNo).toLowerCase().trim() :
    '';

    const iId = issue.studentId ? String(issue.studentId).toLowerCase().trim() : '';
    const iAdm = issue.admissionNo ? String(issue.admissionNo).toLowerCase().trim() : '';

    const isForStudent = Boolean(
      (sAdm && sAdm !== 'n/a' && iAdm && iAdm === sAdm) ||
      (sId && sId !== '1' && sId !== 'stu-1' && sId !== 'n/a' && iId && iId === sId) ||
      (sAdm && sAdm !== 'n/a' && iId && iId === sAdm) ||
      (sId && sId !== '1' && sId !== 'stu-1' && sId !== 'n/a' && iAdm && iAdm === sId)
    );

    return (
      isForStudent &&
      (issue.status as any) !== "Returned" &&
      (issue.status as any) !== "Cancelled"
    );
  });

  const expectedBaseFee =
    getUniformFeeForClass(
      selectedStudent.className,
      selectedStudent.gender,
      financeUniformConfigs,
      feeStructures,
    ) || getUniformPackageFeeByClass(selectedStudent.className) || 0;

  // Update any existing ledger uniform installments with the exact configured uniform fee amount for the
  student's class
  if (expectedBaseFee && expectedBaseFee > 0) {
    combined.forEach((c) => {
      const headLower = (c.feeHeadName || "").toLowerCase();
      const isUnifHead =
        c.feeHeadId === "FH-04" ||
        c.feeHeadId === "FH-UNI-BASE" ||
        headLower.includes("uniform");
      if (isUnifHead && c.status !== "Paid" && !c.id.startsWith("INST-UNIF-UIS-") && !c.id.startsWith("IN
ST-UNIF-EXTRA-")) {
        c.amount = expectedBaseFee;
        c.dueAmount = Math.max(0, expectedBaseFee - (c.paidAmount || 0));
      }
    });
  }

  // A & B: Process all uniform issues for the student into distinct fee installment rows
  if (studentIssues.length > 0) {
    // Remove any generic placeholder base uniform fee from combined when explicit student issues exist
    for (let i = combined.length - 1; i >= 0; i--) {
      const c = combined[i];
      const headLower = (c.feeHeadName || "").toLowerCase();
      const isUnifHead =
        c.id.startsWith("INST-UNIF-BASE-") ||
        c.feeHeadId === "FH-04" ||
        c.feeHeadId === "FH-UNI-BASE" ||
        headLower.includes("uniform");
      if (isUnifHead && !c.id.startsWith("INST-UNIF-UIS-") && !c.id.startsWith("INST-UNIF-EXTRA-")) {
        combined.splice(i, 1);
      }
    }
  }

  studentIssues.forEach((issue) => {

```

```

const itemRawName = (issue.itemCategory || issue.itemName || '').toLowerCase();
const notesLowerCheck = (issue.notes || '').toLowerCase();
const isFabricCloth = itemRawName.includes('cloth') || itemRawName.includes('fabric') || itemRawName.includes('unstitched') || itemRawName.includes('material') || notesLowerCheck.includes('cloth') || notesLowerCheck.includes('fabric');

// Strictly define Base Package: ONLY Boys Base Package, Girls Base Package, or Admission Kit
// Non-package items like Tie, Sweater, Shoes, Socks, Belt, Tracksuit are ALWAYS Additional Purchase

const isBasePkg = !isFabricCloth && (
  issue.type === 'Base Package' ||
  (issue.type as any) === 'base' ||
  itemRawName.includes('boys base package') ||
  itemRawName.includes('girls base package') ||
  itemRawName === 'base package' ||
  itemRawName.includes('admission kit')
) && !itemRawName.includes('tie') && !itemRawName.includes('sweater') && !itemRawName.includes('shoe') && !itemRawName.includes('sock') && !itemRawName.includes('belt') && !itemRawName.includes('tracksuit');

const itemTitle = isFabricCloth ? (issue.itemName || issue.itemCategory || "Uniform Cloth") : (isBasePkg ? (itemRawName.includes('kit') ? "Admission Kit" : (issue.itemName || issue.itemCategory || "Uniform Fee")) : (issue.itemCategory || issue.itemName || "Uniform Item"));

const finalSizeStr = issue.size?.includes('->') ? issue.size.split('->')[1].trim() : (issue.size || 'M');

const configuredItemFee = calculateClothOrItemPrice(
  issue.itemName || issue.itemCategory,
  finalSizeStr,
  issue.price || issue.unitPrice,
  financeUniformConfigs,
  selectedStudent.className,
  selectedStudent.gender
);

const amt = configuredItemFee * (issue.quantity || 1);
const unitFee = configuredItemFee;
const calcQty = (issue.quantity && issue.quantity > 0) ? issue.quantity : 1;
const sizeVal = issue.size || 'M';
const genderVal = issue.gender || selectedStudent?.gender || 'Male';
const detailsStr = ` (Size: ${sizeVal} · ${genderVal})`;

const loopCount = isFabricCloth ? 1 : calcQty;

for (let qIdx = 0; qIdx < loopCount; qIdx++) {
  const subInstId = (loopCount > 1) ? `INST-UNIF-${issue.id}-P${qIdx + 1}` : `INST-UNIF-${issue.id}`;

  if (combined.some((c) => c.id === subInstId)) continue;

  const isFirstBase = isBasePkg && qIdx === 0;
  const prefix = isFabricCloth
    ? (issue.itemName || issue.itemCategory || "Uniform Cloth")
    : (isFirstBase ? "Base Package" : (isBasePkg ? "Additional Base Kit" : "Additional Purchase"));
  const fabricSizeStr = sizeVal ? ` (Size: ${sizeVal})` : '';
  const qtyTag = (isFabricCloth && calcQty > 1) ? ` × ${calcQty}` : ((!isFabricCloth && calcQty > 1) ? ` (#${qIdx + 1})` : '');
  const itemDisplayName = isFabricCloth
    ? `${prefix}${fabricSizeStr}${qtyTag}`.trim()
    : `${prefix} - ${itemTitle}${detailsStr}${qtyTag}`.trim();
  const subAmt = isFabricCloth ? amt : ((calcQty > 1 && amt > 0) ? Math.round(amt / calcQty) : amt);

  const notesLower = (issue.notes || '').toLowerCase();

```

```

    const isExplicitlyPaidNote = (notesLower.includes("fees paid") || notesLower.includes("paid at co
unter") || notesLower.includes("already paid")) &&
    !notesLower.includes("unpaid") && !notesLower.includes("not paid") && !notesLower.includes("to
be paid") && !notesLower.includes("pending");

    const subPaidAmt = (feePayments || []).filter(p => {
        if (p.studentId !== selectedStudent.id && p.studentId !== selectedStudent.admissionNo) return f
alse;

        if (!p.amountPaid || p.amountPaid <= 0) return false;
        const strippedId = issue.id.replace(/-P\d+$/, '');
        return (
            p.selectedInstallmentIds?.includes(subInstId) ||
            (qIdx === 0 && (
                p.selectedInstallmentIds?.includes(`INST-UNIF-${issue.id}`) ||
                p.selectedInstallmentIds?.includes(issue.id) ||
                p.receiptNo?.includes(`UNI-EXTRA-${issue.id}`) ||
                p.selectedInstallmentIds?.includes(`INST-UNIF-${strippedId}`) ||
                p.selectedInstallmentIds?.includes(strippedId) ||
                p.receiptNo?.includes(`UNI-EXTRA-${strippedId}`)
            ))
        );
    }).reduce((acc, curr) => acc + curr.amountPaid, 0);

    const isPaidInFinance = subPaidAmt >= subAmt || ((issue.status as string) === "Paid") || isExplic
itlyPaidNote;

    combined.push({
        id: subInstId,
        studentId: selectedStudent.id,
        academicYear: currentYear,
        feeAssignmentId: `FA-UNIF-${selectedStudent.id}`,
        feeHeadId: 'FH-04',
        feeHeadName: "Uniform & Accessories",
        frequency: "One Time",
        termName: itemDisplayName,
        dueDate: issue.issueDate || new Date().toISOString().split("T")[0],
        amount: subAmt,
        paidAmount: isPaidInFinance ? subAmt : 0,
        dueAmount: isPaidInFinance ? 0 : subAmt,
        status: isPaidInFinance ? "Paid" : "Pending",
        createdAt: issue.issueDate || new Date().toISOString(),
        updatedAt: new Date().toISOString(),
    });
}
});
} else {
    // No explicit uniform issues yet recorded – check if opted at admission
    const admRecord = (admissions || []).find(a => a.id === selectedStudent.id || a.applicationNo === sel
ectedStudent.id || (selectedStudent.admissionNo && (a.id === selectedStudent.admissionNo || a.applicationNo
=== selectedStudent.admissionNo)));
    const optList = admRecord ? admRecord.selectedOptionalFees : null;
    const isExplicitlyOptedOut = Boolean(
        (admRecord as any)?.uniformOpted === false ||
        (admRecord as any)?.isUniformOpted === false ||
        (selectedStudent as any)?.uniformOpted === false ||
        (selectedStudent as any)?.isUniformOpted === false ||
        (Array.isArray(optList) && optList.length > 0 && !optList.some(id => id === 'FH-04' || id === 'FH-0
04' || String(id).toLowerCase().includes('uniform') || String(id).toLowerCase().includes('kit')))
    );

    const hasPaidBaseInFinance = (feePayments || []).some((p) => {
        const isStudentMatch =
            p.studentId === selectedStudent.id ||

```

```

    (selectedStudent.admissionNo && p.studentId === selectedStudent.admissionNo);
    if (!isStudentMatch || !p.amountPaid || p.amountPaid <= 0) return false;
    if (p.paymentAllocation && p.paymentAllocation.length > 0) {
        return p.paymentAllocation.some(alloc => {
            const head = (alloc.feeHeadName || alloc.termName || '').toLowerCase();
            return (head.includes('uniform') || head.includes('package')) && !head.includes('extra');
        });
    }
    return false;
});

const shouldIncludeUniformFee = isExplicitlyOptedIn && expectedBaseFee > 0;

if (shouldIncludeUniformFee && !hasPaidBaseInFinance) {
    const baseUnitFee = (expectedBaseFee && expectedBaseFee > 0) ? expectedBaseFee : 5000;
    const calcQty = (expectedBaseFee > 0 && expectedBaseFee % baseUnitFee === 0 && expectedBaseFee > ba
seUnitFee)
        ? Math.round(expectedBaseFee / baseUnitFee)
        : 1;
    const genderVal = selectedStudent?.gender || 'Male';
    const sizeVal = 'M';
    const detailsStr = ` (Size: ${sizeVal} · ${genderVal})`;

    // Remove any generic placeholder uniform fee terms (e.g. Annual or FH-04) that aren't explicit iss
ued items
    for (let i = combined.length - 1; i >= 0; i--) {
        const c = combined[i];
        if (!c.id.startsWith("INST-UNIF-")) {
            const headLower = (c.feeHeadName || "").toLowerCase();
            const isUnifHead =
                c.feeHeadId === "FH-04" ||
                c.feeHeadId === "FH-UNI-BASE" ||
                headLower.includes("uniform");
            if (isUnifHead && c.status !== "Paid") {
                combined.splice(i, 1);
            }
        }
    }

    for (let qIdx = 0; qIdx < calcQty; qIdx++) {
        const subInstId = calcQty > 1 ? `INST-UNIF-BASE-${selectedStudent.id}-P${qIdx + 1}` : `INST-UNIF-
BASE-${selectedStudent.id}`;
        if (combined.some(c => c.id === subInstId)) continue;

        const isFirst = qIdx === 0;
        const prefix = isFirst ? "Base Package" : "Additional Base Kit";
        const numTag = calcQty > 1 ? ` (${qIdx + 1})` : '';
        const baseTermName = `${prefix} - Admission Kit${detailsStr}${numTag}`;

        const subPaidAmt = (feePayments || []).filter(p => {
            if (p.studentId !== selectedStudent.id && p.studentId !== selectedStudent.admissionNo) return f
alse;

            if (!p.amountPaid || p.amountPaid <= 0) return false;
            return (
                p.selectedInstallmentIds?.includes(subInstId) ||
                (calcQty === 1 && (p.selectedInstallmentIds?.includes(`INST-UNIF-BASE-${selectedStudent.id}`)
|| p.selectedInstallmentIds?.includes('FH-04') || p.selectedInstallmentIds?.includes('FH-UNI-BASE')))
            );
        }).reduce((acc, curr) => acc + curr.amountPaid, 0);

        const isSubPaid = subPaidAmt >= baseUnitFee;

        combined.push({

```

```

        id: subInstId,
        studentId: selectedStudent.id,
        academicYear: currentYear,
        feeAssignmentId: `FA-UNIF-BASE-${selectedStudent.id}`,
        feeHeadId: "FH-04",
        feeHeadName: "Uniform & Accessories",
        frequency: "One Time",
        termName: baseTermName,
        dueDate: new Date().toISOString().split("T")[0],
        amount: baseUnitFee,
        paidAmount: isSubPaid ? baseUnitFee : 0,
        dueAmount: isSubPaid ? 0 : baseUnitFee,
        status: isSubPaid ? "Paid" : "Pending",
        createdAt: new Date().toISOString(),
        updatedAt: new Date().toISOString(),
    });
}
} else {
    // Remove pending base uniform package fee charge for un-opted students without distributed items
    for (let i = combined.length - 1; i >= 0; i--) {
        const c = combined[i];
        if (!c.id.startsWith("INST-UNIF-")) {
            const headLower = (c.feeHeadName || "").toLowerCase();
            const isUnifHead =
                c.feeHeadId === "FH-04" ||
                c.feeHeadId === "FH-UNI-BASE" ||
                headLower.includes("uniform");
            if (isUnifHead && c.status !== "Paid") {
                combined.splice(i, 1);
            }
        }
    }
}
}
}

// 3. Fallback: If still empty, build default fee structure installments for the student's class
if (combined.length === 0 && selectedStudent) {
    const clsName = selectedStudent.className || "";
    const dfs =
        (dynamicFeeStructures || []).find(
            (d) => matchesClassName(d.className, clsName) && (d.status === "Active" || !d.status),
        ) ||
        (dynamicFeeStructures || []).find(
            (d) => matchesClassName(d.className, clsName),
        );

    let defaultHeads: { id: string; name: string; amount: number }[] = [];
    if (dfs && dfs.items && dfs.items.length > 0) {
        defaultHeads = dfs.items.map((i) => ({
            id: i.feeHeadId || `FH-${i.feeHeadName}`,
            name: i.feeHeadName,
            amount: i.amount || 0,
        }));
    } else {
        const applicableHeads = (feeHeads || []).filter((h) =>
            h.status === "Active" &&
            (!h.applicableClasses ||
                h.applicableClasses.length === 0 ||
                h.applicableClasses.some((c) => matchesClassName(c, clsName)) ||
                h.applicableClasses.includes("All")),
        );
        defaultHeads = applicableHeads.map((h) => ({
            id: h.id,

```

```

        name: h.name,
        amount: h.amount || 0,
    }));
}

defaultHeads.forEach((h) => {
    combined.push({
        id: `INST-AUTO-${selectedStudent.id}-${h.id}`,
        studentId: selectedStudent.id,
        academicYear: currentYear,
        feeAssignmentId: `FA-AUTO-${selectedStudent.id}`,
        feeHeadId: h.id,
        feeHeadName: h.name,
        frequency: "One Time",
        termName: h.name,
        dueDate: new Date().toISOString().split("T")[0],
        amount: h.amount,
        paidAmount: 0,
        dueAmount: h.amount,
        status: "Pending",
        createdAt: new Date().toISOString(),
        updatedAt: new Date().toISOString(),
    });
});
});
}

// Synchronize individual uniform fee payment statuses precisely per line item
combined.forEach((inst) => {
    const isUnifHead =
        (inst.feeHeadName || "").toLowerCase().includes("uniform") ||
        inst.feeHeadId === "FH-04";
    if (!isUnifHead || inst.status === "Paid") return;

    const isPaidInReceipts = (feePayments || []).some((p) => {
        const isStudentMatch =
            p.studentId === selectedStudent.id ||
            (selectedStudent.admissionNo &&
                (p.studentId === selectedStudent.admissionNo ||
                    (p.receiptNo &&
                        p.receiptNo.includes(selectedStudent.admissionNo))));
        if (!isStudentMatch || !p.amountPaid || p.amountPaid <= 0) return false;

        // Check if receipt specifically paid this installment ID
        if (
            p.selectedInstallmentIds &&
            (p.selectedInstallmentIds.includes(inst.id) ||
                (inst.id.startsWith("INST-UNIF-") &&
                    p.selectedInstallmentIds.includes(
                        inst.id.replace("INST-UNIF-", ""),
                    )))
        ) {
            return true;
        }

        // Check if receiptNo contains this item's specific ID
        if (
            p.receiptNo &&
            inst.id.startsWith("INST-UNIF-") &&
            p.receiptNo.includes(inst.id.replace("INST-UNIF-", ""))
        ) {
            return true;
        }
    });
});

```

```

// Check allocation by specific item name
if (p.paymentAllocation && p.paymentAllocation.length > 0) {
  return p.paymentAllocation.some((alloc) => {
    if (alloc.installmentId === inst.id) return true;
    const allocName = (alloc.termName || alloc.feeHeadName || "")
      .toLowerCase()
      .trim();
    const instTerm = (inst.termName || "").toLowerCase().trim();
    if (!allocName || !instTerm) return false;

    // Match specific items precisely (e.g. Sports Dress, Sports Tracksuit Kit, Base Package)
    if (
      allocName.includes("sports dress") &&
      instTerm.includes("sports dress")
    )
      return true;
    if (
      allocName.includes("tracksuit") &&
      instTerm.includes("tracksuit")
    )
      return true;
    if (
      (allocName.includes("base package") ||
        allocName.includes("admission kit")) &&
      (instTerm.includes("base package") ||
        instTerm.includes("admission kit"))
    )
      return true;
    if (allocName === instTerm) return true;
    return false;
  });
}
return false;
});

if (isPaidInReceipts) {
  inst.paidAmount = inst.amount;
  inst.dueAmount = 0;
  inst.status = "Paid";
}
});

// Deduplicate Base Uniform terms: Ensure strictly 1 base uniform term exists under Uniform & Accessori
es
const baseIndices: number[] = [];
combined.forEach((c, idx) => {
  const termLower = (c.termName || "").toLowerCase();
  const headLower = (c.feeHeadName || "").toLowerCase();
  const isExplicitExtra =
    termLower.includes("extra") ||
    termLower.includes("sports") ||
    termLower.includes("tracksuit") ||
    termLower.includes("cap") ||
    termLower.includes("shoes") ||
    termLower.includes("socks") ||
    termLower.includes("shirt") ||
    termLower.includes("pant") ||
    termLower.includes("skirt") ||
    termLower.includes("tie") ||
    termLower.includes("belt");

  const isBaseUniform =
    !c.id.startsWith("INST-UNIF-") &&

```

```

    !isExplicitExtra &&
    (c.feeHeadId === "FH-UNI-BASE" ||
      termLower.includes("package") ||
      termLower.includes("admission kit") ||
      termLower.includes("annual"));

    if (isBaseUniform) {
      baseIndices.push(idx);
    }
  });

  if (baseIndices.length > 1) {
    const keepIndex =
      baseIndices.find((i) => combined[i].status === "Paid") ??
      baseIndices.find(
        (i) =>
          combined[i].termName.toLowerCase().includes("admission kit") ||
          combined[i].id.startsWith("INST-UNIF-"),
      ) ??
      baseIndices[0];

    for (let i = combined.length - 1; i >= 0; i--) {
      if (baseIndices.includes(i) && i !== keepIndex) {
        combined.splice(i, 1);
      }
    }
  }

  // Filter out any unpaid fee installments for returned/cancelled uniform items for this student
  const returnedStudentIssues = (studentUniformIssues || []).filter((issue) => {
    const isForStudent =
      (issue.studentId &&
        (issue.studentId === selectedStudent.id ||
          issue.studentId === selectedStudent.admissionNo)) ||
      (issue.admissionNo &&
        (issue.admissionNo === selectedStudent.id ||
          issue.admissionNo === selectedStudent.admissionNo)) ||
      (issue.studentName &&
        `${selectedStudent.firstName} ${selectedStudent.lastName}`
          .toLowerCase()
          .trim() === issue.studentName.toLowerCase().trim());

    return (
      isForStudent &&
      ((issue.status as any) === "Returned" ||
        (issue.status as any) === "Cancelled" ||
        (issue.notes || "").toLowerCase().includes("returned"))
    );
  });

  if (returnedStudentIssues.length > 0) {
    for (let i = combined.length - 1; i >= 0; i--) {
      const c = combined[i];
      const isUnifHead =
        c.feeHeadId === "FH-04" ||
        c.feeHeadId === "FH-UNI-BASE" ||
        c.feeHeadId === "FH-UNI-EXTRA" ||
        (c.feeHeadName || "").toLowerCase().includes("uniform");

      if (!isUnifHead) continue;

      const termLow = (c.termName || "").toLowerCase();

```

```

const matchesReturnedItem = returnedStudentIssues.some((ret) => {
  const notesLower = (ret.notes || "").toLowerCase();
  const isExplicitlyPaidNote =
    (notesLower.includes("fees paid") ||
     notesLower.includes("paid at counter") ||
     notesLower.includes("already paid")) &&
    !notesLower.includes("unpaid") &&
    !notesLower.includes("not paid") &&
    !notesLower.includes("to be paid") &&
    !notesLower.includes("pending");

  const isItemPaid =
    Boolean((ret as any).wasPaid) ||
    (ret.status as string) === "Paid" ||
    isExplicitlyPaidNote;

  // If paid, keep paid item installment in combined
  if (isItemPaid) return false;

  // If unpaid and returned: match by ID or item name
  if (c.id === ret.id || c.id.includes(ret.id)) return true;

  const itemClean = (ret.itemName || ret.itemCategory || "")
    .replace(/\s*(extra)/gi, "")
    .trim()
    .toLowerCase();

  if (itemClean && itemClean.length >= 2 && termLow.includes(itemClean)) {
    return true;
  }

  if (ret.type === "Additional Purchase" || ret.itemName) {
    const cleanCatName = (ret.itemName || "").toLowerCase().trim();
    if (cleanCatName && termLow.includes(cleanCatName)) return true;
  }

  return false;
});

if (matchesReturnedItem && c.status !== "Paid" && c.dueAmount > 0) {
  c.status = "Cancelled";
  c.dueAmount = 0;
  c.amount = 0;
}
}
}

return combined;
}, [
  selectedStudent,
  studentFeeLedgers,
  studentFeeInstallments,
  studentUniformIssues,
  selectedAcademicYear,
  financeSettings,
]);
const currentYearInstallments = allInstallments.filter(
  (i) => i.academicYear === currentYear,
);
const previousYearInstallments = allInstallments.filter(
  (i) => i.academicYear !== currentYear,
);

```

```
const currentYearPending = currentYearInstallments.reduce(
  (sum, i) => sum + i.dueAmount,
  0,
);
const previousYearPending = previousYearInstallments.reduce(
  (sum, i) => sum + i.dueAmount,
  0,
);
const totalOutstanding = currentYearPending + previousYearPending;

const selectedCurrentYearAmount = currentYearInstallments
  .filter((i) => selectedInstallments.includes(i.id))
  .reduce((sum, i) => sum + getInstallmentCollectionAmount(i), 0);

const selectedPreviousYearAmount = previousYearInstallments
  .filter((i) => selectedInstallments.includes(i.id))
  .reduce((sum, i) => sum + getInstallmentCollectionAmount(i), 0);

const amountPaying = selectedCurrentYearAmount + selectedPreviousYearAmount;

const hasAnyAmountError = selectedInstallments.some((id) => {
  const inst = allInstallments.find((i: any) => i.id === id);
  return inst ? Boolean(getInstallmentAmountError(inst)) : false;
});

const isAmountValid =
  selectedInstallments.length > 0 && amountPaying > 0 && !hasAnyAmountError;
const isModeSelected = Boolean(paymentMode);

let isModeFieldsValid = false;
let validationMessage = "";

if (selectedInstallments.length === 0) {
  validationMessage =
    "Select at least one pending fee component checkbox to collect";
} else if (hasAnyAmountError) {
  const errInstId = selectedInstallments.find((id) => {
    const inst = allInstallments.find((i: any) => i.id === id);
    return inst ? Boolean(getInstallmentAmountError(inst)) : false;
  });
  const errInst = allInstallments.find((i: any) => i.id === errInstId);
  validationMessage = errInst
    ? getInstallmentAmountError(errInst) || "Invalid collection amount"
    : "Invalid collection amount";
} else if (!isModeSelected) {
  validationMessage = "Please select a Payment Mode";
} else if (paymentMode === "Cash") {
  isModeFieldsValid = true;
} else if (
  paymentMode === "Online" ||
  paymentMode === "UPI" ||
  paymentMode === "Card" ||
  paymentMode === "Bank Transfer" ||
  paymentMode === "Other"
) {
  isModeFieldsValid = transactionId.trim().length > 0;
  if (!isModeFieldsValid)
    validationMessage = "Transaction Ref / UTR No is required";
} else if (paymentMode === "Cheque") {
  isModeFieldsValid =
    chequeNo.trim().length > 0 &&
    chequeDate.trim().length > 0 &&
    bankName.trim().length > 0;
```

```

    if (!isModeFieldsValid)
      validationMessage =
        "Cheque No, Cheque Date, and Bank Name are required for Cheque payment";
  }

  const isFormValid = isAmountValid && isModeSelected && isModeFieldsValid;

  const toggleInstallmentSelection = (id: string) => {
    setSelectedInstallments((prev) => {
      if (prev.includes(id)) {
        setCustomCollectionAmounts((amtPrev) => {
          const copy = { ...amtPrev };
          delete copy[id];
          return copy;
        });
        return prev.filter((x) => x !== id);
      } else {
        const inst = allInstallments.find((i: any) => i.id === id);
        if (inst) {
          setCustomCollectionAmounts((amtPrev) => ({
            ...amtPrev,
            [id]: String(inst.dueAmount),
          }));
        }
        return [...prev, id];
      }
    });
  };

  const handleSelectAllCurrentYear = () => {
    const pendingInsts = currentYearInstallments.filter((i) => i.dueAmount > 0);
    const pendingIds = pendingInsts.map((i) => i.id);
    const allSelected = pendingIds.every((id) =>
      selectedInstallments.includes(id),
    );
    if (allSelected) {
      setSelectedInstallments((prev) =>
        prev.filter((id) => !pendingIds.includes(id)),
      );
      setCustomCollectionAmounts((prev) => {
        const copy = { ...prev };
        pendingIds.forEach((id) => delete copy[id]);
        return copy;
      });
    } else {
      setSelectedInstallments((prev) =>
        Array.from(new Set([...prev, ...pendingIds])),
      );
      setCustomCollectionAmounts((prev) => {
        const copy = { ...prev };
        pendingInsts.forEach((i) => {
          if (!(i.id in copy)) {
            copy[i.id] = String(i.dueAmount);
          }
        });
      });
      return copy;
    }
  };

  const handleSelectAllPreviousYear = () => {
    const pendingInsts = previousYearInstallments.filter(
      (i) => i.dueAmount > 0,
    );
  }

```

```

    );
    const pendingIds = pendingInsts.map((i) => i.id);
    const allSelected = pendingIds.every((id) =>
      selectedInstallments.includes(id),
    );
    if (allSelected) {
      setSelectedInstallments((prev) =>
        prev.filter((id) => !pendingIds.includes(id)),
      );
      setCustomCollectionAmounts((prev) => {
        const copy = { ...prev };
        pendingIds.forEach((id) => delete copy[id]);
        return copy;
      });
    } else {
      setSelectedInstallments((prev) =>
        Array.from(new Set([...prev, ...pendingIds])),
      );
      setCustomCollectionAmounts((prev) => {
        const copy = { ...prev };
        pendingInsts.forEach((i) => {
          if (!(i.id in copy)) {
            copy[i.id] = String(i.dueAmount);
          }
        });
        return copy;
      });
    }
  };
};

const selectableCurrentIds = currentYearInstallments
  .filter((i) => i.dueAmount > 0)
  .map((i) => i.id);
const allCurrentSelected =
  selectableCurrentIds.length > 0 &&
  selectableCurrentIds.every((id) => selectedInstallments.includes(id));

const selectablePrevIds = previousYearInstallments
  .filter((i) => i.dueAmount > 0)
  .map((i) => i.id);
const allPreviousSelected =
  selectablePrevIds.length > 0 &&
  selectablePrevIds.every((id) => selectedInstallments.includes(id));

const groupedCurrentYear: {
  [headName: string]: typeof currentYearInstallments;
} = {};
currentYearInstallments.forEach((inst) => {
  let groupKey = inst.feeHeadName || "Fee";
  if (
    groupKey.toLowerCase().includes("hostel accommodation") ||
    groupKey.toLowerCase().includes("hostel security") ||
    groupKey.toLowerCase().includes("hostel deposit")
  ) {
    groupKey = "Hostel Fee";
  }
  if (!groupedCurrentYear[groupKey]) {
    groupedCurrentYear[groupKey] = [];
  }
  groupedCurrentYear[groupKey].push(inst);
});

const groupedPreviousYears: {

```

```

    [year: string]: typeof previousYearInstallments;
  } = {};
  previousYearInstallments.forEach((inst) => {
    if (!groupedPreviousYears[inst.academicYear]) {
      groupedPreviousYears[inst.academicYear] = [];
    }
    groupedPreviousYears[inst.academicYear].push(inst);
  });

  if (!selectedStudent || !calcResult) {
    return (
      <div className="space-y-4 animate-in fade-in">
        {/* Search & Filter Bar Card */}
        <div className="glass-card p-4 rounded-2xl">
          <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-3">
            <div className="relative col-span-1 sm:col-span-2">
              <Search className="w-4 h-4 text-slate-400 absolute left-3.5 top-3" />
              <input
                type="text"
                placeholder="Search student by name, admission no, or roll no..."
                value={searchQuery}
                onChange={(e) => setSearchQuery(e.target.value)}
                className="w-full pl-10 pr-4 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 text-xs font-medium text-slate-900 dark:text-white outline-none focus:ring-2 focus:ring-sky-500/20 transition-all"
              />
            </div>

            <div>
              <select
                value={selectedClass}
                onChange={(e) => setSelectedClass(e.target.value)}
                className="w-full px-3 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 text-xs font-bold text-slate-700 dark:text-slate-200 outline-none focus:ring-2 focus:ring-sky-500/20 cursor-pointer"
              >
                <option value="ALL">All Classes</option>
                {uniqueClassNames.map((cls) => (
                  <option key={cls} value={cls}>
                    {cls.startsWith("Class") ? cls : `Class ${cls}`}
                  </option>
                ))}
              </select>
            </div>

            <div>
              <select
                value={selectedSection}
                onChange={(e) => setSelectedSection(e.target.value)}
                className="w-full px-3 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 text-xs font-bold text-slate-700 dark:text-slate-200 outline-none focus:ring-2 focus:ring-sky-500/20 cursor-pointer"
              >
                <option value="ALL">All Sections</option>
                {uniqueSections.map((sec) => (
                  <option key={sec} value={sec}>
                    Section {sec}
                  </option>
                ))}
              </select>
            </div>
          </div>
        </div>
      </div>
    );
  }

```

```

    /* Full-Screen Student Table List */
    {filteredStudents.length === 0 ? (
      <div className="glass-card p-12 text-center rounded-2xl space-y-3">
        <Users className="w-10 h-10 text-slate-300 mx-auto" />
        <h3 className="font-bold text-slate-700 dark:text-slate-300 text-sm">
          No Students Found
        </h3>
        <p className="text-xs text-slate-400">
          Try adjusting your search query or class/section filters.
        </p>
      </div>
    ) : (
      <div className="glass-card rounded-2xl border border-slate-200/80 dark:border-slate-800 overflow-hidden shadow-xs">
        <table className="w-full text-left border-collapse text-xs">
          <thead>
            <tr className="bg-slate-100/80 dark:bg-slate-800/80 text-slate-500 dark:text-slate-400 font-extrabold uppercase text-[10px] tracking-wider border-b border-slate-200 dark:border-slate-700">
              <th className="p-3.5 pl-4">Admission No</th>
              <th className="p-3.5">Student Name</th>
              <th className="p-3.5">Class & Section</th>
              <th className="p-3.5">Student Type</th>
              <th className="p-3.5 text-right font-mono">
                Total Outstanding
              </th>
              <th className="p-3.5 text-center pr-4">Action</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100 dark:divide-slate-800/60 font-medium">
            {filteredStudents.map((st, stIdx) => {
              const isHosteller =
                st.studentType === "Hosteller" ||
                (st as any).studentType === "Residential" ||
                (st as any).residentialStatus === "Residential";

              const dueSummary = getStudentFeeOutstandingSummary
                ? getStudentFeeOutstandingSummary(st.id)
                : null;
              const dfs = (dynamicFeeStructures || []).find((d) =>
                matchesClassName(d.className, st.className),
              );
              const dueAmt = dueSummary !== null && dueSummary !== undefined
                ? dueSummary.totalOutstanding
                : (dfs?.totalAmount ?? 0);

              const displayClassStr = st.className
                ? st.className.toLowerCase().startsWith("class")
                  ? `${st.className}-${st.section}`
                  : `Class ${st.className}-${st.section}`
                : `Section ${st.section}`;

              return (
                <tr
                  key={`-${st.id}-${st.admissionNo} || ''}-${stIdx}`}
                  onClick={() => handleSelectStudent(st)}
                  className="hover:bg-sky-50/50 dark:hover:bg-slate-800/50 transition-colors cursor-pointer group">
                  <td className="p-3.5 pl-4 font-mono font-bold text-slate-800 dark:text-slate-200">
                    {st.admissionNo || st.id}
                  </td>
                  <td className="p-3.5">

```

```

        <div className="flex items-center gap-3">
          <div className="w-8 h-8 rounded-xl bg-gradient-to-br from-sky-500 to-brand-600 text-white font-black text-xs flex items-center justify-center shrink-0 shadow-xs">
            {st.firstName?.[0] || "S"}
          </div>
          <span className="font-extrabold text-slate-900 dark:text-white group-hover:text-sky-600 transition-colors">
            {st.firstName} {st.lastName}
          </span>
        </div>
      </td>
      <td className="p-3.5 font-bold text-slate-800 dark:text-slate-200">
        {displayClassStr}
      </td>
      <td className="p-3.5">
        <span
          className={`px-2.5 py-0.5 rounded-full text-[10px] font-extrabold uppercase ${
            isHosteller
              ? "bg-purple-100 text-purple-700 dark:bg-purple-950 dark:text-purple-300"
              : "bg-sky-100 text-sky-700 dark:bg-sky-950 dark:text-sky-300"
          }`}
        >
          {isHosteller ? "Residential" : "Day Scholar"}
        </span>
      </td>
      <td>
        <div
          className={`p-3.5 text-right font-mono text-xs ${
            dueAmt > 0
              ? "font-black text-rose-600 dark:text-rose-400"
              : "font-bold text-emerald-600 dark:text-emerald-400"
            }`}
        >
          {formatCurrency(dueAmt)}
        </div>
      <td className="p-3.5 text-center pr-4">
        <button
          type="button"
          onClick={e => {
            e.stopPropagation();
            handleSelectStudent(st);
          }}
          className="px-3.5 py-1.5 rounded-xl bg-sky-50 dark:bg-sky-950/60 text-sky-600 dark:text-sky-400 font-extrabold text-xs group-hover:bg-sky-600 group-hover:text-white transition-all inline-flex items-center gap-1 shadow-xs"
        >
          Collect Fee <ArrowRight className="w-3.5 h-3.5" />
        </button>
      </td>
    </tr>
  </tbody>
</table>
</div>
  </div>
);
}

return (
  <div className="space-y-6 animate-in fade-in">
    <div className="space-y-4">
      <div className="glass-card p-4 rounded-2xl space-y-3 bg-slate-50/80 dark:bg-slate-900/60 border bor

```

```

der-slate-200/80 dark:border-slate-800">
  <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 border-b border-slate-200/80 dark:border-slate-800 pb-3">
    <div className="flex items-center gap-3">
      <button
        type="button"
        onClick={() => {
          setSelectedStudent(null);
          if (onClearInitialStudent) {
            onClearInitialStudent();
          }
        }}
        className="px-3.5 py-2 rounded-xl bg-white dark:bg-slate-800 border border-slate-200 dark:border-slate-700 hover:bg-slate-100 dark:hover:bg-slate-700 text-slate-700 dark:text-slate-200 text-xs font-extrabold flex items-center gap-2 transition-all cursor-pointer shadow-xs"
      >
        <ArrowLeft className="w-4 h-4 text-sky-500" /> Back to Student List
      </button>
      <div className="h-6 w-[1px] bg-slate-200 dark:bg-slate-700 hidden sm:block" />
      <div className="flex items-center gap-3">
        <div className="w-10 h-10 rounded-xl bg-sky-500 text-white font-black text-sm flex items-center justify-center shadow-md shadow-sky-500/20">
          {selectedStudent.firstName?.[0] || "S"}
        </div>
        <div>
          <h3 className="font-extrabold text-sm text-slate-900 dark:text-white">
            {selectedStudent.firstName} {selectedStudent.lastName}
          </h3>
          <p className="text-[11px] text-slate-500 dark:text-slate-400">
            Class {selectedStudent.className.replace(/^Class\s+/i, "")}-
            {selectedStudent.section} • Adm No:{ " "}
            <span className="font-mono font-bold">
              {selectedStudent.admissionNo}
            </span>
          </p>
        </div>
      </div>
    </div>
    <div className="text-right">
      <p className="text-[10px] uppercase font-bold text-slate-400 tracking-wider">
        Total Outstanding Balance
      </p>
      <div className="flex items-center justify-end gap-2 mt-0.5">
        <h4 className="text-xl font-black text-rose-600 dark:text-rose-400">
          {formatCurrency(totalOutstanding)}
        </h4>
        {totalOutstanding === 0 && (
          <span className="px-2 py-0.5 rounded-full bg-emerald-100 text-emerald-800 dark:bg-emerald-950 dark:text-emerald-300 font-black text-[9px] tracking-wider uppercase">
            ✓ Settled
          </span>
        )}
      </div>
    </div>
  </div>
  <div className="grid grid-cols-3 gap-2 text-xs pt-0.5">
    <div className="bg-white dark:bg-slate-800/80 p-2 rounded-xl border border-slate-200/60 dark:border-slate-700/60">
      <p className="text-[10px] text-slate-400 font-bold uppercase">
        Current AY Dues
      </p>
      <p className="font-bold text-slate-900 dark:text-white mt-0.5">

```

```

        {formatCurrency(currentYearPending)}}
      </p>
    </div>
    <div className="bg-white dark:bg-slate-800/80 p-2 rounded-xl border border-slate-200/60 dark:bo
rder-slate-700/60">
      <p className="text-[10px] text-slate-400 font-bold uppercase">
        Previous Years Arrears
      </p>
      <p className="font-bold text-amber-600 mt-0.5">
        +{formatCurrency(previousYearPending)}
      </p>
    </div>
    <div className="bg-white dark:bg-slate-800/80 p-2 rounded-xl border border-slate-200/60 dark:bo
rder-slate-700/60">
      <p className="text-[10px] text-slate-400 font-bold uppercase">
        Total Concessions
      </p>
      <p className="font-bold text-emerald-600 dark:text-emerald-400 mt-0.5">
        -
        {formatCurrency(
          (calcResult?.scholarshipDeduction || 0) +
          (calcResult?.discountDeduction || 0),
        )}
      </p>
    </div>
  </div>
  </div>

  {(totalOutstanding > 0 ||
    (calcResult?.scholarshipDeduction || 0) > 0 ||
    (calcResult?.discountDeduction || 0) > 0) && (
    <div className="glass-card p-3 rounded-2xl space-y-2 bg-slate-50/40 dark:bg-slate-900/20">
      <h4 className="font-bold text-xs uppercase text-slate-400">
        Apply Concessions (Before Payment)
      </h4>
      <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
        <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 p-2.
5 rounded-xl space-y-1.5">
          {(calcResult?.scholarshipDeduction || 0) > 0 ? (
            <div className="space-y-1.5">
              <div className="flex justify-between items-center text-[10px]">
                <span className="font-black text-emerald-600">
                  ✓ Scholarship Active
                </span>
                <button
                  type="button"
                  onClick={handleRemoveScholarship}
                  className="text-rose-500 font-extrabold hover:underline"
                >
                  Remove
                </button>
              </div>
              <p className="font-bold text-slate-800 dark:text-slate-200 text-xs truncate">
                {calcResult?.scholarshipName}
              </p>
              <p className="text-[10px] text-slate-400 font-bold">
                -{formatCurrency(calcResult?.scholarshipDeduction || 0)}
              </p>
            </div>
          ) : (
            <div className="space-y-1">
              <label className="block text-[10px] text-slate-400 font-bold">
                Scholarship Concession
              </label>
            </div>
          )}
        </div>
      </div>
    </div>
  )}

```

```

        </label>
        <div className="flex items-center gap-2">
          <select
            value={tempScholarshipId}
            onChange={(e) => setTempScholarshipId(e.target.value)}
            className="flex-1 min-w-0 w-full px-3.5 py-1.5 rounded-xl bg-slate-50 dark:bg-slate
-800 border border-slate-200 dark:border-slate-700 text-xs font-bold text-slate-900 dark:text-white outline
-none cursor-pointer focus:ring-2 focus:ring-sky-500/20"
          >
            <option value="">Select...</option>
            {scholarships
              .filter((s) => s.status === "Active")
              .map((s) => (
                <option key={s.id} value={s.id}>
                  {s.name} (
                    {s.discountType === "Percentage"
                      ? `${s.percentage}%`
                      : formatCurrency(s.fixedAmount || 0)}
                  )
                </option>
              ))}
          </select>
          <button
            type="button"
            onClick={() =>
              tempScholarshipId &&
              handleApplyScholarship(tempScholarshipId)
            }
            className="px-4 py-1.5 bg-sky-600 hover:bg-sky-500 text-white font-extrabold text-x
s rounded-xl cursor-pointer shadow-xs transition-colors shrink-0"
          >
            Apply
          </button>
        </div>
      </div>
    )}
  </div>
  <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 p-2.
5 rounded-xl space-y-1.5">
    {(calcResult?.discountDeduction || 0) > 0 ? (
      <div className="space-y-1.5">
        <div className="flex justify-between items-center text-[10px]">
          <span className="font-black text-emerald-600">
            ✓ Discount Active
          </span>
          <button
            type="button"
            onClick={handleRemoveDiscount}
            className="text-rose-500 font-extrabold hover:underline"
          >
            Remove
          </button>
        </div>
        <p className="font-bold text-slate-800 dark:text-slate-200 text-xs truncate">
          {calcResult?.discountName}
        </p>
        <p className="text-[10px] text-slate-400 font-bold">
          -{formatCurrency(calcResult?.discountDeduction || 0)}
        </p>
      </div>
    ) : (
      <div className="space-y-1">
        <label className="block text-[10px] text-slate-400 font-bold">

```

```

        General Discount
      </label>
      <div className="flex items-center gap-2">
        <select
          value={tempDiscountId}
          onChange={(e) => setTempDiscountId(e.target.value)}
          className="flex-1 min-w-0 w-full px-3.5 py-1.5 rounded-xl bg-slate-50 dark:bg-slate
-800 border border-slate-200 dark:border-slate-700 text-xs font-bold text-slate-900 dark:text-white outline
-none cursor-pointer focus:ring-2 focus:ring-sky-500/20"
        >
          <option value="">Select...</option>
          {discounts
            .filter((d) => d.status === "Active")
            .map((d) => (
              <option key={d.id} value={d.id}>
                {d.name} (
                  {d.mode === "Percentage"
                    ? `${d.value}%`
                    : formatCurrency(d.value)}
                )
              </option>
            ))}
        </select>
        <button
          type="button"
          onClick={() =>
            tempDiscountId && handleApplyDiscount(tempDiscountId)
          }
          className="px-4 py-1.5 bg-sky-600 hover:bg-sky-500 text-white font-extrabold text-x
s rounded-xl cursor-pointer shadow-xs transition-colors shrink-0"
        >
          Apply
        </button>
      </div>
    </div>
  )}
</div>

  { /* Late Fee Management Card */ }
  <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 p-2.
5 rounded-xl space-y-1.5 col-span-full">
    <div className="flex justify-between items-center text-[10px]">
      <span className="font-extrabold text-slate-700 dark:text-slate-300 uppercase">
        Late Fee / Fine Rule
      </span>
      {isFineWaived ? (
        <button
          type="button"
          onClick={handleReapplyFine}
          className="text-sky-600 dark:text-sky-400 font-extrabold hover:underline cursor-point
er"
        >
          + Re-apply Fine
        </button>
      ) : calcResult.fineAmount > 0 ? (
        <button
          type="button"
          onClick={handleWaiveFine}
          className="text-rose-500 font-extrabold hover:underline cursor-pointer"
        >
          ✕ Remove / Waive Late Fee
        </button>
      ) : null}
    </div>
  </div>

```

```

    </div>

    {isFineWaived ? (
      <div className="p-2 rounded-lg bg-emerald-50 dark:bg-emerald-950/40 border border-emerald
-200 dark:border-emerald-900/50 flex justify-between items-center">
        <div>
          <p className="font-bold text-emerald-800 dark:text-emerald-300 text-xs">
            ✓ Late Fee Waived / Removed
          </p>
          <p className="text-[10px] text-emerald-600 dark:text-emerald-400">
            Fine amount set to ₹0 for this collection.
          </p>
        </div>
        <span className="font-black text-emerald-600 dark:text-emerald-400 text-xs">
          ₹0
        </span>
      </div>
    ) : calcResult.fineAmount > 0 ? (
      <div className="p-2 rounded-lg bg-rose-50 dark:bg-rose-950/40 border border-rose-200 dar
k:border-rose-900/50 flex justify-between items-center">
        <div>
          <p className="font-bold text-rose-900 dark:text-rose-200 text-xs">
            {calcResult.fineDetails?.ruleName ||
              "Automatic Late Fine"}{" "}
            ({calcResult.fineDetails?.daysOverdue || 0} Days
            Overdue)
          </p>
          <p className="text-[10px] text-rose-500 font-medium">
            Calculated automatically based on active Fine Rule
            configuration.
          </p>
        </div>
        <span className="font-black text-rose-600 dark:text-rose-400 text-xs font-mono">
          +{formatCurrency(calcResult.fineAmount)}
        </span>
      </div>
    ) : (
      <div className="p-2 rounded-lg bg-slate-50 dark:bg-slate-800/60 border border-slate-200/6
0 dark:border-slate-700/60 text-[11px] text-slate-500 font-medium">
        No late fine applicable (Within grace period).
      </div>
    )}
  </div>
</div>
)}

<div className="glass-card p-4 rounded-2xl space-y-3">
  <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-850
pb-2">
    <h4 className="font-black text-xs uppercase tracking-wider text-slate-800 dark:text-slate-250 f
lex items-center gap-1.5">
      <CheckCircle className="w-4 h-4 text-sky-500" /> Current Academic
      Year Fees ({currentYear})
    </h4>
    {selectableCurrentIds.length > 0 && (
      <button
        type="button"
        onClick={handleSelectAllCurrentYear}
        className="text-[10px] text-brand-600 dark:text-brand-400 font-bold hover:underline cursor-
pointer"
      >
        {allCurrentSelected

```

```

      ? "Deselect All Current Year"
      : "Select All Current Year"}
    </button>
  )}
</div>

{currentYearInstallments.length === 0 ? (
  <p className="text-xs text-slate-400 italic">
    No fee structure assigned for current academic year.
  </p>
) : (
  <div className="space-y-3">
    {Object.entries(groupedCurrentYear).map(([headName, list]) => {
      const groupTotal = list.reduce((sum, i) => sum + i.amount, 0);
      const groupPaid = list.reduce(
        (sum, i) => sum + i.paidAmount,
        0,
      );
      const groupRemaining = list.reduce(
        (sum, i) => sum + i.dueAmount,
        0,
      );
      const isMultiTerm = list.length > 1;
      const hasCheckedItem = list.some((inst) =>
        selectedInstallments.includes(inst.id),
      );
      const isExpanded = isMultiTerm
        ? (expandedGroups[headName] ?? hasCheckedItem ?? false)
        : true;

      return (
        <div
          key={headName}
          className="p-3 bg-slate-50/50 dark:bg-slate-900/40 border border-slate-200/50 dark:bord
er-slate-800 rounded-xl space-y-2"
        >
          <div
            onClick={() =>
              isMultiTerm && toggleGroupExpansion(headName)
            }
            className={`flex justify-between items-center border-b border-slate-200/30 dark:borde
r-slate-800 pb-1 text-xs select-none ${
              isMultiTerm
                ? "cursor-pointer hover:opacity-80 transition-opacity"
                : ""
            }`}
          >
            <div className="flex items-center gap-2">
              <span className="font-extrabold text-slate-900 dark:text-white text-xs">
                {headName}
              </span>
              {isMultiTerm && !headName.toLowerCase().includes('uniform')} && (
                <span className="text-[10px] bg-slate-200/70 dark:bg-slate-800 text-slate-600 dar
k:text-slate-300 font-bold px-2 py-0.5 rounded-full flex items-center gap-1">
                  {list.length} Terms
                  {isExpanded ? (
                    <ChevronUp className="w-3.5 h-3.5 text-sky-500" />
                  ) : (
                    <ChevronDown className="w-3.5 h-3.5 text-sky-500" />
                  )}
                </span>
              )}
            </div>
          </div>
        )
      )}
    )}
  </div>

```

```

    <span className="text-[10px] text-slate-500 font-bold">
      Total: {formatCurrency(groupTotal)} • Paid:{" "}
      {formatCurrency(groupPaid)} • Remaining:{" "}
    <span className="text-rose-600 font-bold">
      {formatCurrency(groupRemaining)}
    </span>
  </span>
</div>

{isExpanded && (
  <div className="divide-y divide-slate-100 dark:divide-slate-800/60 pl-1 animate-in fa
de-in">

    {list.map((inst) => {
      const isPaid = inst.dueAmount <= 0;
      const isChecked = selectedInstallments.includes(
        inst.id,
      );
      const status = getInstallmentStatus(
        inst.dueAmount,
        inst.dueDate,
        inst.status
      );

      let displayStatus = status;
      let badgeClass =
        "bg-amber-100 text-amber-800 dark:bg-amber-950 dark:text-amber-300";
      if (status === "PAID") {
        badgeClass =
          "bg-emerald-100 text-emerald-800 dark:bg-emerald-950 dark:text-emerald-300";
      } else if (status === "CANCELLED") {
        badgeClass =
          "bg-slate-200 text-slate-600 dark:bg-slate-800 dark:text-slate-400";
      } else if (status === "OVERDUE") {
        badgeClass =
          "bg-rose-100 text-rose-800 dark:bg-rose-950 dark:text-rose-350";
      } else if (status === "UPCOMING") {
        displayStatus = "FEE PENDING";
        badgeClass =
          "bg-amber-100 text-amber-800 dark:bg-amber-950 dark:text-amber-300";
      }

      const isOneTimeOrAnnual =
        inst.frequency === "One Time" ||
        inst.frequency === "Annual" ||
        inst.termName === "One Time" ||
        inst.termName === "Annual" ||
        inst.termId === "ONETIME" ||
        inst.termId === "ANNUAL";

      return (
        <div
          key={inst.id}
          className="flex items-center justify-between py-2.5 first:pt-1 last:pb-1"
        >
          <div className="flex items-center gap-3">
            {isPaid ? (
              <input
                type="checkbox"
                disabled
                checked={false}
                className="w-4 h-4 rounded border-slate-200 bg-slate-100 cursor-not-all
owed opacity-40"
              />

```

```

    ) : (
      <input
        type="checkbox"
        checked={isChecked}
        onChange={() =>
          toggleInstallmentSelection(inst.id)
        }
        className="w-4 h-4 rounded text-sky-600 border-slate-350 dark:border-slate-700 bg-white focus:ring-sky-500 cursor-pointer shrink-0"
      />
    )}
  <div>
    <div className="flex items-center gap-2 flex-wrap">
      <p className="font-bold text-slate-800 dark:text-slate-200 text-xs">
        {{{() => {
          const head = inst.feeHeadName || "";
          const rawTerm = inst.termName || inst.termId || "Installment";
          // Clean any duplicate "Uniform & Accessories - " prefix and size
          let cleanTerm = rawTerm.replace(/^(Uniform & Accessories\s*[\s\u2013-4\u2013-]\s*)+/gi, '').replace(/\/s*\(\Size:([^\)]*)\/gi, '').trim();
          if (head.toLowerCase().includes('uniform')) {
            const lowerClean = cleanTerm.toLowerCase();
            if (lowerClean.includes('cloth') || lowerClean.includes('fabric') || lowerClean.includes('unstitched') || lowerClean.includes('material')) {
              return cleanTerm;
            }
            if (cleanTerm.startsWith('Base Package') || cleanTerm.startsWith('Additional Purchase') || cleanTerm.startsWith('Additional Base Kit')) {
              return cleanTerm;
            }
            const isBasePkg = lowerClean.includes('package') || lowerClean.includes('admission kit') || lowerClean.includes('base') || lowerClean.includes('uniform fee') || lowerClean.includes('uniform') || lowerClean.includes('kit');
            return isBasePkg ? (lowerClean.includes('kit') ? `Base Package – Admission Kit` : `Base Package – ${cleanTerm}`) : `Additional Purchase – ${cleanTerm}`;
          }
          if (inst.feeHeadName.includes("Additional Purchases") || cleanTerm.startsWith(head)) {
            return cleanTerm;
          }
          return `${head} – ${cleanTerm}`;
        }}}()}}
      </p>
    </div>
    <p className="text-[10px] text-slate-400 font-medium mt-0.5">
      Due: {inst.dueDate}
    </p>
    {isChecked && !isPaid && (
      <div className="mt-1.5 flex flex-col gap-0.5">
        <div className="flex items-center gap-1.5">
          <span className="text-[11px] font-semibold text-slate-600 dark:text-slate-300">
            Collection Amount:
          </span>
          <div className="relative flex items-center">
            <span className="absolute left-2 text-xs font-bold text-slate-400">
              ₹
            </span>
            <input
              type="text"
              value={
                customCollectionAmounts[

```

```

        inst.id
      ] ?? String(inst.dueAmount)
    }
    onChange={(e) =>
      handleCustomAmountChange(
        inst.id,
        e.target.value,
      )
    }
    className={`w-28 pl-5 pr-2 py-1 rounded-lg border text-right font-mono font-bold text-xs outline-none transition-all ${
      getInstallmentAmountError(inst)
        ? "border-rose-400 bg-rose-50 dark:bg-rose-950/40 text-rose-600 dark:text-rose-400 focus:ring-2 focus:ring-rose-500/20"
        : "border-slate-300 dark:border-slate-700 bg-white dark:bg-slate-900 text-sky-600 dark:text-sky-400 focus:ring-2 focus:ring-sky-500/20"
    }`}
  />
</div>
</div>
{getInstallmentAmountError(inst) && (
  <p className="text-[10px] text-rose-500 font-bold mt-0.5">
    {getInstallmentAmountError(inst)}
  </p>
)}
</div>
)}
</div>
</div>

<div className="flex items-center gap-4 text-right">
  <div className="text-[11px] font-medium text-slate-500 space-y-0.5">
    {inst.paidAmount > 0 && (
      <p>
        Paid:{" "}
        <span className="font-mono text-emerald-600 font-bold">
          {formatCurrency(inst.paidAmount)}
        </span>
      </p>
    )}
    {!isPaid && (
      <p>
        className={
          isPaid
            ? "text-slate-400 font-medium"
            : "font-semibold text-slate-900 dark:text-white"
        }
      >
        Pending:{" "}
        <span className="font-mono">
          {formatCurrency(inst.dueAmount)}
        </span>
      </p>
    )}
  </div>
  <span>
    className={`px-2.5 py-0.5 rounded text-[8px] font-black uppercase tracking-wider ${badgeClass} min-w-[80px] text-center`}
    >
      {displayStatus}
    </span>
  </div>
</div>

```

```

    });
    }}}
  </div>
  })
</div>
);
}}
</div>
})
</div>

{previousYearInstallments.length > 0 && (
  <div className="glass-card p-4 rounded-2xl space-y-3 border-amber-250 dark:border-amber-950/60 bg
-amber-50/10">
    <div className="flex items-center justify-between border-b border-amber-150/60 pb-2">
      <button
        type="button"
        onClick={() => setIsPreviousDuesOpen((prev) => !prev)}
        className="font-black text-xs uppercase tracking-wider text-amber-700 dark:text-amber-400 f
lex items-center gap-2 hover:opacity-85 transition-all cursor-pointer select-none"
      >
        <AlertCircle className="w-4 h-4 text-amber-600 shrink-0" />
        <span>Previous Academic Year Dues</span>
        {!isPreviousDuesOpen && (
          <span className="text-[10px] font-bold text-rose-600 bg-amber-100 dark:bg-amber-950/60 px
-2 py-0.5 rounded-full font-mono">
            (Pending: {formatCurrency(previousYearPending)})
          </span>
        )}
        {isPreviousDuesOpen ? (
          <ChevronUp className="w-4 h-4 text-amber-600 shrink-0" />
        ) : (
          <ChevronDown className="w-4 h-4 text-amber-600 shrink-0" />
        )}
      </button>

      <div className="flex items-center gap-3">
        {selectablePrevIds.length > 0 && (
          <button
            type="button"
            onClick={handleSelectAllPreviousYear}
            className="text-[10px] text-amber-700 dark:text-amber-400 font-bold hover:underline cur
sor-pointer"
          >
            {allPreviousSelected
              ? "Deselect All Previous Years"
              : "Select All Previous Years"}
          </button>
        )}
        <button
          type="button"
          onClick={() => setIsPreviousDuesOpen((prev) => !prev)}
          className="px-2 py-1 rounded-lg bg-amber-100/80 dark:bg-amber-900/40 hover:bg-amber-200 d
ark:hover:bg-amber-800/60 text-amber-800 dark:text-amber-300 cursor-pointer transition-all flex items-cente
r gap-1 text-[11px] font-bold"
          title={
            isPreviousDuesOpen
              ? "Close Previous Academic Dues"
              : "Open Previous Academic Dues"
          }
        >
          <span>{isPreviousDuesOpen ? "Close" : "Open"}</span>
          {isPreviousDuesOpen ? (

```

```

        <ChevronUp className="w-3.5 h-3.5" />
      ) : (
        <ChevronDown className="w-3.5 h-3.5" />
      )}
    </button>
  </div>
</div>

{isPreviousDuesOpen && (
  <div className="space-y-4 animate-in fade-in duration-200">
    {Object.entries(groupedPreviousYears).map(([year, list]) => {
      const yearDueTotal = list.reduce(
        (sum, i) => sum + i.dueAmount,
        0,
      );
      return (
        <div
          key={year}
          className="p-3 bg-slate-50/50 dark:bg-slate-900/40 border border-slate-200/50 dark:bo
rder-slate-800 rounded-xl space-y-2"
        >
          <div className="flex justify-between items-center border-b border-slate-200/30 dark:b
order-slate-800 pb-1 text-xs">
            <span className="font-extrabold text-slate-900 dark:text-white text-xs">
              Academic Year: {year}
            </span>
            <span className="text-[10px] text-slate-550 font-bold">
              Pending:{" "}
              <span className="text-rose-600 font-bold font-mono">
                {formatCurrency(yearDueTotal)}
              </span>
            </span>
          </div>
          <div className="divide-y divide-slate-100 dark:divide-slate-800/60 pl-1">
            {list.map((inst) => {
              const isPaid = inst.dueAmount <= 0;
              const isChecked = selectedInstallments.includes(
                inst.id,
              );
              const status = getInstallmentStatus(
                inst.dueAmount,
                inst.dueDate,
                inst.status
              );
              let badgeClass =
                "bg-slate-100 text-slate-700 dark:bg-slate-800 dark:text-slate-300";
              if (status === "PAID") {
                badgeClass =
                  "bg-emerald-100 text-emerald-800 dark:bg-emerald-950 dark:text-emerald-300";
              } else if (status === "CANCELLED") {
                badgeClass =
                  "bg-slate-200 text-slate-600 dark:bg-slate-800 dark:text-slate-400";
              } else if (status === "OVERDUE") {
                badgeClass =
                  "bg-rose-100 text-rose-800 dark:bg-rose-950 dark:text-rose-350";
              }
              return (
                <div
                  key={inst.id}
                  className="flex items-center justify-between py-2.5 first:pt-1 last:pb-1"
                >

```

```

<div className="flex items-center gap-3">
  {isPaid ? (
    <input
      type="checkbox"
      disabled
      checked={false}
      className="w-4 h-4 rounded border-slate-200 bg-slate-100 cursor-not-all
owed opacity-40"
    />
  ) : (
    <input
      type="checkbox"
      checked={isChecked}
      onChange={() =>
        toggleInstallmentSelection(inst.id)
      }
      className="w-4 h-4 rounded text-amber-600 border-slate-350 dark:border-
slate-700 bg-white focus:ring-amber-500 cursor-pointer shrink-0"
    />
  )}
<div>
  <p className="font-bold text-slate-800 dark:text-slate-200 text-xs">
    {inst.feeHeadName} -{" "}
    {inst.termName ||
      inst.termId ||
      "Installment"}
  </p>
  <p className="text-[10px] text-slate-400 font-medium mt-0.5">
    Due: {inst.dueDate}
  </p>
  {isChecked && !isPaid && (
    <div className="mt-1.5 flex flex-col gap-0.5">
      <div className="flex items-center gap-1.5">
        <span className="text-[11px] font-semibold text-slate-600 dark:text
-slate-300">
          Collection Amount:
        </span>
        <div className="relative flex items-center">
          <span className="absolute left-2 text-xs font-bold text-slate-40
0">
            ₹
          </span>
          <input
            type="text"
            value={
              customCollectionAmounts[
                inst.id
              ] ?? String(inst.dueAmount)
            }
            onChange={(e) =>
              handleCustomAmountChange(
                inst.id,
                e.target.value,
              )
            }
            className={`w-28 pl-5 pr-2 py-1 rounded-lg border text-right fo
nt-mono font-bold text-xs outline-none transition-all ${
              getInstallmentAmountError(inst)
                ? "border-rose-400 bg-rose-50 dark:bg-rose-950/40 text-rose
-600 dark:text-rose-400 focus:ring-2 focus:ring-rose-500/20"
                : "border-slate-300 dark:border-slate-700 bg-white dark:bg-
slate-900 text-sky-600 dark:text-sky-400 focus:ring-2 focus:ring-sky-500/20"
            }`
          </div>
        </div>
      </div>
    )}
  </div>

```

```

        />
      </div>
    </div>
    {getInstallmentAmountError(inst) && (
      <p className="text-[10px] text-rose-500 font-bold mt-0.5">
        {getInstallmentAmountError(inst)}
      </p>
    )}
  </div>
)}
</div>
</div>

<div className="flex items-center gap-4 text-right">
  <div className="text-[11px] font-medium text-slate-500 space-y-0.5">
    <p>
      Paid:{" "}
      <span className="font-mono">
        {formatCurrency(inst.paidAmount)}
      </span>
    </p>
    <p>
      <span>
        className={
          isPaid
            ? "text-slate-400 font-medium"
            : "font-semibold text-slate-905 dark:text-white"
        }
      >
      Pending:{" "}
      <span className="font-mono">
        {formatCurrency(inst.dueAmount)}
      </span>
    </p>
  </div>
  <span>
    className={`px-2.5 py-0.5 rounded text-[8px] font-black uppercase trackin
g-wider ${badgeClass} min-w-[70px] text-center`}
    >
    {status}
  </span>
</div>
</div>
);
}}
</div>
</div>
);
}}
</div>
)}
</div>
)}

<div className="glass-card p-4 rounded-2xl grid grid-cols-1 md:grid-cols-5 gap-6">
  <div className="md:col-span-2 space-y-3 pr-0 md:pr-4 md:border-r border-slate-100 dark:border-sla
te-850">
    <h4 className="font-black text-xs uppercase text-slate-400 tracking-wider">
      Payment Summary
    </h4>
    <div className="space-y-2 text-xs font-bold text-slate-600 dark:text-slate-400">
      <div className="flex justify-between">
        <span>Current Academic Year Pending:</span>
        <span className="font-mono text-slate-900 dark:text-white font-extrabold">

```

```

        {formatCurrency(currentYearPending)}}
      </span>
    </div>
    <div className="flex justify-between">
      <span>Previous Academic Year Pending:</span>
      <span className="font-mono text-slate-955 dark:text-white font-extrabold">
        {formatCurrency(previousYearPending)}
      </span>
    </div>
    <hr className="border-slate-150 dark:border-slate-800" />
    <div className="flex justify-between text-slate-905 dark:text-white font-extrabold">
      <span>Total Outstanding:</span>
      <span className="font-mono text-rose-600 font-black text-sm">
        {formatCurrency(totalOutstanding)}
      </span>
    </div>
    <hr className="border-slate-150 dark:border-slate-800 border-dashed" />
    {availableReturnCredit > 0 && (
      <label className="flex items-center justify-between text-xs font-bold text-emerald-800 dark:text-emerald-300 cursor-pointer py-1">
        <span className="flex items-center gap-2">
          <input
            type="checkbox"
            checked={applyReturnCredit}
            onChange={(e) => setApplyReturnCredit(e.target.checked)}
            className="w-4 h-4 rounded text-emerald-600 border-emerald-300 dark:border-emerald-700 bg-white focus:ring-emerald-500 cursor-pointer shrink-0"
          />
          <span>Apply Return Credit</span>
        </span>
        <span className="font-mono font-extrabold text-emerald-700 dark:text-emerald-400">
          -{formatCurrency(availableReturnCredit)}
        </span>
      </label>
    )}
    <div className="flex justify-between items-center bg-emerald-50 dark:bg-emerald-950/20 p-2.5 rounded-xl border border-emerald-100 dark:border-emerald-900/50 text-emerald-800 dark:text-emerald-400 mt-2 font-black text-[13px]">
      <span>Selected for Collection:</span>
      <span className="font-mono text-base font-black">
        {formatCurrency(
          Math.max(
            0,
            amountPaying -
              (applyReturnCredit ? availableReturnCredit : 0),
          ),
        )}
      </span>
    </div>
  </div>
</div>
<div className="md:col-span-3">
  <form
    onSubmit={handleSubmitPayment}
    className="space-y-3.5 text-xs"
  >
    <div>
      <label className="block font-semibold text-slate-700 dark:text-slate-300 mb-1">
        Payment Method <span className="text-rose-500 font-bold ml-0.5">*</span></label>
      <select
        value={paymentMode}
        onChange={(e) => {
          setPaymentMode(e.target.value as any);

```

```

        setTransactionId("");
        setChequeNo("");
        setChequeDate("");
        setBankName("");
    }}
    className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-850 border font-bold tex
t-slate-900 dark:text-white cursor-pointer outline-none focus:border-brand-500"
  >
    <option value="">-- Select Payment Method --</option>
    <option value="Cash">Cash</option>
    <option value="Card">Card</option>
    <option value="UPI">UPI / QR Scan</option>
    <option value="Bank Transfer">Bank Transfer</option>
    <option value="Cheque">Cheque</option>
    <option value="Other">Other Configured Mode</option>
  </select>
</div>
{{(paymentMode === "Online" ||
  paymentMode === "Card" ||
  paymentMode === "UPI" ||
  paymentMode === "Bank Transfer" ||
  paymentMode === "Other") && (
  <div className="animate-in slide-in-from-top-1">
    <label className="block font-semibold text-slate-700 dark:text-slate-300 mb-1">
      Transaction Ref / UTR No <span className="text-rose-500 font-bold ml-0.5">*</span></lab
el>

    <input
      type="text"
      required
      placeholder="Enter UTR reference code or Approval ID..."
      value={transactionId}
      onChange={(e) => setTransactionId(e.target.value)}
      className="w-full px-3 py-1.5 rounded-xl bg-slate-50 dark:bg-slate-850 border text-slat
e-900 dark:text-white font-mono outline-none"
    />
  </div>
  )}
  {paymentMode === "Cheque" && (
    <div className="grid grid-cols-1 sm:grid-cols-3 gap-2.5 animate-in slide-in-from-top-1">
      <div>
        <label className="block font-semibold text-slate-700 dark:text-slate-300 mb-1">
          Cheque No <span className="text-rose-500 font-bold ml-0.5">*</span></label>
        <input
          type="text"
          required
          placeholder="6-digit No."
          value={chequeNo}
          onChange={(e) => setChequeNo(e.target.value)}
          className="w-full px-3 py-1.5 rounded-xl bg-slate-50 dark:bg-slate-850 border text-sl
ate-900 dark:text-white font-mono outline-none"
        />
      </div>
      <div>
        <label className="block font-semibold text-slate-700 dark:text-slate-300 mb-1">
          Cheque Date <span className="text-rose-500 font-bold ml-0.5">*</span></label>
        <input
          type="date"
          required
          value={chequeDate}
          onChange={(e) => setChequeDate(e.target.value)}
          className="w-full px-3 py-1.5 rounded-xl bg-slate-50 dark:bg-slate-850 border text-sl
ate-900 dark:text-white outline-none"
        />
      </div>
    </div>
  )}

```

```

    </div>
    <div>
      <label className="block font-semibold text-slate-700 dark:text-slate-300 mb-1">
        Bank Name <span className="text-rose-500 font-bold ml-0.5">*</span></label>
      <input
        type="text"
        required
        placeholder="e.g. SBI / ICICI"
        value={bankName}
        onChange={(e) => setBankName(e.target.value)}
        className="w-full px-3 py-1.5 rounded-xl bg-slate-50 dark:bg-slate-850 border text-slate-900 dark:text-white outline-none"
      />
    </div>
  </div>
)}
<div>
  <label className="block font-semibold text-slate-700 dark:text-slate-300 mb-1">
    Payment Remarks (Optional)
  </label>
  <input
    type="text"
    placeholder="Add internal verification notes..."
    value={remarks}
    onChange={(e) => setRemarks(e.target.value)}
    className="w-full px-3 py-1.5 rounded-xl bg-slate-50 dark:bg-slate-850 border text-slate-900 dark:text-white outline-none"
  />
</div>
{!isFormValid && validationMessage && (
  <p className="text-[11px] font-bold text-amber-600 dark:text-amber-400 bg-amber-50 dark:bg-amber-950/40 p-2 rounded-xl border border-amber-200/60 dark:border-amber-900/60 flex items-center gap-1.5 animate-pulse">
    ⚠ {validationMessage}
  </p>
)}
<button
  type="submit"
  disabled={!isFormValid}
  className={`w-full py-2.5 rounded-xl font-black text-xs shadow-md flex items-center justify-center gap-2 transition-all ${
    isFormValid
      ? "bg-emerald-600 hover:bg-emerald-500 text-white shadow-emerald-500/20 cursor-pointer"
      : "bg-slate-200 dark:bg-slate-800 text-slate-400 dark:text-slate-650 cursor-not-allowed opacity-75"
  }`}
>
  <IndianRupee className="w-4 h-4" /> Collect Payment & Issue Receipt
</button>
</form>
</div>
</div>

<div className="glass-card p-3.5 rounded-2xl space-y-2.5">
  <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-1.5">
    <h4 className="font-bold text-xs uppercase tracking-wider text-slate-400 flex items-center gap-1.5">
      <History className="w-4 h-4 text-sky-500" /> Recorded Payment Receipts (
      {
        feePayments.filter((p) => p.studentId === selectedStudent.id)

```

```

        .length
      }
    )
  </h4>
</div>
<div className="space-y-2 text-xs max-h-60 overflow-y-auto pr-0.5">
  {feePayments.filter((p) => p.studentId === selectedStudent.id)
    .length === 0 ? (
      <p className="text-[11px] text-slate-400 italic">
        No payment receipts recorded yet.
      </p>
    ) : (
      feePayments
        .filter((p) => p.studentId === selectedStudent.id)
        .map((p) => (
          <div
            key={p.id}
            className="flex items-center justify-between p-2.5 rounded-xl bg-slate-50 dark:bg-slate-850 border border-slate-100 dark:border-slate-800"
          >
            <div>
              <div className="flex items-center gap-2">
                <span className="font-mono font-black text-sky-600 dark:text-sky-400 text-xs">
                  {p.receiptNo}
                </span>
                <span className="px-2 py-0.5 rounded-full bg-emerald-100 text-emerald-800 dark:bg-emerald-950 dark:text-emerald-300 font-extrabold text-[9px]">
                  ✓ PAID ({p.paymentMode})
                </span>
              </div>
              <p className="text-[10px] text-slate-400 font-mono mt-0.5">
                Date: {p.paymentDate}{ " " }
                {p.transactionId ? `• Ref: ${p.transactionId}` : ""}
              </p>
            </div>
            <div className="flex items-center gap-3">
              <span className="font-black text-emerald-600 text-xs">
                {formatCurrency(p.amountPaid)}
              </span>
              <button
                type="button"
                onClick={() => onPrintReceipt(p)}
                className="px-2.5 py-1 rounded-lg bg-sky-600 hover:bg-sky-500 text-white font-bold text-[10px] flex items-center gap-1 shadow-sm cursor-pointer"
              >
                <Printer className="w-3.5 h-3.5" /> Print Receipt
              </button>
            </div>
          </div>
        ))
    )}
</div>
</div>
</div>

{showPaymentConfirmModal && selectedStudent && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/60 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-2xl max-w-lg w-full p-5 shadow-2xl space-y-4">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-3">
        <div className="flex items-center gap-2.5">

```

```

<div className="p-2 rounded-xl bg-emerald-100 dark:bg-emerald-950/80 text-emerald-600 dark:
text-emerald-400">
  <IndianRupee className="w-5 h-5" />
</div>
<div>
  <h3 className="font-extrabold text-sm text-slate-900 dark:text-white">
    Confirm Official Receipt & Fee Payment
  </h3>
  <p className="text-[11px] text-slate-500">
    Verify details before recording payment
  </p>
</div>
</div>
<button
  type="button"
  onClick={() => setShowPaymentConfirmModal(false)}
  className="p-1 rounded-lg text-slate-405 hover:text-slate-600 dark: hover:text-slate-200 cur
sor-pointer"
>
  <X className="w-4 h-4" />
</button>
</div>
<div className="space-y-2 text-xs bg-slate-50 dark:bg-slate-800/40 p-3.5 rounded-xl border bord
er-slate-200/60 dark:border-slate-700/60">
  <div className="flex justify-between border-b border-slate-200/60 dark:border-slate-700/60 pb
-1.5">
    <span className="text-slate-500 font-medium">
      Student Name:
    </span>
    <span className="font-extrabold text-slate-900 dark:text-white">
      {selectedStudent.firstName} {selectedStudent.lastName} (
      {selectedStudent.admissionNo})
    </span>
  </div>
  <div className="flex justify-between border-b border-slate-200/60 dark:border-slate-700/60 pb
-1.5">
    <span className="text-slate-500 font-medium">
      Class & Section:
    </span>
    <span className="font-bold text-slate-800 dark:text-slate-200">
      {selectedStudent.className}-{selectedStudent.section}
    </span>
  </div>
  <div className="flex justify-between border-b border-slate-200/60 dark:border-slate-700/60 pb
-1.5">
    <span className="text-slate-500 font-medium">
      Payment Mode / Type:
    </span>
    <span className="font-extrabold text-brand-600 dark:text-brand-400 uppercase tracking-wide
r">
      {paymentMode}
    </span>
  </div>
  {paymentMode !== "Cash" && transactionId && (
    <div className="flex justify-between border-b border-slate-200/60 dark:border-slate-700/60
pb-1.5">
      <span className="text-slate-500 font-medium">
        Transaction / UTR Ref:
      </span>
      <span className="font-mono font-bold text-slate-800 dark:text-slate-200">
        {transactionId}
      </span>
    </div>
  )
}

```

```

    })
    {remarks && (
      <div className="flex justify-between border-b border-slate-200/60 dark:border-slate-700/60
pb-1.5">
        <span className="text-slate-500 font-medium">
          Payment Remarks:
        </span>
        <span className="font-medium text-slate-700 dark:text-slate-300 italic">
          {remarks}
        </span>
      </div>
    })
    <div className="flex justify-between items-center pt-1.5 font-black text-slate-900 dark:text-
white text-sm">
      <span>Total Amount to Collect:</span>
      <span className="text-lg text-emerald-600 dark:text-emerald-400 font-mono font-black">
        {formatCurrency(amountPaying)}
      </span>
    </div>
  </div>
  <div className="p-3.5 bg-sky-50/60 dark:bg-sky-950/40 rounded-xl border border-sky-200/80 dark:
border-sky-800/80 space-y-1.5 text-xs max-h-48 overflow-y-auto">
    <p className="text-[10px] font-extrabold uppercase text-sky-900 dark:text-sky-200 tracking-wi
der border-b border-sky-100 dark:border-sky-900 pb-1 mb-1">
      Payment Allocation Details
    </p>
    {{{() => {
      const selectedInstObjects = allInstallments.filter((i) =>
        selectedInstallments.includes(i.id),
      );
      const groupedByYear: {
        [year: string]: typeof selectedInstObjects;
      } = {};
      selectedInstObjects.forEach((inst) => {
        if (!groupedByYear[inst.academicYear]) {
          groupedByYear[inst.academicYear] = [];
        }
        groupedByYear[inst.academicYear].push(inst);
      });
      return Object.entries(groupedByYear).map(([year, list]) => (
        <div key={year} className="space-y-0.5">
          <p className="font-extrabold text-[10px] text-slate-500 uppercase">
            {year}
          </p>
          <ul className="list-disc list-inside space-y-0.5 pl-1 text-[11px] text-slate-700 dark:t
ext-slate-300">
            {list.map((i) => (
              <li key={i.id}>
                {i.feeHeadName} {i.termName ? ` - ${i.termName}` : ""}:{" "}
                <span className="font-mono font-extrabold">
                  {formatCurrency(i.dueAmount)}
                </span>
              </li>
            ))}
          </ul>
        </div>
      ));
    }}}()
  </div>
  <div className="flex gap-2 pt-2">
    <button
      type="button"
      onClick={() => setShowPaymentConfirmModal(false)}
    >

```

```

        className="flex-1 py-2.5 rounded-xl border border-slate-200 dark:border-slate-700 font-bold text-slate-600 dark:text-slate-300 hover:bg-slate-50 dark:hover:bg-slate-800 text-xs transition-all cursor-pointer"
      >
        Cancel
      </button>
      <button
        type="button"
        onClick={executeProcessPayment}
        className="flex-1 py-2.5 rounded-xl bg-emerald-600 hover:bg-emerald-500 text-white font-bl
ck text-xs shadow-md shadow-emerald-500/20 flex items-center justify-center gap-1.5 transition-all cursor-p
ointer"
      >
        <Receipt className="w-4 h-4" /> Confirm & Record Payment
      </button>
    </div>
  </div>
</div>
)}

{showPaymentHistoryModal && selectedStudent && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/70 backdrop-bl
ur-xs animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-
3xl max-w-3xl w-full p-6 shadow-2xl space-y-4 max-h-[88vh] flex flex-col">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-8
00 pb-4 shrink-0">
        <div className="flex items-center gap-3">
          <div className="p-2.5 rounded-2xl bg-amber-100 dark:bg-amber-950 text-amber-700 dark:text-a
mber-300">
            <History className="w-6 h-6" />
          </div>
          <div>
            <h3 className="font-black text-base text-slate-900 dark:text-white">
              Fee Payment & Allocation History Log
            </h3>
            <p className="text-xs text-slate-500">
              {selectedStudent.firstName} {selectedStudent.lastName} • Adm
              No:{" "}
              <span className="font-mono font-bold">
                {selectedStudent.admissionNo}
              </span>
            </p>
          </div>
        </div>
        <button
          type="button"
          onClick={() => setShowPaymentHistoryModal(false)}
          className="p-1.5 rounded-xl text-slate-400 hover:text-slate-600 dark:hover:text-slate-200 h
over:bg-slate-100 dark:hover:bg-slate-800 transition-colors cursor-pointer"
        >
          <X className="w-5 h-5" />
        </button>
      </div>
    <div className="flex-1 overflow-y-auto space-y-4 pr-1 text-xs">
      {(() => {
        const studentPayments = feePayments.filter(
          (p) => p.studentId === selectedStudent.id,
        );
        if (studentPayments.length === 0) {
          return (
            <div className="p-8 text-center text-slate-400 font-medium italic border border-dashed
border-slate-200 dark:border-slate-800 rounded-2xl">

```

```

        No historical fee payment receipts found for this student.
      </div>
    );
  }
  return (
    <div className="rounded-2xl border border-slate-200 dark:border-slate-800 overflow-hidde
n">
      <table className="w-full text-left text-xs border-collapse">
        <thead>
          <tr className="bg-slate-100 dark:bg-slate-800/80 font-extrabold uppercase text-[10p
x] text-slate-500 border-b border-slate-200 dark:border-slate-800">
            <th className="p-3">Receipt No</th>
            <th className="p-3">Payment Date</th>
            <th className="p-3">Mode / Ref</th>
            <th className="p-3 font-mono">Amount Paid</th>
            <th className="p-3 text-right">Receipt</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100 dark:divide-slate-800 font-medium">
          {studentPayments.map((p) => (
            <tr
              key={p.id}
              className="hover:bg-slate-50/60 dark:hover:bg-slate-800/40"
            >
              <td className="p-3 font-mono font-bold text-sky-600 dark:text-sky-400">
                {p.receiptNo}
              </td>
              <td className="p-3 font-mono text-slate-600 dark:text-slate-300">
                {p.paymentDate}
              </td>
              <td className="p-3">
                <span className="font-bold text-slate-900 dark:text-white uppercase text-[11p
x]">
                  {p.paymentMode}
                </span>
                {p.transactionId && (
                  <span className="block text-[10px] text-slate-400 font-mono">
                    Ref: {p.transactionId}
                  </span>
                )}
              </td>
              <td className="p-3 font-mono font-black text-emerald-600 dark:text-emerald-40
0">
                {formatCurrency(p.amountPaid)}
              </td>
              <td className="p-3 text-right">
                <button
                  type="button"
                  onClick={() => onPrintReceipt(p)}
                  className="px-2.5 py-1 rounded-lg bg-sky-50 dark:bg-sky-950 text-sky-700 da
rk:text-sky-300 font-bold hover:bg-sky-100 flex items-center gap-1 ml-auto text-[11px] border border-sky-20
0 dark:border-sky-800 transition-colors cursor-pointer"
                >
                  <Printer className="w-3.5 h-3.5" /> Print
                </button>
              </td>
            </tr>
          )})}
        </tbody>
      </table>
    </div>
  );
}
}

```

```
    </div>
    <div className="pt-3 border-t border-slate-100 dark:border-slate-800 flex justify-end shrink-
0">
      <button
        type="button"
        onClick={() => setShowPaymentHistoryModal(false)}
        className="px-5 py-2 rounded-xl bg-slate-100 dark:bg-slate-800 font-bold text-slate-700 dar
k:text-slate-300 text-xs hover:bg-slate-200 transition-colors cursor-pointer"
      >
        Close History Log
      </button>
    </div>
  </div>
</div>
)}
</div>
);
};
```

 Finance/DueFeesView.tsx

TSX

```

import React, { useState } from 'react';
import { formatCurrency } from '../../utils/currency';
import { Clock, Search, AlertCircle, IndianRupee, Filter, Calendar } from 'lucide-react';
import { Student } from '../../types';
import { useData } from '../../context/DataContext';
import { useAuth } from '../../context/AuthContext';
import { matchesClassName } from '../../utils/classSorter';

interface DueFeesViewProps {
  onCollectStudentFee: (student: Student) => void;
}

export const DueFeesView: React.FC<DueFeesViewProps> = ({ onCollectStudentFee }) => {
  const {
    students,
    getStudentFeeLedger,
    academicClasses,
    academicYearFeeSchedules,
    feeHeads,
    financeSettings,
    dynamicFeeStructures,
    studentFeeAssignments,
    studentTransports,
    studentHostels,
  } = useData();

  const { selectedAcademicYear } = useAuth();

  const activeAYDefault = selectedAcademicYear || financeSettings?.academicYear || "2026-2027";

  const [query, setQuery] = useState('');
  const [selectedAY, setSelectedAY] = useState(activeAYDefault);
  const [selectedClass, setSelectedClass] = useState('All');
  const [selectedSection, setSelectedSection] = useState('All');
  const [selectedTerm, setSelectedTerm] = useState('All');
  const [selectedFeeHead, setSelectedFeeHead] = useState('All');
  const [selectedDueDate, setSelectedDueDate] = useState('All');
  const [timelineFilter, setTimelineFilter] = useState<'All' | 'Overdue' | 'Current' | 'Upcoming'>('All');

  const uniqueSections = Array.from(new Set(students.map(st => st.section))).filter(Boolean).sort();
  const scheduleForAY = academicYearFeeSchedules.find(s => s.academicYear === selectedAY);
  const termsList = scheduleForAY ? scheduleForAY.terms : [];
  const uniqueDueDates = Array.from(new Set(termsList.map((t: any) => t.dueDate))).filter(Boolean).sort();

  const todayStr = new Date().toISOString().split("T")[0];

  // 1. Gather student-wise due summaries for the selected academic year
  const studentDueList: {
    student: Student;
    academicYear: string;
    totalAmount: number;
    totalPaid: number;
    totalOutstanding: number;
    overdueAmount: number;
    earliestDueDate: string;
    isOverdue: boolean;
    unpaidCount: number;
  }[] = [];

```

```
students.forEach((st) => {
  // Basic class & section filters
  if (selectedClass !== "All" && st.className !== selectedClass) return;
  if (selectedSection !== "All" && st.section !== selectedSection) return;

  // Search query filter
  const matchesQuery =
    query.trim() === "" ||
    `${st.firstName} ${st.lastName}`.toLowerCase().includes(query.toLowerCase()) ||
    st.admissionNo.toLowerCase().includes(query.toLowerCase());
  if (!matchesQuery) return;

  // Fast skip for students who have NO dynamic fee structure configured in MySQL,
  // NO student fee assignment, and NO transport/hostel services assigned
  const hasStructure = dynamicFeeStructures.some(
    (d) => matchesClassName(d.className, st.className) && (d.status === "Active" || !d.status),
  );
  const hasAssignment = studentFeeAssignments.some(
    (a) => a.studentId === st.id && a.status === "Active",
  );
  const hasTransport = studentTransports.some(
    (t) => t.studentId === st.id && t.status === "Active",
  );
  const hasHostel = studentHostels.some(
    (h) => h.studentId === st.id && h.status === "Active",
  );

  if (!hasStructure && !hasAssignment && !hasTransport && !hasHostel) {
    return;
  }

  const ledger = getStudentFeeLedger(st.id, selectedAY);
  if (!ledger || !ledger.installments || ledger.installments.length === 0) return;

  const unpaidInsts = ledger.installments.filter((inst) => inst.dueAmount > 0);
  if (unpaidInsts.length === 0) return;

  const totalAmount = ledger.installments.reduce((sum, inst) => sum + inst.amount, 0);
  const totalPaid = ledger.installments.reduce((sum, inst) => sum + inst.paidAmount, 0);
  const totalOutstanding = unpaidInsts.reduce((sum, inst) => sum + inst.dueAmount, 0);
  if (totalAmount <= 0 || totalOutstanding <= 0) return;

  const overdueInsts = unpaidInsts.filter((inst) => inst.dueDate <= todayStr);
  const overdueAmount = overdueInsts.reduce((sum, inst) => sum + inst.dueAmount, 0);

  const dueDates = unpaidInsts.map((inst) => inst.dueDate).sort();
  const earliestDueDate = dueDates[0] || todayStr;
  const isOverdue = overdueAmount > 0 || (dueDates.length > 0 && dueDates[0] < todayStr);

  // Timeline Filter
  if (timelineFilter === "Overdue" && !isOverdue) return;
  if (timelineFilter === "Upcoming" && isOverdue) return;

  studentDueList.push({
    student: st,
    academicYear: selectedAY,
    totalAmount,
    totalPaid,
    totalOutstanding,
    overdueAmount,
    earliestDueDate,
    isOverdue,
    unpaidCount: unpaidInsts.length,
  });
});
```

```

});
});

// 2. Sort by Highest Outstanding Dues First at Top
studentDueList.sort((a, b) => b.totalOutstanding - a.totalOutstanding);

const totalOutstandingSum = studentDueList.reduce((acc, item) => acc + item.totalOutstanding, 0);
const totalOverdueSum = studentDueList.reduce((acc, item) => acc + item.overdueAmount, 0);

return (
  <div className="space-y-6 animate-in fade-in">
    { /* Summary KPI Banner */ }
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <div className="glass-card p-5 rounded-2xl flex items-center justify-between border-1-4 border-1-rose-500 bg-white dark:bg-slate-900 shadow-xs">
        <div>
          <p className="text-xs font-semibold text-slate-500 uppercase">Total Outstanding Dues</p>
          <h3 className="text-2xl font-black text-rose-600 dark:text-rose-400 mt-1">
            {formatCurrency(totalOutstandingSum)}
          </h3>
          <p className="text-[10px] text-slate-400 mt-0.5">
            Across {studentDueList.length} students with pending dues
          </p>
        </div>
        <div className="p-3 rounded-xl bg-rose-50 text-rose-600 dark:bg-rose-950 dark:text-rose-400">
          <AlertCircle className="w-6 h-6" />
        </div>
      </div>

      <div className="glass-card p-5 rounded-2xl flex items-center justify-between border-1-4 border-1-amber-500 bg-white dark:bg-slate-900 shadow-xs">
        <div>
          <p className="text-xs font-semibold text-slate-500 uppercase">Overdue Amount (Due to Date)</p>
          <h3 className="text-2xl font-black text-amber-600 dark:text-amber-400 mt-1">
            {formatCurrency(totalOverdueSum)}
          </h3>
          <p className="text-[10px] text-slate-400 mt-0.5">
            Immediate payable amount past due dates
          </p>
        </div>
        <div className="p-3 rounded-xl bg-amber-50 text-amber-600 dark:bg-amber-950 dark:text-amber-400">
          <Clock className="w-6 h-6" />
        </div>
      </div>
    </div>

    { /* Filter Box */ }
    <div className="glass-card p-5 rounded-3xl bg-white dark:bg-slate-900 border border-slate-200 dark:bor-der-slate-800 space-y-4 shadow-xs">
      <h4 className="text-xs font-extrabold uppercase tracking-wider text-slate-900 dark:text-white flex items-center gap-1.5">
        <Filter className="w-4 h-4 text-sky-500" /> Filter Dues Directory
      </h4>

      <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-4 gap-3.5">
        { /* Search Query */ }
        <div className="space-y-1">
          <label className="text-[10px] font-bold text-slate-500">Search Student</label>
          <div className="relative">
            <Search className="w-3.5 h-3.5 text-slate-400 absolute left-3 top-2.5" />
            <input
              type="text"
              placeholder="Name or Admission No..."
            >
          </div>
        </div>
      </div>
    </div>
  </div>
);

```

```

        value={query}
        onChange={(e) => setQuery(e.target.value)}
        className="w-full pl-8 pr-4 py-2 rounded-xl bg-slate-50 dark:bg-slate-950 border border-slate-200 dark:border-slate-800 text-xs font-medium text-slate-900 dark:text-white outline-none"
      />
    </div>
  </div>

  { /* Academic Year */ }
  <div className="space-y-1">
    <label className="text-[10px] font-bold text-slate-500">Academic Year</label>
    <select
      value={selectedAY}
      onChange={(e) => setSelectedAY(e.target.value)}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-950 border border-slate-200 dark:border-slate-800 text-xs font-bold text-slate-900 dark:text-white cursor-pointer"
    >
      {academicYearFeeSchedules.map((s) => (
        <option key={s.id} value={s.academicYear}>
          {s.academicYear}
        </option>
      ))}
    </select>
  </div>

  { /* Class */ }
  <div className="space-y-1">
    <label className="text-[10px] font-bold text-slate-500">Class Grade</label>
    <select
      value={selectedClass}
      onChange={(e) => setSelectedClass(e.target.value)}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-950 border border-slate-200 dark:border-slate-800 text-xs font-bold text-slate-900 dark:text-white cursor-pointer"
    >
      <option value="All">All Classes</option>
      {academicClasses.map((c) => (
        <option key={c.id} value={c.name}>
          {c.name}
        </option>
      ))}
    </select>
  </div>

  { /* Section */ }
  <div className="space-y-1">
    <label className="text-[10px] font-bold text-slate-500">Section</label>
    <select
      value={selectedSection}
      onChange={(e) => setSelectedSection(e.target.value)}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-950 border border-slate-200 dark:border-slate-800 text-xs font-bold text-slate-900 dark:text-white cursor-pointer"
    >
      <option value="All">All Sections</option>
      {uniqueSections.map((sec) => (
        <option key={sec} value={sec}>
          {sec}
        </option>
      ))}
    </select>
  </div>

  { /* Timeline Filter */ }
  <div className="space-y-1 sm:col-span-2">

```

```

<label className="text-[10px] font-bold text-slate-500">Dues Status Filter</label>
<select
  value={timelineFilter}
  onChange={(e) => setTimelineFilter(e.target.value as any)}
  className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-950 border border-slate-200
dark:border-slate-800 text-xs font-bold text-slate-900 dark:text-white cursor-pointer"
>
  <option value="All">All Statuses (Overdue & Upcoming)</option>
  <option value="Overdue">Overdue Dues Only (Past Due Date)</option>
  <option value="Upcoming">Upcoming Dues Only</option>
</select>
</div>
</div>
</div>

{/* Student-Wise Dues Table */}
<div className="glass-card rounded-2xl overflow-hidden border border-slate-200/80 dark:border-slate-800
bg-white dark:bg-slate-900 shadow-xs">
  <div className="overflow-x-auto">
    <table className="w-full text-left border-collapse text-xs">
      <thead>
        <tr className="bg-slate-100/70 dark:bg-slate-800/60 text-slate-500 font-bold uppercase tracking-wider border-b border-slate-200 dark:border-slate-800 text-[10px]">
          <th className="py-3.5 px-4">Adm No</th>
          <th className="py-3.5 px-4">Student Name</th>
          <th className="py-3.5 px-4">Class & Section</th>
          <th className="py-3.5 px-4">Student Type</th>
          <th className="py-3.5 px-4">Earliest Due Date</th>
          <th className="py-3.5 px-4 font-mono text-right">Total Fee</th>
          <th className="py-3.5 px-4 font-mono text-right">Amount Paid</th>
          <th className="py-3.5 px-4 font-mono text-right">Overdue to Date</th>
          <th className="py-3.5 px-4 font-mono text-right">Total Outstanding</th>
          <th className="py-3.5 px-4 text-center">Status</th>
          <th className="py-3.5 px-4 text-right">Action</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-100 dark:divide-slate-800/80 font-medium">
        {studentDueList.length === 0 ? (
          <tr>
            <td colspan={11} className="py-8 text-center text-slate-400 text-xs font-bold italic">
              No student due records found matching the configured filters.
            </td>
          </tr>
        ) : (
          studentDueList.map((item) => {
            const st = item.student;
            const isPrePrimary = /^(nursery|lkg|ukg|playgroup|pre-kg)/i.test(st.className || "");
            const displayClassStr = st.className
              ? st.className.toLowerCase().startsWith("class") || isPrePrimary
                ? `${st.className}-${st.section}`
                : `Class ${st.className}-${st.section}`
              : `Section ${st.section}`;

            const isHosteller =
              st.studentType === "Hosteller" ||
              st.studentType === "Residential" ||
              (st as any).residentialStatus === "Residential" ||
              (studentHostels || []).some(
                (h) => h.studentId === st.id && h.status === "Active",
              );

            return (
              <tr key={st.id} className="hover:bg-slate-50/80 dark:hover:bg-slate-800/40 transition-c

```

```

olors">
    <td className="py-3 px-4 font-mono font-bold text-slate-700 dark:text-slate-300">
        {st.admissionNo || st.id}
    </td>
    <td className="py-3 px-4">
        <div className="flex items-center gap-2.5">
            <div className="w-8 h-8 rounded-xl bg-gradient-to-br from-sky-500 to-brand-600 text-white font-black text-xs flex items-center justify-center shrink-0 shadow-xs">
                {st.firstName?.[0] || "S"}
            </div>
            <div>
                <span className="font-extrabold text-slate-900 dark:text-white block">
                    {st.firstName} {st.lastName}
                </span>
                <span className="text-[10px] text-slate-400 font-medium">
                    {item.unpaidCount} Pending Term(s)
                </span>
            </div>
        </div>
    </td>
    <td className="py-3 px-4 text-slate-800 dark:text-slate-200 font-bold">
        {displayClassStr}
    </td>
    <td className="py-3 px-4">
        <span
            className={`px-2.5 py-0.5 rounded-full text-[10px] font-extrabold uppercase ${
                isHosteller
                    ? "bg-purple-100 text-purple-700 dark:bg-purple-950 dark:text-purple-300"
                    : "bg-sky-100 text-sky-700 dark:bg-sky-950 dark:text-sky-300"
            }`}
        >
            {isHosteller ? "Residential" : "Day Scholar"}
        </span>
    </td>
    <td className="py-3 px-4 font-mono text-slate-500">
        <div className="flex items-center gap-1">
            <Calendar className="w-3.5 h-3.5 text-slate-400" />
            <span>{item.earliestDueDate}</span>
        </div>
    </td>
    <td className="py-3 px-4 font-mono text-right text-slate-600 dark:text-slate-400">
        {formatCurrency(item.totalAmount)}
    </td>
    <td className="py-3 px-4 font-mono text-right text-emerald-600 dark:text-emerald-400 font-bold">
        {formatCurrency(item.totalPaid)}
    </td>
    <td className="py-3 px-4 font-mono text-right text-amber-600 dark:text-amber-400 font-bold">
        {formatCurrency(item.overdueAmount)}
    </td>
    <td className="py-3 px-4 font-mono text-right text-rose-600 dark:text-rose-400 font-black text-xs">
        {formatCurrency(item.totalOutstanding)}
    </td>
    <td className="py-3 px-4 text-center">
        <span
            className={`px-2.5 py-0.5 rounded text-[9px] font-black uppercase tracking-wider ${
                item.isOverdue
                    ? "bg-rose-100 text-rose-800 dark:bg-rose-950 dark:text-rose-300"
                    : "bg-sky-100 text-sky-800 dark:bg-sky-950 dark:text-sky-300"
            }`}
        >

```

```

        >
        {item.isOverdue ? "OVERDUE" : "PENDING"}
      </span>
    </td>
    <td className="py-3 px-4 text-right">
      <button
        onClick={() => onCollectStudentFee(st)}
        className="px-3 py-1 rounded-xl bg-emerald-600 hover:bg-emerald-500 text-white font-bold text-xs shadow flex items-center gap-1 ml-auto cursor-pointer transition-all"
      >
        <IndianRupee className="w-3.5 h-3.5" /> Collect
      </button>
    </td>
  </tr>
);
})
})
</tbody>
</table>
</div>
</div>
</div>
);
};

```

 Finance/TransactionsMasterLedgerView.tsx

TSX

```
import React, { useState } from 'react';
import {
  FileSpreadsheet, ArrowUpRight, ArrowDownLeft, Plus, Search, Filter, Calendar,
  Building2, CreditCard, Eye, RotateCcw, AlertTriangle, ShieldCheck, CheckCircle2,
  Printer, Download, FileText, IndianRupee, Layers, SlidersHorizontal, BookOpen, Clock,
  ChevronRight, ExternalLink, X, Paperclip, Lock, RefreshCw, PieChart, Sparkles, TrendingUp
} from 'lucide-react';
import { formatCurrency } from '../../utils/currency';
import { FinanceTransaction, FinancialAccount, FinancialCategory, FinancialBudget, TransactionType } from '../../types';
import { useData } from '../../context/DataContext';
import { useAuth } from '../../context/AuthContext';
import { useToast } from '../../context/ToastContext';
import { exportToExcel } from '../../utils/excelExport';

export const TransactionsMasterLedgerView: React.FC = () => {
  const {
    financeTransactions, addFinanceTransaction, reverseFinanceTransaction, cancelFinanceTransaction,
    financialAccounts, addFinancialAccount, updateFinancialAccount,
    financialCategories, addFinancialCategory, updateFinancialCategory,
    financialBudgets, updateFinancialBudget,
    students, staff
  } = useData();

  const { selectedAcademicYear, selectedBranch } = useAuth();
  const { addToast } = useToast();

  // Active Sub-Tab: 'ledger' | 'manual' | 'categories-accounts' | 'budget' | 'audit'
  const [activeSubTab, setActiveSubTab] = useState<'ledger' | 'categories-accounts' | 'budget'>('ledger');

  // Filter States
  const [searchQuery, setSearchQuery] = useState('');
  const [dateRangeFilter, setDateRangeFilter] = useState<string>('All');
  const [typeFilter, setTypeFilter] = useState<'All' | 'Income' | 'Expense'>('All');
  const [sourceModuleFilter, setSourceModuleFilter] = useState<string>('All');
  const [categoryFilter, setCategoryFilter] = useState<string>('All');
  const [paymentModeFilter, setPaymentModeFilter] = useState<string>('All');
  const [statusFilter, setStatusFilter] = useState<string>('All');
  const [sortBy, setSortBy] = useState<'date-desc' | 'date-asc' | 'amount-desc' | 'amount-asc'>('date-desc');

  // Modals
  const [isManualModalOpen, setIsManualModalOpen] = useState(false);
  const [selectedTxnForDetail, setSelectedTxnForDetail] = useState<FinanceTransaction | null>(null);
  const [txnToReverse, setTxnToReverse] = useState<FinanceTransaction | null>(null);
  const [reversalReason, setReversalReason] = useState('');

  // Category / Account Modals
  const [isAddCategoryOpen, setIsAddCategoryOpen] = useState(false);
  const [isAddAccountOpen, setIsAddAccountOpen] = useState(false);

  // Manual Entry Form State
  const [manualForm, setManualForm] = useState({
    type: 'Income' as TransactionType,
    category: 'Donations & Grants',
    description: '',
    amount: '',
    paymentMode: 'Bank Transfer' as any,
    account: 'Main Bank Account' as any,
  });
}
```

```

    date: new Date().toISOString().split('T')[0],
    branch: 'Main Campus',
    academicYear: '2025-2026',
    notes: '',
    attachmentName: ''
  });

  // New Category Form State
  const [newCatForm, setNewCatForm] = useState({
    name: '',
    type: 'Income' as TransactionType,
    sourceModule: 'Manual'
  });

  // New Account Form State
  const [newAccForm, setNewAccForm] = useState({
    accountName: '',
    accountType: 'Main Bank Account' as any,
    accountNumber: '',
    bankName: '',
    branchName: '',
    currentBalance: ''
  });

  // Summary Metrics Computation
  const filteredTxns = financeTransactions.filter(t => {
    // Search
    const matchesSearch =
      (t.transactionId || '').toLowerCase().includes(searchQuery.toLowerCase()) ||
      (t.description || '').toLowerCase().includes(searchQuery.toLowerCase()) ||
      (t.referenceNumber || '').toLowerCase().includes(searchQuery.toLowerCase()) ||
      (t.category || '').toLowerCase().includes(searchQuery.toLowerCase()) ||
      (t.createdBy || '').toLowerCase().includes(searchQuery.toLowerCase());

    // Type
    const matchesType = typeFilter === 'All' || t.type === typeFilter;

    // Module
    const mod = String(t.sourceModule || '');
    const matchesModule =
      sourceModuleFilter === 'All' ||
      t.sourceModule === sourceModuleFilter ||
      (sourceModuleFilter === 'Student Fee Collection' && (mod === 'Fees' || mod === 'Fee Collection' || mod === 'Student Fee Collection')) ||
      ((sourceModuleFilter === 'Manual' || sourceModuleFilter === 'Manual Entries') && (mod === 'Manual' || mod === 'Manual Entries'));

    // Category
    const matchesCategory = categoryFilter === 'All' || t.category === categoryFilter;

    // Payment Mode
    const matchesMode = paymentModeFilter === 'All' || t.paymentMode === paymentModeFilter;

    // Status
    const matchesStatus = statusFilter === 'All' || t.status === statusFilter;

    // Academic Year (Global Header filter)
    const matchesAcademicYear = !selectedAcademicYear || selectedAcademicYear === 'All' || selectedAcademicYear === 'All Years' || t.academicYear === selectedAcademicYear || !t.academicYear;

    return matchesSearch && matchesType && matchesModule && matchesCategory && matchesMode && matchesStatus && matchesAcademicYear;
  }).sort((a, b) => {

```

```
if (sortBy === 'date-desc') return new Date(b.date).getTime() - new Date(a.date).getTime();
if (sortBy === 'date-asc') return new Date(a.date).getTime() - new Date(b.date).getTime();
if (sortBy === 'amount-desc') return b.amount - a.amount;
if (sortBy === 'amount-asc') return a.amount - b.amount;
return 0;
});

const totalIncome = financeTransactions
  .filter(t => t.type === 'Income' && t.status === 'Completed')
  .reduce((sum, t) => sum + t.amount, 0);

const totalExpenses = financeTransactions
  .filter(t => t.type === 'Expense' && t.status === 'Completed')
  .reduce((sum, t) => sum + t.amount, 0);

const netBalance = totalIncome - totalExpenses;

const todayStr = new Date().toISOString().split('T')[0];
const todayIncome = financeTransactions
  .filter(t => t.type === 'Income' && t.status === 'Completed' && t.date === todayStr)
  .reduce((sum, t) => sum + t.amount, 0);

const todayExpenses = financeTransactions
  .filter(t => t.type === 'Expense' && t.status === 'Completed' && t.date === todayStr)
  .reduce((sum, t) => sum + t.amount, 0);

const cashBalance = financialAccounts
  .filter(a => a.accountType === 'Cash' || a.accountType === 'Petty Cash Account')
  .reduce((sum, a) => sum + a.currentBalance, 0);

const bankBalance = financialAccounts
  .filter(a => a.accountType !== 'Cash' && a.accountType !== 'Petty Cash Account')
  .reduce((sum, a) => sum + a.currentBalance, 0);

// Handle Manual Entry Submit
const handleManualSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  if (!manualForm.description || !manualForm.amount || isNaN(Number(manualForm.amount))) {
    addToast('warning', 'Validation Error', 'Please enter a valid description and amount.');
```

```
    return;
  }

  const amt = parseFloat(manualForm.amount);
  const newTxn = addFinanceTransaction({
    date: manualForm.date,
    time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
    type: manualForm.type,
    category: manualForm.category,
    sourceModule: 'Manual',
    referenceNumber: `MNL-${Date.now().toString().slice(-6)}`,
    description: manualForm.description,
    amount: amt,
    paymentMode: manualForm.paymentMode,
    account: manualForm.account,
    branch: manualForm.branch,
    academicYear: manualForm.academicYear,
    status: 'Completed',
    createdBy: 'Finance Admin',
    approvedBy: 'Chief Accountant',
    notes: manualForm.notes,
    attachments: manualForm.attachmentName ? [manualForm.attachmentName] : []
  });
```

```

addToast('success', 'Transaction Recorded', `Master Ledger entry ${newTxn.transactionId} created.`);
setIsManualModalOpen(false);
setManualForm({
  type: 'Income',
  category: 'Donations & Grants',
  description: '',
  amount: '',
  paymentMode: 'Bank Transfer',
  account: 'Main Bank Account',
  date: new Date().toISOString().split('T')[0],
  branch: 'Main Campus',
  academicYear: '2025-2026',
  notes: '',
  attachmentName: ''
});
});

// Handle Reversal Submit
const handleReversalSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  if (!txnToReverse || !reversalReason.trim()) return;

  reverseFinanceTransaction(txnToReverse.transactionId, reversalReason, 'Finance Administrator');
  addToast('info', 'Transaction Reversed', `Transaction ${txnToReverse.transactionId} has been reversed.`);
  setTxnToReverse(null);
  setReversalReason('');
};

// Handle Add Category Submit
const handleSaveCategory = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCatForm.name.trim()) return;

  addFinancialCategory({
    name: newCatForm.name.trim(),
    type: newCatForm.type,
    sourceModule: newCatForm.sourceModule,
    status: 'Active',
    isSystem: false
  });

  addToast('success', 'Category Created', `Category '${newCatForm.name}' added.`);
  setIsAddCategoryOpen(false);
  setNewCatForm({ name: '', type: 'Income', sourceModule: 'Manual' });
};

// Handle Add Account Submit
const handleSaveAccount = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newAccForm.accountName.trim()) return;

  addFinancialAccount({
    accountName: newAccForm.accountName.trim(),
    accountType: newAccForm.accountType,
    accountNumber: newAccForm.accountNumber,
    bankName: newAccForm.bankName,
    branchName: newAccForm.branchName,
    currentBalance: parseFloat(newAccForm.currentBalance) || 0,
    currency: 'INR',
    status: 'Active'
  });
};

```

$p-2'' >$

```

bg-slate-700 text-slate-700 dark:text-slate-200 text-xs font-bold flex items-center gap-2 transition-all border border-slate-200 dark:border-slate-700"
    >
    <Download className="w-4 h-4 text-emerald-600" /> Export Excel / CSV
  </button>

  <button
    onClick={() => setIsManualModalOpen(true)}
    className="px-4 py-2.5 rounded-xl bg-sky-600 hover:bg-sky-500 text-white text-xs font-bold shadow-lg shadow-sky-600/20 flex items-center gap-2 transition-all"
    >
    <Plus className="w-4 h-4" /> Add Manual Transaction
  </button>
</div>
</div>

{/* SUMMARY KPI CARDS */}
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
  {/* Total Income */}
  <div className="p-4 rounded-2xl bg-gradient-to-br from-emerald-50 to-slate-50 dark:from-emerald-950/20 dark:to-slate-900 border border-emerald-200/80 dark:border-emerald-900/40 shadow-xs space-y-2">
    <div className="flex items-center justify-between">
      <span className="text-[10px] font-black uppercase text-emerald-600 tracking-wider">Total Income</span>
      <div className="p-2 rounded-xl bg-emerald-100 dark:bg-emerald-900/60 text-emerald-700 dark:text-emerald-300">
        <ArrowUpRight className="w-4 h-4" />
      </div>
    </div>
    <p className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white">{formatCurrency(totalIncome)}</p>
    <div className="flex items-center justify-between text-[11px] text-slate-500">
      <span>Today: <strong className="text-emerald-600">+{formatCurrency(todayIncome)}</strong></span>
      <span className="text-emerald-600 font-bold">● Active Ledger</span>
    </div>
  </div>

  {/* Total Expenses */}
  <div className="p-4 rounded-2xl bg-gradient-to-br from-rose-50 to-slate-50 dark:from-rose-950/20 dark:to-slate-900 border border-rose-200/80 dark:border-rose-900/40 shadow-xs space-y-2">
    <div className="flex items-center justify-between">
      <span className="text-[10px] font-black uppercase text-rose-600 tracking-wider">Total Expenses</span>
      <div className="p-2 rounded-xl bg-rose-100 dark:bg-rose-900/60 text-rose-700 dark:text-rose-300">
        <ArrowDownLeft className="w-4 h-4" />
      </div>
    </div>
    <p className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white">{formatCurrency(totalExpenses)}</p>
    <div className="flex items-center justify-between text-[11px] text-slate-500">
      <span>Today: <strong className="text-rose-600">-{formatCurrency(todayExpenses)}</strong></span>
      <span className="text-rose-600 font-bold">● Verified Payroll/Bills</span>
    </div>
  </div>

  {/* Net Surplus / Balance */}
  <div className="p-4 rounded-2xl bg-gradient-to-br from-sky-50 to-slate-50 dark:from-sky-950/20 dark:to-slate-900 border border-sky-200/80 dark:border-sky-900/40 shadow-xs space-y-2">
    <div className="flex items-center justify-between">
      <span className="text-[10px] font-black uppercase text-sky-600 tracking-wider">Net Financial Balance</span>

```

```

        <div className="p-2 rounded-xl bg-sky-100 dark:bg-sky-900/60 text-sky-700 dark:text-sky-300">
          <TrendingUp className="w-4 h-4" />
        </div>
      </div>
      <p className={`text-xl sm:text-2xl font-black ${netBalance >= 0 ? 'text-emerald-600 dark:text-eme
rald-400' : 'text-rose-600'}`}>
        {formatCurrency(netBalance)}
      </p>
      <div className="flex items-center justify-between text-[11px] text-slate-500">
        <span>Overall Cash Flow</span>
        <span className="font-bold text-slate-700 dark:text-slate-300">{netBalance >= 0 ? 'Surplus' :
'Deficit'}</span>
      </div>
    </div>

    { /* Liquidity Accounts */ }
    <div className="p-4 rounded-2xl bg-gradient-to-br from-sky-50 to-slate-50 dark:from-sky-950/20 dar
k:to-slate-900 border border-sky-200/80 dark:border-sky-900/40 shadow-xs space-y-2">
      <div className="flex items-center justify-between">
        <span className="text-[10px] font-black uppercase text-sky-600 tracking-wider">Liquidity Accoun
ts</span>
        <div className="p-2 rounded-xl bg-sky-100 dark:bg-sky-900/60 text-sky-700 dark:text-sky-300">
          <CreditCard className="w-4 h-4" />
        </div>
      </div>
      <div className="space-y-1">
        <div className="flex justify-between text-xs font-bold text-slate-900 dark:text-white">
          <span className="text-slate-500 font-medium">Bank Balance:</span>
          <span className="text-sky-600">{formatCurrency(bankBalance)}</span>
        </div>
        <div className="flex justify-between text-xs font-bold text-slate-900 dark:text-white">
          <span className="text-slate-500 font-medium">Cash Vault:</span>
          <span className="text-emerald-600">{formatCurrency(cashBalance)}</span>
        </div>
      </div>
    </div>

    { /* SUB-MODULE TABS */ }
    <div className="flex items-center gap-1.5 p-1 rounded-2xl bg-slate-100 dark:bg-slate-800/60 max-w-xl
border border-slate-200/40 dark:border-slate-800">
      <button
        onClick={() => setActiveSubTab('ledger')}
        className={`flex-1 py-2 px-3 rounded-xl text-xs font-bold transition-all flex items-center justif
y-center gap-1.5 ${
          activeSubTab === 'ledger'
            ? 'bg-white dark:bg-slate-950 text-sky-600 dark:text-sky-400 shadow-sm'
            : 'text-slate-500 hover:text-slate-900 dark:dark: hover:text-white'
          }`}
      >
        <FileSpreadsheet className="w-3.5 h-3.5" /> Master Ledger ({financeTransactions.length})
      </button>

      <button
        onClick={() => setActiveSubTab('categories-accounts')}
        className={`flex-1 py-2 px-3 rounded-xl text-xs font-bold transition-all flex items-center justif
y-center gap-1.5 ${
          activeSubTab === 'categories-accounts'
            ? 'bg-white dark:bg-slate-950 text-sky-600 dark:text-sky-400 shadow-sm'
            : 'text-slate-500 hover:text-slate-900 dark:dark: hover:text-white'
          }`}
      >
        <Layers className="w-3.5 h-3.5" /> Categories & Accounts
      </button>
    </div>
  </div>
</div>

```

```

</button>

<button
  onClick={() => setActiveSubTab('budget')}
  className={`flex-1 py-2 px-3 rounded-xl text-xs font-bold transition-all flex items-center justify
y-center gap-1.5 ${
    activeSubTab === 'budget'
      ? 'bg-white dark:bg-slate-950 text-sky-600 dark:text-sky-400 shadow-sm'
      : 'text-slate-500 hover:text-slate-900 dark:hover:text-white'
    }`}
>
  <PieChart className="w-3.5 h-3.5" /> Budget Integration
</button>
</div>

{/* SUB-TAB 1: MASTER LEDGER TABLE & ADVANCED FILTERS */}
{activeSubTab === 'ledger' && (
  <div className="space-y-4">

    {/* Filter Bar */}
    <div className="p-4 rounded-2xl bg-white dark:bg-slate-900 border border-slate-200 dark:border-sl
ate-800 shadow-xs">
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-3 text-xs">

        {/* Search (Spans 2 columns for wide input view) */}
        <div className="relative sm:col-span-2 lg:col-span-2">
          <Search className="w-4 h-4 absolute left-3 top-2.5 text-slate-400" />
          <input
            type="text"
            placeholder="Search Txn ID, Description, Ref No..."
            value={searchQuery}
            onChange={e => setSearchQuery(e.target.value)}
            className="w-full pl-9 pr-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-s
late-200 dark:border-slate-700 text-slate-900 dark:text-white font-medium outline-none focus:ring-2 focus:r
ing-sky-500/20"
          />
        </div>

        {/* Type Filter */}
        <div>
          <select
            value={typeFilter}
            onChange={e => setTypeFilter(e.target.value as any)}
            className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-
200 dark:border-slate-700 text-slate-900 dark:text-white font-semibold outline-none cursor-pointer"
          >
            <option value="All">All Transaction Types</option>
            <option value="Income">Income (+)</option>
            <option value="Expense">Expense (-)</option>
          </select>
        </div>

        {/* Source Module Filter */}
        <div>
          <select
            value={sourceModuleFilter}
            onChange={e => setSourceModuleFilter(e.target.value)}
            className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-
200 dark:border-slate-700 text-slate-900 dark:text-white font-semibold outline-none cursor-pointer"
          >
            <option value="All">All School Sections</option>
            <option value="Student Fee Collection">Student Fee Collection</option>
            <option value="Admissions">Admissions</option>
          </select>
        </div>
      </div>
    </div>
  )}

```

```

        <option value="Payroll">Payroll</option>
        <option value="Hostel">Hostel</option>
        <option value="Transport">Transport</option>
        <option value="Library">Library</option>
        <option value="Inventory">Inventory</option>
        <option value="Manual">Manual Entries</option>
    </select>
</div>

{/* Category Filter */}
<div>
    <select
        value={categoryFilter}
        onChange={e => setCategoryFilter(e.target.value)}
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-
200 dark:border-slate-700 text-slate-900 dark:text-white font-semibold outline-none cursor-pointer"
    >
        <option value="All">All Categories</option>
        {financialCategories.map(c => (
            <option key={c.id} value={c.name}>{c.name} ({c.type})</option>
        ))}
    </select>
</div>

{/* Payment Mode */}
<div>
    <select
        value={paymentModeFilter}
        onChange={e => setPaymentModeFilter(e.target.value)}
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-
200 dark:border-slate-700 text-slate-900 dark:text-white font-semibold outline-none cursor-pointer"
    >
        <option value="All">All Payment Modes</option>
        <option value="Cash">Cash</option>
        <option value="Bank Transfer">Bank Transfer</option>
        <option value="Cheque">Cheque</option>
        <option value="UPI">UPI</option>
        <option value="Card">Card</option>
        <option value="Online">Online</option>
    </select>
</div>

{/* Status Filter */}
<div>
    <select
        value={statusFilter}
        onChange={e => setStatusFilter(e.target.value)}
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-
200 dark:border-slate-700 text-slate-900 dark:text-white font-semibold outline-none cursor-pointer"
    >
        <option value="All">All Statuses</option>
        <option value="Completed">Completed</option>
        <option value="Pending">Pending</option>
        <option value="Approved">Approved</option>
        <option value="Reversed">Reversed</option>
    </select>
</div>

{/* Sort By */}
<div>
    <select
        value={sortBy}
        onChange={e => setSortBy(e.target.value as any)}

```

```

        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 text-slate-900 dark:text-white font-semibold outline-none cursor-pointer"
      >
        <option value="date-desc">Sort: Date (Newest First)</option>
        <option value="date-asc">Sort: Date (Oldest First)</option>
        <option value="amount-desc">Sort: Amount (High to Low)</option>
        <option value="amount-asc">Sort: Amount (Low to High)</option>
      </select>
    </div>

  </div>
</div>

{ /* Master Ledger Table */}
<div className="bg-white dark:bg-slate-900 rounded-3xl border border-slate-200 dark:border-slate-800 shadow-sm overflow-hidden">
  <div className="overflow-x-auto">
    <table className="w-full text-left border-collapse text-xs">
      <thead>
        <tr className="bg-slate-50 dark:bg-slate-800/60 border-b border-slate-200 dark:border-slate-700 text-slate-500 font-bold uppercase text-[10px] tracking-wider">
          <th className="p-4">Txn ID & Date</th>
          <th className="p-4">Type & Category</th>
          <th className="p-4">Source Module</th>
          <th className="p-4">Reference No.</th>
          <th className="p-4">Description</th>
          <th className="p-4">Amount</th>
          <th className="p-4">Mode & Account</th>
          <th className="p-4">Status</th>
          <th className="p-4 text-right">Actions</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-100 dark:divide-slate-800">
        {filteredTxns.length === 0 ? (
          <tr>
            <td colspan={9} className="p-8 text-center text-slate-400 italic">
              No financial transactions found matching the selected filters.
            </td>
          </tr>
        ) : (
          filteredTxns.map(txn => (
            <tr key={txn.id} className="hover:bg-slate-50/80 dark:hover:bg-slate-800/40 transition-colors">

              { /* Txn ID & Date */}
              <td className="p-4 font-mono font-bold text-slate-900 dark:text-white">
                <span className="block text-sky-600 dark:text-sky-400">{txn.transactionId}</span>
                <span className="text-[10px] text-slate-400 font-normal">{txn.date} {txn.time || ''}</span>
              </td>

              { /* Type & Category */}
              <td className="p-4">
                <div className="space-y-1">
                  <span className={`inline-block px-2 py-0.5 rounded-full text-[9px] font-black uppercase ${
                    txn.type === 'Income'
                      ? 'bg-emerald-100 text-emerald-800 dark:bg-emerald-950 dark:text-emerald-300'
                      : 'bg-rose-100 text-rose-800 dark:bg-rose-950 dark:text-rose-300'
                    }`}>
                    {txn.type}
                  </span>
                </div>
              </td>
            </tr>
          )
        )
      </tbody>
    </table>
  </div>

```

```

        <p className="font-bold text-slate-900 dark:text-white">{txn.category}</p>
      </div>
    </td>

    {/* Source Module */}
    <td className="p-4">
      <span className="px-2.5 py-1 rounded-xl bg-slate-100 dark:bg-slate-800 text-slate
-700 dark:text-slate-300 font-semibold text-[11px] inline-flex items-center gap-1 border">
        {txn.sourceModule}
      </span>
    </td>

    {/* Reference No */}
    <td className="p-4 font-mono font-bold text-slate-700 dark:text-slate-300">
      {txn.referenceNumber}
    </td>

    {/* Description */}
    <td className="p-4 max-w-xs text-slate-700 dark:text-slate-300 font-medium">
      <p className="truncate" title={txn.description}>{txn.description}</p>
    </td>

    {/* Amount */}
    <td className="p-4 font-mono font-black text-sm">
      <span className={txn.type === 'Income' ? 'text-emerald-600 dark:text-emerald-400'
: 'text-rose-600 dark:text-rose-400'}>
        {txn.type === 'Income' ? '+' : '-'}{formatCurrency(txn.amount)}
      </span>
    </td>

    {/* Mode & Account */}
    <td className="p-4">
      <p className="font-bold text-slate-900 dark:text-white">{txn.paymentMode}</p>
      <p className="text-[10px] text-slate-400">{txn.account}</p>
    </td>

    {/* Status */}
    <td className="p-4">
      <span className={`px-2.5 py-0.5 rounded-full text-[10px] font-bold ${
txn.status === 'Completed' ? 'bg-emerald-100 text-emerald-800 dark:bg-emerald-9
50 dark:text-emerald-300' :
txn.status === 'Reversed' ? 'bg-rose-100 text-rose-800 dark:bg-rose-950 dark:te
xt-rose-300' :
'bg-amber-100 text-amber-800 dark:bg-amber-950 dark:text-amber-300'
}`}>
        {txn.status}
      </span>
    </td>

    {/* Actions */}
    <td className="p-4 text-right space-x-1">
      <button
        onClick={() => setSelectedTxnForDetail(txn)}
        className="p-1.5 rounded-lg text-sky-600 hover:bg-sky-50 dark:hover:bg-sky-950
border border-sky-200 dark:border-sky-800"
        title="View Transaction Details"
      >
        <Eye className="w-3.5 h-3.5" />
      </button>

      {txn.status !== 'Reversed' && (
        <button
          onClick={() => setTxnToReverse(txn)}

```

```

        className="p-1.5 rounded-lg text-rose-600 hover:bg-rose-50 dark:hover:bg-rose-950 border border-rose-200 dark:border-rose-800"
        title="Reverse Transaction"
      >
        <RotateCcw className="w-3.5 h-3.5" />
      </button>
    ))
  </td>
</tr>
))
</tbody>
</table>
</div>
</div>
</div>
))

{/* SUB-TAB 2: CATEGORIES & ACCOUNTS MANAGEMENT */}
{activeSubTab === 'categories-accounts' && (
  <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">

    {/* Categories Box */}
    <div className="p-5 rounded-3xl bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm space-y-4">
      <div className="flex items-center justify-between border-b pb-3">
        <div>
          <h3 className="font-bold text-sm text-slate-900 dark:text-white flex items-center gap-2">
            <Layers className="w-4 h-4 text-sky-600" />
            Financial Transaction Categories
          </h3>
          <p className="text-[11px] text-slate-400">Income & Expense classification master</p>
        </div>
        <button
          onClick={() => setIsAddCategoryOpen(true)}
          className="px-3 py-1.5 rounded-xl bg-sky-600 text-white font-bold text-xs flex items-center gap-1 shadow-xs"
        >
          <Plus className="w-3.5 h-3.5" /> Add Category
        </button>
      </div>

      <div className="space-y-2 max-h-[450px] overflow-y-auto pr-1">
        {financialCategories.map(cat => (
          <div key={cat.id} className="p-3 rounded-2xl bg-slate-50 dark:bg-slate-800/60 border flex items-center justify-between text-xs">
            <div>
              <div className="flex items-center gap-2">
                <p className="font-bold text-slate-900 dark:text-white">{cat.name}</p>
                <span className={`px-2 py-0.5 rounded-full text-[9px] font-black ${cat.type === 'Income' ? 'bg-emerald-100 text-emerald-800' : 'bg-rose-100 text-rose-800'}`}>
                  {cat.type}
                </span>
              </div>
              <p className="text-[10px] text-slate-400">Source: {cat.sourceModule || 'Manual'}</p>
            </div>
            <span className="px-2 py-0.5 rounded-md text-[10px] font-bold bg-slate-200 dark:bg-slate-700 text-slate-600 dark:text-slate-300">
              {cat.status}
            </span>
          </div>
        ))}
      </div>
    </div>
  </div>
)

```

```

    ))}
  </div>
</div>

  { /* Accounts Box */ }
  <div className="p-5 rounded-3xl bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm space-y-4">
    <div className="flex items-center justify-between border-b pb-3">
      <div>
        <h3 className="font-bold text-sm text-slate-900 dark:text-white flex items-center gap-2">
          <CreditCard className="w-4 h-4 text-emerald-600" />
          Financial Accounts & Liquidity Vaults
        </h3>
        <p className="text-[11px] text-slate-400">Cash vaults, bank accounts & department accounts
      </p>
      </div>
      <button
        onClick={() => setIsAddAccountOpen(true)}
        className="px-3 py-1.5 rounded-xl bg-emerald-600 text-white font-bold text-xs flex items-center gap-1 shadow-xs"
      >
        <Plus className="w-3.5 h-3.5" /> Add Account
      </button>
    </div>

    <div className="space-y-3 max-h-[450px] overflow-y-auto pr-1">
      {financialAccounts.map(acc => (
        <div key={acc.id} className="p-4 rounded-2xl bg-slate-50 dark:bg-slate-800/60 border flex items-center justify-between text-xs space-y-1">
          <div>
            <h4 className="font-extrabold text-slate-900 dark:text-white text-xs">{acc.accountName}
          </h4>
            <p className="text-[10px] text-slate-400">{acc.accountType} {acc.accountNumber ? `• ${acc.accountNumber}` : ''}</p>
            {acc.bankName && <p className="text-[10px] text-slate-500">{acc.bankName} ({acc.branchName})}</p>}
          </div>
          <div className="text-right">
            <span className="text-[10px] font-bold uppercase text-slate-400">Current Balance</span>
            <p className="font-black text-sm text-emerald-600">{formatCurrency(acc.currentBalance)}
          </p>
          </div>
        </div>
      ))}
    </div>
  </div>
)}

  { /* SUB-TAB 3: BUDGET INTEGRATION */ }
  {activeSubTab === 'budget' && (
    <div className="space-y-4">
      <div className="p-5 rounded-3xl bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm space-y-4">
        <div className="flex items-center justify-between border-b pb-3">
          <div>
            <h3 className="font-bold text-sm text-slate-900 dark:text-white flex items-center gap-2">
              <PieChart className="w-4 h-4 text-sky-600" />
              Budget Allocation & Consumption Tracking
            </h3>
            <p className="text-[11px] text-slate-400">Automated budget deduction based on Master Ledger expenses</p>

```

```

    </div>
  </div>

  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    {financialBudgets.map(b => {
      const percent = Math.min(100, Math.round((b.consumedAmount / (b.allocatedAmount || 1)) * 10
0));
      return (
        <div key={b.id} className="p-4 rounded-2xl bg-slate-50 dark:bg-slate-800/60 border space-
y-3">
          <div className="flex justify-between items-start">
            <div>
              <h4 className="font-bold text-xs text-slate-900 dark:text-white">{b.categoryName}</
h4>
              <p className="text-[10px] text-slate-400">{b.branch} • AY {b.academicYear}</p>
            </div>
            <span className={`px-2.5 py-0.5 rounded-full text-[10px] font-bold ${
              percent >= 90 ? 'bg-rose-100 text-rose-800' : 'bg-emerald-100 text-emerald-800'
            }`} >
              {percent}% Consumed
            </span>
          </div>

          <div className="w-full bg-slate-200 dark:bg-slate-700 h-2.5 rounded-full overflow-hidde
n">
            <div
              className={`h-full transition-all ${percent >= 90 ? 'bg-rose-500' : 'bg-sky-600'}`}
              style={{ width: `${percent}%` }}
            />
          </div>

          <div className="grid grid-cols-3 text-center text-xs pt-1 border-t border-slate-200/80
dark:border-slate-700/60">
            <div>
              <span className="block text-[9px] font-bold text-slate-400 uppercase">Allocated</sp
an>
              <span className="font-extrabold text-slate-800 dark:text-slate-200">{formatCurrency
(b.allocatedAmount)}</span>
            </div>
            <div>
              <span className="block text-[9px] font-bold text-slate-400 uppercase">Consumed</spa
n>
              <span className="font-extrabold text-rose-600">{formatCurrency(b.consumedAmount)}</
span>
            </div>
            <div>
              <span className="block text-[9px] font-bold text-slate-400 uppercase">Remaining</sp
an>
              <span className="font-extrabold text-emerald-600">{formatCurrency(b.remainingAmoun
t)}</span>
            </div>
          </div>
        </div>
      );
    })}
  </div>
</div>
)}

{/* TRANSACTION DETAIL VIEW MODAL */}
{selectedTxnForDetail && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/70 backdrop-bl

```

```

ur-sm animate-in fade-in">
  <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-
3xl max-w-xl w-full p-6 shadow-2xl space-y-4 max-h-[90vh] flex flex-col text-xs">

    {/* Header */}
    <div className="flex items-center justify-between border-b pb-3">
      <div>
        <h3 className="font-black text-sm text-slate-900 dark:text-white flex items-center gap-2">
          <FileText className="w-4 h-4 text-sky-600" />
          Transaction Details: {selectedTxnForDetail.transactionId}
        </h3>
        <p className="text-[10px] text-slate-400">Master Ledger Central Audit Entry</p>
      </div>
      <button onClick={() => setSelectedTxnForDetail(null)} className="p-1 text-slate-400 hover:tex
t-slate-600"><X className="w-5 h-5" /></button>
    </div>

    {/* Content Body */}
    <div className="flex-1 overflow-y-auto space-y-4 pr-1">

      <div className="p-4 rounded-2xl bg-slate-50 dark:bg-slate-800/60 border flex items-center jus
tify-between">
        <div>
          <span className="text-[10px] font-bold uppercase text-slate-400">Transaction Type & Amoun
t</span>

          <div className="flex items-center gap-2">
            <span className={`px-2 py-0.5 rounded-full text-[10px] font-black ${
              selectedTxnForDetail.type === 'Income' ? 'bg-emerald-100 text-emerald-800' : 'bg-rose
-100 text-rose-800'
            }`}>
              {selectedTxnForDetail.type}
            </span>
            <span className="font-black text-lg text-slate-900 dark:text-white">
              {formatCurrency(selectedTxnForDetail.amount)}
            </span>
          </div>
        </div>

        <span className="px-3 py-1 rounded-full font-bold text-xs bg-emerald-100 text-emerald-800">
          {selectedTxnForDetail.status}
        </span>
      </div>

      <div className="grid grid-cols-2 gap-3">
        <div className="p-3 rounded-xl bg-slate-50 dark:bg-slate-800/50 border">
          <span className="text-[10px] font-bold text-slate-400 uppercase">Category</span>
          <p className="font-bold text-slate-900 dark:text-white">{selectedTxnForDetail.category}</p>
        </div>
        <div className="p-3 rounded-xl bg-slate-50 dark:bg-slate-800/50 border">
          <span className="text-[10px] font-bold text-slate-400 uppercase">Source Module</span>
          <p className="font-bold text-sky-600">{selectedTxnForDetail.sourceModule}</p>
        </div>
        <div className="p-3 rounded-xl bg-slate-50 dark:bg-slate-800/50 border">
          <span className="text-[10px] font-bold text-slate-400 uppercase">Reference Number</span>
          <p className="font-mono font-bold text-slate-900 dark:text-white">{selectedTxnForDetail.r
eferenceNumber}</p>
        </div>
        <div className="p-3 rounded-xl bg-slate-50 dark:bg-slate-800/50 border">
          <span className="text-[10px] font-bold text-slate-400 uppercase">Payment Mode & Account</
span>
          <p className="font-bold text-slate-900 dark:text-white">{selectedTxnForDetail.paymentMod
e} ({selectedTxnForDetail.account})</p>
        </div>
      </div>
    </div>
  </div>

```

```

    </div>
    <div className="p-3 rounded-xl bg-slate-50 dark:bg-slate-800/50 border">
      <span className="text-[10px] font-bold text-slate-400 uppercase">Date & Time</span>
      <p className="font-bold text-slate-900 dark:text-white">{selectedTxnForDetail.date} {selectedTxnForDetail.time || ''}</p>
    </div>
    <div className="p-3 rounded-xl bg-slate-50 dark:bg-slate-800/50 border">
      <span className="text-[10px] font-bold text-slate-400 uppercase">Branch & Academic Year</span>
      <p className="font-bold text-slate-900 dark:text-white">{selectedTxnForDetail.branch} ({selectedTxnForDetail.academicYear})</p>
    </div>
  </div>

  <div className="p-3 rounded-xl bg-slate-50 dark:bg-slate-800/50 border">
    <span className="text-[10px] font-bold text-slate-400 uppercase">Description</span>
    <p className="font-medium text-slate-800 dark:text-slate-200 mt-0.5">{selectedTxnForDetail.description}</p>
  </div>

  {/* Audit Log Timeline */}
  {selectedTxnForDetail.auditTrail && selectedTxnForDetail.auditTrail.length > 0 && (
    <div className="space-y-2 pt-2 border-t">
      <h4 className="font-bold uppercase text-slate-400 text-[10px]">Audit Trail Timeline</h4>
      <div className="space-y-2">
        {selectedTxnForDetail.auditTrail.map(log => (
          <div key={log.id} className="p-2.5 rounded-xl bg-slate-100 dark:bg-slate-800 border text-[10px] space-y-0.5">
            <div className="flex justify-between font-bold">
              <span className="text-sky-600">{log.action}</span> by <span>{log.user}</span>
              <span className="text-slate-400 font-normal">{log.timestamp}</span>
            </div>
            <p>{log.notes} && <span className="text-slate-500 italic">{log.notes}</span></p>
          </div>
        ))}
      </div>
    </div>
  )}

  </div>

  <div className="flex justify-end pt-2 border-t">
    <button onClick={() => setSelectedTxnForDetail(null)} className="px-4 py-2 font-bold bg-slate-800 text-white rounded-xl">Close</button>
  </div>
</div>
)}

{/* REVERSAL CONFIRMATION MODAL */}
{txnToReverse && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/70 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-3xl max-w-md w-full p-6 shadow-2xl space-y-4 text-xs">
      <div className="flex items-center justify-between border-b pb-3">
        <h3 className="font-bold text-rose-600 flex items-center gap-2">
          <RotateCcw className="w-4 h-4" />
          Reverse Transaction {txnToReverse.transactionId}
        </h3>
        <button onClick={() => setTxnToReverse(null)} className="p-1 text-slate-400">X<span className="w-5 h-5" /></button>
      </div>
    </div>
  </div>
)}

```

```

    <p className="text-slate-600 dark:text-slate-300">
      Reversing this transaction will post an offsetting reversal entry in the Master Ledger to pre
      serve financial integrity and audit history.
    </p>

    <form onSubmit={handleReversalSubmit} className="space-y-3">
      <div>
        <label className="block font-bold mb-1">Reason for Reversal <span className="text-rose-500
font-bold ml-0.5">*</span></label>
        <textarea
          required
          rows={3}
          placeholder="e.g. Duplicate entry, Incorrect account, Returned payment..."
          value={reversalReason}
          onChange={e => setReversalReason(e.target.value)}
          className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
        />
      </div>

      <div className="flex justify-end gap-2 pt-2 border-t">
        <button type="button" onClick={() => setTxnToReverse(null)} className="px-4 py-2 font-semib
old bg-slate-200 dark:bg-slate-800 rounded-xl">Cancel</button>
        <button type="submit" className="px-4 py-2 font-bold bg-rose-600 text-white rounded-xl shad
ow-md">Confirm Reversal</button>
      </div>
    </form>
  </div>
</div>
)}}

{/* MANUAL TRANSACTION CREATION MODAL */}
{isManualModalOpen && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/70 backdrop-bl
ur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-
3xl max-w-xl w-full p-6 shadow-2xl space-y-4 max-h-[90vh] flex flex-col text-xs">
      <div className="flex items-center justify-between border-b pb-3">
        <h3 className="font-black text-sm text-slate-900 dark:text-white flex items-center gap-2">
          <Plus className="w-4 h-4 text-sky-600" />
          Add Manual Financial Transaction
        </h3>
        <button onClick={() => setIsManualModalOpen(false)} className="p-1 text-slate-400"><X classNa
me="w-5 h-5" /></button>
      </div>

      <form onSubmit={handleManualSubmit} className="flex-1 overflow-y-auto space-y-3 pr-1">
        <div className="grid grid-cols-2 gap-3">
          <div>
            <label className="block font-bold mb-1">Transaction Type <span className="text-rose-500 f
ont-bold ml-0.5">*</span></label>
            <select
              value={manualForm.type}
              onChange={e => setManualForm({ ...manualForm, type: e.target.value as any })}
              className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold t
ext-sky-600"
            >
              <option value="Income">Income (+)</option>
              <option value="Expense">Expense (-)</option>
            </select>
          </div>
          <div>
            <label className="block font-bold mb-1">Category <span className="text-rose-500 font-bold

```

```

ml-0.5">*</span></label>
    <select
      value={manualForm.category}
      onChange={e => setManualForm({ ...manualForm, category: e.target.value })}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-semibo
ld"
    >
      {financialCategories
        .filter(c => c.type === manualForm.type)
        .map(c => (
          <option key={c.id} value={c.name}>{c.name}</option>
        ))}
    </select>
  </div>
</div>

<div>
  <label className="block font-bold mb-1">Description <span className="text-rose-500 font-bol
d ml-0.5">*</span></label>
  <input
    type="text"
    required
    placeholder="e.g. Alumni Endowment Fund Donation, Emergency Plumbing Repair"
    value={manualForm.description}
    onChange={e => setManualForm({ ...manualForm, description: e.target.value })}
    className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
  />
</div>

<div className="grid grid-cols-2 gap-3">
  <div>
    <label className="block font-bold mb-1">Amount (INR) <span className="text-rose-500 font-
bold ml-0.5">*</span></label>
    <input
      type="number"
      required
      min="1"
      placeholder="e.g. 5000"
      value={manualForm.amount}
      onChange={e => setManualForm({ ...manualForm, amount: e.target.value })}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-mono f
ont-bold"
    />
  </div>
  <div>
    <label className="block font-bold mb-1">Payment Mode <span className="text-rose-500 font-
bold ml-0.5">*</span></label>
    <select
      value={manualForm.paymentMode}
      onChange={e => setManualForm({ ...manualForm, paymentMode: e.target.value as any })}
      className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-semibo
ld"
    >
      <option value="Cash">Cash</option>
      <option value="Bank Transfer">Bank Transfer</option>
      <option value="Cheque">Cheque</option>
      <option value="UPI">UPI</option>
      <option value="Card">Card</option>
      <option value="Online">Online</option>
    </select>
  </div>
</div>

```

```

        <div className="grid grid-cols-2 gap-3">
          <div>
            <label className="block font-bold mb-1">Linked Financial Account <span className="text-rose-500 font-bold ml-0.5">*</span></label>
            <select
              value={manualForm.account}
              onChange={e => setManualForm({ ...manualForm, account: e.target.value as any })}
              className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-semibold"
            >
              {financialAccounts.map(a => (
                <option key={a.id} value={a.accountType}>{a.accountName} ({formatCurrency(a.currentBalance)})</option>
              ))}
            </select>
          </div>
          <div>
            <label className="block font-bold mb-1">Transaction Date <span className="text-rose-500 font-bold ml-0.5">*</span></label>
            <input
              type="date"
              required
              value={manualForm.date}
              onChange={e => setManualForm({ ...manualForm, date: e.target.value })}
              className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-mono"
            />
          </div>
        </div>

        <div>
          <label className="block font-bold mb-1">Notes / Remarks (Optional)</label>
          <textarea
            rows={2}
            placeholder="Enter audit notes or approval details..."
            value={manualForm.notes}
            onChange={e => setManualForm({ ...manualForm, notes: e.target.value })}
            className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border"
          />
        </div>

        <div className="flex justify-end gap-2 pt-2 border-t">
          <button type="button" onClick={() => setIsManualModalOpen(false)} className="px-4 py-2 font-semibold bg-slate-200 dark:bg-slate-800 rounded-xl">Cancel</button>
          <button type="submit" className="px-5 py-2 font-bold bg-sky-600 text-white rounded-xl shadow-md">Post to Master Ledger</button>
        </div>
      </form>
    </div>
  </div>
)}

{/* ADD CATEGORY SUB-MODAL */}
{isAddCategoryOpen && (
  <div className="fixed inset-0 z-60 flex items-center justify-center p-4 bg-slate-950/70 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border rounded-3xl max-w-md w-full p-6 shadow-2xl space-y-3 text-xs">
      <div className="flex justify-between border-b pb-2">
        <h3 className="font-bold text-slate-900 dark:text-white">Add Financial Category</h3>
        <button onClick={() => setIsAddCategoryOpen(false)}>X</button>
      </div>
      <form onSubmit={handleSaveCategory} className="space-y-3">
        <div>

```

```

        <label className="block font-semibold mb-1">Category Name <span className="text-rose-500 font-bold ml-0.5">*</span></label>
        <input type="text" required value={newCatForm.name} onChange={e => setNewCatForm({ ...newCatForm, name: e.target.value })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border" />
      </div>
      <div>
        <label className="block font-semibold mb-1">Type <span className="text-rose-500 font-bold ml-0.5">*</span></label>
        <select value={newCatForm.type} onChange={e => setNewCatForm({ ...newCatForm, type: e.target.value as any })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border">
          <option value="Income">Income</option>
          <option value="Expense">Expense</option>
        </select>
      </div>
      <div className="flex justify-end gap-2 pt-2 border-t">
        <button type="button" onClick={() => setIsAddCategoryOpen(false)} className="px-3 py-1.5 rounded-xl bg-slate-200">Cancel</button>
        <button type="submit" className="px-4 py-1.5 rounded-xl bg-sky-600 text-white font-bold">Save Category</button>
      </div>
    </form>
  </div>
</div>
)}

{/* ADD ACCOUNT SUB-MODAL */}
{isAddAccountOpen && (
  <div className="fixed inset-0 z-60 flex items-center justify-center p-4 bg-slate-950/70 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border rounded-3xl max-w-md w-full p-6 shadow-2xl space-y-3 text-xs">
      <div className="flex justify-between border-b pb-2">
        <h3 className="font-bold text-slate-900 dark:text-white">Add Financial Account</h3>
        <button onClick={() => setIsAddAccountOpen(false)}><X className="w-4 h-4" /></button>
      </div>
      <form onSubmit={handleSaveAccount} className="space-y-3">
        <div>
          <label className="block font-semibold mb-1">Account Name <span className="text-rose-500 font-bold ml-0.5">*</span></label>
          <input type="text" required placeholder="e.g. HDFC Hostel Operations" value={newAccForm.accountName} onChange={e => setNewAccForm({ ...newAccForm, accountName: e.target.value })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border" />
        </div>
        <div>
          <label className="block font-semibold mb-1">Account Type <span className="text-rose-500 font-bold ml-0.5">*</span></label>
          <select value={newAccForm.accountType} onChange={e => setNewAccForm({ ...newAccForm, accountType: e.target.value as any })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border">
            <option value="Cash">Cash</option>
            <option value="Main Bank Account">Main Bank Account</option>
            <option value="Salary Account">Salary Account</option>
            <option value="Hostel Account">Hostel Account</option>
            <option value="Transport Account">Transport Account</option>
            <option value="Petty Cash Account">Petty Cash Account</option>
          </select>
        </div>
        <div>
          <label className="block font-semibold mb-1">Opening Balance (INR) <span className="text-rose-500 font-bold ml-0.5">*</span></label>
          <input type="number" required value={newAccForm.currentBalance} onChange={e => setNewAccForm({ ...newAccForm, currentBalance: e.target.value })} className="w-full px-3 py-2 rounded-xl bg-slate-50 da

```

```
rk:bg-slate-800 border font-mono" />
    </div>
    <div className="flex justify-end gap-2 pt-2 border-t">
      <button type="button" onClick={() => setIsAddAccountOpen(false)} className="px-3 py-1.5 rounded-xl bg-slate-200">Cancel</button>
      <button type="submit" className="px-4 py-1.5 rounded-xl bg-emerald-600 text-white font-bold">Save Account</button>
    </div>
  </form>
</div>
</div>
)}

</div>
);
};
export default TransactionsMasterLedgerView;
```

 Finance/ScholarshipsView.tsx

TSX

```
// @ts-nocheck
import React, { useState, useMemo } from "react";
import { formatCurrency } from "../../utils/currency";
import {
  Gift,
  Plus,
  Search,
  Edit,
  Trash2,
  CheckCircle2,
  UserPlus,
  X,
  Filter,
  GraduationCap,
  Award,
} from "lucide-react";
import {
  Scholarship,
  ScholarshipType,
  StudentScholarship,
} from "../../types";
import { useData } from "../../context/DataContext";
import { useToast } from "../../context/ToastContext";
import { Badge } from "../../common/Badge";
import { ExportButton } from "../../common/ExportButton";
import { ConfirmModal } from "../../common/ConfirmModal";
import { compareClassesAscending } from "../../utils/classSorter";

const SCHOLARSHIP_TYPES: ScholarshipType[] = [
  "Merit",
  "Government",
  "Minority",
  "Sports",
  "Staff Child",
  "Management",
  "Financial Aid",
];

export const ScholarshipsView: React.FC = () => {
  const {
    scholarships,
    studentScholarships,
    feeHeads,
    students,
    addScholarship,
    updateScholarship,
    deleteScholarship,
    assignScholarshipToStudent,
    revokeStudentScholarship,
    academicClasses,
  } = useData();
  const { addToast } = useToast();

  const [activeTab, setActiveTab] = useState<"master" | "allocated">("master");
  const [query, setQuery] = useState("");
  const [selectedType, setSelectedType] = useState<string>("All");
  const [allocatedClassFilter, setAllocatedClassFilter] =
    useState<string>("All");
```

```

const [isModalOpen, setIsModalOpen] = useState(false);
const [editingSch, setEditingSch] = useState<Scholarship | null>(null);
const [deletingSch, setDeletingSch] = useState<Scholarship | null>(null);

// Allocation Modal with Class Filter & Search
const [isAllocModalOpen, setIsAllocModalOpen] = useState(false);
const [allocClassFilter, setAllocClassFilter] = useState<string>("All");
const [allocSearchQuery, setAllocSearchQuery] = useState<string>("");
const [allocStudentId, setAllocStudentId] = useState("");
const [allocScholarshipId, setAllocScholarshipId] = useState(
  scholarships[0]?.id || "",
);

const sortedClasses = useMemo(() => {
  return [...academicClasses].sort((a, b) =>
    compareClassesAscending(a.name, b.name),
  );
}, [academicClasses]);

// Filtered Students for the Award Modal
const filteredStudentsForAward = useMemo(() => {
  return students.filter((st) => {
    const matchesClass =
      allocClassFilter === "All" || st.className === allocClassFilter;
    const q = allocSearchQuery.trim().toLowerCase();
    const matchesSearch =
      !q ||
      `${st.firstName} ${st.lastName}`.toLowerCase().includes(q) ||
      (st.admissionNo && st.admissionNo.toLowerCase().includes(q)) ||
      (st.rollNumber && st.rollNumber.toLowerCase().includes(q));
    return matchesClass && matchesSearch;
  });
}, [students, allocClassFilter, allocSearchQuery]);

const selectedStudentDetails = useMemo(() => {
  return students.find((s) => s.id === allocStudentId);
}, [students, allocStudentId]);

const selectedScholarshipDetails = useMemo(() => {
  return scholarships.find((s) => s.id === allocScholarshipId);
}, [scholarships, allocScholarshipId]);

const [formData, setFormData] = useState<Partial<Scholarship>>>({
  name: "",
  code: "",
  type: "Merit",
  discountType: "Percentage",
  percentage: 15,
  fixedAmount: 0,
  applicableFeeHeadIds: ["FH-001"],
  applicableClasses: ["Class 9", "Class 10", "Class 11", "Class 12"],
  startDate: "2026-04-01",
  endDate: "2027-03-31",
  eligibility: "GPA >= 3.8",
  description: "Academic excellence grant",
  status: "Active",
});

const filteredScholarships = scholarships.filter((s) => {
  const matchesQuery =
    s.name.toLowerCase().includes(query.toLowerCase()) ||
    s.code.toLowerCase().includes(query.toLowerCase());
  const matchesType = selectedType === "All" || s.type === selectedType;

```

```
    return matchesQuery && matchesType;
  });

const filteredAwardedList = studentScholarships.filter((ss) => {
  const q = query.trim().toLowerCase();
  const matchesQuery =
    !q ||
    ss.studentName.toLowerCase().includes(q) ||
    ss.scholarshipName.toLowerCase().includes(q);
  const matchesClass =
    allocatedClassFilter === "All" ||
    (ss as any).className === allocatedClassFilter;
  return matchesQuery && matchesClass;
});

const handleOpenAdd = () => {
  setEditingSch(null);
  setFormData({
    name: "",
    code: "SCH-" + Math.floor(100 + Math.random() * 900),
    type: "Merit",
    discountType: "Percentage",
    percentage: 15,
    fixedAmount: 0,
    applicableFeeHeadIds: feeHeads.slice(0, 1).map((h) => h.id),
    applicableClasses: academicClasses.map((c) => c.name),
    startDate: "2026-04-01",
    endDate: "2027-03-31",
    eligibility: "Merit criteria",
    description: "Educational Grant",
    status: "Active",
  });
  setIsModalOpen(true);
};

const handleOpenEdit = (s: Scholarship) => {
  setEditingSch(s);
  setFormData(s);
  setIsModalOpen(true);
};

const handleOpenAlloc = () => {
  setAllocClassFilter("All");
  setAllocSearchQuery("");
  setAllocStudentId("");
  if (scholarships.length > 0) {
    setAllocScholarshipId(scholarships[0].id);
  }
  setIsAllocModalOpen(true);
};

const handleSubmit = async (e: React.SyntheticEvent) => {
  e.preventDefault();
  if (!formData.name || !formData.code) {
    addToast(
      "warning",
      "Validation Error",
      "Scholarship name and code are required.",
    );
    return;
  }
}

if (editingSch) {
```

```

    await updateScholarship(editingSch.id, formData);
    addToast("success", "Scholarship Updated", `Updated ${formData.name}`);
  } else {
    await addScholarship(formData as Omit<Scholarship, "id">);
    addToast("success", "Scholarship Created", `Created ${formData.name}`);
  }
  setIsModalOpen(false);
};

const handleAllocSubmit = async (e: React.SyntheticEvent) => {
  e.preventDefault();
  if (!allocStudentId || !allocScholarshipId) {
    addToast(
      "warning",
      "Selection Required",
      "Select both student and scholarship.",
    );
    return;
  }
  await assignScholarshipToStudent(allocStudentId, allocScholarshipId);
  addToast(
    "success",
    "Scholarship Awarded",
    "Awarded scholarship to student successfully.",
  );
  setIsAllocModalOpen(false);
};

return (
  <div className="space-y-6 animate-in fade-in">
    {/* Header */}
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
      <div>
        <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center gap-2">
          <Gift className="w-6 h-6 text-sky-500" /> Scholarships
        </h2>
      </div>

      <div className="flex items-center gap-3">
        <button
          onClick={handleOpenAdd}
          className="px-4 py-2.5 rounded-xl bg-sky-600 hover:bg-sky-500 text-white text-xs font-bold shadow-lg shadow-sky-500/20 flex items-center gap-2 transition-all"
          >
          <Plus className="w-4 h-4" /> Create Scholarship
        </button>
        <ExportButton data={activeTab === 'master' ? filteredScholarships : filteredAwardedList} filename="scholarships" />
      </div>
    </div>

    {/* Tabs & Search Filter Bar */}
    <div className="glass-card p-4 rounded-2xl flex flex-col sm:flex-row items-center justify-between gap-4">
      <div className="flex items-center gap-1 bg-slate-100 dark:bg-slate-800 p-1 rounded-xl w-full sm:w-auto">
        <button
          onClick={() => setActiveTab("master")}
          className={`px-4 py-2 rounded-lg text-xs font-bold transition-all ${
            activeTab === "master"
              ? "bg-white dark:bg-slate-900 text-brand-600 shadow-sm"
              : "text-slate-500"
          }`
        >

```

```

    }}
  >
    Scholarship Master Schemes ({scholarships.length})
  </button>
  <button
    onClick={() => setActiveTab("allocated")}
    className={`px-4 py-2 rounded-lg text-xs font-bold transition-all ${
      activeTab === "allocated"
        ? "bg-white dark:bg-slate-900 text-brand-600 shadow-sm"
        : "text-slate-500"
      }`}
  >
    Student Awarded Scholarships ({studentScholarships.length})
  </button>
</div>

<div className="flex items-center gap-3 w-full sm:w-auto">
  {activeTab === "allocated" && (
    <select
      value={allocatedClassFilter}
      onChange={(e) => setAllocatedClassFilter(e.target.value)}
      className="px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-slate-900 d
ark:text-white outline-none font-medium"
    >
      <option value="All">All Class Grades</option>
      {sortedClasses.map((c) => (
        <option key={c.id} value={c.name}>
          {c.name}
        </option>
      ))}
    </select>
  )}
</div>

<div className="relative w-full sm:w-64">
  <Search className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
  <input
    type="text"
    placeholder={
      activeTab === "master"
        ? "Search scholarship name..."
        : "Search student or scholarship..."
    }
    value={query}
    onChange={(e) => setQuery(e.target.value)}
    className="w-full pl-9 pr-4 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text
-slate-900 dark:text-white outline-none font-medium"
  />
</div>
</div>
</div>

{/* Tab 1: Master Schemes */}
{activeTab === "master" && (
  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    {filteredScholarships.map((s) => (
      <div
        key={s.id}
        className="glass-card p-5 rounded-2xl space-y-3 hover:shadow-md transition-shadow"
      >
        <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate
-800 pb-2">
          <div>
            <span className="text-[10px] font-bold px-2 py-0.5 rounded bg-emerald-50 text-emerald-700

```

```

dark:bg-emerald-950 dark:text-emerald-300">
    {s.type} Type • Code: {s.code}
  </span>
  <h3 className="font-bold text-sm text-slate-900 dark:text-white mt-1">
    {s.name}
  </h3>
</div>

<div className="flex items-center gap-1">
  <button
    onClick={() => handleOpenEdit(s)}
    className="p-1.5 rounded-lg hover:bg-slate-100 dark:hover:bg-slate-800 text-sky-600"
  >
    <Edit className="w-4 h-4" />
  </button>
  <button
    onClick={() => setDeletingSch(s)}
    className="p-1.5 rounded-lg hover:bg-rose-50 dark:hover:bg-rose-950/40 text-rose-600"
  >
    <Trash2 className="w-4 h-4" />
  </button>
</div>
</div>

<div className="space-y-1.5 text-xs text-slate-600 dark:text-slate-300">
  <div className="flex justify-between">
    <span>Benefit Value:</span>
    <span className="font-bold text-emerald-600 dark:text-emerald-400">
      {s.discountType === "Percentage"
        ? `${s.percentage}% Fee Waiver`
        : `${formatCurrency(s.fixedAmount || 0)} Grant`}
    </span>
  </div>
  <div className="flex justify-between">
    <span>Validity Period:</span>
    <span>
      {s.startDate} to {s.endDate}
    </span>
  </div>
  <p className="text-[11px] text-slate-500 pt-1 italic">
    {s.description}
  </p>
</div>
</div>
  )})
</div>
})

/* Tab 2: Awarded List */
{activeTab === "allocated" && (
  <div className="glass-card rounded-2xl overflow-hidden border border-slate-200/80 dark:border-slate-800">
    <div className="overflow-x-auto">
      <table className="w-full text-left border-collapse text-xs">
        <thead>
          <tr className="bg-slate-100/70 dark:bg-slate-800/60 text-slate-500 font-bold uppercase border-b border-slate-200 dark:border-slate-800">
            <th className="py-3.5 px-4">STUDENT NAME</th>
            <th className="py-3.5 px-4">SCHOLARSHIP AWARDED</th>
            <th className="py-3.5 px-4">DISCOUNT APPLIED</th>
            <th className="py-3.5 px-4">DATE AWARDED</th>
            <th className="py-3.5 px-4 text-right">REVOKE</th>
          </tr>

```

ap-2">

```

<div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-3">
  <h3 className="text-base font-bold text-slate-900 dark:text-white">
    {editingSch
      ? "Edit Scholarship Scheme"
      : "Add Scholarship Master"}
  </h3>
  <button
    onClick={() => setIsModalOpen(false)}
    className="text-slate-400 hover:text-slate-600"
  >
    ✕
  </button>
</div>

<form onSubmit={handleSubmit} className="space-y-3 text-xs">
  <div className="grid grid-cols-2 gap-3">
    <div>
      <label className="block font-semibold mb-1">
        Scholarship Name{" "}
        <span className="text-rose-500 font-bold ml-0.5">*</span>
      </label>
      <input
        type="text"
        required
        value={formData.name}
        onChange={(e) =>
          setFormData({ ...formData, name: e.target.value })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border outline-none"
      />
    </div>
    <div>
      <label className="block font-semibold mb-1">
        Scholarship Code{" "}
        <span className="text-rose-500 font-bold ml-0.5">*</span>
      </label>
      <input
        type="text"
        required
        value={formData.code}
        onChange={(e) =>
          setFormData({ ...formData, code: e.target.value })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-mono outline-none"
      />
    </div>
  </div>

  <div className="grid grid-cols-2 gap-3">
    <div>
      <label className="block font-semibold mb-1">
        Scheme Type
      </label>
      <select
        value={formData.type}
        onChange={(e) =>
          setFormData({ ...formData, type: e.target.value as any })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border outline-none font-medium"
      >
    </div>
  </div>

```

```

        >
        {SCHOLARSHIP_TYPES.map((t) => (
          <option key={t} value={t}>
            {t}
          </option>
        ))}
      </select>
    </div>
    <div>
      <label className="block font-semibold mb-1">
        Discount Mode
      </label>
      <select
        value={formData.discountType}
        onChange={(e) =>
          setFormData({
            ...formData,
            discountType: e.target.value as any,
          })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border outline-non
e font-medium"
      >
        <option value="Percentage">Percentage (%)</option>
        <option value="Fixed Amount">Fixed Amount (₹)</option>
      </select>
    </div>
  </div>

  {formData.discountType === "Percentage" ? (
    <div>
      <label className="block font-semibold mb-1">
        Percentage Waiver (%)
      </label>
      <input
        type="number"
        value={formData.percentage}
        onChange={(e) =>
          setFormData({
            ...formData,
            percentage: Number(e.target.value),
          })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold t
ext-emerald-600 outline-none"
      />
    </div>
  ) : (
    <div>
      <label className="block font-semibold mb-1">
        Fixed Grant Amount (₹)
      </label>
      <input
        type="number"
        value={formData.fixedAmount}
        onChange={(e) =>
          setFormData({
            ...formData,
            fixedAmount: Number(e.target.value),
          })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold t
ext-emerald-600 outline-none"
      />
    </div>
  )
}

```

```

        />
      </div>
    )}

    <div className="flex items-center justify-end gap-3 pt-3 border-t border-slate-100 dark:borde
r-slate-800">
      <button
        type="button"
        onClick={() => setIsModalOpen(false)}
        className="px-4 py-2 font-semibold bg-slate-100 dark:bg-slate-800 rounded-xl"
      >
        Cancel
      </button>
      <button
        type="submit"
        className="px-4 py-2 font-bold bg-sky-600 text-white rounded-xl"
      >
        Save Scheme
      </button>
    </div>
  </form>
</div>
</div>
)}

{/* Award Student Scholarship Modal - WITH CLASS FILTER & SEARCH */}
{isAllocModalOpen && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/60 backdrop-bl
ur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-
3xl max-w-lg w-full p-6 shadow-2xl space-y-4">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-8
00 pb-3">
        <div className="flex items-center gap-2">
          <div className="p-2 rounded-xl bg-emerald-500/10 text-emerald-600">
            <Award className="w-5 h-5" />
          </div>
          <div>
            <h3 className="text-base font-bold text-slate-900 dark:text-white">
              Award Student Scholarship
            </h3>
            <p className="text-[11px] text-slate-500">
              Filter students by class grade or search by name & admission
              no
            </p>
          </div>
        </div>
        <button
          onClick={() => setIsAllocModalOpen(false)}
          className="text-slate-400 hover:text-slate-600"
        >
          ✕
        </button>
      </div>

      <form onSubmit={handleAllocSubmit} className="space-y-4 text-xs">
        {/* Filter Controls: Class Dropdown + Search Student */}
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3 p-3 bg-slate-50 dark:bg-slate-800/50 ro
unded-2xl border border-slate-200/60 dark:border-slate-800">
          <div>
            <label className="block font-bold text-slate-700 dark:text-slate-300 mb-1 flex items-cent
er gap-1">
              <Filter className="w-3 h-3 text-sky-500" /> Filter Class

```

```

    </label>
    <select
      value={allocClassFilter}
      onChange={(e) => {
        setAllocClassFilter(e.target.value);
        setAllocStudentId("");
      }}
      className="w-full px-3 py-2 rounded-xl bg-white dark:bg-slate-900 border border-slate-2
00 dark:border-slate-700 font-semibold text-slate-900 dark:text-white outline-none"
    >
      <option value="All">All Classes ({students.length})</option>
      {sortedClasses.map((c) => {
        const count = students.filter(
          (s) => s.className === c.name,
        ).length;
        return (
          <option key={c.id} value={c.name}>
            {c.name} ({count})
          </option>
        );
      })}
    </select>
  </div>

  <div>
    <label className="block font-bold text-slate-700 dark:text-slate-300 mb-1 flex items-cent
er gap-1">
      <Search className="w-3 h-3 text-sky-500" /> Search Student
    </label>
    <input
      type="text"
      placeholder="Search name or adm no..."
      value={allocSearchQuery}
      onChange={(e) => {
        setAllocSearchQuery(e.target.value);
        setAllocStudentId("");
      }}
      className="w-full px-3 py-2 rounded-xl bg-white dark:bg-slate-900 border border-slate-2
00 dark:border-slate-700 font-medium text-slate-900 dark:text-white outline-none"
    />
  </div>
</div>

{/* Select Student Selector */}
<div>
  <div className="flex items-center justify-between mb-1">
    <label className="block font-bold text-slate-800 dark:text-slate-200">
      Select Student{" "}
      <span className="text-rose-500 font-bold ml-0.5">*</span>
    </label>
    <span className="text-[11px] font-bold text-sky-600 dark:text-sky-400">
      {filteredStudentsForAward.length} student
      {filteredStudentsForAward.length !== 1 ? "s" : ""} available
    </span>
  </div>
  <select
    value={allocStudentId}
    onChange={(e) => setAllocStudentId(e.target.value)}
    className="w-full px-3 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slat
e-200 dark:border-slate-700 font-bold text-slate-900 dark:text-white outline-none"
    required
  >
    <option value="">

```

```

-- Choose Student ({filteredStudentsForAward.length}{ " "
matching) --
</option>
{filteredStudentsForAward.map((st) => (
  <option key={st.id} value={st.id}>
    {st.firstName} {st.lastName} ({st.className}
    {st.section ? `-${st.section}` : ""} • Adm:{ " "
    {st.admissionNo || st.id})
  </option>
))}
</select>
</div>

{/* Selected Student Details Card */}
{selectedStudentDetails && (
  <div className="p-3 rounded-xl bg-sky-50 dark:bg-sky-950/30 border border-sky-200 dark:bord
er-sky-800/50 flex items-center justify-between">
    <div>
      <div className="font-bold text-slate-900 dark:text-white">
        {selectedStudentDetails.firstName}{ " "
        {selectedStudentDetails.lastName}
      </div>
      <div className="text-[11px] text-slate-500">
        Class:{ " "
        <span className="font-semibold text-slate-700 dark:text-slate-300">
          {selectedStudentDetails.className}-
          {selectedStudentDetails.section}
        </span>{ " "
        • Adm No:{ " "
        <span className="font-semibold text-slate-700 dark:text-slate-300">
          {selectedStudentDetails.admissionNo}
        </span>
      </div>
    </div>
    <span className="px-2.5 py-1 rounded-full text-[10px] font-bold bg-sky-100 dark:bg-sky-90
0/60 text-sky-700 dark:text-sky-300">
      Eligible
    </span>
  </div>
)}

{/* Select Scholarship Scheme */}
<div>
  <label className="block font-bold text-slate-800 dark:text-slate-200 mb-1">
    Select Scholarship Scheme{ " "
    <span className="text-rose-500 font-bold ml-0.5">*</span>
  </label>
  <select
    value={allocScholarshipId}
    onChange={(e) => setAllocScholarshipId(e.target.value)}
    className="w-full px-3 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slat
e-200 dark:border-slate-700 font-bold text-slate-900 dark:text-white outline-none"
    required
  >
    {scholarships.map((s) => (
      <option key={s.id} value={s.id}>
        {s.name} (
        {s.discountType === "Percentage"
        ? `${s.percentage}% Fee Waiver`
        : `${formatCurrency(s.fixedAmount || 0)} Grant`}{ " "
        • {s.type}}
      </option>
    ))}
  </select>

```

```

    </select>
  </div>

  { /* Scheme Value Preview */ }
  { selectedScholarshipDetails && (
    <div className="p-3 rounded-xl bg-emerald-50 dark:bg-emerald-950/30 border border-emerald-200 dark:border-emerald-800/50 flex items-center justify-between">
      <div className="space-y-0.5">
        <span className="text-[10px] font-bold text-emerald-700 dark:text-emerald-400 uppercase tracking-wider">
          Applied Concession
        </span>
        <div className="font-black text-emerald-700 dark:text-emerald-300 text-sm">
          { selectedScholarshipDetails.discountType === "Percentage"
            ? ` ${selectedScholarshipDetails.percentage}% Fee Waiver`
            : ` ${formatCurrency(selectedScholarshipDetails.fixedAmount || 0)} Off Base Fee` }
        </div>
      </div>
      <span className="text-[11px] text-emerald-600 dark:text-emerald-400 font-medium">
        { selectedScholarshipDetails.type }
      </span>
    </div>
  ) }

  <div className="flex items-center justify-end gap-3 pt-3 border-t border-slate-100 dark:borde
r-slate-800">
    <button
      type="button"
      onClick={() => setIsAllocModalOpen(false)}
      className="px-4 py-2.5 font-bold text-slate-600 dark:text-slate-300 bg-slate-100 hover:bg
-slate-200 dark:bg-slate-800 dark:hover:bg-slate-700 rounded-xl transition-colors"
    >
      Cancel
    </button>
    <button
      type="submit"
      className="px-5 py-2.5 font-bold bg-emerald-600 hover:bg-emerald-500 text-white rounded-x
1 shadow-md shadow-emerald-600/20 transition-all flex items-center gap-1.5"
    >
      <UserPlus className="w-4 h-4" /> Award Scholarship
    </button>
  </div>
</form>
</div>
</div>
)}

<ConfirmModal
  isOpen={!deletingSch}
  title="Delete Scheme"
  message={`Are you sure you want to delete ${deletingSch?.name}?`}
  onConfirm={async () => {
    if (deletingSch) {
      await deleteScholarship(deletingSch.id);
      addToast("success", "Scholarship Scheme Deleted");
      setDeletingSch(null);
    }
  }}
  onCancel={() => setDeletingSch(null)}
/>
</div>

```

```
);  
};
```

 Finance/DiscountsView.tsx

TSX

```
// @ts-nocheck
import React, { useState, useMemo } from "react";
import { formatCurrency } from "../../utils/currency";
import {
  Percent,
  Plus,
  Search,
  Edit,
  Trash2,
  UserPlus,
  Filter,
  GraduationCap,
  Sparkles,
  CheckCircle2,
} from "lucide-react";
import { Discount, DiscountType, StudentDiscount } from "../../types";
import { useData } from "../../context/DataContext";
import { useToast } from "../../context/ToastContext";
import { ExportButton } from "../../common/ExportButton";
import { ConfirmModal } from "../../common/ConfirmModal";
import { compareClassesAscending } from "../../utils/classSorter";

const DISCOUNT_TYPES: DiscountType[] = [
  "Sibling Discount",
  "Employee Discount",
  "Early Payment Discount",
  "Special Approval",
  "Custom",
];

export const DiscountsView: React.FC = () => {
  const {
    discounts,
    studentDiscounts,
    students,
    addDiscount,
    updateDiscount,
    deleteDiscount,
    assignDiscountToStudent,
    removeStudentDiscount,
    academicClasses,
  } = useData();
  const { addToast } = useToast();

  const [activeTab, setActiveTab] = useState<"master" | "allocated">("master");
  const [query, setQuery] = useState("");
  const [allocatedClassFilter, setAllocatedClassFilter] =
    useState<string>("All");

  const [isModalOpen, setIsModalOpen] = useState(false);
  const [editingDisc, setEditingDisc] = useState<Discount | null>(null);
  const [deletingDisc, setDeletingDisc] = useState<Discount | null>(null);

  // Grant Concession Modal with Class Filter & Search
  const [isAllocOpen, setIsAllocOpen] = useState(false);
  const [allocClassFilter, setAllocClassFilter] = useState<string>("All");
  const [allocSearchQuery, setAllocSearchQuery] = useState<string>("");
  const [allocStudentId, setAllocStudentId] = useState("");
  const [allocDiscountId, setAllocDiscountId] = useState(
```

```

    discounts[0]?.id || "",
  );

  const sortedClasses = useMemo(() => {
    return [...academicClasses].sort((a, b) =>
      compareClassesAscending(a.name, b.name),
    );
  }, [academicClasses]);

  // Filtered Students for the Grant Concession Modal
  const filteredStudentsForGrant = useMemo(() => {
    return students.filter((st) => {
      const matchesClass =
        allocClassFilter === "All" || st.className === allocClassFilter;
      const q = allocSearchQuery.trim().toLowerCase();
      const matchesSearch =
        !q ||
        `${st.firstName} ${st.lastName}`.toLowerCase().includes(q) ||
        (st.admissionNo && st.admissionNo.toLowerCase().includes(q)) ||
        (st.rollNumber && st.rollNumber.toLowerCase().includes(q));
      return matchesClass && matchesSearch;
    });
  }, [students, allocClassFilter, allocSearchQuery]);

  const selectedStudentDetails = useMemo(() => {
    return students.find((s) => s.id === allocStudentId);
  }, [students, allocStudentId]);

  const selectedDiscountDetails = useMemo(() => {
    return discounts.find((d) => d.id === allocDiscountId);
  }, [discounts, allocDiscountId]);

  const [formData, setFormData] = useState<Partial<Discount>>>({
    name: "",
    code: "",
    type: "Sibling Discount",
    mode: "Percentage",
    value: 10,
    status: "Active",
  });

  const filteredDiscounts = discounts.filter(
    (d) =>
      d.name.toLowerCase().includes(query.toLowerCase()) ||
      d.code.toLowerCase().includes(query.toLowerCase()),
  );

  const filteredAwardedList = studentDiscounts.filter((sd) => {
    const st = students.find((s) => s.id === sd.studentId);
    const studentName = st ? `${st.firstName} ${st.lastName}` : sd.studentId;
    const q = query.trim().toLowerCase();
    const matchesQuery =
      !q ||
      studentName.toLowerCase().includes(q) ||
      sd.discountName.toLowerCase().includes(q);
    const matchesClass =
      allocatedClassFilter === "All" ||
      (st && st.className === allocatedClassFilter);
    return matchesQuery && matchesClass;
  });

  const handleOpenAdd = () => {
    setEditingDisc(null);
  }

```

```
setFormData({
  name: "",
  code: "DSC-" + Math.floor(100 + Math.random() * 900),
  type: "Sibling Discount",
  mode: "Percentage",
  value: 10,
  status: "Active",
});
setIsModalOpen(true);
};

const handleOpenEdit = (d: Discount) => {
  setEditingDisc(d);
  setFormData(d);
  setIsModalOpen(true);
};

const handleOpenAlloc = () => {
  setAllocClassFilter("All");
  setAllocSearchQuery("");
  setAllocStudentId("");
  if (discounts.length > 0) {
    setAllocDiscountId(discounts[0].id);
  }
  setIsAllocOpen(true);
};

const handleSubmit = async (e: React.SyntheticEvent) => {
  e.preventDefault();
  if (!formData.name || !formData.code) {
    addToast("warning", "Validation Error", "Name and code are required.");
    return;
  }

  if (editingDisc) {
    await updateDiscount(editingDisc.id, formData);
    addToast("success", "Discount Updated", `Updated ${formData.name}`);
  } else {
    await addDiscount(formData as Omit<Discount, "id">);
    addToast("success", "Discount Created", `Created ${formData.name}`);
  }
  setIsModalOpen(false);
};

const handleAllocSubmit = async (e: React.SyntheticEvent) => {
  e.preventDefault();
  if (!allocStudentId || !allocDiscountId) {
    addToast(
      "warning",
      "Selection Required",
      "Select both student and discount rule.",
    );
    return;
  }
  await assignDiscountToStudent(allocStudentId, allocDiscountId);
  addToast(
    "success",
    "Concession Granted",
    "Applied concession to student successfully.",
  );
  setIsAllocOpen(false);
};
```

```

return (
  <div className="space-y-6 animate-in fade-in">
    { /* Header */ }
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
      <div>
        <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center gap-2">
          <Percent className="w-6 h-6 text-sky-500" /> Discounts & Concessions
        </h2>
      </div>

      <div className="flex items-center gap-3">
        <button
          onClick={handleOpenAdd}
          className="px-4 py-2.5 rounded-xl bg-sky-600 hover:bg-sky-500 text-white text-xs font-bold shadow-lg shadow-sky-500/20 flex items-center gap-2 transition-all">
          <Plus className="w-4 h-4" /> Create Discount
        </button>
        <ExportButton data={activeTab === 'master' ? filteredDiscounts : filteredAwardedList} filename="discounts" />
      </div>
    </div>

    { /* Tabs & Filter Bar */ }
    <div className="glass-card p-4 rounded-2xl flex flex-col sm:flex-row items-center justify-between gap-4">
      <div className="flex items-center gap-1 bg-slate-100 dark:bg-slate-800 p-1 rounded-xl w-full sm:w-auto">
        <button
          onClick={() => setActiveTab("master")}
          className={`px-4 py-2 rounded-lg text-xs font-bold transition-all ${
            activeTab === "master"
              ? "bg-white dark:bg-slate-900 text-brand-600 shadow-sm"
              : "text-slate-500"
          }`}
          >
          Discount Rules ({discounts.length})
        </button>
        <button
          onClick={() => setActiveTab("allocated")}
          className={`px-4 py-2 rounded-lg text-xs font-bold transition-all ${
            activeTab === "allocated"
              ? "bg-white dark:bg-slate-900 text-brand-600 shadow-sm"
              : "text-slate-500"
          }`}
          >
          Granted Student Concessions ({studentDiscounts.length})
        </button>
      </div>

      <div className="flex items-center gap-3 w-full sm:w-auto">
        {activeTab === "allocated" && (
          <select
            value={allocatedClassFilter}
            onChange={(e) => setAllocatedClassFilter(e.target.value)}
            className="px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text-slate-900 dark:text-white outline-none font-medium">
            >
            <option value="All">All Class Grades</option>
            {sortedClasses.map((c) => (
              <option key={c.id} value={c.name}>
                {c.name}
              </option>
            ))}
          </select>
        )}
      </div>
    </div>
  </div>
)

```

```

        </option>
      )}}
    </select>
  )}

  <div className="relative w-full sm:w-64">
    <Search className="w-4 h-4 text-slate-400 absolute left-3 top-3" />
    <input
      type="text"
      placeholder={
        activeTab === "master"
          ? "Search discount name..."
          : "Search student or concession..."
      }
      value={query}
      onChange={(e) => setQuery(e.target.value)}
      className="w-full pl-9 pr-4 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border text-xs text
-slate-900 dark:text-white outline-none font-medium"
    />
  </div>
</div>
</div>

{/* Tab 1: Discount Rules */}
{activeTab === "master" && (
  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    {filteredDiscounts.map((d) => (
      <div
        key={d.id}
        className="glass-card p-5 rounded-2xl flex items-center justify-between hover:shadow-md trans
ition-shadow"
      >
        <div>
          <span className="text-[10px] font-bold px-2 py-0.5 rounded bg-sky-50 text-sky-700 dark:bg-s
ky-950 dark:text-sky-300">
            {d.type} • {d.code}
          </span>
          <h3 className="font-bold text-sm text-slate-900 dark:text-white mt-1">
            {d.name}
          </h3>
          <p className="text-xs font-extrabold text-emerald-600 dark:text-emerald-400 mt-1">
            {d.mode === "Percentage"
              ? `${d.value}% Concession`
              : `${formatCurrency(d.value)} Flat Off`}
          </p>
        </div>

        <div className="flex items-center gap-1">
          <button
            onClick={() => handleOpenEdit(d)}
            className="p-1.5 rounded-lg hover:bg-slate-100 dark:hover:bg-slate-800 text-sky-600"
          >
            <Edit className="w-4 h-4" />
          </button>
          <button
            onClick={() => setDeletingDisc(d)}
            className="p-1.5 rounded-lg hover:bg-rose-50 dark:hover:bg-rose-950/40 text-rose-600"
          >
            <Trash2 className="w-4 h-4" />
          </button>
        </div>
      </div>
    )}
  </div>
)}}

```

```

    </div>
  })

  { /* Tab 2: Granted Concessions List */ }
  { activeTab === "allocated" && (
    <div className="glass-card rounded-2xl overflow-hidden border border-slate-200/80 dark:border-slate-800">
      <div className="overflow-x-auto">
        <table className="w-full text-left border-collapse text-xs">
          <thead>
            <tr className="bg-slate-100/70 dark:bg-slate-800/60 text-slate-500 font-bold uppercase border-b border-slate-200 dark:border-slate-800">
              <th className="py-3.5 px-4">STUDENT NAME</th>
              <th className="py-3.5 px-4">GRANTED CONCESSION</th>
              <th className="py-3.5 px-4">DISCOUNT VALUE</th>
              <th className="py-3.5 px-4">APPLIED DATE</th>
              <th className="py-3.5 px-4 text-right">REMOVE</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100 dark:divide-slate-800/80 font-medium">
            { filteredAwardedList.length === 0 ? (
              <tr>
                <td colspan={5} className="py-8 text-center text-slate-400">
                  No student concessions granted yet. Click "+ Grant Student Concession" to apply a rule.
                </td>
              </tr>
            ) : (
              filteredAwardedList.map((sd) => {
                const st = students.find((s) => s.id === sd.studentId);
                const disc = discounts.find((d) => d.id === sd.discountId);
                return (
                  <tr
                    key={sd.id}
                    className="hover:bg-slate-50/80 dark:hover:bg-slate-800/40">
                    <td className="py-3 px-4 font-bold text-slate-900 dark:text-white flex items-center gap-2">
                      <GraduationCap className="w-4 h-4 text-sky-500" />
                      {st ? `${st.firstName} ${st.lastName}` : sd.studentId}
                      {st?.className && (
                        <span className="text-[10px] font-semibold text-slate-400">
                          ({st.className}-{st.section})
                        </span>
                      )}
                    </td>
                    <td className="py-3 px-4 font-semibold text-sky-600 dark:text-sky-400">
                      {sd.discountName}
                    </td>
                    <td className="py-3 px-4 font-bold text-emerald-600 dark:text-emerald-400">
                      {disc
                        ? disc.mode === "Percentage"
                        ? `${disc.value}% Waiver`
                        : `${formatCurrency(disc.value)} Off`
                        : "Active"}
                    </td>
                    <td className="py-3 px-4 text-slate-500">
                      {sd.appliedDate}
                    </td>
                    <td className="py-3 px-4 text-right">
                      <button
                        onClick={async () => {
                          await removeStudentDiscount(sd.id);

```

```

        addToast(
            "info",
            "Concession Removed",
            "Removed concession record.",
        );
    }}
    className="p-1.5 text-rose-500 hover:bg-rose-50 dark:hover:bg-rose-950/40 round
ed-lg transition-colors"
    title="Remove Concession"
  >
    <Trash2 className="w-4 h-4" />
  </button>
</td>
</tr>
</tbody>
</table>
</div>
</div>
)}}

{/* Create / Edit Discount Modal */}
{isModalOpen && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/60 backdrop-bl
ur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-
3xl max-w-md w-full p-6 shadow-2xl space-y-4">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-8
00 pb-3">
        <h3 className="text-base font-bold text-slate-900 dark:text-white">
          {editingDisc ? "Edit Discount" : "Add Discount"}
        </h3>
        <button
          onClick={() => setIsModalOpen(false)}
          className="text-slate-400 hover:text-slate-600"
        >
          ✕
        </button>
      </div>

      <form onSubmit={handleSubmit} className="space-y-3 text-xs">
        <div className="grid grid-cols-2 gap-3">
          <div>
            <label className="block font-semibold mb-1">
              Discount Name{" "}
              <span className="text-rose-500 font-bold ml-0.5">*</span>
            </label>
            <input
              type="text"
              required
              value={formData.name}
              onChange={(e) =>
                setFormData({ ...formData, name: e.target.value })
              }
              className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border outline-non
e font-medium"
            />
          </div>
          <div>
            <label className="block font-semibold mb-1">
              Code{" "}

```

```

        <span className="text-rose-500 font-bold ml-0.5">*</span>
      </label>
      <input
        type="text"
        required
        value={formData.code}
        onChange={(e) =>
          setFormData({ ...formData, code: e.target.value })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-mono o
utline-none"
      />
    </div>
  </div>

  <div className="grid grid-cols-2 gap-3">
    <div>
      <label className="block font-semibold mb-1">
        Discount Type
      </label>
      <select
        value={formData.type}
        onChange={(e) =>
          setFormData({ ...formData, type: e.target.value as any })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border outline-non
e font-medium"
      >
        {DISCOUNT_TYPES.map((t) => (
          <option key={t} value={t}>
            {t}
          </option>
        ))}
      </select>
    </div>
    <div>
      <label className="block font-semibold mb-1">Mode</label>
      <select
        value={formData.mode}
        onChange={(e) =>
          setFormData({ ...formData, mode: e.target.value as any })
        }
        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border outline-non
e font-medium"
      >
        <option value="Percentage">Percentage (%)</option>
        <option value="Fixed Amount">Fixed Amount (₹)</option>
      </select>
    </div>
  </div>

  <div>
    <label className="block font-semibold mb-1">
      Value ({formData.mode === "Percentage" ? "%" : "₹"}){" "}
      <span className="text-rose-500 font-bold ml-0.5">*</span>
    </label>
    <input
      type="number"
      required
      value={formData.value}
      onChange={(e) =>
        setFormData({ ...formData, value: Number(e.target.value) })
      }
    >
  </div>

```

```

        className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold text-emerald-600 outline-none"
      />
    </div>

    <div className="flex items-center justify-end gap-3 pt-3 border-t border-slate-100 dark:borde
r-slate-800">
      <button
        type="button"
        onClick={() => setIsModalOpen(false)}
        className="px-4 py-2 font-semibold bg-slate-100 dark:bg-slate-800 rounded-xl"
      >
        Cancel
      </button>
      <button
        type="submit"
        className="px-4 py-2 font-bold bg-sky-600 text-white rounded-xl"
      >
        Save Discount
      </button>
    </div>
  </form>
</div>
</div>
)}}

{/* Grant Concession Modal - WITH CLASS FILTER & SEARCH */}
{isAllocOpen && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/60 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-3xl max-w-lg w-full p-6 shadow-2xl space-y-4">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-3">
        <div className="flex items-center gap-2">
          <div className="p-2 rounded-xl bg-emerald-500/10 text-emerald-600">
            <Percent className="w-5 h-5" />
          </div>
          <div>
            <h3 className="text-base font-bold text-slate-900 dark:text-white">
              Grant Student Concession
            </h3>
            <p className="text-[11px] text-slate-500">
              Filter students by class grade or search by name & admission
            </p>
          </div>
        </div>
        <button
          onClick={() => setIsAllocOpen(false)}
          className="text-slate-400 hover:text-slate-600"
        >
          ✕
        </button>
      </div>

      <form onSubmit={handleAllocSubmit} className="space-y-4 text-xs">
        {/* Filter Controls: Class Dropdown + Search Student */}
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3 p-3 bg-slate-50 dark:bg-slate-800/50 rounded-2xl border border-slate-200/60 dark:border-slate-800">
          <div>
            <label className="block font-bold text-slate-700 dark:text-slate-300 mb-1 flex items-center gap-1">

```

```

        <Filter className="w-3 h-3 text-sky-500" /> Filter Class
      </label>
      <select
        value={allocClassFilter}
        onChange={(e) => {
          setAllocClassFilter(e.target.value);
          setAllocStudentId("");
        }}
        className="w-full px-3 py-2 rounded-xl bg-white dark:bg-slate-900 border border-slate-2
00 dark:border-slate-700 font-semibold text-slate-900 dark:text-white outline-none"
      >
        <option value="All">All Classes ({students.length})</option>
        {sortedClasses.map((c) => {
          const count = students.filter(
            (s) => s.className === c.name,
          ).length;
          return (
            <option key={c.id} value={c.name}>
              {c.name} ({count})
            </option>
          );
        })}
      </select>
    </div>

    <div>
      <label className="block font-bold text-slate-700 dark:text-slate-300 mb-1 flex items-cent
er gap-1">

        <Search className="w-3 h-3 text-sky-500" /> Search Student
      </label>
      <input
        type="text"
        placeholder="Search name or adm no..."
        value={allocSearchQuery}
        onChange={(e) => {
          setAllocSearchQuery(e.target.value);
          setAllocStudentId("");
        }}
        className="w-full px-3 py-2 rounded-xl bg-white dark:bg-slate-900 border border-slate-2
00 dark:border-slate-700 font-medium text-slate-900 dark:text-white outline-none"
      />
    </div>
  </div>

  {/* Select Student Selector */}
  <div>
    <div className="flex items-center justify-between mb-1">
      <label className="block font-bold text-slate-800 dark:text-slate-200">
        Select Student{" "}
        <span className="text-rose-500 font-bold ml-0.5">*</span>
      </label>
      <span className="text-[11px] font-bold text-sky-600 dark:text-sky-400">
        {filteredStudentsForGrant.length} student
        {filteredStudentsForGrant.length !== 1 ? "s" : ""} available
      </span>
    </div>
    <select
      value={allocStudentId}
      onChange={(e) => setAllocStudentId(e.target.value)}
      className="w-full px-3 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slat
e-200 dark:border-slate-700 font-bold text-slate-900 dark:text-white outline-none"
      required
    >

```

```

<option value="">
  -- Choose Student ({filteredStudentsForGrant.length}{ " "
  matching) --
</option>
{filteredStudentsForGrant.map((st) => (
  <option key={st.id} value={st.id}>
    {st.firstName} {st.lastName} ({st.className}
    {st.section ? `-${st.section}` : ""} • Adm:{ " "
    {st.admissionNo || st.id})
  </option>
))}
</select>
</div>

{/* Selected Student Details Card */}
{selectedStudentDetails && (
  <div className="p-3 rounded-xl bg-sky-50 dark:bg-sky-950/30 border border-sky-200 dark:bord
er-sky-800/50 flex items-center justify-between">
    <div>
      <div className="font-bold text-slate-900 dark:text-white">
        {selectedStudentDetails.firstName}{ " "
        {selectedStudentDetails.lastName}
      </div>
      <div className="text-[11px] text-slate-500">
        Class:{ " "
        <span className="font-semibold text-slate-700 dark:text-slate-300">
          {selectedStudentDetails.className}-
          {selectedStudentDetails.section}
        </span>{ " "
        • Adm No:{ " "
        <span className="font-semibold text-slate-700 dark:text-slate-300">
          {selectedStudentDetails.admissionNo}
        </span>
      </div>
    </div>
    <span className="px-2.5 py-1 rounded-full text-[10px] font-bold bg-sky-100 dark:bg-sky-90
0/60 text-sky-700 dark:text-sky-300">
      Eligible
    </span>
  </div>
)}

{/* Select Discount Rule */}
<div>
  <label className="block font-bold text-slate-800 dark:text-slate-200 mb-1">
    Select Discount Rule{ " "
    <span className="text-rose-500 font-bold ml-0.5">*</span>
  </label>
  <select
    value={allocDiscountId}
    onChange={(e) => setAllocDiscountId(e.target.value)}
    className="w-full px-3 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800 border border-slat
e-200 dark:border-slate-700 font-bold text-slate-900 dark:text-white outline-none"
    required
  >
    {discounts.map((d) => (
      <option key={d.id} value={d.id}>
        {d.name} (
          {d.mode === "Percentage"
            ? `${d.value}% Concession`
            : `${formatCurrency(d.value)} Flat Off`}
          )
      </option>
    )
  </div>

```

```

    )))
  </select>
</div>

  { /* Concession Value Preview */
  {selectedDiscountDetails && (
    <div className="p-3 rounded-xl bg-emerald-50 dark:bg-emerald-950/30 border border-emerald-200 dark:border-emerald-800/50 flex items-center justify-between">
      <div className="space-y-0.5">
        <span className="text-[10px] font-bold text-emerald-700 dark:text-emerald-400 uppercase tracking-wider">
          Applied Concession
        </span>
        <div className="font-black text-emerald-700 dark:text-emerald-300 text-sm">
          {selectedDiscountDetails.mode === "Percentage"
            ? `${selectedDiscountDetails.value}% Concession on Fee`
            : `${formatCurrency(selectedDiscountDetails.value)} Flat Deduction`}
          </div>
        </div>
        <span className="text-[11px] text-emerald-600 dark:text-emerald-400 font-medium">
          {selectedDiscountDetails.type}
        </span>
      </div>
    </div>

    <div className="flex items-center justify-end gap-3 pt-3 border-t border-slate-100 dark:borde
r-slate-800">
      <button
        type="button"
        onClick={() => setIsAllocOpen(false)}
        className="px-4 py-2.5 font-bold text-slate-600 dark:text-slate-300 bg-slate-100 hover:bg
-slate-200 dark:bg-slate-800 dark:hover:bg-slate-700 rounded-xl transition-colors"
      >
        Cancel
      </button>
      <button
        type="submit"
        className="px-5 py-2.5 font-bold bg-emerald-600 hover:bg-emerald-500 text-white rounded-x
1 shadow-md shadow-emerald-600/20 transition-all flex items-center gap-1.5"
      >
        <UserPlus className="w-4 h-4" /> Grant Concession
      </button>
    </div>
  </form>
</div>
</div>
  )}

  <ConfirmModal
    isOpen={!deletingDisc}
    title="Delete Discount"
    message={`Are you sure you want to delete ${deletingDisc?.name}?`}
    onConfirm={async () => {
      if (deletingDisc) {
        await deleteDiscount(deletingDisc.id);
        addToast("success", "Discount Deleted");
        setDeletingDisc(null);
      }
    }}
    onCancel={() => setDeletingDisc(null)}
  />
</div>

```

```
);  
};
```

 Finance/FineRulesView.tsx

TSX

```
import React, { useState } from 'react';
import { formatCurrency } from '../../../utils/currency';
import { AlertTriangle, Plus, Edit, Trash2, CheckCircle2, Calendar } from 'lucide-react';
import { FineRule } from '../../../types';
import { useData } from '../../../context/DataContext';
import { useToast } from '../../../context/ToastContext';
import { ConfirmModal } from '../../../common/ConfirmModal';

export const FineRulesView: React.FC = () => {
  const { fineRules, addFineRule, updateFineRule, deleteFineRule } = useData();
  const { addToast } = useToast();

  const [isModalOpen, setIsModalOpen] = useState(false);
  const [editingRule, setEditingRule] = useState<FineRule | null>(null);
  const [deletingRule, setDeletingRule] = useState<FineRule | null>(null);

  const [formData, setFormData] = useState<Partial<FineRule>>>({
    ruleName: '',
    dueDate: '2026-08-15',
    graceDays: 5,
    fineType: 'Daily Fine',
    dailyFine: 50,
    fixedFine: 200,
    maximumFine: 1500,
    status: 'Active'
  });

  const handleOpenAdd = () => {
    setEditingRule(null);
    setFormData({
      ruleName: '',
      dueDate: new Date().toISOString().split('T')[0],
      graceDays: 5,
      fineType: 'Daily Fine',
      dailyFine: 50,
      fixedFine: 200,
      maximumFine: 1500,
      status: 'Active'
    });
    setIsModalOpen(true);
  };

  const handleOpenEdit = (r: FineRule) => {
    setEditingRule(r);
    setFormData(r);
    setIsModalOpen(true);
  };

  const handleSubmit = (e: React.SyntheticEvent) => {
    e.preventDefault();
    if (!formData.ruleName) return;

    if (editingRule) {
      updateFineRule(editingRule.id, formData);
      addToast('success', 'Fine Rule Updated', `Updated ${formData.ruleName}`);
    } else {
      addFineRule(formData as Omit<FineRule, 'id'>);
      addToast('success', 'Fine Rule Configured', `Configured ${formData.ruleName}`);
    }
  }
}
```

```

    setIsModalOpen(false);
  };

  return (
    <div className="space-y-6 animate-in fade-in">
      { /* Header */ }
      <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
        <div>
          <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center gap-2">
            <AlertTriangle className="w-6 h-6 text-rose-500" /> Fine Rules
          </h2>
        </div>

        <button
          onClick={handleOpenAdd}
          className="px-4 py-2 rounded-xl bg-sky-600 hover:bg-sky-500 text-white text-xs font-bold shadow-lg shadow-sky-500/20 flex items-center gap-2"
        >
          <Plus className="w-4 h-4" /> Add Late Fine
        </button>
      </div>

      { /* Grid */ }
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        {fineRules.map(r => (
          <div key={r.id} className="glass-card p-5 rounded-2xl space-y-3">
            <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-2">
              <div>
                <span className={`text-[10px] font-bold px-2 py-0.5 rounded ${
                  r.status === 'Active' ? 'bg-emerald-50 text-emerald-700 dark:bg-emerald-950 dark:text-emerald-300' : 'bg-slate-100 text-slate-500'
                }`} >
                  {r.status} Rule
                </span>
                <h3 className="font-bold text-sm text-slate-900 dark:text-white mt-1">{r.ruleName}</h3>
              </div>

              <div className="flex items-center gap-1">
                <button onClick={() => handleOpenEdit(r)} className="p-1.5 rounded hover:bg-slate-100 text-sky-600"><Edit className="w-4 h-4" /></button>
                <button onClick={() => setDeletingRule(r)} className="p-1.5 rounded hover:bg-rose-50 text-rose-600"><Trash2 className="w-4 h-4" /></button>
              </div>
            </div>

            <div className="space-y-1.5 text-xs text-slate-600 dark:text-slate-300">
              <div className="flex justify-between"><span>Due Date:</span><span className="font-bold text-slate-900 dark:text-white">{r.dueDate}</span></div>
              <div className="flex justify-between"><span>Grace Days:</span><span className="font-semibold text-sky-600">{r.graceDays} Days</span></div>
              <div className="flex justify-between"><span>Fine Type:</span><span className="font-semibold">{r.fineType}</span></div>
              {r.fineType === 'Daily Fine' ? (
                <div className="flex justify-between"><span>Daily Rate:</span><span className="font-bold text-rose-500">{formatCurrency(r.dailyFine || 0)}/day</span></div>
              ) : (
                <div className="flex justify-between"><span>Fixed Rate:</span><span className="font-bold text-rose-500">{formatCurrency(r.fixedFine || 0)}</span></div>
              )}
              <div className="flex justify-between"><span>Maximum Fine Cap:</span><span className="font-bold text-slate-900 dark:text-white">{formatCurrency(r.maximumFine || 0)}</span></div>
            </div>
          </div>
        ))}
      </div>
    </div>
  );

```

```

    </div>
  </div>
  )}}
</div>

{/* Modal */}
{isModalOpen && (
  <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-950/60 backdrop-blur-sm animate-in fade-in">
    <div className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-3xl max-w-md w-full p-6 shadow-2xl space-y-4">
      <div className="flex items-center justify-between border-b border-slate-100 dark:border-slate-800 pb-3">
        <h3 className="text-base font-bold text-slate-900 dark:text-white">
          {editingRule ? 'Edit Fine' : 'Add Late Fee'}
        </h3>
        <button onClick={() => setIsModalOpen(false)} className="text-slate-400">X</button>
      </div>

      <form onSubmit={handleSubmit} className="space-y-3 text-xs">
        <div>
          <label className="block font-semibold mb-1">Rule Name <span className="text-rose-500 font-bold ml-0.5">*</span></label>
          <input type="text" required value={formData.ruleName} onChange={e => setFormData({ ...formData, ruleName: e.target.value })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border" />
        </div>

        <div className="grid grid-cols-2 gap-3">
          <div>
            <label className="block font-semibold mb-1">Standard Due Date <span className="text-rose-500 font-bold ml-0.5">*</span></label>
            <input type="date" required value={formData.dueDate} onChange={e => setFormData({ ...formData, dueDate: e.target.value })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold" />
          </div>
          <div>
            <label className="block font-semibold mb-1">Grace Days</label>
            <input type="number" value={formData.graceDays} onChange={e => setFormData({ ...formData, graceDays: Number(e.target.value) })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold" />
          </div>
        </div>

        <div className="grid grid-cols-2 gap-3">
          <div>
            <label className="block font-semibold mb-1">Fine Mode</label>
            <select value={formData.fineType} onChange={e => setFormData({ ...formData, fineType: e.target.value as any })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border">
              <option value="Daily Fine">Daily Rate</option>
              <option value="Fixed Fine">Fixed Fine</option>
            </select>
          </div>
          <div>
            <label className="block font-semibold mb-1">Max Fine Limit (₹)</label>
            <input type="number" value={formData.maximumFine} onChange={e => setFormData({ ...formData, maximumFine: Number(e.target.value) })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold text-rose-500" />
          </div>
        </div>

        {formData.fineType === 'Daily Fine' ? (
          <div><label className="block font-semibold mb-1">Daily Fine Amount (₹/day)</label><input type="

```

```

pe="number" value={formData.dailyFine} onChange={e => setFormData({ ...formData, dailyFine: Number(e.target.value) })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold text-rose-500" /></div>
    ) : (
      <div><label className="block font-semibold mb-1">Fixed Fine Amount (₹)</label><input type="number" value={formData.fixedFine} onChange={e => setFormData({ ...formData, fixedFine: Number(e.target.value) })} className="w-full px-3 py-2 rounded-xl bg-slate-50 dark:bg-slate-800 border font-bold text-rose-500" /></div>
    )}

    <div className="flex items-center justify-end gap-3 pt-3 border-t border-slate-100 dark:border-slate-800">
      <button type="button" onClick={() => setIsModalOpen(false)} className="px-4 py-2 font-semibold bg-slate-100 dark:bg-slate-800 rounded-xl">Cancel</button>
      <button type="submit" className="px-4 py-2 font-bold bg-sky-600 text-white rounded-xl">Save</button>
    </div>
  </form>
</div>
</div>
)}

<ConfirmModal
  isOpen={!deletingRule}
  title="Delete Fine Rule"
  message={`Are you sure you want to delete ${deletingRule?.ruleName}?`}
  onConfirm={() => {
    if (deletingRule) {
      deleteFineRule(deletingRule.id);
      addToast('success', 'Fine Rule Removed');
      setDeletingRule(null);
    }
  }}
  onCancel={() => setDeletingRule(null)}
/>
</div>
);
};

```

 Finance/FinanceDashboardView.tsx

TSX

```
import React, { useState, useEffect, useCallback } from "react";
import { formatCurrency } from "../../utils/currency";
import {
  IndianRupee,
  AlertCircle,
  CheckCircle,
  Bus,
  Home,
  Gift,
  AlertTriangle,
  TrendingUp,
  PieChart,
  BarChart2,
  Shirt,
  RefreshCw,
  Clock,
} from "lucide-react";
import { useData } from "../../context/DataContext";
import * as FinanceAPI from "../../api/finance";
import { useToast } from "../../context/ToastContext";

export const FinanceDashboardView: React.FC = () => {
  const {
    students,
    feePayments,
    studentTransports,
    studentHostels,
    studentScholarships,
    feeHeads,
    academicClasses,
    studentUniformIssues = [],
    uniforms = [],
    getStudentFeeOutstandingSummary,
    fetchFinanceData,
  } = useData();

  const { addToast } = useToast();
  const [apiStats, setApiStats] = useState<any>(null);
  const [isLoading, setIsLoading] = useState<boolean>(false);
  const [lastUpdated, setLastUpdated] = useState<Date>(new Date());

  const fetchFinanceDataRef = React.useRef(fetchFinanceData);
  fetchFinanceDataRef.current = fetchFinanceData;
  const addToastRef = React.useRef(addToast);
  addToastRef.current = addToast;

  const loadDashboardData = useCallback(async (isManual = false) => {
    setIsLoading(true);
    try {
      const [dashRes] = await Promise.allSettled([
        FinanceAPI.fetchFinanceDashboardStatsApi(),
        fetchFinanceDataRef.current ? fetchFinanceDataRef.current() : Promise.resolve(),
      ]);

      if (dashRes.status === "fulfilled" && dashRes.value?.data) {
        setApiStats(dashRes.value.data);
      }
      setLastUpdated(new Date());
      if (isManual) {

```

```

    addToastRef.current("success", "Dashboard Refreshed", "Loaded latest financial metrics and transactions.");
  }
} catch (err) {
  console.warn("Failed to load dashboard data", err);
} finally {
  setIsLoading(false);
}
}, []);

// Initial load on mount and 5-minute recurring auto-refresh timer
useEffect(() => {
  loadDashboardData(false);
  const interval = setInterval(() => {
    loadDashboardData(false);
  }, 5 * 60 * 1000); // 5 minutes

  return () => clearInterval(interval);
}, [loadDashboardData]);

// Calculations purely from live stats
const totalCollected = Number(apiStats?.totalCollectedRevenue || 0);
const totalPending = Number(apiStats?.totalOutstandingDues || 0);
const totalExpected = Number(apiStats?.totalExpectedRevenue || 0);
const todaysCollection = Number(apiStats?.todayCollectionAmount || 0);
const transportCollection = Number(apiStats?.transportRevenue || 0);
const hostelCollection = Number(apiStats?.hostelRevenue || 0);
const uniformCollection = Number(apiStats?.uniformRevenue || 0);
const scholarshipAmount = Number(apiStats?.scholarshipsGranted || 0);
const fineCollection = Number(apiStats?.fineCollected || 0);

return (
  <div className="space-y-6 animate-in fade-in">
    {/* Header */}
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
      <div>
        <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center gap-2">
          <TrendingUp className="w-6 h-6 text-sky-500" /> Finance Dashboard
        </h2>
      </div>
    </div>

    {/* KPI Cards Grid */}
    <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
      <div className="glass-card p-5 rounded-3xl space-y-2 border-1-4 border-1-sky-500">
        <div className="flex items-center justify-between">
          <span className="text-xs font-bold text-slate-500 uppercase">
            Total Expected Collection
          </span>
          <IndianRupee className="w-5 h-5 text-sky-500" />
        </div>
        <h3 className="text-2xl font-black text-slate-900 dark:text-white">
          {formatCurrency(totalExpected)}
        </h3>
        <p className="text-[10px] text-slate-400">Target baseline revenue</p>
      </div>

      <div className="glass-card p-5 rounded-3xl space-y-2 border-1-4 border-1-emerald-500">
        <div className="flex items-center justify-between">
          <span className="text-xs font-bold text-slate-500 uppercase">
            Total Collected
          </span>

```

```

        <CheckCircle className="w-5 h-5 text-emerald-500" />
      </div>
      <h3 className="text-2xl font-black text-emerald-600 dark:text-emerald-400">
        {formatCurrency(totalCollected)}
      </h3>
      <p className="text-[10px] text-emerald-500 font-semibold">
        Realized revenue
      </p>
    </div>

    <div className="glass-card p-5 rounded-3xl space-y-2 border-l-4 border-l-rose-500">
      <div className="flex items-center justify-between">
        <span className="text-xs font-bold text-slate-500 uppercase">
          Total Pending Dues
        </span>
        <AlertCircle className="w-5 h-5 text-rose-500" />
      </div>
      <h3 className="text-2xl font-black text-rose-600 dark:text-rose-400">
        {formatCurrency(totalPending)}
      </h3>
      <p className="text-[10px] text-rose-500 font-semibold">
        Action required
      </p>
    </div>

    <div className="glass-card p-5 rounded-3xl space-y-2 border-l-4 border-l-sky-500">
      <div className="flex items-center justify-between">
        <span className="text-xs font-bold text-slate-500 uppercase">
          Today's Collection
        </span>
        <TrendingUp className="w-5 h-5 text-sky-500" />
      </div>
      <h3 className="text-2xl font-black text-sky-600 dark:text-sky-400">
        {formatCurrency(todaysCollection)}
      </h3>
      <p className="text-[10px] text-slate-400">Daily receipt total</p>
    </div>
  </div>

  {/* Secondary Service KPI Grid */}
  <div className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-5 gap-3.5">
    <div className="glass-card p-4 rounded-2xl flex items-center justify-between">
      <div>
        <p className="text-[11px] font-bold text-slate-500 uppercase">
          Transport Revenue
        </p>
        <h4 className="text-lg font-black text-slate-900 dark:text-white mt-0.5">
          {formatCurrency(transportCollection)}
        </h4>
      </div>
    </div>
    <div className="p-2.5 rounded-xl bg-sky-50 text-sky-600 dark:bg-sky-950 dark:text-sky-400">
      <Bus className="w-5 h-5" />
    </div>
  </div>

  <div className="glass-card p-4 rounded-2xl flex items-center justify-between">
    <div>
      <p className="text-[11px] font-bold text-slate-500 uppercase">
        Hostel Revenue
      </p>
      <h4 className="text-lg font-black text-slate-900 dark:text-white mt-0.5">
        {formatCurrency(hostelCollection)}
      </h4>
    </div>
  </div>

```

```

    </div>
    <div className="p-2.5 rounded-xl bg-sky-50 text-sky-600 dark:bg-sky-950 dark:text-sky-400">
      <Home className="w-5 h-5" />
    </div>
  </div>

  <div className="glass-card p-4 rounded-2xl flex items-center justify-between">
    <div>
      <p className="text-[11px] font-bold text-slate-500 uppercase">
        Uniform Revenue
      </p>
      <h4 className="text-lg font-black text-slate-900 dark:text-white mt-0.5">
        {formatCurrency(uniformCollection)}
      </h4>
    </div>
    <div className="p-2.5 rounded-xl bg-purple-50 text-purple-600 dark:bg-purple-950 dark:text-purple-400">
      <Shirt className="w-5 h-5" />
    </div>
  </div>

  <div className="glass-card p-4 rounded-2xl flex items-center justify-between">
    <div>
      <p className="text-[11px] font-bold text-slate-500 uppercase">
        Scholarships Granted
      </p>
      <h4 className="text-lg font-black text-slate-900 dark:text-white mt-0.5">
        {formatCurrency(scholarshipAmount)}
      </h4>
    </div>
    <div className="p-2.5 rounded-xl bg-emerald-50 text-emerald-600 dark:bg-emerald-950 dark:text-emerald-400">
      <Gift className="w-5 h-5" />
    </div>
  </div>

  <div className="glass-card p-4 rounded-2xl flex items-center justify-between">
    <div>
      <p className="text-[11px] font-bold text-slate-500 uppercase">
        Fine Collected
      </p>
      <h4 className="text-lg font-black text-slate-900 dark:text-white mt-0.5">
        {formatCurrency(fineCollection)}
      </h4>
    </div>
    <div className="p-2.5 rounded-xl bg-rose-50 text-rose-600 dark:bg-rose-950 dark:text-rose-400">
      <AlertTriangle className="w-5 h-5" />
    </div>
  </div>
</div>

/* Visual Charts Summary */
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
  /* Class Wise Breakdown */
  <div className="glass-card p-6 rounded-3xl space-y-4">
    <h3 className="font-bold text-base text-slate-900 dark:text-white flex items-center gap-2">
      <BarChart2 className="w-5 h-5 text-sky-500" /> Class-wise Revenue
    </h3>
    <div className="space-y-3">
      {apiStats?.classWiseRevenue && apiStats.classWiseRevenue.length > 0 ? (
        apiStats.classWiseRevenue.map((item: any, idx: number) => {
          const collected = Number(item.collectedAmount || 0);
          const expected = Number(item.expectedAmount || 0);

```

```

    const pct = expected > 0 ? Math.min(100, Math.round((collected / expected) * 100)) : (colle
    cted > 0 ? 100 : 0);
    return (
      <div key={idx} className="space-y-1">
        <div className="flex justify-between text-xs font-bold">
          <span className="text-slate-900 dark:text-white">
            {item.className}
          </span>
          <span className="text-slate-500">
            Collected: {formatCurrency(collected)} / Target: {formatCurrency(expected)} ({pc
            t}%)
          </span>
        </div>
        <div className="w-full bg-slate-100 dark:bg-slate-800 h-2.5 rounded-full overflow-hidde
        n">
          <div
            className="bg-sky-500 h-full rounded-full transition-all duration-500"
            style={{ width: `${pct}%` }}
          />
        </div>
      </div>
    );
  })
) : academicClasses && academicClasses.length > 0 ? (
  academicClasses.map((c) => (
    <div key={c.id} className="space-y-1">
      <div className="flex justify-between text-xs font-bold">
        <span className="text-slate-900 dark:text-white">{c.name}</span>
        <span className="text-slate-500">Collected: ₹0 / Target: ₹0 (0%)</span>
      </div>
      <div className="w-full bg-slate-100 dark:bg-slate-800 h-2.5 rounded-full overflow-hidde
      n">
        <div className="bg-sky-500 h-full rounded-full" style={{ width: "0%" }} />
      </div>
    </div>
  ))
) : (
  <div className="py-8 text-center text-slate-400 dark:text-slate-500 text-xs">
    No classes or fee assignments configured yet.
  </div>
)
</div>
</div>

{/* Fee Head Wise Breakdown */}
<div className="glass-card p-6 rounded-3xl space-y-4">
  <h3 className="font-bold text-base text-slate-900 dark:text-white flex items-center gap-2">
    <PieChart className="w-5 h-5 text-sky-500" /> Fee Collection by Category
  </h3>
  <div className="space-y-3">
    {feeHeads && feeHeads.length > 0 ? (
      feeHeads.slice(0, 5).map((h) => (
        <div
          key={h.id}
          className="p-3 rounded-2xl bg-slate-50 dark:bg-slate-800/60 flex items-center justify-bet
          ween text-xs"
        >
          <div>
            <p className="font-bold text-slate-900 dark:text-white">
              {h.name}
            </p>
            <p className="text-[10px] text-slate-400">
              {h.category} • {h.frequency}
            </p>
          </div>
        </div>
      ))
    ) : (
      <div>
        <p>No fee heads configured yet.</p>
      </div>
    )
  )
}

```

```
        </p>
      </div>
      <span className="px-2.5 py-1 rounded-lg bg-sky-100 text-sky-700 dark:bg-sky-950 dark:text
-sky-300 font-bold">
        {h.mandatory ? "Mandatory" : "Optional"}
      </span>
    </div>
  ))
) : (
  <div className="py-8 text-center text-slate-400 dark:text-slate-500 text-xs">
    No fee categories configured yet. Create fee heads in Fee Masters.
  </div>
  )}
</div>
</div>
</div>
</div>
);
};
```

 Finance/FinanceReportsView.tsx

TSX

```
// @ts-nocheck
import React, { useState, useEffect } from 'react';
import { formatCurrency } from '../../utils/currency';
import {
  FileSpreadsheet, Printer, Search, IndianRupee,
  BarChart3, CheckCircle2, AlertCircle, Gift, Tag, Bus, Home, Info
} from 'lucide-react';
import { useData } from '../../context/DataContext';
import { useAuth } from '../../context/AuthContext';
import { resolveMediaUrl } from '../../utils/mediaUtils';
import { ExportButton } from '../../common/ExportButton';
import { SchoolPrintHeader } from '../../common/SchoolPrintHeader';
import * as FinanceAPI from '../../api/finance';

const REPORT_CATEGORIES = [
  'Collection Reports',
  'Student Reports',
  'Fee Reports',
  'Service Reports'
] as const;

type ReportCategory = typeof REPORT_CATEGORIES[number];

const REPORTS_BY_CATEGORY: Record<ReportCategory, string[]> = {
  'Collection Reports': [
    'Daily Collection',
    'Monthly Collection',
    'Yearly Collection',
    'Head-wise Collection',
    'Defaulter List',
    'Class-wise Collection',
    'Outstanding Summary'
  ],
  'Student Reports': [
    'Concession Report',
    'Discount Report',
    'Scholarship Report',
    'Fee Structure Assign',
    'Student Ledger'
  ],
  'Fee Reports': [
    'Structure Matrix',
    'Schedule Status',
    'Fine Collection',
    'Head Master'
  ],
  'Service Reports': [
    'Hostel Collection',
    'Transport Collection'
  ]
};

export const FinanceReportsView: React.FC = () => {
  const {
    feePayments,
    students,
    studentTransports,
    studentHostels,
    studentScholarships,
  }
```

```

    getStudentFeeOutstandingSummary,
    getStudentFeeLedger,
    financeSettings,
    schoolProfile
  } = useData();

const { selectedAcademicYear } = useAuth();

const [selectedCategory, setSelectedCategory] = useState<ReportCategory>('Collection Reports');
const [selectedReport, setSelectedReport] = useState<string>('Daily Collection');
const [isGenerated, setIsGenerated] = useState<boolean>(false);
const [generatedReportType, setGeneratedReportType] = useState<string>('');
const [currentData, setCurrentData] = useState<any[]>([]);
const [apiSummary, setApiSummary] = useState<any>(null);

useEffect(() => {
  FinanceAPI.fetchFinanceReportsSummaryApi(selectedAcademicYear || '2025-2026')
    .then(res => {
      if (res?.data) {
        setApiSummary(res.data);
      }
    })
    .catch(err => console.warn('Failed to load reports summary', err));
}, [selectedAcademicYear]);

const handleCategoryChange = (newCategory: ReportCategory) => {
  setSelectedCategory(newCategory);
  const availableReports = REPORTS_BY_CATEGORY[newCategory] || [];
  const firstReport = availableReports[0] || '';
  setSelectedReport(firstReport);
  setIsGenerated(false);
};

const handleReportChange = (newReport: string) => {
  setSelectedReport(newReport);
  setIsGenerated(false);
};

// Search & Pagination States
const [searchQuery, setSearchQuery] = useState<string>('');
const [currentPage, setCurrentPage] = useState<number>(1);
const [pageSize, setPageSize] = useState<number>(10);

// Common Filter States
const [filterClass, setFilterClass] = useState<string>('All');
const [filterSection, setFilterSection] = useState<string>('All');
const [filterStartDate, setFilterStartDate] = useState<string>('');
const [filterEndDate, setFilterEndDate] = useState<string>('');

// Optional/Conditional Filter States
const [filterFeeHead, setFilterFeeHead] = useState<string>('All');
const [filterPaymentMode, setFilterPaymentMode] = useState<string>('All');
const [filterHostelName, setFilterHostelName] = useState<string>('All');
const [filterRouteName, setFilterRouteName] = useState<string>('All');

// Reset pagination on report type or query change
useEffect(() => {
  setCurrentPage(1);
}, [selectedReport, searchQuery]);

// Distinct values for filter dropdowns
const paymentModes = Array.from(new Set(feePayments.map(p => p.paymentMode))).filter(Boolean);
const hostelNames = Array.from(new Set(studentHostels.map(h => h.hostelName))).filter(Boolean);

```

```

const routeNames = Array.from(new Set(studentTransports.map(t => t.routeName))).filter(Boolean);
const classesList = Array.from(new Set(students.map(s => s.className))).filter(Boolean);
const sectionsList = Array.from(new Set(students.map(s => s.section))).filter(Boolean);

// Conditional filters visibility check
const showFeeHeadFilter = selectedReport === 'Fee Head Wise Collection';
const showPaymentModeFilter = ['Daily Collection', 'Monthly Collection', 'Yearly Collection', 'Collection Summary', 'Cash Book'].includes(selectedReport);
const showHostelFilter = selectedReport === 'Hostel Collection';
const showRouteFilter = selectedReport === 'Transport Collection';

// KPI Calculations
const todayStr = new Date().toISOString().split('T')[0];
const todayCollection = apiSummary?.todayCollection ?? feePayments.filter(p => p.paymentDate === todayStr).reduce((sum, p) => sum + Number(p.amountPaid || p.amount || 0), 0);

const thisMonthStr = new Date().toISOString().substring(0, 7); // YYYY-MM
const monthlyCollection = apiSummary?.monthlyCollection ?? feePayments.filter(p => p.paymentDate && p.paymentDate.startsWith(thisMonthStr)).reduce((sum, p) => sum + Number(p.amountPaid || p.amount || 0), 0);

const pendingFees = apiSummary?.pendingDues ?? students.reduce((sum, s) => sum + (getStudentFeeOutstandingSummary ? getStudentFeeOutstandingSummary(s.id).totalOutstanding : 0), 0);
const distinctPaidStudents = apiSummary?.studentsPaidCount ?? new Set(feePayments.map(p => p.studentId)).size;
const totalScholarshipsAmount = apiSummary?.scholarshipsAndDiscounts ?? studentScholarships.reduce((sum, s) => sum + Number(s.discountValue || 0), 0);
const totalDiscountsAmount = feePayments.reduce((sum, p) => sum + Number(p.discount || 0), 0);

const transportRevenue = apiSummary?.transportAndHostel ?? studentTransports.reduce((sum, t) => sum + Number(t.feeAmount || 0), 0);
const hostelRevenue = studentHostels.reduce((sum, h) => sum + Number(h.feeAmount || 0), 0);

const handleGenerate = () => {
  let result: any[] = [];
  switch (selectedReport) {
    case 'Transport Collection':
      result = studentTransports.map(t => ({
        studentName: t.studentName,
        admissionNo: t.admissionNo,
        routeName: t.routeName,
        feePlan: t.feePlan,
        amount: t.feeAmount
      }));
      break;
    case 'Hostel Collection':
      result = studentHostels.map(h => ({
        studentName: h.studentName,
        hostelName: h.hostelName,
        roomNo: h.roomNo,
        amount: h.feeAmount
      }));
      break;
    case 'Scholarship Report':
      result = studentScholarships.map(s => ({
        studentName: s.studentName,
        scholarshipName: s.scholarshipName,
        discount: `${formatCurrency(s.discountValue)} (${s.discountType})`,
        appliedDate: s.appliedDate
      }));
      break;
    case 'Pending Fees':
      result = students.map(s => ({ s, summary: getStudentFeeOutstandingSummary(s.id) })).filter(item => item.summary.totalOutstanding > 0).map(({ s, summary }) => ({

```

```

        studentName: `${s.firstName} ${s.lastName}`,
        classSection: `${s.className}-${s.section}`,
        currentYearDue: summary.currentYearDue,
        previousYearsDue: summary.previousYearsDue,
        totalOutstanding: summary.totalOutstanding
    }));
    break;
case 'Term-wise Collection':
    students.forEach(st => {
        const ledger = getStudentFeeLedger(st.id, selectedAcademicYear || financeSettings?.academicYear |
| '2026-2027');
        if (ledger && ledger.installments) {
            ledger.installments.forEach(inst => {
                if (inst.paidAmount > 0) {
                    result.push({
                        studentName: `${st.firstName} ${st.lastName}`,
                        admissionNo: st.admissionNo,
                        classSection: `${st.className}-${st.section}`,
                        termName: inst.termName,
                        feeHead: inst.feeHeadName,
                        dueDate: inst.dueDate,
                        amount: inst.paidAmount
                    });
                }
            });
        }
    });
    break;
case 'Term-wise Outstanding':
    students.forEach(st => {
        const ledger = getStudentFeeLedger(st.id, selectedAcademicYear || financeSettings?.academicYear |
| '2026-2027');
        if (ledger && ledger.installments) {
            ledger.installments.forEach(inst => {
                if (inst.dueAmount > 0) {
                    result.push({
                        studentName: `${st.firstName} ${st.lastName}`,
                        admissionNo: st.admissionNo,
                        classSection: `${st.className}-${st.section}`,
                        termName: inst.termName,
                        feeHead: inst.feeHeadName,
                        dueDate: inst.dueDate,
                        amount: inst.dueAmount
                    });
                }
            });
        }
    });
    break;
case 'Overdue Collection':
    {
        const todayStr = new Date().toISOString().split('T')[0];
        students.forEach(st => {
            const ledger = getStudentFeeLedger(st.id, selectedAcademicYear || financeSettings?.academicYear
| '2026-2027');
            if (ledger && ledger.installments) {
                ledger.installments.forEach(inst => {
                    if (inst.dueAmount > 0 && inst.dueDate < todayStr) {
                        result.push({
                            studentName: `${st.firstName} ${st.lastName}`,
                            admissionNo: st.admissionNo,
                            classSection: `${st.className}-${st.section}`,
                            termName: inst.termName,

```

```

        feeHead: inst.feeHeadName,
        dueDate: inst.dueDate,
        amount: inst.dueAmount
    });
    }
    });
    }
    });
    }
    break;
case 'Upcoming Dues':
    {
        const currentToday = new Date().toISOString().split('T')[0];
        students.forEach(st => {
            const ledger = getStudentFeeLedger(st.id, selectedAcademicYear || financeSettings?.academicYear || '2026-2027');
            if (ledger && ledger.installments) {
                ledger.installments.forEach(inst => {
                    if (inst.dueAmount > 0 && inst.dueDate >= currentToday) {
                        result.push({
                            studentName: `${st.firstName} ${st.lastName}`,
                            admissionNo: st.admissionNo,
                            classSection: `${st.className}-${st.section}`,
                            termName: inst.termName,
                            feeHead: inst.feeHeadName,
                            dueDate: inst.dueDate,
                            amount: inst.dueAmount
                        });
                    }
                });
            }
        });
    }
    break;
case 'Academic Year Collection':
    result = feePayments.filter(p => p.academicYear === (selectedAcademicYear || financeSettings?.academicYear || '2026-2027')).map(p => ({
        receiptNo: p.receiptNo,
        studentName: p.studentName,
        classSection: p.className,
        paymentMode: p.paymentMode,
        paymentDate: p.paymentDate,
        amount: p.amountPaid
    }));
    break;
case 'Outstanding by Academic Year':
    students.forEach(st => {
        const ledger = getStudentFeeLedger(st.id, selectedAcademicYear || financeSettings?.academicYear || '2026-2027');
        if (ledger && ledger.dueBalance > 0) {
            result.push({
                studentName: `${st.firstName} ${st.lastName}`,
                admissionNo: st.admissionNo,
                classSection: `${st.className}-${st.section}`,
                academicYear: selectedAcademicYear || financeSettings?.academicYear || '2026-2027',
                amount: ledger.dueBalance
            });
        }
    });
    break;
default:
    // Daily Collection, Monthly Collection, Yearly Collection, Branch Wise Collection,
    // Class Wise Collection, Section Wise Collection, Fee Head Wise Collection,

```

```

// Fine Report (Fine Collection), Collection Summary, Cash Book
result = feePayments.map(p => ({
  receiptNo: p.receiptNo,
  studentName: p.studentName,
  classSection: p.className,
  amount: p.amountPaid,
  paymentMode: p.paymentMode,
  paymentDate: p.paymentDate
})));
break;
}

// Apply Filters
if (filterClass !== 'All') {
  result = result.filter(item => {
    const val = item.classSection || item.class || '';
    return val.toLowerCase().includes(filterClass.toLowerCase());
  });
}
if (filterSection !== 'All') {
  result = result.filter(item => {
    const val = item.classSection || item.class || '';
    return val.toLowerCase().includes(`-${filterSection}`.toLowerCase()) || val.toLowerCase().endsWith(
(filterSection.toLowerCase());
  });
}
if (filterStartDate) {
  result = result.filter(item => {
    const d = item.paymentDate || item.appliedDate || '';
    return !d || new Date(d) >= new Date(filterStartDate);
  });
}
if (filterEndDate) {
  result = result.filter(item => {
    const d = item.paymentDate || item.appliedDate || '';
    return !d || new Date(d) <= new Date(filterEndDate);
  });
}
if (showPaymentModeFilter && filterPaymentMode !== 'All') {
  result = result.filter(item => item.paymentMode === filterPaymentMode);
}
if (showHostelFilter && filterHostelName !== 'All') {
  result = result.filter(item => item.hostelName === filterHostelName);
}
if (showRouteFilter && filterRouteName !== 'All') {
  result = result.filter(item => item.routeName === filterRouteName);
}

setCurrentData(result);
setGeneratedReportType(selectedReport);
setIsGenerated(true);
};

// Search filter
const displayedData = currentData.filter(item => {
  const q = searchQuery.toLowerCase();
  return (
    (item.studentName && item.studentName.toLowerCase().includes(q)) ||
    (item.admissionNo && item.admissionNo.toLowerCase().includes(q)) ||
    (item.receiptNo && item.receiptNo.toLowerCase().includes(q)) ||
    (item.routeName && item.routeName.toLowerCase().includes(q)) ||
    (item.hostelName && item.hostelName.toLowerCase().includes(q))
  );
});

```

```

});

// Totals calculations
const totalAmountSum = displayedData.reduce((sum, item) => sum + (item.amount || item.dueFee || item.fee || 0), 0);

// Pagination calculations
const totalRecords = displayedData.length;
const totalPages = Math.ceil(totalRecords / pageSize);
const paginatedData = displayedData.slice((currentPage - 1) * pageSize, currentPage * pageSize);

const handlePrint = () => {
  const printWindow = window.open('', '_blank');
  if (!printWindow) return;

  const logoUrl = resolveMediaUrl(schoolProfile?.logoUrl);
  const schoolName = schoolProfile?.name || 'Pirnav Educational Institutions';
  const tagline = schoolProfile?.tagline || 'Empowering Minds, Shaping Tomorrow';
  const address = schoolProfile?.address || 'Jain Sadguru Images Capital Park502B, Capital Pk Rd, VIP Hills, Madhapur, HITEC City, Hyderabad, Telangana 500081';
  const phone = schoolProfile?.phone || '+91 9123456789';
  const email = schoolProfile?.email || 'contact@pirnavschools.edu';
  const academicYearStr = schoolProfile?.academicYear || '2026-2027';

  const headers = Object.keys(currentData[0] || {}).slice(0, 7);
  const rowsHtml = displayedData.map(row => `
    <tr>
      ${headers.map(h => `<td style="padding:8px; border:1px solid #ddd;">${row[h] !== undefined ? row[h] : ''}</td>`).join('')}
    </tr>
  `).join('');

  printWindow.document.write(`
    <html>
      <head>
        <title>${generatedReportType} Report</title>
        <style>
          body { font-family: 'Segoe UI', sans-serif; padding: 20px; color: #1e293b; }
          .header-container { display: flex; align-items: center; justify-content: space-between; gap: 20px; border-bottom: 2px solid #0f172a; padding-bottom: 14px; margin-bottom: 20px; }
          .school-logo { width: 110px; height: 110px; object-fit: contain; flex-shrink: 0; border-radius: 14px; }
          .school-details { flex: 1; text-align: center; }
          .school-name { font-size: 24px; font-weight: 900; color: #0f172a; text-transform: uppercase; letter-spacing: 0.8px; margin: 0; text-align: center; }
          .school-tagline { font-size: 12px; font-weight: 700; color: #0369a1; font-style: italic; margin-top: 2px; text-align: center; }
          .school-address { font-size: 11px; font-weight: 600; color: #475569; margin-top: 3px; text-align: center; }
          .school-meta { font-size: 10px; font-weight: 700; color: #64748b; margin-top: 4px; display: flex; justify-content: center; align-items: center; gap: 8px; flex-wrap: wrap; text-align: center; }
          .report-title-bar { margin-top: 14px; padding-top: 10px; border-top: 1px solid #e2e8f0; text-align: center; }
          .report-title { font-size: 14px; font-weight: 900; text-transform: uppercase; color: #0f172a; }
          .report-subtitle { font-size: 11px; color: #64748b; margin-top: 2px; }
          table { width: 100%; border-collapse: collapse; font-size: 11px; margin-top: 16px; }
          th { background: #f1f5f9; padding: 8px; border: 1px solid #cbd5e1; text-align: left; font-weight: 800; }
        </style>
      </head>
      <body>
        <div class="header-container">
          ${logoUrl ? `` : `<div style="width: 110px; height: 110px; border-radius: 14px; background-color: #e2e8f0; display: flex; align-items: center; justify-content: center; font-size: 10px; font-weight: 700; color: #64748b; text-align: center; margin: 0; text-align: center;`

```

```

idth:100px; height:100px; background:#0284c7; color:#ffffff; border-radius:16px; display:flex; align-items:
center; justify-content:center; font-weight:900; font-size:36px; flex-shrink:0;">P</div>`}
    <div class="school-details">
        <h1 class="school-name">${schoolName}</h1>
        ${tagline ? `<div class="school-tagline">${tagline}</div>` : ''}
        <div class="school-address">${address}</div>
        <div class="school-meta">Ph: ${phone} • Email: ${email} • Acad. Year: ${academicYearStr}</div>
    >

    </div>
</div>
<div class="report-title-bar">
    <div class="report-title">Finance Report – ${generatedReportType}</div>
    <div class="report-subtitle">Generated Date: ${new Date().toLocaleDateString()} • Total Record
s: ${totalRecords} • Filtered Total: ${formatCurrency(totalAmountSum)}</div>
</div>
<table>
    <thead>
        <tr>${headers.map(h => `<th>${h.toUpperCase()}</th>`).join('')}</tr>
    </thead>
    <tbody>
        ${rowsHtml}
    </tbody>
</table>
<script>
    window.onload = function() { window.print(); }
</script>
</body>
</html>
`);
printWindow.document.close();
};

return (
    <div className="space-y-8 animate-in fade-in">
        {/* Title */}
        <div className="no-print">
            <h2 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white flex items-center gap-
2">
                <FileSpreadsheet className="w-6 h-6 text-sky-500" />Finance Reports
            </h2>
        </div>

        {/* KPI Cards (Hidden in Print) */}
        <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-4 no-print">
            <div className="glass-card p-4.5 rounded-3xl space-y-1.5 border-l-4 border-l-sky-500 bg-white dark:
bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm">
                <span className="text-[10px] font-extrabold text-slate-400 uppercase">Today's Collection</span>
                <h4 className="text-sm sm:text-base font-black text-slate-900 dark:text-white truncate">${formatCu
rrency(todayCollection)}</h4>
                <span className="text-[9px] text-sky-500 font-bold">Daily Receipts</span>
            </div>

            <div className="glass-card p-4.5 rounded-3xl space-y-1.5 border-l-4 border-l-emerald-500 bg-white d
ark:bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm">
                <span className="text-[10px] font-extrabold text-slate-400 uppercase">Monthly Collection</span>
                <h4 className="text-sm sm:text-base font-black text-emerald-600 dark:text-emerald-400 truncate">
${formatCurrency(monthlyCollection)}</h4>
                <span className="text-[9px] text-emerald-500 font-bold">Monthly Receipts</span>
            </div>

            <div className="glass-card p-4.5 rounded-3xl space-y-1.5 border-l-4 border-l-rose-500 bg-white dar
k:bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm">
                <span className="text-[10px] font-extrabold text-slate-400 uppercase">Pending Dues</span>
            </div>
        </div>
    </div>
)

```

```

        <h4 className="text-sm sm:text-base font-black text-rose-600 dark:text-rose-400 truncate">{format
Currency(pendingFees)}</h4>
        <span className="text-[9px] text-rose-500 font-bold">Outstanding dues</span>
    </div>

    <div className="glass-card p-4.5 rounded-3xl space-y-1.5 border-l-4 border-l-sky-500 bg-white dark:
bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm">
        <span className="text-[10px] font-extrabold text-slate-400 uppercase">Students Paid</span>
        <h4 className="text-sm sm:text-base font-black text-sky-600 dark:text-sky-400 truncate">{distinct
PaidStudents} Students</h4>
        <span className="text-[9px] text-slate-400 font-bold">Distinct Payees</span>
    </div>

    <div className="glass-card p-4.5 rounded-3xl space-y-1.5 border-l-4 border-l-sky-500 bg-white dark:
bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm">
        <span className="text-[10px] font-extrabold text-slate-400 uppercase">Scholarships & Discounts</s
pan>
        <h4 className="text-sm sm:text-base font-black text-sky-600 dark:text-sky-400 truncate">{formatCu
rrency(totalScholarshipsAmount + totalDiscountsAmount)}</h4>
        <span className="text-[9px] text-slate-400 font-bold">Total Concessions</span>
    </div>

    <div className="glass-card p-4.5 rounded-3xl space-y-1.5 border-l-4 border-l-amber-500 bg-white dar
k:bg-slate-900 border border-slate-200 dark:border-slate-800 shadow-sm">
        <span className="text-[10px] font-extrabold text-slate-400 uppercase">Transport & Hostel</span>
        <h4 className="text-sm sm:text-base font-black text-amber-600 dark:text-amber-400 truncate">{form
atCurrency(transportRevenue + hostelRevenue)}</h4>
        <span className="text-[9px] text-slate-400 font-bold">Services Collection</span>
    </div>
</div>

    {/* Report Selector & Filter Panel (Hidden in Print) */}
    <div className="glass-card p-6 rounded-3xl bg-white dark:bg-slate-900 border border-slate-200 dark:bo
rder-slate-800 shadow-sm space-y-6 no-print">
        <h3 className="text-xs font-extrabold text-slate-400 uppercase tracking-wider">Report Selection & P
arameters</h3>

        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
            {/* Level 1: Report Category Dropdown */}
            <div className="space-y-1">
                <label className="text-[10px] font-bold text-slate-500 block">Report Category</label>
                <select
                    value={selectedCategory}
                    onChange={e => handleCategoryChange(e.target.value as ReportCategory)}
                    className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border borde
r-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer
shadow-xs transition hover:border-slate-300 dark:hover:border-slate-600"
                >
                    {REPORT_CATEGORIES.map(cat => (
                        <option key={cat} value={cat}>{cat}</option>
                    ))}
                </select>
            </div>

            {/* Level 2: Report Dropdown */}
            <div className="space-y-1">
                <label className="text-[10px] font-bold text-slate-500 block">Report</label>
                <select
                    value={selectedReport}
                    onChange={e => handleReportChange(e.target.value)}
                    className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border borde
r-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer
shadow-xs transition hover:border-slate-300 dark:hover:border-slate-600"
                >

```

```

    >
    {{(REPORTS_BY_CATEGORY[selectedCategory] || []).map(rep => (
      <option key={rep} value={rep}>{rep}</option>
    ))}}
  </select>
</div>

{/* Class */}
<div className="space-y-1">
  <label className="text-[10px] font-bold text-slate-500 block">Class</label>
  <select
    value={filterClass}
    onChange={e => setFilterClass(e.target.value)}
    className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-
r-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer"
  >
    <option value="All">All Classes</option>
    {classesList.map(c => (
      <option key={c} value={c}>{c}</option>
    ))}
  </select>
</div>

{/* Section */}
<div className="space-y-1">
  <label className="text-[10px] font-bold text-slate-500 block">Section</label>
  <select
    value={filterSection}
    onChange={e => setFilterSection(e.target.value)}
    className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-
r-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer"
  >
    <option value="All">All Sections</option>
    {sectionsList.map(s => (
      <option key={s} value={s}>{s}</option>
    ))}
  </select>
</div>
</div>

{/* Dynamic / Optional Filters Panel */}
{
  (showPaymentModeFilter || showHostelFilter || showRouteFilter || showFeeHeadFilter || filterStartDate || filterEndDate) && (
    <div className="border-t border-slate-100 dark:border-slate-800 pt-4 grid grid-cols-1 md:grid-col
s-2 lg:grid-cols-4 gap-4 animate-in fade-in duration-200">
      {/* Payment Mode */}
      {showPaymentModeFilter && (
        <div className="space-y-1">
          <label className="text-[10px] font-bold text-slate-500 block">Payment Mode</label>
          <select
            value={filterPaymentMode}
            onChange={e => setFilterPaymentMode(e.target.value)}
            className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border b
order-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-poin
ter"
          >
            <option value="All">All Modes</option>
            {paymentModes.map(m => (
              <option key={m} value={m}>{m}</option>
            ))}
          </select>
        </div>
      )}
    </div>
  )}

```

```

    { /* Hostel Selector */ }
    { showHostelFilter && (
      <div className="space-y-1">
        <label className="text-[10px] font-bold text-slate-500 block">Select Hostel</label>
        <select
          value={filterHostelName}
          onChange={e => setFilterHostelName(e.target.value)}
          className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer"
        >
          <option value="All">All Hostels</option>
          {hostelNames.map(h => (
            <option key={h} value={h}>{h}</option>
          ))}
        </select>
      </div>
    ) }

    { /* Route Selector */ }
    { showRouteFilter && (
      <div className="space-y-1">
        <label className="text-[10px] font-bold text-slate-500 block">Select Route</label>
        <select
          value={filterRouteName}
          onChange={e => setFilterRouteName(e.target.value)}
          className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer"
        >
          <option value="All">All Routes</option>
          {routeNames.map(r => (
            <option key={r} value={r}>{r}</option>
          ))}
        </select>
      </div>
    ) }

    { /* Fee Head (for Fee Head Wise Collection) */ }
    { showFeeHeadFilter && (
      <div className="space-y-1">
        <label className="text-[10px] font-bold text-slate-500 block">Select Fee Head</label>
        <select
          value={filterFeeHead}
          onChange={e => setFilterFeeHead(e.target.value)}
          className="w-full px-3.5 py-2.5 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer"
        >
          <option value="All">All Fee Types</option>
          <option value="Tuition Fee">Tuition Fee</option>
          <option value="Admission Fee">Admission Fee</option>
          <option value="Books Fee">Books Fee</option>
          <option value="Hostel Fee">Hostel Fee</option>
          <option value="Transport Fee">Transport Fee</option>
        </select>
      </div>
    ) }

    { /* Date Range Start */ }
    <div className="space-y-1">
      <label className="text-[10px] font-bold text-slate-500 block">Start Date</label>

```

```

        <input
          type="date"
          value={filterStartDate}
          onChange={e => setFilterStartDate(e.target.value)}
          className="w-full px-3.5 py-2 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-
r-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer"
        />
      </div>

      { /* Date Range End */ }
      <div className="space-y-1">
        <label className="text-[10px] font-bold text-slate-500 block">End Date</label>
        <input
          type="date"
          value={filterEndDate}
          onChange={e => setFilterEndDate(e.target.value)}
          className="w-full px-3.5 py-2 text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-
r-slate-200 dark:border-slate-700 font-bold outline-none text-slate-800 dark:text-slate-200 cursor-pointer"
        />
      </div>
    </div>
  )}

  { /* Generate Primary Button */ }
  <div className="flex justify-end pt-2">
    <button
      onClick={handleGenerate}
      className="px-6 py-2.5 text-xs font-bold text-white bg-sky-600 hover:bg-sky-500 rounded-xl shad
ow-md shadow-sky-500/25 transition-all flex items-center gap-2"
    >
      <BarChart3 className="w-4 h-4" /> Generate Report
    </button>
  </div>
</div>

{ /* Generated Report Output */ }
{isGenerated ? (
  <div className="glass-card p-6 rounded-3xl bg-white dark:bg-slate-900 border border-slate-200 dark:
border-slate-800 shadow-sm space-y-4">
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 border-b border-s
late-100 dark:border-slate-800 pb-3.5">
      <div>
        <h3 className="font-extrabold text-base text-slate-900 dark:text-white flex items-center gap-
2">
          <BarChart3 className="w-5 h-5 text-sky-500" /> {generatedReportType}
        </h3>
        <p className="text-[10px] text-slate-400 font-semibold mt-0.5">Total Records Found: {totalRec
ords} • Current Page: {currentPage} / {totalPages} || 1</p>
      </div>

      <div className="flex items-center gap-3">
        <button
          onClick={handlePrint}
          className="px-3.5 py-2 rounded-xl bg-slate-100 dark:bg-slate-800 hover:bg-slate-200 dark:ho
ver:bg-slate-700 text-slate-700 dark:text-slate-200 text-xs font-bold transition-all flex items-center gap-
2 border border-slate-200 dark:border-slate-700"
        >
          <Printer className="w-4 h-4 text-slate-500" /> Print
        </button>
        <ExportButton data={displayedData} filename={`report_${generatedReportType.toLowerCase().repl
ace(/\s+/g, '_')}`} />
      </div>
    </div>
  )}

```

```

    { /* Search Box & Page Size */ }
    <div className="flex flex-col sm:flex-row items-center justify-between gap-4">
      <div className="relative w-full sm:w-72">
        <span className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none">
          <Search className="h-3.5 w-3.5 text-slate-400" />
        </span>
        <input
          type="text"
          value={searchQuery}
          onChange={e => setSearchQuery(e.target.value)}
          placeholder="Search name, receipt, admission..."
          className="pl-9 pr-4 py-2 w-full text-xs rounded-xl bg-slate-50 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 outline-none font-bold text-slate-800 dark:text-slate-200"
        />
      </div>

      <div className="flex items-center gap-2 text-xs font-bold text-slate-500 self-end sm:self-auto">
        <span>Show</span>
        <select
          value={pageSize}
          onChange={e => {
            setPageSize(Number(e.target.value));
            setCurrentPage(1);
          }}
          className="px-2.5 py-1.5 rounded-lg bg-slate-50 dark:bg-slate-800 border border-slate-200 dark:border-slate-700 outline-none font-extrabold text-slate-800 dark:text-slate-200"
        >
          <option value={5}>5 Rows</option>
          <option value={10}>10 Rows</option>
          <option value={25}>25 Rows</option>
          <option value={50}>50 Rows</option>
        </select>
      </div>
    </div>

    { /* Dynamic Table */ }
    <div id="printable-content" className="overflow-x-auto border border-slate-100 dark:border-slate-800 rounded-2xl p-4 print:p-0 print:border-none">
      <SchoolPrintHeader
        title={`Finance Report - ${generatedReportType}`}
        subtitle={`Total Filtered Records: ${currentData.length}`}
      />
      <table className="w-full text-left border-collapse text-xs">
        <thead>
          <tr className="bg-slate-50 dark:bg-slate-800/50 text-slate-500 font-bold uppercase tracking-wider border-b border-slate-100 dark:border-slate-800">
            {Object.keys(currentData[0] || { Record: '' }).map(key => (
              <th key={key} className="py-3 px-4 capitalize">{key.replace(/([A-Z])/g, ' $1')}</th>
            ))}
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-100 dark:divide-slate-800/80 font-medium">
          {paginatedData.length === 0 ? (
            <tr>
              <td colspan={10} className="py-8 text-center text-slate-400 font-bold">No records matching search query.</td>
            </tr>
          ) : (
            paginatedData.map((row, idx) => (
              <tr key={idx} className="hover:bg-slate-50/50 dark:hover:bg-slate-800/40">
                {Object.entries(row).map(([key, val]: any, valIdx) => (

```

```

                <td key={valIdx} className="py-3.5 px-4 font-bold text-slate-900 dark:text-white">
                    {key.toLowerCase().includes('amount') || key.toLowerCase().includes('fee') || key
                    === 'cost' ? formatCurrency(val) : String(val)}
                </td>
            )}}
        </tr>
    )}
  </tbody>
</table>
</div>

  {/* Table Totals Summary Bar */}
  {displayedData.length > 0 && (
    <div className="p-4 rounded-2xl bg-sky-50/50 dark:bg-sky-950/20 border border-sky-100/60 dark:b
order-sky-900/30 flex items-center justify-between text-xs font-bold text-sky-700 dark:text-sky-300">
      <span>Filtered Summary (Sum of Amounts):</span>
      <span className="font-black text-sm">{formatCurrency(totalAmountSum)}</span>
    </div>
  )}

  {/* Pagination Controls */}
  {totalPages > 1 && (
    <div className="flex items-center justify-between pt-2 border-t border-slate-100 dark:border-sl
ate-800">
      <span className="text-[11px] font-bold text-slate-400">Showing {Math.min(totalRecords, (curre
ntPage - 1) * pageSize + 1)}-{Math.min(totalRecords, currentPage * pageSize)} of {totalRecords} records</sp
an>

      <div className="flex items-center gap-1.5">
        <button
          disabled={currentPage === 1}
          onClick={() => setCurrentPage(currentPage - 1)}
          className="px-3 py-1.5 text-[10px] font-bold rounded-lg border bg-white dark:bg-slate-900
disabled:opacity-40 hover:bg-slate-50"
        >
          Previous
        </button>
        {Array.from({ length: totalPages }).map((_, idx) => (
          <button
            key={idx}
            onClick={() => setCurrentPage(idx + 1)}
            className={`w-7 h-7 text-[10px] font-bold rounded-lg transition-all ${
              currentPage === idx + 1
                ? 'bg-sky-600 text-white'
                : 'border bg-white dark:bg-slate-900 hover:bg-slate-50'
            }`}
          >
            {idx + 1}
          </button>
        ))}
        <button
          disabled={currentPage === totalPages}
          onClick={() => setCurrentPage(currentPage + 1)}
          className="px-3 py-1.5 text-[10px] font-bold rounded-lg border bg-white dark:bg-slate-900
disabled:opacity-40 hover:bg-slate-50"
        >
          Next
        </button>
      </div>
    </div>
  )}
</div>
) : (

```

```
/* Empty State */
<div className="glass-card p-12 text-center rounded-3xl border border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900 flex flex-col items-center justify-center space-y-3 shadow-sm">
  <div className="p-3.5 rounded-2xl bg-sky-50 dark:bg-sky-950/60 text-sky-600 dark:text-sky-400">
    <Info className="w-6 h-6" />
  </div>
  <div>
    <h4 className="font-bold text-slate-800 dark:text-white text-sm">No Report Generated Yet</h4>
    <p className="text-xs text-slate-400 max-w-sm mt-1">Select a report type, configure target search parameters/filters, and click "Generate Report" to view analytics.</p>
  </div>
</div>
  </div>
);
};
```